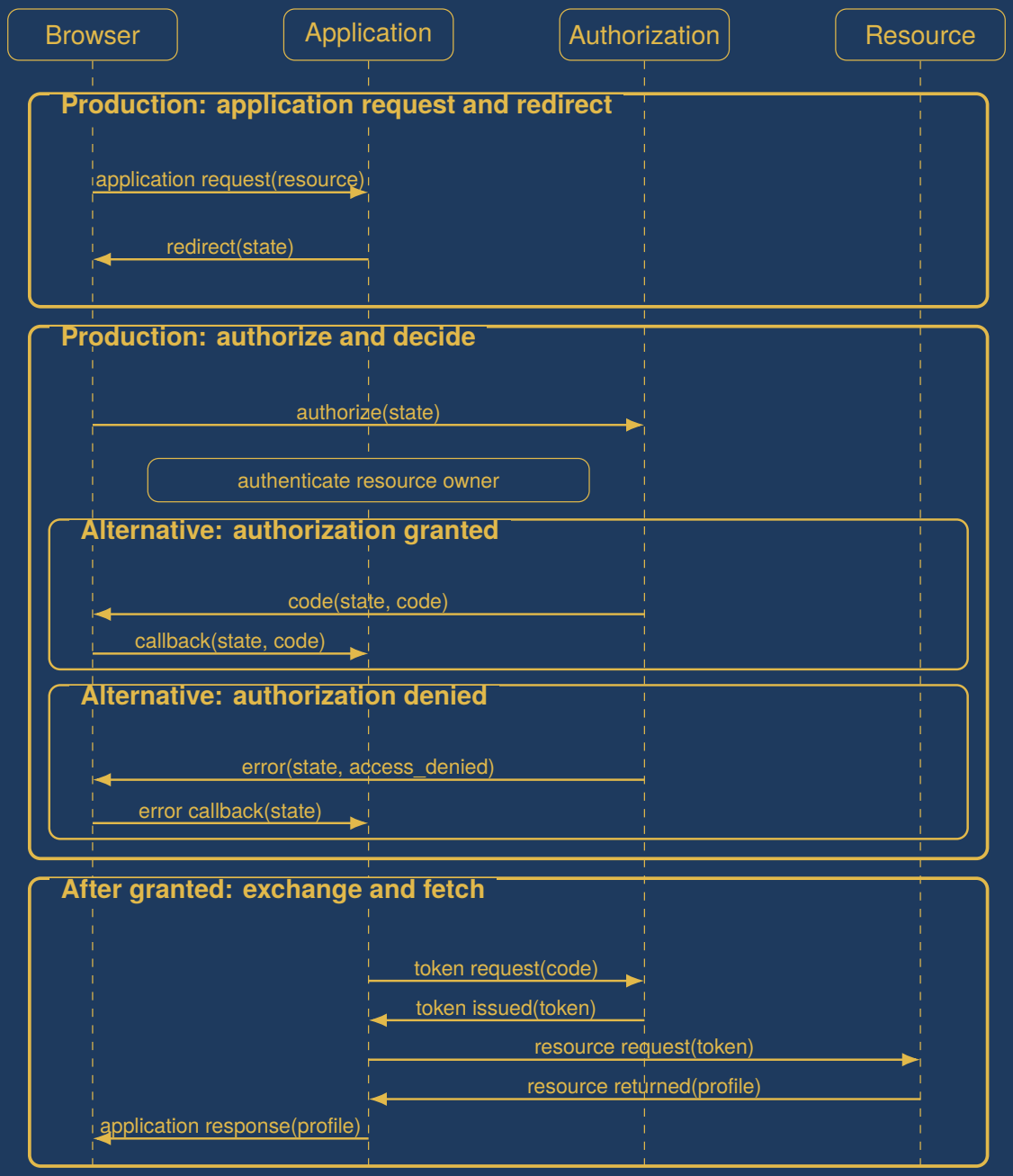
L∃∀NFM Software Engineering

Session types with bounded queues and unreliable messages



Rob Fielding and ChatGPT

Working draft—November 5, 2026

Preface

Every specification effort tries to turn what people said into an explicit record of what was understood. LeanFM does this for one subject, the *behavior* of software at its boundaries: who sends what to whom, in what order, within what time and with what probability, and what must always or never happen.

The record LeanFM keeps is a *multiparty session type*: one global description of a conversation between roles, from which each role's own obligations are derived. Session types come from programming-language research, where they guarantee that the parties to a conversation never send a message the other cannot handle and never wait for one that will not come. LeanFM uses them as a software engineering method, for messages between network nodes and for calls between APIs alike, and adds what the classic theory leaves out: every queue has a declared size, and every channel declares how it fails. Messages are lost, duplicated, reordered and late; timers fire early; a retried request arrives twice. Those facts are part of what a protocol means, and LeanFM checks the protocol together with them. Its distinguishing feature is that the record is not passive. It answers questions with generated examples and counterexamples, so misunderstandings surface while the specification is being written rather than after the code ships.

LeanFM is a way of building software by arguing about it first. You describe what a system should do in ordinary language; an LLM turns that into precise requirements; and LeanFM checks those requirements and shows you what they mean—as sequence diagrams, state machines, event traces and counterexamples. You push back, the requirements change, and the pictures change with them. Only when you are satisfied does any code get written.

The precise part is Lean. Every requirement is a Lean definition: which roles exist, which messages they exchange down to the protobuf bytes, which orders of messages are legal, and what must always or never happen. Because these are definitions rather than prose, they can be checked, and the diagrams are generated from them rather than drawn by hand. When two requirements contradict each other—say A must reference B, B needs a C, and policy forbids that C—LeanFM finds the contradiction before anyone builds it.

Not everything worth requiring is a message on a wire. Some requirements are about values and relationships, such as an identifier that must refer to a real object. LeanFM states those as predicates in the requirements and then says, in a separate implementation mapping, how the code will enforce them. Requirements over hidden state that no observer can see are flagged, so they can be restated in terms of events that can actually be tested.

Three layers, reviewed in git

Work proceeds in three layers: first the *requirements*, the observable behaviour and the views generated from it; then the *implementation mapping*, how each requirement will be realized and

tested; then the code, written in one pass. Each layer is checked by LeanFM before the next is built on it. Review is what you already do with git: you read the diff and commit it. LeanFM has no approval step of its own and never makes you click to continue. A passing check is evidence for your review; it is not the review.

Nothing lives only in the chat

A conversation with an LLM is temporary. Everything that matters is written into the project: the book, the requirements, the protobuf schema, the implementation mapping and the source. Each project has one directory that holds all of its state, so you can delete the conversation, delete a project, or start a new session and pick up exactly where the files leave off. This repository is itself such a project; the book you are reading is part of its specification and is read by the agents that work on it.

How to read this book

Chapter 1 sets out the method: session types, what they assume about channels, and what LeanFM declares instead. The chapters that follow build it up: events and scenarios as recorded runs of a session, a worked bakery example, global types written as grammars, the TFTP example in which declaring the physics turns a perfect session type into a famous bug, and the authoring language. Chapter 8 covers the LeanFM server, including the MCP interface that lets an agent consult LeanFM's logic directly. Later chapters apply temporal and modal logic to security and legal examples, and Chapter 19 defines exactly how a new project is set up.

LeanFM is a working draft. The method and its tools are still changing, and the book records what exists today alongside what is planned; where it makes a claim, the evidence labels in Chapter 1 say what kind of evidence stands behind it—compiled, tested, proposed or planned. The source is public [7].

Contents

Preface i

I Session Types as a Software Engineering Method 1

1 Session Types as a Software Engineering Method 2
- 1.1 Session types in one page . . . 3
- 1.2 What the theory assumes, and what LeanFM declares . . . 5
- 1.3 Every boundary is a channel . . . 5
  - 1.3.1 Masked and surfaced failures . . . 7
  - 1.3.2 A hierarchy of interfaces . . . 7
  - 1.3.3 The deployment is a tree . . . 7
  - 1.3.4 Collapsed interaction diagrams . . . 10
  - 1.3.5 Cut and un-cut: what HTTP allows . . . 12
- 1.4 Choice is always internal . . . 13
- 1.5 The requirement is the global type . . . 14
- 1.6 Comparison with TLA+ . . . 14
  - 1.6.1 An executable queue model . . . 14
  - 1.6.2 The cost of irrelevant state . . . 15
  - 1.6.3 Proposed message-first notation . . . 16
  - 1.6.4 Requirements at the message boundary . . . 17
- 1.7 Discovery first, consolidation before commitment . . . 17
- 1.8 Four ingredients . . . 18
- 1.9 A test for every abstraction . . . 18

2 Session Histories Made of Events 19
- 2.1 Compute everything from a fork/join event list . . . 20
  - 2.1.1 Nested joins: choose a winner, then cancel the rest . . . 20
  - 2.1.2 Durations from boundary events . . . 22
- 2.2 Derived state . . . 24
- 2.3 Time is more than a timestamp . . . 24
- 2.4 Estimating completion from an event stream . . . 25
- 2.5 Charts as declarative trace reductions . . . 25
- 2.6 Round trip through bytes . . . 26

3 Scenarios **27**
3.1 Event envelope and scenario grouping . . . 27
3.2 Derived outputs and provenance . . . 27
3.3 Example: minimal two-player game scenario . . . 27

4 The Bakery: Declared and Measured **29**
4.1 The morning, generated . . . 29
4.2 A sale: what can happen . . . 29
4.3 What we expected, and what the corpus shows . . . 30
4.4 The same questions under both overlays . . . 31
4.5 The deadline lesson: a mean hides the spread . . . 31
4.6 Generated runs . . . 32
4.7 Money and waste . . . 32

5 Global Types as Grammars **34**
5.1 The protocol syntax . . . 34
5.2 A BNF for composable interactions . . . 36
5.3 Parameters carry information . . . 39
5.4 Projection: from the global type to each role . . . 39
5.5 Declared physics . . . 40
5.6 Local types run on actors with bounded queues . . . 41

6 Declared Physics: TFTP and the Sorcerer's Apprentice **44**
6.1 The grammar . . . 44
6.2 The physics . . . 45
6.3 The question . . . 45
6.4 Liveness needs fairness . . . 46
6.5 What the classic session-type reading leaves out . . . 47

7 The Scenario Authoring Language **48**
7.1 Proposed scenario notation . . . 48
7.2 Worked MDP: a stochastic user and deterministic server . . . 49
7.3 Actor specifications and instances . . . 51
7.4 Non-functional requirements: rates and scalability . . . 51
7.4.1 The three USL regimes and capacity . . . 52
7.5 Outages, recovery, and replica populations . . . 56
7.5.1 Redundant, load-balanced runtimes and blast radius . . . 56
7.6 Infrastructure cost and demand-constrained supply . . . 60
7.7 Specified dwell time . . . 61
7.8 Composing requirements means intersecting constraints . . . 62

8 Arguing Requirements through the Lean Web Server **63**
8.1 The argument loop . . . 63
8.2 The server is a staging environment . . . 64
8.2.1 The code that creates the TCP listener . . . 64
8.3 Using the agent in Codex . . . 65
8.4 MCP: the agent-facing entrance . . . 65

8.5 Consulting MCP about logic . . . 65
8.5.1 The tools . . . 67
8.5.2 Formulas as data . . . 68
8.5.3 Five verdicts . . . 69
8.5.4 Requirements without observable behavior . . . 69
8.5.5 The design loop . . . 69
8.5.6 Real answers . . . 70
8.6 A tour of the LeanFM chat . . . 72
8.6.1 Progress, cancellation and the Operational trace . . . 73
8.6.2 A private fault barrier and actionable errors . . . 74
8.6.3 Requirements properties and implementation provenance . . . 75
8.6.4 How a chat turn becomes a durable edit . . . 76
8.6.5 Example: ask to see MCP move . . . 77
8.6.6 What should appear in the response . . . 77
8.6.7 Screenshot-first documentation . . . 78
8.7 Local credentials and visible billing identity . . . 79
8.8 Cursor as an agent client . . . 79
8.9 One project, one directory . . . 80
8.9.1 Shared workspace and fresh start . . . 80
8.9.2 Credential accounts and project roots . . . 81
8.10 What the Lean artifact must settle . . . 81
8.11 Interrogate, characterize, synthesize, replay . . . 81
8.11.1 Characterization readiness . . . 81
8.12 The protobuf artifact is part of the requirement . . . 82
8.13 A ladder of checks . . . 82
8.14 How to argue well . . . 83
8.15 Client-driven bounded work . . . 83
8.15.1 Persistent application launch . . . 83

9 Using the LeanFM Chat UI 85
9.1 Open the service and unlock your account . . . 85
9.2 Choose the project before writing the prompt . . . 88
9.3 Send a useful prompt and understand the queue . . . 88
9.4 Read the result before retrying . . . 91
9.5 Keep the book and inspect the generated behavior . . . 92
9.6 Review the specification, then inspect the implementation . . . 94
9.7 Start, open, and stop the app . . . 96
9.8 Reproduce and review the screenshots . . . 98

10 Worked Example: OAuth Authorization Code Flow 99
10.1 Actors, values, and observable events . . . 99
10.2 Production 1: application request and redirect . . . 100
10.3 Production 2: authorize, then grant or deny . . . 101
10.4 Scenario 3: exchange the code and use the token . . . 102
10.5 Reading the scenarios together . . . 103
10.6 The parallel protobuf alphabet . . . 103

10.7 Generate one legal trace and its artifacts . . . 105
10.7.1 Envelope stream and concrete bytes . . . 107
10.7.2 Event graph with stored backpointers . . . 107
10.7.3 Actor projections . . . 107
10.7.4 Event order, queues, and memory . . . 107
10.7.5 Measurements, plots, and verdicts . . . 107
10.8 Interaction diagrams by scenario . . . 109
10.9 Requirements as observable constraints . . . 109
10.9.1 Who knows the password? . . . 111
10.9.2 Byte-level refinement without choosing a transport . . . 111
10.10 Probabilities and charts . . . 111
10.11 Residual state machine . . . 112

11 CTL over Generated Futures 114
11.1 Basic modal logic . . . 114
11.2 LTL: time along one path . . . 115
11.3 CTL: path modality combined with time . . . 115
11.3.1 Least upper bounds and greatest lower bounds . . . 116
11.3.2 Asymptotic cost . . . 116
11.4 Projecting to LTL-style F and G . . . 117
11.5 Persistence and recurrence . . . 118
11.6 Knowledge, obligation, and permission . . . 118
11.7 Possibility means preparedness . . . 120
11.8 Strong implication and the Chewbacca argument . . . 120
11.9 Weak until: AW and EW . . . 121
11.10 Past claims . . . 122
11.11 Fairness and time bounds . . . 122

12 A Legal Example: Lawful, Right, Known, Required 123
12.1 Three meanings of “legal” . . . 123
12.2 The Model Tenancy Deposit Act and a declared code . . . 124
12.3 Knowledge from the causal past . . . 124
12.4 Violation and reparation . . . 125
12.5 Lawful, right, and the four quadrants . . . 125
12.6 What CTL says, and what it does not . . . 126
12.7 Computing what is owed: Catala . . . 128
12.8 Status . . . 128

13 Generated Examples and Probability Overlays 129
13.1 The core says what can happen . . . 129
13.2 The overlay says how likely and how long . . . 129
13.3 Questions and their generated answers . . . 131
13.3.1 The usual run . . . 131
13.3.2 A precedent: the client gives up . . . 131
13.3.3 A counterexample: the late success . . . 132
13.4 The distribution behind the numbers . . . 132
13.5 What comes next . . . 133

14 Feature Completion as Positive and Negative Proof 134
14.1 The logical shape of a feature . . . 134
14.2 Stability review before security review . . . 135
14.3 Security as forbidden observable behavior . . . 135
14.4 The feature-completion ledger . . . 136
14.5 Worked example: the project Source view . . . 136
14.6 Worked example: chat auto-scroll . . . 137
14.7 When the result is not complete . . . 137

15 Lean as the Rendered Language 138
15.1 An unchecked exploratory fixture . . . 138
15.2 From specification to service . . . 142

16 Scope and Open Questions 143
16.1 Proof obligations . . . 143

II Security Protocol Exercises 145

17 Protocol Exercises: Roles, Attackers, and Knowledge 146
17.1 Roles compose by sharing events . . . 146
17.2 A transport-neutral byte algebra . . . 147
17.3 Example 1: an on-call handoff . . . 148
17.3.1 Three roles, written separately . . . 148
17.3.2 What composition catches . . . 149
17.4 Example 2: a sealed-bid auction . . . 149
17.4.1 Secrecy holds, and the auction is still broken . . . 150
17.4.2 The fix: bind the bidder into the commitment . . . 150
17.5 Knowledge and agreement as theorems . . . 151
17.5.1 Calculated knowledge and policy inside CTL . . . 152
17.6 Example 3: pairing a speaker by a code on its screen . . . 153
17.7 Where each modality attaches . . . 154
17.8 Needham–Schroeder, checked . . . 155
17.8.1 Two sessions: reflection . . . 158
17.8.2 A defender on the wire . . . 159
17.8.3 Reassembly . . . 160
17.9 What these exercises demand from the library . . . 161

III Machine Reference 162

18 The LeanFM Web-Server Protocol 163
18.1 One exchange across four representations . . . 163
18.1.1 Conservative default HTTP projection . . . 164
18.2 The conversion, field by field . . . 164
18.3 The running adapter . . . 165
18.4 The accepted HTTP projection . . . 166

18.5 Accepted message endpoints . . . 168
18.5.1 Canonical byte schema . . . 171
18.6 Complete LeanFM task grammar . . . 175
18.7 Authenticate an individual user . . . 175
18.8 Create an account using a user-supplied provider API key . . . 176
18.9 List, create, open, and delete owned projects . . . 176
18.10 Turn a fresh project conversation into checked requirements and implementation . . . 176
18.11 Select user-funded or platform-funded model access . . . 177
18.12 Conduct one durable project-scoped requirements turn . . . 177
18.13 Compact a project conversation into durable specifications before clearing it . . . 178
18.14 Sketch and promote multimodal requirement artifacts . . . 178
18.15 Inspect current, accepted, superseded, and stale artifacts . . . 178
18.16 Inspect one task's linked interaction, state machine, constraints, and protobuf events 179
18.17 Search the public web, follow supplied HTTP URLs, and return clickable citations . . . 179
18.18 Recall diagram syntax or propose AI-generated detail for one markerboard revision . 180
18.19 The grammar as an executable canvas . . . 180
18.20 What is checked, and what remains . . . 181

19 The LeanFM Project Bootstrap Contract 182
19.1 Required project inputs . . . 182
19.2 Conventional generated interface . . . 183
19.3 Required commands and order . . . 183
19.4 Three layers, reviewed in git . . . 184
19.5 Construction is not a runtime dependency . . . 184
19.6 Build, runtime, and publication dependency inventory . . . 184
19.7 Local and server-hosted references . . . 186

Selected Books and Further Reading 187

Bibliography 192

Index 193

Part I

Session Types as a Software Engineering Method

Chapter 1

Session Types as a Software Engineering Method

LeanFM is a session-type method for engineering software. A *multiparty session type* is a global description of a conversation: which roles take part, which messages pass between them, in what order, and where the conversation branches [12]. From that one global description each role's *local type* is derived by projection, and a role built to its local type cannot send a message nobody expects, or wait forever for one that nobody will send. That is exactly the contract a team needs at every boundary between two pieces of software that it does not want to read each other's code.

Session types grew up as a type discipline for programming languages [10, 11]. LeanFM uses the same idea one level up, as the requirements. The global type is what the stakeholders argue about; the local types are what each implementer owes; recorded production events are checked against both. Formal methods are then not an extra stage after the code: the shortest path to a running service passes through the checked conversation.

LeanFM departs from the textbook theory in one deliberate way: *its channels have physics*. Standard asynchronous session types assume that every message eventually arrives, exactly once, in order, through a queue that never fills. Real channels are bounded. UDP drops and reorders. An HTTP call times out and is retried, so the server sees it twice. A Kafka consumer replays a batch after a rebalance. A timer cannot tell a slow answer from a lost one. LeanFM makes the capacity and failure behaviour of every channel part of the requirement, and checks the protocol together with it. Chapter 6 shows the consequence on a real protocol: the same grammar is a perfect session type, and one of its two implementations has a famous bug that appears only once the physics is declared.

The same method covers two kinds of boundary that are usually designed with different tools: messages between network nodes, and calls between APIs. Both are channels between roles; they differ only in their declared physics.

Evidence labels used throughout: proposed notation (uncompiled); executable code (builds or runs, not proof); finite-model check (exhaustive only within declared bounds); test (named finite cases); generated analysis (named inputs and reducer); Lean proof (kernel-checked under stated assumptions); future interface (planned, unavailable). Labels scope evidence, not rank it; several may apply to one artifact.

1.1 Session types in one page

A global type is written once, from the point of view of nobody in particular. This is a purchase between three roles, in the Scribble style of Yoshida et al. [25], written in LeanFM's session-type syntax. It is an ordinary Lean module: `protocol` comes from the `LeanFM.Session` library, and the listing is taken from `SessionExample.lean`, where it is checked every time the file is elaborated (Chapter 5 gives the whole syntax).

```
protocol Purchase (Client, Shop, Bank) {
  o: Order from Client to Shop;
  choice {
    q: Quote from Shop to Client cause o;
    p: Pay   from Client to Bank cause q where p.amount = q.price;
       Paid  from Bank to Shop cause p;
       Shipped from Shop to Client;
  } or {
       SoldOut from Shop to Client cause o;
  }
}
```

Each line is one message: a label, a sender, a receiver, and optionally the earlier message that caused it. A label is the name of a Lean structure, so the message's payload is that structure's fields: `Order` has an `item`, `Pay` an `amount`, and `where p.amount = q.price` is type-checked Lean. `choice` says that exactly one branch happens; the shop decides which, because the shop starts both. Projection keeps, for one role, the lines that role sends or receives. LeanFM computes it; this is the shop's:

```
receive Order from Client;
select {
  send Quote to Client;
  receive Paid from Bank;
  send Shipped to Client;
} or {
  send SoldOut to Client;
}
```

The theory then asks two questions of the global type, and LeanFM asks the same two.

One chooser per choice.
Every branch must be started by the same role, and that role is the one that decides. If Client could start one branch while Shop starts another, both could act at once and the run would leave the protocol. This is the realizability (non-local choice) condition.

Everyone who must know, learns.
A role whose later behaviour differs between branches must be told which branch was taken, by some message it receives. In the `soldOut` branch of `Purchase` the bank is never told anything.

Classic projection rejects `Purchase` for that, and asks for an extra message to the bank. LeanFM accepts it, for a reason that comes from how servers are built: a role whose part in the session has not yet started owes nothing to a branch that never contacts it. The bank's part starts when `pay` arrives, as a thread forked for that session; in the `soldOut` branch that thread simply never exists.

Each such thread ingests only the messages its session expects and sends only the ones it owes, so checking it on its own is exact, not an approximation. The bank's projection shows it: the sold-out branch is empty.

Client

```
send Order to Shop;
offer {
  receive Quote from Shop;
  send Pay to Bank;
  receive Shipped from Shop;
} or {
  receive SoldOut from Shop;
}
```

Shop

```
receive Order from Client;
select {
  send Quote to Client;
  receive Paid from Bank;
  send Shipped to Client;
} or {
  send SoldOut to Client;
}
```

Bank

```
offer {
  receive Pay from Client;
  send Paid to Shop;
} or {
}
```

The rule still bites a role that has already taken part. If the client first placed a hold with the bank and the shop then sold out, the bank's session would be open with money held and nobody to tell it to release:

```
protocol PurchaseWithHold (Client, Shop, Bank) {
  h: Hold from Client to Bank;
  o: Order from Client to Shop;
  choice {
    q: Quote from Shop to Client cause o;
    p: Pay   from Client to Bank cause q;
  } or {
       SoldOut from Shop to Client cause o;
  }
}
```

LeanFM reports:

```
Bank cannot tell which branch Shop chose
```

Both cases are checked in `SessionExample.lean`.

When the global type passes, the theory guarantees three things for roles that follow their local types: *communication safety* (no message arrives that the receiver cannot handle), *session fidelity*

(the run follows the global type), and *progress* (no role waits forever on a well-formed session). Each guarantee holds *under the theory's assumptions about channels*. The next section lists those assumptions, because they are exactly where real systems fail.

1.2 What the theory assumes, and what LeanFM declares

Concern	Standard asynchronous session types	LeanFM
Queue size	unbounded	declared capacity, and what happens when full: block, reject, drop newest, or drop oldest
Delivery	exactly once, in order per pair of roles	declared per channel: ordering, sharding, loss and duplication
Time	absent	timers that may fire early; deadlines on responses
Failure	absent in the core theory; extensions add crash-stop roles [2]	declared: lost messages, restarted roles, expired sessions
Choice	one chooser per branch	the same, and the chooser may act only on its own history
Properties	safety, fidelity, progress by construction	those, plus CTL questions over the product of roles and channels, with fairness stated as a requirement
Evidence	a program that type-checks	also conformance of recorded production event streams

Bounded queues are not only more realistic; they are what makes exhaustive checking possible. Roles that exchange messages through unbounded FIFO queues are communicating finite-state machines, and for those most verification questions are undecidable [4]. Session types avoid the problem by restricting the shape of the protocol; the correspondence is made precise by Deniélou and Yoshida [6]. LeanFM takes the other exit: it bounds the queues. The product of finite roles and finite queues is a finite transition system, and every CTL question asked of it is answered exhaustively, with a witness or a counterexample.

1.3 Every boundary is a channel

A channel is any path a message can take between two roles. Every channel declares its physics along six axes. These declarations are **proposed notation**; the TFTP chapter builds its UDP channels by hand today.

Axis	Values	What it adds to the checked product
capacity	a positive number of messages	a send to a full channel takes the *when full* policy
when full	block, reject, drop newest, drop oldest	block: the sender waits; reject: the sender receives an observable refusal (an HTTP 503 or 429); the drops lose a message silently
ordering	FIFO, per key, none	which queued message may be delivered next
sharding	none, or a key field and a partition count	the channel becomes several queues; ordering holds within one partition only
loss	never, may drop	a transition that removes a queued message
duplication	never, may duplicate	a transition that delivers a message and leaves a copy queued

Most channels are one of a few familiar kinds, so LeanFM names them as presets. A declaration starts from a preset and overrides only what differs, as in `udp (capacity := 2)`.

Preset	Typical use	Physics
`reliable`	internal APIs, in-process calls, Go channels	FIFO, never lost, never duplicated, blocks when full
`http`	calls between services	each request and its response may be lost, so the caller has a deadline; a retried request may arrive twice; concurrent requests have no order; a busy server rejects
`udp`	datagrams	no order, may drop, may duplicate, drops the newest message when full
`stream`	TCP, TLS, websockets	FIFO and never lost while open; it may close after any message, and both sides observe the close as the end of the session
`kafka`	event topics	sharded by a key field; FIFO within a partition; never lost within retention; delivered at least once, so may duplicate; the producer blocks when full

Physics never appears inside the global type. The type is written once, as if every channel were `reliable`, and stays as readable as the Purchase protocol above; the declarations sit beside it, one line per channel, and `reliable` is the default for a channel nobody declares. A lossy channel costs lines in the type only where the protocol must answer it: a timer and a resend for `udp`, a deadline and a retry for `http`. Declaring everything `udp` would not make the type harder to read, but it would make the protocol wrong until those recovery lines were written, and the checker would say so. Declare each channel as what it is: most internal boundaries are `reliable`, and the few that cross a network are where the physics, and the bugs, live.

`tls` is a modifier, not a preset: `tls http` or `tls stream` adds an authenticated peer identity as an observable field of every message, and changes nothing else.

Sharding deserves its own rule. A sharded channel is several queues, and a message goes to the partition chosen by hashing its key field; order holds only between messages with the same key. A session therefore keeps its order across a sharded channel only if every message of the session carries the same key, such as the session identifier or an account number fixed at the start. LeanFM can check this from the declarations alone: a global type that relies on two of its messages arriving in order, over a channel sharded by a field that differs between them, is rejected before any state is explored.

1.3.1 Masked and surfaced failures

A lower layer can hide a failure from the layer above by retrying, and then the failure is not gone: it has become latency. TCP retransmits lost segments and reorders nothing, so a protocol over a `stream` sees no loss, only delay, until the delay is long enough that the connection closes. UDP hides nothing, so a protocol over a `datagram` must handle loss, reordering and duplicates itself, as TFTP does. Kafka hides loss inside its retention window and surfaces duplicates instead. Every interface kind therefore says two things: which conditions it *surfaces*, which the protocol must handle, and which it *masks* as latency, which the protocol's deadlines must allow for.

1.3.2 A hierarchy of interfaces

An actor does not have one physics; it has interfaces, and each interface has a kind. Kinds form a hierarchy: a kind inherits every condition its parent surfaces, and adds or removes a few. The table is a **generated analysis** from `TopologyExample.lean`:

kind	surfaces	hidden as latency	notes
`call`	–	–	synchronous; one process
`queue`	–	–	bounded, blocks when full; one machine
`stream`	–	loss and reorder become latency	TCP; a close ends the session
↪ `http`	+timeout, +duplicate	–	request and response; a retry may arrive twice
↪ `api`	+reject	–	a busy server answers 429 or 503
↪ `ingressHttp`	+hostile, +readable	–	from outside; anyone can send or read
↪ `ingressHttps`	−readable	–	TLS: peer identity observable, bytes private
`datagram`	+loss, +reorder, +duplicate	–	UDP; nothing is hidden
`topic`	+duplicate, +per-key order	loss becomes latency within retention	Kafka; at least once, ordered per key

`ingressHttps` is `ingressHttp` with TLS: it removes `readable`, because a hostile network can no longer read the bytes, and keeps `hostile`, because anyone can still send them.

1.3.3 The deployment is a tree

The interface kind gives the conditions of the protocol stack. Where the two ends run adds the conditions of the network between them, and decides whether they may talk at all. LeanFM describes a deployment as one tree: islands that may be partitioned from each other, datacenters and machines inside them, groups for replicated and sharded roles, actors, and under each actor its interfaces grouped by *scope*. A node's name is its path, so nothing is named twice (**proposed notation**; `TopologyExample.lean` holds the same tree as Lean values):

```
world {
  outside { browser }
```

```
  us {
    datacenter dc1 {
      machine m1 { alice { ingress { https }  internal { api } } }
      replicated x3 by raft { ledger { internal { api } } }
    }
  }
  eu {
    datacenter dc1 {
      machine m1 { bob { ingress { http }  peering { api } } }
      machine m2 { charles { ingress { https }  internal { api } }
                   audit   { local { syslog } } }
    }
  }
  sharded by account across { us, eu } { accounts { peering { topic } } }
}
```

The implicit name of an interface keeps the island, the actor, the scope and the interface, and leaves the containers between them out: `us.alice.ingress.https`, `eu.audit.local.syslog`. Moving `alice` to another machine in `us` does not rename her.

Scopes constrain who may talk. Before any physics, the tree says which channels may exist. A scope is a promise about callers:

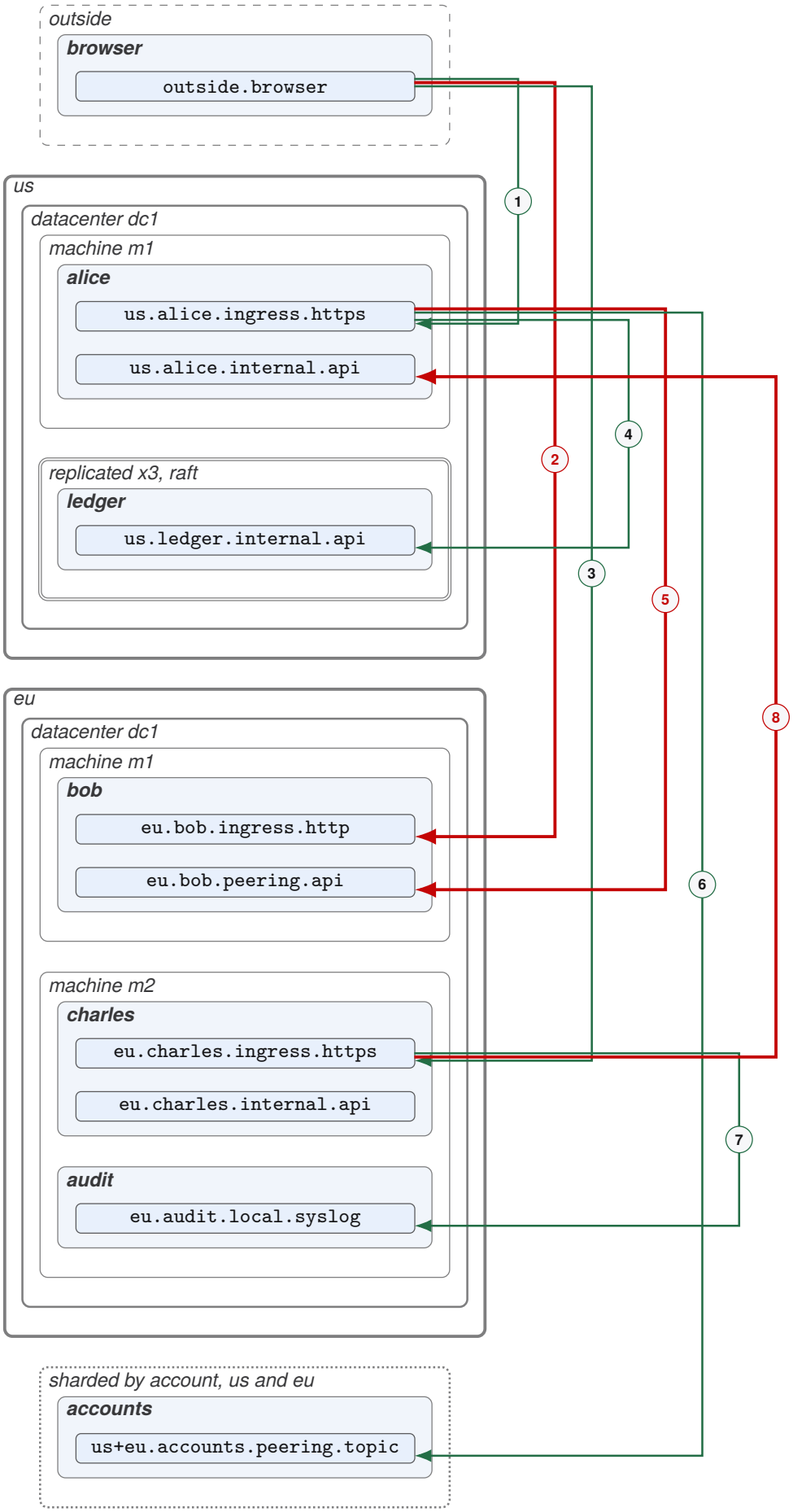
scope	may be called from
`ingress`	anyone, including outside
`peering`	any island, not outside
`internal`	the same island
`local`	the same machine

Collapsing the tree to its top level gives the whole deployment's permission structure on one page. Each cell lists the scopes in the column's subtree that some actor in the row's subtree may reach. The table is computed from the actors and interfaces actually in the tree, not from the rule above:

from \ to	`outside`	`us`	`eu`	`us+eu`
`outside`	–	ingress	ingress	–
`us`	–	ingress, internal	ingress, peering	peering
`eu`	–	ingress	ingress, peering, internal, local	peering
`us+eu`	–	ingress	ingress, peering	–

Placement adds the network. A channel whose ends are in different islands may be `partitioned`: for a while, nothing is delivered in either direction. An end outside every island makes the channel `hostile`. A channel into a replicated group may be rejected (no leader, no quorum), and the consensus protocol inside the group is hidden. A channel into a sharded group keeps order only per key.

The requirements say which conditions each channel's protocol handles. The generated graph places every interface in the tree and draws each channel from its sending actor to the receiving interface. A channel is red when its scope forbids it, or when it surfaces a condition its protocol does not handle.


outside
browser
outside.browser
us
datacenter dc1
machine m1
alice
us.alice.ingress.https
us.alice.internal.api
replicated x3, raft
ledger
us.ledger.internal.api
eu
datacenter dc1
machine m1
bob
eu.bob.ingress.http
eu.bob.peering.api
machine m2
charles
eu.charles.ingress.https
eu.charles.internal.api
audit
eu.audit.local.syslog
sharded by account, us and eu
accounts
us+eu.accounts.peering.topic
1
2
3
4
5
6
7
8

	from	to	conditions surfaced	verdict
1	browser	`us.alice.ingress.https`	timeout, duplicate, hostile	handled
2	browser	`eu.bob.ingress.http`	timeout, duplicate, hostile, readable	unhandled: readable
3	browser	`eu.charles.ingress.https`	timeout, duplicate, hostile	handled
4	alice	`us.ledger.internal.api`	timeout, duplicate, reject	handled
5	alice	`eu.bob.peering.api`	timeout, duplicate, reject, partition	unhandled: partition
6	alice	`us+eu.accounts.peering.topic`	duplicate, per-key order, partition	handled
7	charles	`eu.audit.local.syslog`	loss, reorder, duplicate	handled
8	charles	`us.alice.internal.api`	timeout, duplicate, reject, partition	not reachable: internal to us

Table 1.1: Each channel's reachability and the conditions it surfaces, against what its protocol handles.

Three channels are findings, one of each kind. Channel 8, from `charles` in `eu` to `us.alice.internal.api`, is not allowed to exist: `internal` means the same island. Either `alice` exports that API under `peering`, or `charles` goes through her ingress. Channel 5, from `alice` to `eu.bob.peering.api`, is allowed but crosses islands and may be partitioned, which nothing in its protocol answers. Channel 2 is plain `http` from outside, which a hostile network can read; either the protocol carries nothing that matters and says so by handling `readable`, or the interface should be `https`.

1.3.4 Collapsed interaction diagrams

The tree also gives every interaction diagram a zoom level. Collapsing a subtree into one lifeline hides every message whose two ends are inside it, which is exactly how a requirement hides implementation: what is not crossing the boundary is not stated. This is the key to navigating a global type too large to read whole: start at the top of the tree and open one node at a time.

The `cause` pointers decide how the remaining arrows are drawn. A message that crosses from X to Y is the *return* of the nearest unanswered request from Y to X in its cause chain, however many hidden messages lie between them; otherwise it is a request. Requests are solid, returns dashed, and the callee's lifeline carries a bar from each call to its return. Here is one order at the level of actors:

and then with the tree collapsed to its top level. The reservation with the ledger vanishes, because both ends are in `us`; the return `201 Created` is still paired with `POST /orders`, through a cause chain that passes through the hidden reservation and through `eu`. What remains is what one island promises another:

A local type is a production. Projecting the global type onto one node of the tree gives that node's local type, and it is an ordinary production of the grammar. It is cut off exactly where the node's tasks leave its boundary: a call to another node is one terminal, `call eu: ship(order) returns shipped`, and whatever `eu` does to produce `shipped` is `eu`'s own production. Folding the crossings of the scenario above, with the tree cut at its top level:

```
outside ::=
```

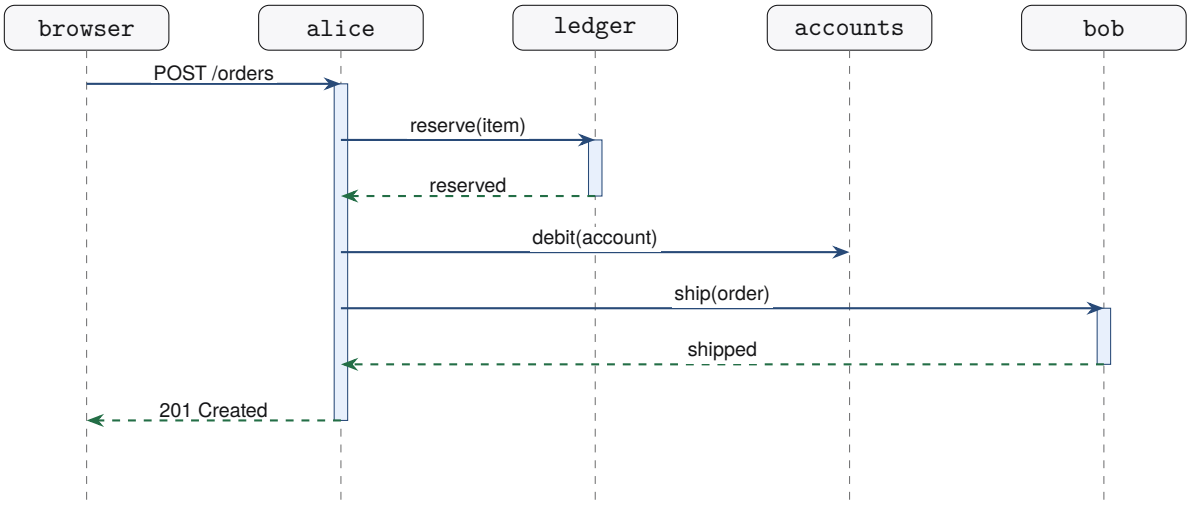


Figure 1.2: One order at the level of actors.

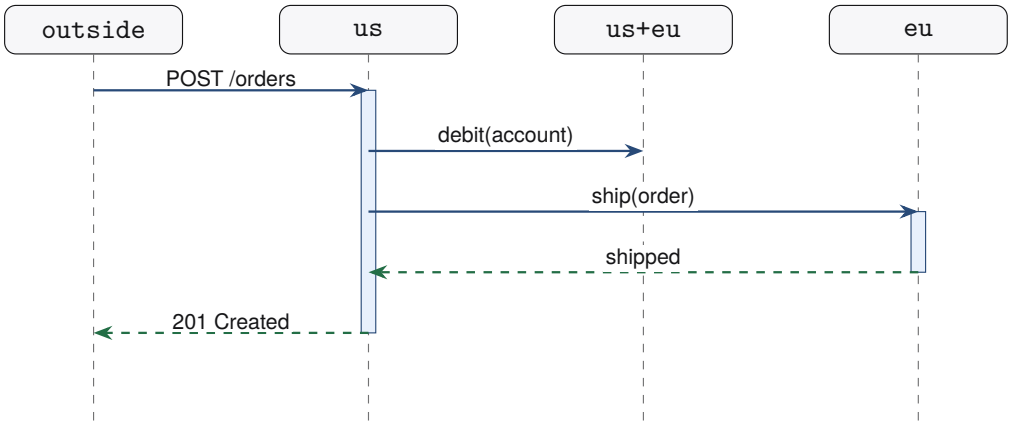


Figure 1.3: The same order with the tree collapsed to islands: five of its seven messages cross a boundary at this level.

```
  call us: POST /orders returns 201 Created

us ::=
  on outside: POST /orders {
    send us+eu: debit(account)
    call eu: ship(order) returns shipped
  } return 201 Created

us+eu ::=
  on us: debit(account)

eu ::=
  on us: ship(order) return shipped
```

Each production can be read as if it were the top-level protocol, which is what lets requirements be stacked: `us` is specified against the promises of `eu` and `us+eu`, not against their insides. Opening a node replaces its one production by the productions of its children, and the children's crossings must fold back into the parent's.

The collapsed diagram is also the high-level requirement. A team can agree on the islands view first, with its scopes and its partition handling, and refine each island later; a refinement is acceptable when collapsing it again gives back the agreed view.

The placement is an implementation decision, so the tree's containers belong in `Implementation.lean`, while the scopes and what each channel's protocol handles are requirements. Moving `bob` into `us` would remove the partition from channel 5, and moving `ledger` out of `us` would make channel 4 unreachable. Either is a checked change: the views are regenerated, and the move is acceptable only if no channel turns red. Regenerate with `lake env lean -run TopologyExample.lean`.

With channel declarations, inter-API design is the same activity as network-protocol design. A browser call that passes a reverse proxy and reaches a service is a three-role session:

```
q: tls http GET /users/{id}       from Browser to Proxy;
r: http GET /users/{id}           from Proxy to Users   cause q
     where header USER_DN = peer.dn(q);
s: http 200 {user}                from Users to Proxy   cause r;
   http 200 {user}                from Proxy to Browser cause s;
```

The requirement names only what crosses a boundary: the request, the header the proxy adds, the response. Whether `Users` answers from a cache, a database or a replica is hidden by not being written down. Hiding is how a requirement leaves an implementation free; it is not a separate refinement step. The negative requirements follow the same rule. "A request without a client certificate never reaches `Users`" is a statement about observable messages on two channels, and can be checked and tested as one.

1.3.5 Cut and un-cut: what HTTP allows

A global type may say that a reply comes from someone other than the role that was asked:

```
fq: X() from A to B;
fr: Y() from C to A   cause fq;
```

Over HTTP this cannot happen. A response travels back on the connection its request came in on, so `C` cannot answer a request that `A` sent to `B`. The `http` channel kind therefore adds a rule:

every message is either a request to an interface of its receiver, or the response to a pending request that its receiver sent to its sender. A protocol that breaks the rule has to be *un-cut*, the way a proof with a cut is expanded into one without it: the missing calls and returns are written out.

```
fq:  X()  from A to B;
fbc: X'() from B to C   cause fq;
fcb: Y'() from C to B   cause fbc;
fr:  Y()  from B to A   cause fcb;
```

Collapse `B` and `C` into one lifeline, as in figure 1.3, and the un-cut form becomes `A` to `BC`, `BC` to `A`: an ordinary call and its return. So does the cut form. A cut is a statement made at a coarser zoom level than the names in it suggest, and it is sound exactly when the roles it jumps between sit inside one node of the tree that the caller sees as a single HTTP server. `Purchase` at the start of this chapter is cut twice: `paid` answers the client's payment to the shop, and `shipped` answers a request the client never made, because `quote` already answered `order`. Un-cut for HTTP, with the client as the only role that starts calls to the shop:

```
o: http POST /orders {item}          from Client to Shop;
choice {
  q: http 200 {quote}                from Shop to Client  cause o;
  p: http POST /payments {quote}     from Client to Bank  cause q;
  r: http 201 {receipt}              from Bank to Client  cause p;
  c: http POST /orders/{id} {receipt} from Client to Shop cause r;
  v: http POST /receipts/verify      from Shop to Bank    cause c;
  w: http 200 {paid}                 from Bank to Shop    cause v;
     http 200 {shipped}              from Shop to Client  cause w;
} or {
     http 409 {soldOut}              from Shop to Client  cause o;
}
```

Every `cause` now names a message its sender received, and every response goes back to the role that asked. The other un-cut is a callback: the bank calls an interface of the shop, which needs the shop to expose one that the bank's island may reach (table 1.1). Which one to use is a design decision; LeanFM's job is to reject the cut form on an `http` channel and to show both. That check is a **future interface**; the rule above is its definition.

1.4 Choice is always internal

Inside the product, every role is an actor reading its own queue. Session-type theory distinguishes *internal* choice (a role selects which label to send) from *external* choice (a role offers to receive any of several labels). In the actor view only the first is a decision: an external choice is just this role's queue delivering whatever another role decided. So every choice in a LeanFM protocol is made by exactly one role, from exactly what that role can know: the messages it has sent and the messages that have arrived in its queue.

LeanFM makes that history explicit. Every recorded event carries `priorIds`, the earlier events it follows, and `causeId`, the one event that made it happen (Chapter 2). The realizability rule is then a rule about pointers: *a role's event may cause-point only to events that role sent or received.* A protocol in which a role reacts to something it never saw cannot be implemented by anyone, and

is rejected. LeanFM does not yet perform this validation (a **future interface**); the rule is how the validation is defined.

1.5 The requirement is the global type

The primary object is the set of permitted, visible histories: who communicated with whom, which message, what it carried, and which earlier messages it follows. Message bytes, their order, and persistent disk formats become durable compatibility boundaries once another process, customer or stored data set depends on them.

Threads, actors, databases, caches, continuations and retry machinery inside a role are realization choices unless they change the observable conversation. Modelling every internal choice couples requirements to harmless refactoring and creates a product state space that grows exponentially with independently varying components. The channels between roles are different: their capacity and failure behaviour *are* observable, because they decide which histories can occur. So LeanFM requires the boundary and its physics, and leaves the inside of each role alone until it changes an observable promise.

1.6 Comparison with TLA+

TLA+ is the specification language most engineers meet first, so the contrast is worth making precisely. LeanFM is not an extension of TLA+ and does not try to replace it. TLA+ commonly describes a system with variables, an initial-state predicate, a next-state relation, and temporal formulas over the resulting state behaviors [15]. A message queue can be represented as a sequence- valued variable, and an action can append to or remove from that sequence. This is expressive and makes internal state transitions available to invariants and model checking.

1.6.1 An executable queue model

This TLA+ module sends an order to Bob's inbox, then lets Bob consume it and change his control state:

```
---------------------------- MODULE OrderQueue ----------------------------
EXTENDS Integers, Sequences, FiniteSets, TLC
CONSTANTS Clients, Servers, Orders, QueueBound, CacheBits, ClockBound
Actors == Clients \cup Servers
Messages == [kind : {"placeOrder", "placedOrder"},
             order : Orders, replyTo : Clients]
NoMessage == [kind |-> "none", order |-> "none", replyTo |-> "none"]
VARIABLES inbox, pc, current, sent, placed, received, scratch, tick
vars == <<inbox, pc, current, sent, placed, received, scratch, tick>>

Init == /\ inbox = [a \in Actors |-> <<>>]
        /\ pc = [b \in Servers |-> "idle"]
        /\ current = [b \in Servers |-> NoMessage]
        /\ sent = {} /\ placed = {} /\ received = {}
        /\ scratch = [i \in 1..CacheBits |-> FALSE]
        /\ tick = 0

Send(a, b, o) ==
```

```
  /\ o \notin sent /\ Len(inbox[b]) < QueueBound
  /\ inbox' = [inbox EXCEPT ![b] = Append(@,
       [kind |-> "placeOrder", order |-> o, replyTo |-> a])]
  /\ sent' = sent \cup {o}
  /\ UNCHANGED <<pc, current, placed, received, scratch, tick>>

Receive(b) ==
  /\ pc[b] = "idle" /\ Len(inbox[b]) > 0
  /\ current' = [current EXCEPT ![b] = Head(inbox[b])]
  /\ inbox' = [inbox EXCEPT ![b] = Tail(@)]
  /\ pc' = [pc EXCEPT ![b] = "busy"]
  /\ UNCHANGED <<sent, placed, received, scratch, tick>>
```

The excerpt above defines actions but does not yet connect them to a behavior. That connection appears later in the same module:

```
Next == (\E a \in Clients, b \in Servers, o \in Orders: Send(a,b,o))
     \/ (\E b \in Servers: Receive(b) \/ Finish(b))
     \/ (\E a \in Clients: ReadReply(a)) \/ CacheStep \/ ClockStep
Spec == Init /\ [][Next]_vars
```

TLA+ does not call `Send` or `Receive` like procedures. `Next` is a relation between the current variables and their primed next values. Its disjunction says that one step may be a `Send` for some client, server and order; a `Receive` or `Finish` for some server; a `ReadReply` for some client; a cache step; or a clock step. A candidate action whose guards are false is disabled. TLC explores every enabled choice.

Read `Spec == Init /\ [][Next]_vars` as: "the first state satisfies `Init`; thereafter, every state transition satisfies `Next` or leaves every variable in `vars` unchanged." The configuration closes the final loop with `SPECIFICATION Spec`; it tells TLC which temporal formula to explore.

Read `inbox'` as "the next value of inbox," `@` as "the old value at this selected entry," and `UNCHANGED` as "these variables retain their values." `Append` adds a message; `Head` selects the first message; `Tail` removes it. Alice's send updates Bob's inbox, not Bob's private control state. Bob's receive performs the latter transition. The configuration instantiates `Clients = {"alice"}` and `Servers = {"bob"}`; functions indexed by those sets hold instance state. The full module commits the order, sends a reply, and checks that no reply precedes placement. No eventual-reply property is checked.

1.6.2 The cost of irrelevant state

The module includes optional cache-bit toggles and a clock increment that cannot affect an order message. TLC nevertheless distinguishes their values:

Model	Cache bits	Clock bound	Distinct states
Baseline	0	0	26
Cache4	4	0	416
Cache8	8	0	6,656
Clock7	0	7	208

These exhaustive finite-model safety-check counts come from TLC 1.7.4 via `make tla-check`; evidence and provenance are in `book/generated/tla-state-space.json`. With S protocol states,

b independent Boolean cache bits and a counter ranging from zero through k, the example has

$$S\,2^b(k+1)$$

states. Read: “S times two to the b times k plus one.” S is the protocol state count, b the cache-bit count and k the counter bound. Each irrelevant bit doubles this model’s state count. `ClockBound = -1` allows ticks 0, 1, 2 and onward: infinitely many states with the same order behavior. Exhaustive explicit-state search cannot finish that variant. Unbounded queues or retained histories can also make a model infinite.

This is checker state growth, not measured application memory. More states usually require more exploration time and storage; the table gives no fixed bytes-per-state or timing law. TLA+ supports abstraction, hiding and refinement. The lesson is to omit irrelevant state, not to blame the language for its inclusion.

1.6.3 Proposed message-first notation

This is invented TLA+-inspired syntax, not standard TLA+ or a compiled LeanFM feature:

```
// Proposed TLA+-inspired notation. Not standard TLA+, not compiled by LeanFM.
message placeOrder(orderId: OrderId, replyTo: ClientId)
message placedOrder(orderId: OrderId)

actor class Client {
  input placedOrder
  output placeOrder
}
actor class Shop {
  input placeOrder
  output placedOrder
  inbox FIFO(capacity = 2)
  on placeOrder(orderId, replyTo) {
    require commitRecorded(orderId)
    self:replyTo ! placedOrder(orderId)
  }
}

alice = new Client()
bob = new Shop()
alice:bob ! placeOrder(o1, replyTo = alice)
bob ? placeOrder(o1, replyTo = alice)
// Shop handler records the commit and sends placedOrder.
alice ? placedOrder(o1)

web = instances(WebServer, count = 3, route = roundRobin)
documents = shards(DocumentStore, count = 8,
                   key = documentPartition, replicas = 3,
                   writeQuorum = 2)
```

Read `alice:bob ! placeOrder` as “Alice sends placeOrder to Bob,” and `bob ? placeOrder` as “Bob receives placeOrder.” Bob consumes the request `placeOrder`; Alice consumes the reply `placedOrder`. The proposal treats inboxes as first-class actor-owned queue state, not arbitrary private variables. Blocking and capacity semantics are specified in Chapter 5; this proposed syntax does not implement them. The handler’s commit precondition is an observable obligation, not a database algorithm.

Actor classes describe behavior; instances have distinct identities, queues and local state. WebServer instances share a class, not an inbox. DocumentStore shards partition ownership by key; replicas protect each shard, and a quorum controls write acknowledgment. These deployment roles are placeholders, not classes implemented by the order module. First-class syntax still needs capacity, routing, failure and scheduling rules; it does not eliminate their state space.

1.6.4 Requirements at the message boundary

LeanFM starts from the conversation: a global type over decoded production events, whose messages are correlated and whose order is classified as conforming or a counterexample without pretending to observe private variables. The global type describes legal traces and generates scenarios; state machines are projections of visible histories. The executable grammar limits are documented in Chapter 5. A property dependent on hidden state needs an observable event or justified abstraction; otherwise its status is indeterminate.

This is a workflow choice, not a claim that TLA+ cannot express queues or traces. Our initial TLA+-style attempt required modeling internal state before an ordinary observed scenario could be recorded, so LeanFM makes production-visible messages the starting point.

The trade is deliberate:

Question	TLA+-style state specification	LeanFM session specification
Primary behavior	sequence of states	a global type, and its runs as partial-order graphs of events
Queue representation	usually an explicit variable	a channel with declared capacity and failure behaviour
Conformance input	modeled states or action correspondence	decoded production event stream
Hidden variables	available when declared	omitted unless they affect an observable contract
Generation	behaviors satisfying the state relation	runs generated by the global type and its channel physics
Probabilities	require an additional probabilistic interpretation or tool	declared and measured overlays over the same core
Main risk	state model may drift from instrumentation	event model may omit a hidden cause needed for a property

This boundary separates the durable specifications: `Requirements.lean` defines observable behavior, while `Implementation.lean` maps it to generation and runtime choices. Existing source does not redefine requirements; Chapter 19 defines the project-root handoff.

1.7 Discovery first, consolidation before commitment

During discovery, the book may contain alternatives that are not accepted. Before a specification is committed and again before implementation, classify every normative idea as a requirement, implementation decision, deferred item, or material to remove. Preserve technical claims and qualifications; replace prose that mainly enumerates roles, messages, states, ordering, dependencies,

or quantities with a generated view from typed artifacts and a short interpretation. Each view retains its prompt, question, event source, reducer, unit, and boundary. An idea's survival in the book does not make it binding; only committed, checked artifacts do.

1.8 Four ingredients

The method combines four traditions.

1. Session types give the backbone: one global description of a conversation, projected to the local obligations of each role, with well-formedness conditions that rule out unimplementable choices [11, 12, 25].

2. Language theory gives compact generative descriptions of legal histories: a global type is a grammar whose terminals are messages, and its sequence, choice, parallel composition and recursion are the grammar's productions. Dana Richards and Henry Hamburger, the author's college professors, are cited in Richards and Hamburger [21] as the source from which the author learned these methods, not as their originators. For the underlying habits of symbolization, validity, proof, quantification, and modal reasoning, we also draw on the accessible progression of Gensler [8].

3. Protocol analysis contributes explicit participants, adversarial choices, trace semantics, and tool-supported checking. An attacker is one more role, or a channel whose physics an adversary controls. The CSP treatment of Ryan et al. [22], including Steve Schneider, is cited as the source from which the author learned this treatment.

4. Temporal logic turns expectations into queries over the product of roles and channels. CTL is central because it distinguishes claims about some possible future from claims about every possible future [5].

1.9 A test for every abstraction

Each abstraction in this book should answer a practical question:

> Can a developer state a conversation once, with the physics of its channels, compose it with other conversations, check its consequences, and keep it stable while the implementation of each role changes?

If an abstraction does not improve that workflow, it does not belong in the core library.

Chapter 2

Session Histories Made of Events

A global type says which conversations are legal. An *event* is one message of one actual conversation: a single instance of a session, recorded as it happened. Every execution is represented by a list of such events, and checking an implementation means checking that list against the global type and its declared physics. A deliberately small common envelope makes histories comparable across protocols:

```
structure Event (Actor Session Task Kind Payload : Type) where
  timeAt  : ClockTimestamp
  id      : String
  priorIds : List String
  causeId : Option String
  session : Session
  task    : Task
  from    : Actor
  to      : Actor
  kind    : Kind
  payload : Payload

abbrev Trace (Actor Session Task Kind Payload : Type) :=
  List (Event Actor Session Task Kind Payload)
```

The fields have different jobs. `id` identifies a visible fact; `priorIds` records zero or more immediate predecessor backpointers; `causeId` optionally identifies the one direct causal event (the MCP `check_trace` tool spells these `prevIds` and a list `causeIds`, and the protobuf envelope `prior_ids` and `cause_id`); `from` and `to` name the sending and receiving roles; `session` names the instance of the global type, and `(session, task)` correlates independently progressing conversations; and `kind` supplies a stable discriminator. The payload is protocol-specific and carries the parameters that flow from one production to the next. `timeAt` is sampled from the model's shared monotonic clock. Its differences measure elapsed time; it is not a civil-time date and its origin is immaterial.

The predecessor links form a directed acyclic graph rather than a singly linked list. An initial event has no prior identifiers. A fork is represented by several later events that each name the same prior event. A join is represented by one event whose `priorIds` names every branch on which it depends. An end event of any kind may also name its corresponding start event. That backpointer makes elapsed time simply `end.timeAt - start.timeAt`, even when several attempts with the same `(session, task)` overlap. A backpointer to an already-ancestral start is a redundant ordering

edge, but it is useful correlation data and does not alter the partial order. The edge says “ordered after,” not “caused by.” A causal interpretation requires a separate grammar or domain assertion.

Causality uses a different backpointer. An event’s optional `causeId` names the event that directly caused it and does not add or remove permitted interleavings. It is not restricted to terminal events. A task-start event may cause intermediate messages; those messages may cause nested task starts; and terminal events may point to the event whose work they finish. Following the reverse `causeId` relation produces a causal subtree. A renderer places that subtree inside a labeled box, recursively, so nested work remains visible without pretending that concurrent causal children have a fixed order.

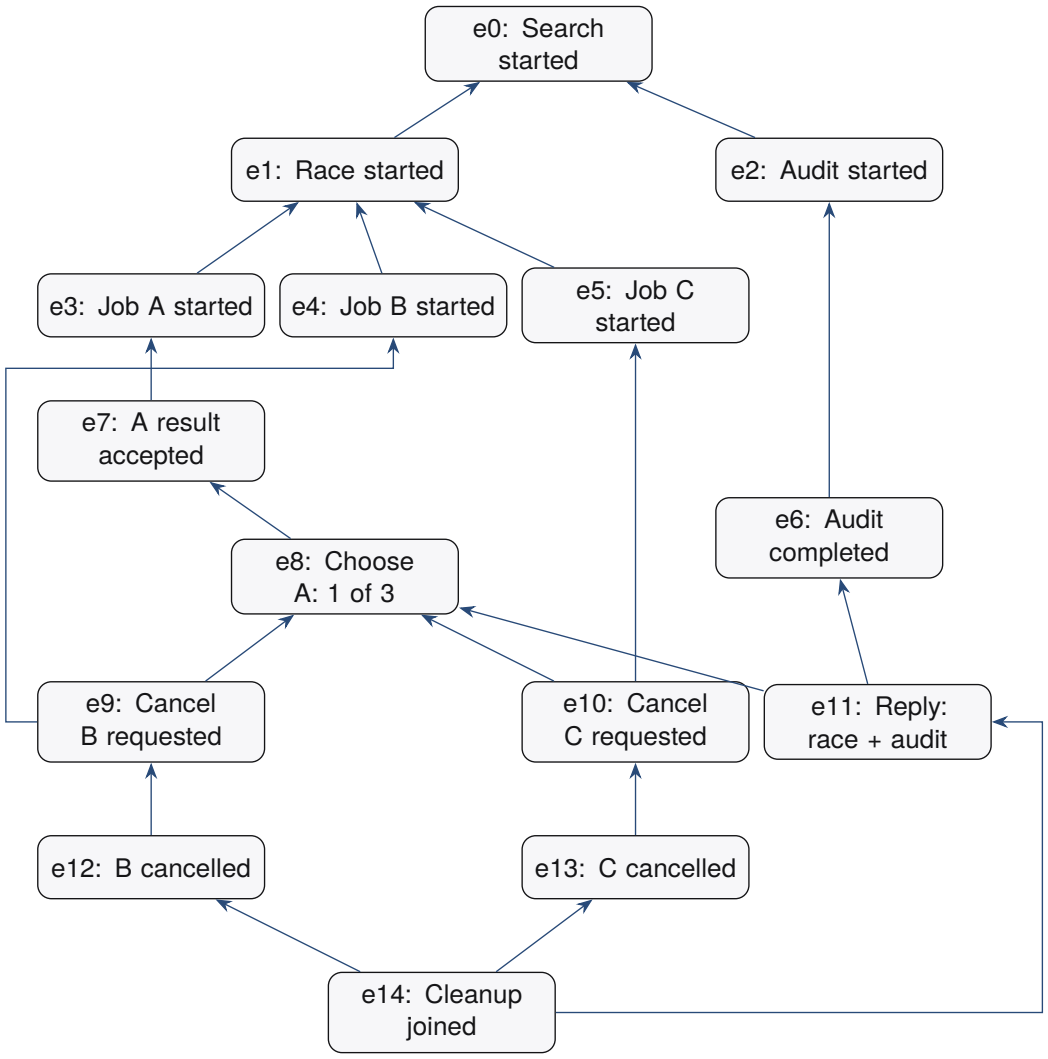
The two pointers are also what makes a choice checkable. A role may decide only from its own history, so an event’s `causeId` should name an event its sender sent or received. An event that points across to a message its sender never saw records a decision nobody could have made (Chapter 1).

For a request/response production, the response’s `causeId` is the request’s `id`; read it as “this response answers this request.” The rule remains unambiguous when many requests are in flight because correlation follows event identity rather than adjacency or arrival order. `priorIds` remains exclusively the partial-order relation.

For an m-of-n join, the event additionally records m, n, and the exact m branch-terminal IDs that satisfied the threshold. The selected IDs must be distinct members of `priorIds`. Merely seeing two predecessors is not enough to distinguish a two-of-three threshold from an ordinary two-branch all-of join.

2.1 Compute everything from a fork/join event list

2.1.1 Nested joins: choose a winner, then cancel the rest



Arrows show predecessor order with redundant transitive edges removed; full backpointers remain in the event list. The event list and provenance are generated from `NestedForkJoinExample.lean`; this is not a claim about Google's implementation.

The winner event records threshold 1, population 3 and selected terminal `e7`. Its predecessors are `[e1,e7]`, not the results of all three jobs. The outer reply records `[e0,e8,e6]`: task start, chosen result and completed audit. Reply time is 12 synthetic milliseconds; cleanup ends at 18. These times illustrate ordering, not measured search performance.

For a general n-of-m join, choose n distinct qualifying terminals from m declared branches. Selection is atomic, so racing results cannot select two winners for a 1-of-m task. "First" means the coordinator's observed order, with a declared tie rule, not unsynchronized worker clocks. A cancellation request is not a cancellation acknowledgment; late results must not replace the chosen result. The requirement must say whether a reply waits for losing-task cleanup. Here it does not. Selected-ID and threshold metadata belongs in the join event, not inferred from the picture's number of incoming arrows.

2.1.2 Durations from boundary events

Each event is one observation at one instant. Starting and completing are different messages, so an event does not contain an interval. Durations are derived by selecting the two boundary events and subtracting their timestamps.

```
structure AnalysisEvent where
  id       : EventId
  priorIds : List EventId
  timeAt   : ClockTimestamp
  bytes    : ByteArray
  reveals  : List ValueRef
```

Consider the first retail purchase in `examples/bakery-events.jsonl`. Its bread was baked and shipped before the storefront opened. The purchase itself reserves that inventory, tenders payment, and completes in about a second:

Event	Prior backpointers	`timeAt`	Meaning	Bytes	Reveals
e_0	$[]$	28859991	purchase requested	59	product and quantity
e_1	$[e_0]$	28860343	inventory reserved	62	batch allocation
e_2	$[e_1]$	28861004	payment tendered	55	amount
e_3	$[e_1, e_2]$	28861248	sale completed	72	bread and payment

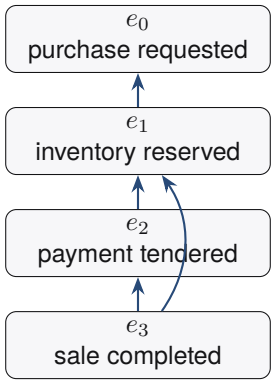


Figure 2.1: The table's immediate predecessor backpointers rendered as an order graph. Each arrow points from an event to an ID in its `priorIds` list: for example, $e_2 \to e_1$. The extra $e_3 \to e_1$ edge preserves direct correlation with the inventory allocation even though e_1 is already an ancestor through e_2.

The list order is one serialization convenient for storage. Event order is the transitive closure of `priorIds`. This sale is sequential; baking and shipping belong to the morning production stream, not to the customer purchase. The following quantities are folds or graph queries over this one list:

$$\begin{aligned}
\mathrm{reservationTime}(e_0, e_1) &= 28860343 - 28859991 = 352 \text{ ms}, \\
\mathrm{paymentTime}(e_1, e_2) &= 28861004 - 28860343 = 661 \text{ ms}, \\
\mathrm{purchaseLatency}(e_0, e_3) &= 28861248 - 28859991 = 1257 \text{ ms}, \\
\mathrm{wireBytes} &= 59 + 62 + 55 + 72 = 248.
\end{aligned}$$

The subscripted arguments identify each duration's exact start and end events.

Two events are concurrent in this order when neither is an ancestor of the other:

```
def concurrent (events : List AnalysisEvent) (a b : EventId) : Bool :=
  !reachable events a b && !reachable events b a
```

The maximum concurrency permitted by the trace is the width of this partial order: the size of its largest antichain. This purchase is a chain, so its width is one. Other purchases and the morning production and transport streams may still be concurrent. Incomparable events may commute in a linear serialization, provided each event still appears after all of its named predecessors. A timestamped point event has no active interval; runtime concurrency requires separate start and completion messages. This is stronger than counting adjacent list elements and different from measuring average runtime concurrency.

Start and completion boundaries make nested concurrent work especially readable as an order graph. In the following trace, the request forks into two task attempts. Each completion points back to its own start, and the response joins the two completions. The shaded columns are visual grouping only; the arrows are still exactly the stored predecessor pointers.

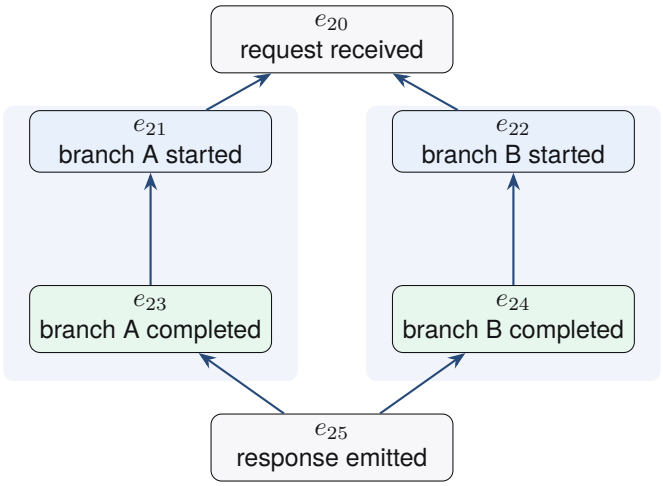


Figure 2.2: A separate synthetic example of task boundaries nested inside a concurrent fork/join. The intervals $e_{21} \ldots e_{23}$ and $e_{22} \ldots e_{24}$ may overlap, while the two branches remain incomparable in predecessor order. Either branch may appear first in a linear serialization without changing the graph.

Timestamps and predecessor links also give elapsed time along a selected path:

```
edgeTime(prior, event) := event.timeAt - prior.timeAt
elapsed(started, completed) := completed.timeAt - started.timeAt
```

The retail critical path is e_0, e_1, e_2, e_3, with elapsed time 1257 ms. The direct $e_3 \to e_1$ correlation edge is redundant for reachability and does not change that duration. Queue occupancy, queued bytes, active `(session,task)` keys, and memory pressure are computed by adding enqueue/dequeue events to the same alphabet and folding them in clock order.

The entire checked-in corpus contains 20,000 orders and 81,039 events. Streaming that file produces the following model inputs; the numbers in this table are generated LaTeX macros, not independently typed constants:

Reduction over `bakery-events.jsonl`	Observed value
sales completed / purchase requests	20,000/20,000
stockouts / purchase requests	0/20,000
morning batches baked	140
FIFO shipments dequeued / enqueued	140/140
maximum simultaneously active purchases	1

The sale and stockout ratios are frequency estimates for this corpus; the other rows are counts and maxima, not probabilities. Regeneration with a different source stream changes the macros and therefore changes every book example that consumes them.

Knowledge is another event reduction. For a value v, calculate the actors that can derive it from initial knowledge and the bytes visible to them:

```
def knowers (actors : List Actor) (v : ValueRef)
    (events : List AnalysisEvent) : List Actor :=
  actors.filter fun actor =>
    Derives (knowledgeFrom actor events) v
```

Encryption, signing, projection, key possession, and compromise contribute rules to `Derives`. The primary result is the set `knowers v events`. “Only these actors know v” is a comparison between that computed set and an allowed set, not a separate primitive called secrecy.

2.2 Derived state

A current session, an authenticated principal, or an outstanding request can often be computed by folding the trace. Such a fact need not also exist as primitive model state. This discipline has two benefits: evidence for a fact remains visible, and the state representation does not multiply independently of the history that justifies it.

Some implementations necessarily maintain private state. The model includes it only when two executions with the same visible trace may make different future behaviour legal. Even then, the first remedy is to expose the relevant distinction as an event or a grammar parameter rather than copying an implementation object graph into the specification.

2.3 Time is more than a timestamp

Timing constraints pair explicit boundaries; changing a timeout bound changes the temporal requirement, not the event-order or causal relations defined above.

For two events from the same `(session, task)`, elapsed time is defined when the first event’s `timeAt` value does not exceed the second’s:

```
def elapsed (start finish : Event) : Option Duration :=
  if start.session = finish.session &&
     start.task = finish.task &&
     start.timeAt <= finish.timeAt
  then some (finish.timeAt - start.timeAt)
  else none
```

Latency queries name their boundary event kinds and use the terminal's `priorIds` backpointer to select the start. Matching `(session, task)` values alone cannot disambiguate overlapping attempts, so reducers never pair by adjacency. Completion, failure, cancellation, timeout, and other declared terminals use the same explicit pairing for elapsed time, byte deltas, and other measurements.

This pairing is how a view may elide detail without becoming hand-waving. A collapsed production retains its boundary event IDs, complete predecessor links, parameter assignment, aggregate deltas, and validation status. Opening it reveals the intervening events. Closing it again changes only presentation, not the history or the evidence supporting the displayed duration, bytes, or outcome.

2.4 Estimating completion from an event stream

The same boundary-event model can consume observed events. The `LeanFM.StreamStats` implementation reconstructs attempts from stored terminal-to-start predecessor links and computes completion, conditional-success, and completed-latency statistics.

For an observation window containing M starts, C matched terminal events, S successful terminals, and completed latencies $L_1, \ldots, L_C$, the basic estimates are

$$\widehat{P}(\text{complete}) = \frac{C}{M}, \qquad \widehat{P}(\text{success} \mid \text{complete}) = \frac{S}{C}, \qquad \widehat{E}[L \mid \text{complete}] = \frac{\sum_i L_i}{C}.$$

The vertical bar means "given"; hats mark estimates from the observation window. Attempts without a terminal event remain *in flight*. They contribute to the number started and to queue/load measurements, but not to completed-task latency or to the conditional-success denominator. Calling them failures would bias both probability and latency whenever the observation window ends before they do.

The reducer can group these sufficient statistics by task, actor, time bucket, maximum concurrency, or queue occupancy and feed them to charts. Specified probabilities and dwell times are predictions; stream-derived estimates are measurements to compare with them.

2.5 Charts as declarative trace reductions

The proposed chart declaration names an observable event source, a grouping or x expression, and an aggregate or y expression. It describes the reduction, not rendering code; this syntax is illustrative, not an implemented renderer.

```
def outcomeShare : ChartSpec :=
  pie "quote outcomes" do
    source events
    where_ (kind == "quote accepted" || kind == "quote declined")
    groupBy kind
    value count

def responseLatency : ChartSpec :=
  line "quote response latency" do
    source (correlate events by "quoteId")
    x clock
    y (elapsed "quote offered" "customer decision")
```

```
    series "p50" median

def latencyByConcurrency : ChartSpec :=
  xyplot "latency by concurrent sessions" do
    source completedRuns
    x maxConcurrentSessions
    y (mean (elapsed firstEvent terminalEvent))

def queueByOfferedLoad : ChartSpec :=
  xyplot "queue occupancy by offered load" do
    source completedRuns
    x requestsPerSecond
    y (mean totalQueueOccupancy)

def propagatedGrantProbability : ChartSpec :=
  xyplot "user probability propagated through deterministic servers" do
    source parameterSweep
    x userGrantProbability
    y terminalGrantProbability
```

An XY plot preserves two numeric axes. It differs from a time series, whose x-axis is specifically an event clock, and from a categorical chart, whose x values are labels. The first two examples reduce each completed model run to one point. The third sweeps a model parameter and checks that deterministic downstream processing transports the user's probability mass.

Apply the provenance and implementation-status rule in Section 3 to each generated chart.

2.6 Round trip through bytes

A generated scenario is not complete until it can cross the byte boundary in both directions. `ScenarioEvent` in `Requirements.proto` carries the event identity, complete `prior` list, correlation key, actors, `timeAt`, and the selected protobuf atom. Encoding a `Scenario` therefore produces bytes for both its ordering envelope and payload. Decoding those bytes must reconstruct the same ordered event list. The executable round-trip check compares the decoded protobuf value with the generated value and separately checks the reconstructed `prior` graph; loss of a fork or join fails the build.

Chapter 3

Scenarios

A scenario is LeanFM's named, bounded stream of timestamped events for one run of a session (for example, a quote request, OAuth exchange, or 9×9 game). The global type says which runs are legal; a scenario is one of them, either generated from the type or recorded from production, and it is the concrete thing people point at while arguing about the type.

3.1 Event envelope and scenario grouping

The shared envelope and semantics of `id`, `priorIds`, `causeId`, actor endpoints, correlation and monotonic `timeAt` are defined in Chapter 2. A serialized `scenario` key or aliases such as `src/message` are canonical only if the accepted schema and reducers declare them.

3.2 Derived outputs and provenance

Question-driven outputs include interaction diagrams, residual state machines, latency/throughput charts and outcome distributions. Each `DesiredOutput` must bind its prompt, precise question, event source, reducer, unit and scenario boundary. A listed output is not implemented until its renderer exists; when a generated view carries the explanation, prefer it to repeated prose and add a short interpretation.

3.3 Example: minimal two-player game scenario

Below is an illustrative event fragment for a 9×9 game. Times are milliseconds on the model's monotonic clock:

```
{ "id":"evt-0010", "priorIds":[], "causeId":null,
  "timeAt":30000, "session":"s-g1", "task":"game-g1",
  "from":"alice","to":"server","kind":"MoveRequested",
  "payload":{"move":"E5","color":"B"} }
{ "id":"evt-0011", "priorIds":["evt-0010"], "causeId":"evt-0010",
  "timeAt":30001, "session":"s-g1", "task":"game-g1",
  "from":"server","to":"alice","kind":"MoveAccepted",
  "payload":{"move":"E5"} }
```

The latency pair is `evt-0010` to `evt-0011`: one millisecond.

Prefer runnable scenario traces to prose-only examples, declare their expected derived views and provenance, and keep transport mappings in `Implementation.lean` once the requirements check.

Chapter 4

The Bakery: Declared and Measured

The bakery is the quantitative worked example. Its tasks are written as a *core*, a session between customers, staff and the oven that says only what can happen, plus *overlays*: separate tables of branch probabilities and delays. A *declared* overlay states what we expect; a *measured* overlay is reduced from recorded events. Both answer the same questions about the same unchanged core, and Chapter 13 develops the idea in general. The corpus is the 81,039-event stream `examples/bakery-events.jsonl`, a reproducible synthetic population generated by `scripts/generate_bakery_ev` its numbers describe that corpus, not a real bakery. Every figure and table below is regenerated by `lake env lean -run BakeryFlowExample.lean`.

4.1 The morning, generated

Bread is baked to a schedule, not to order. Each batch is baked, queued on a truck, and stocked at the storefront before opening. The stocking task's core lists only those four messages; the overlay measured from the 140 recorded batches supplies how long each step takes. The figure is the most likely stocking run under that overlay.

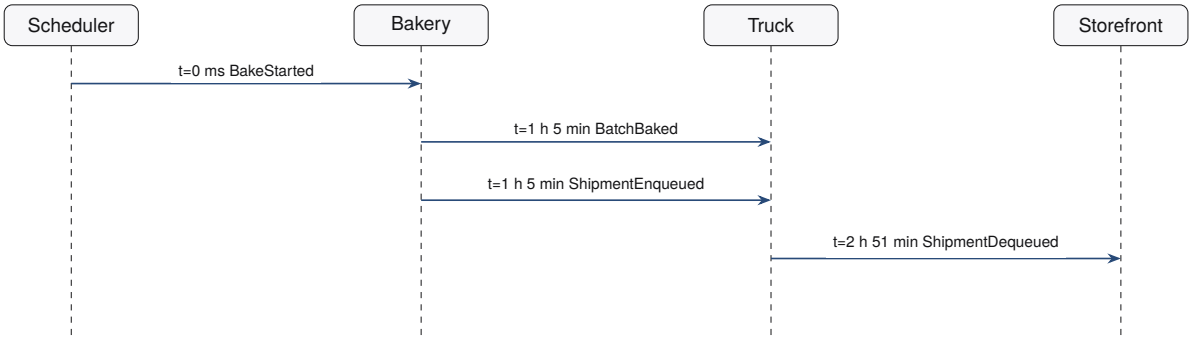


Figure 4.1: One batch from oven to shelf, with mean step times measured from the corpus.

4.2 A sale: what can happen

A customer asks for bread. The storefront either reserves stocked inventory or turns the customer away; a reserved order is paid for, and the payment either completes the sale or is declined. The

core states this and nothing more. It already proves that every sale attempt ends and that a customer *can* be turned away.

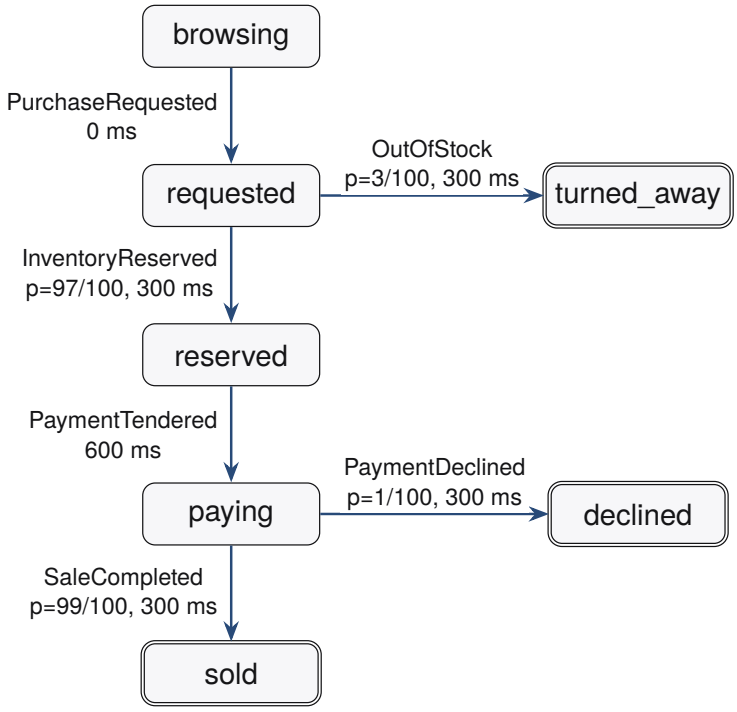


Figure 4.2: The retail-sale core, with the bakery's declared expectations drawn on its edges. Double borders mark terminal states.

4.3 What we expected, and what the corpus shows

The declared overlay was written before looking at any data: a 3% stockout rate, a 1% payment-decline rate, and round-number delays. The measured overlay has exactly the same shape. `LeanFM.Overlay.measure` walks each of the 20,000 recorded sales through the core, counts how often each branch was taken, and averages the time since the previous message. A recorded session that left the grammar would be reported rather than counted.

Transition	Declared p	Declared delay	Measured count	Mean delay
browsing → `PurchaseRequested`	1	0 ms	20000 / 20000	0 ms
requested → `InventoryReserved`	97/100	300 ms	20000 / 20000	289 ms
requested → `OutOfStock`	3/100	300 ms	0 / 20000	—
reserved → `PaymentTendered`	1	600 ms	20000 / 20000	574 ms
paying → `SaleCompleted`	99/100	300 ms	20000 / 20000	300 ms
paying → `PaymentDeclined`	1/100	300 ms	0 / 20000	—

Table 4.1: Declared expectations against branch counts and mean delays reduced from the corpus.

4.4 The same questions under both overlays

Question	Formula	Declared	Measured
Does every sale attempt end?	`AF terminal`	proved	(core, no overlay)
Can a customer be turned away?	`EF turned_away`	proved	(core, no overlay)
How likely is a sale?	`P=? [F sold]`	9603/10000 (0.9603)	1
Sold within 1.2 s?	`P=? [F<=1200ms sold]`	9603/10000 (0.9603)	1; raw data 0.5454
How often out of stock?	branch `OutOfStock`	3/100	0 of 20000; 95% bound $\leq$ 3/20000
Expected sale duration	`E[duration]`	1173.0 ms	1163.0 ms

Table 4.2: One unchanged core, two overlays. The first two rows need no numbers at all.

Two rows deserve a closer reading. The corpus contains no stockout in 20,000 sales, but "never observed" is not "impossible": with zero occurrences in n trials the usual 95% upper bound on the rate is about $3/n$, so the honest measured answer is a bound, not zero. The core, meanwhile, proves that turning a customer away is possible, so the path must still be designed and tested even though the data never exercised it.

4.5 The deadline lesson: a mean hides the spread

The measured overlay records each step's *mean* delay. Its predicted sale takes 1163 ms, inside the 1.2 s deadline, so the model answers $P[F_{\leq 1200\,\mathrm{ms}}\ sold] = 1$. The recorded sales say otherwise: only 0.545 of them finished within 1.2 s. Real sales took between 348 and 1956 ms.

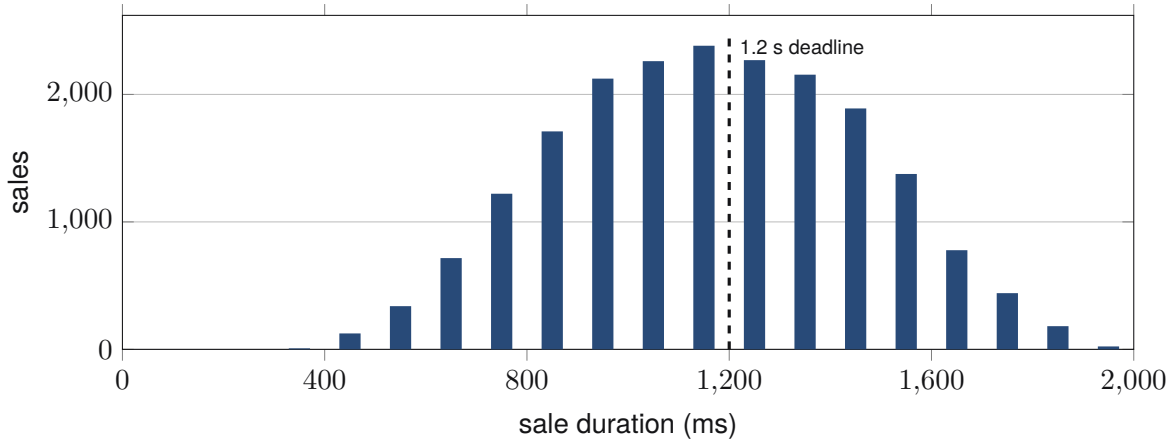


Figure 4.3: Duration of every recorded sale. The mean sits just inside the deadline; almost half of the sales do not.

This is why a measured overlay must eventually carry distributions, not just means, as FizzBee's performance files do: the deadline question is about the spread, and a point estimate answers it with false confidence. Until then, the raw-data column keeps the model honest by comparing its prediction with the recorded fraction.

4.6 Generated runs

The most likely sale under the measured overlay is the one a newcomer should see first. The most likely *failing* sale comes from the declared overlay, because the measured one gives every failure probability zero: it is the stockout, the case the corpus never shows and the core says can happen.

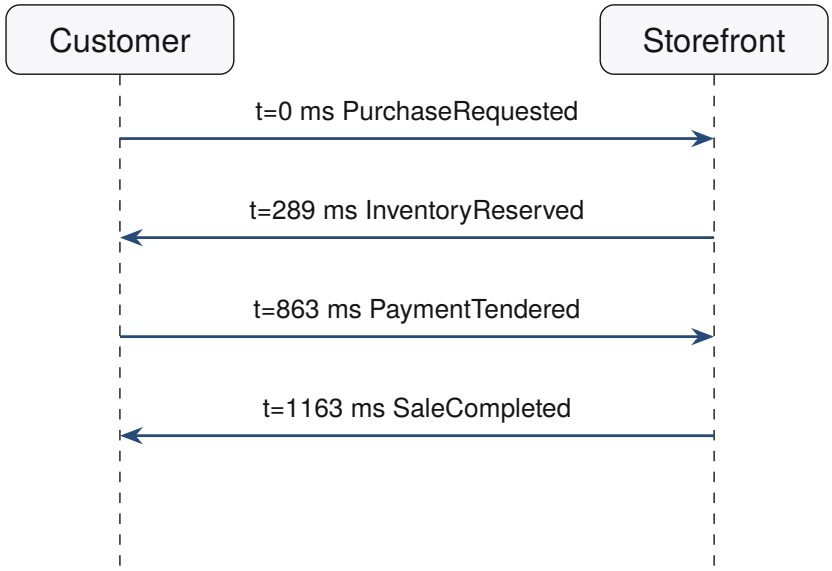


Figure 4.4: The usual sale, with mean times measured from the corpus.

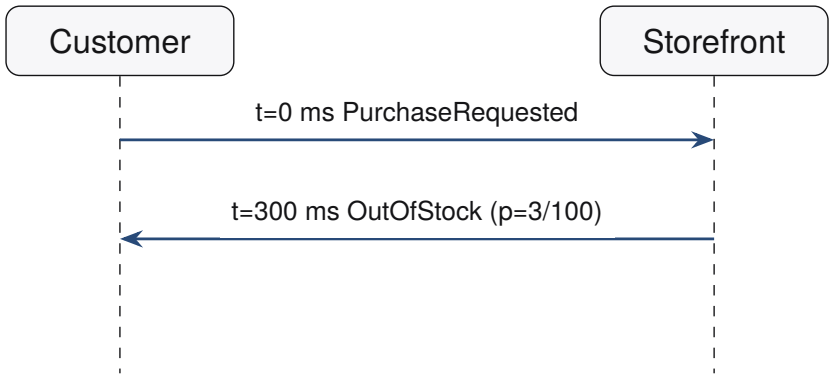


Figure 4.5: The most likely failing sale under the declared overlay: a stockout, probability $3/100$, never seen in the corpus.

4.7 Money and waste

The same corpus answers economic questions through ordinary reducers. For each calendar day, net revenue is the sum of `SaleCompleted` amounts less refunds; ingredient cost comes from `BatchBaked`; employee pay from `ShiftPaid`; and modeled contribution is revenue less ingredients and pay. Donated bread is already in ingredient cost, so its value is reported separately rather than subtracted twice, and it is not a tax saving until tax rules are supplied. Costs that have no events in the corpus are excluded, so this is contribution, not business profit.

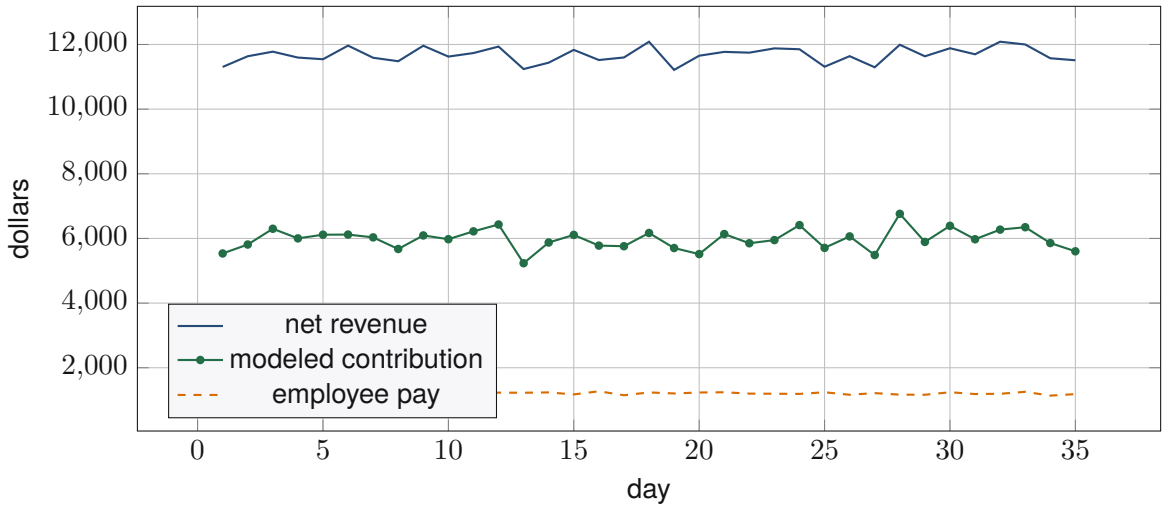


Figure 4.6: Daily economics reduced from sale, batch and payroll events.

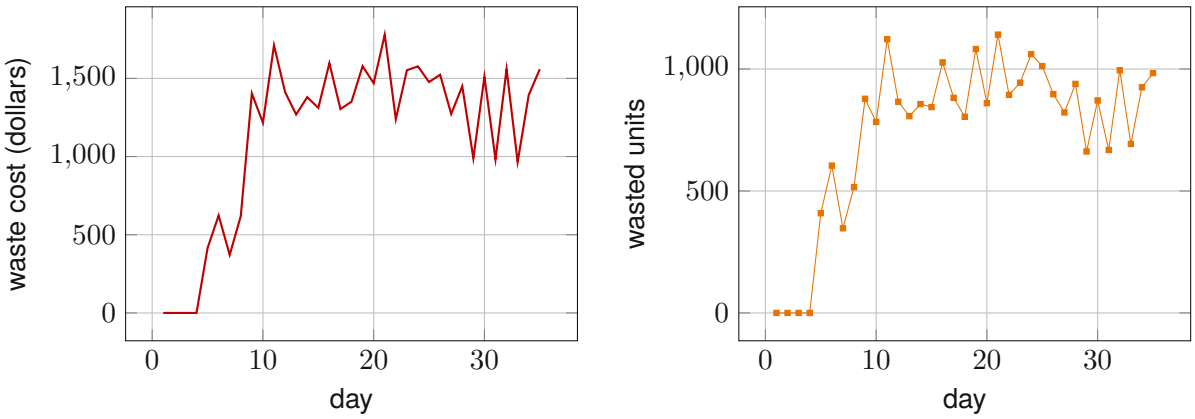


Figure 4.7: Donation-eligible cost and units by day: inventory left when a batch's freshness window closes. Cost and units differ because a donated sourdough loaf and a donated croissant are not worth the same.

Grouping the same events differently—by product, batch age, shift, or order volume—gives further charts without changing the corpus. The axes are always named reductions over events, which is what lets a measured overlay and an economic chart come from one source of truth. How these specifications drive code generation is the subject of Chapter 19.

Chapter 5

Global Types as Grammars

A global session type is a grammar. Its terminal symbols are messages, each with a sender and a receiver; its productions are the type's sequence, choice, parallel composition and recursion; the language it generates is the set of legal conversations. LeanFM writes global types this way so that requirements can be stacked: every protocol can be read as if it were the top-level one, and a larger protocol is built by including smaller ones.

Global types are written in Lean, with the `protocol` syntax of the `LeanFM.Session` library. A protocol elaborates to an ordinary Lean value of type `Global R`, for any type `R` of roles, and the library checks it: who decides each choice, what each cause may point to, and each role's projection. `SessionExample.lean` holds every protocol in this chapter and runs those checks whenever it is elaborated. For durable artifacts a protocol translates to the older `GrammarExpr` form (`event`, `seq`, `choice`, `parallel`, `thresholdJoin`, `repeat`), which the rest of LeanFM's validation consumes. The queue rules later in this chapter are model semantics, not a proof of equivalence to a Go runtime.

5.1 The protocol syntax

The vocabulary comes first. Every message is a Lean structure, and its fields are its payload:

```
structure Order    where item : String  deriving TypeName
structure Quote    where price : Nat    deriving TypeName
structure Pay      where amount : Nat   deriving TypeName
structure Paid     where                deriving TypeName
```

A misspelt label is then an unknown identifier, not a new message. A protocol names its roles and lists statements:

```
protocol Purchase (Client, Shop, Bank) {
  o: Order from Client to Shop;
  choice {
    q: Quote from Shop to Client cause o;
    p: Pay   from Client to Bank cause q where p.amount = q.price;
       Paid  from Bank to Shop cause p;
       Shipped from Shop to Client;
  } or {
       SoldOut from Shop to Client cause o;
  }
```

```
}
```

`o: Order from Client to Shop;`
One message. The optional name `o` lets later messages say `cause o` and lets guards read `o.item`.

`cause q`
This message answers `q`. It becomes the event's cause link, and the checker requires that the sender has sent or received `q`: a role decides only from its own history.

`where p.amount = q.price`
A Lean `Bool` over the named messages in scope, the guarded one included. It is type-checked when the protocol is written and evaluated over decoded payloads when a trace is checked.

`choice { ...} or { ...}`
Exactly one branch happens. Who decides is not written: it is the role that starts every branch, here the shop.

`Timeout at Sender;`
An event at one role only, such as its timer firing; nobody else observes it.

`rec Block { ... continue Block; }`
A named block that `continue` repeats. `continue` ends its branch, so repetition is always in tail position and the state space stays finite.

`include Replicate Primary A B C;`
Another protocol, run here with these roles; whatever follows runs where it finishes.

`par { ...} and { ...}`
Both sides, interleaved, each keeping its own order.

`join 2 of { ...} and { ...} and { ...}`
Any two of the branches, then the rest of the protocol.

The words are not reserved, so importing the library takes no identifiers away from Lean code. `protocol Purchase (Client, Shop, Bank)` defines `Purchase : R → R → R → Global R`, so the same protocol is instantiated, and included, with whatever roles a larger one has. Each statement elaborates to one constructor of the global type:

```
inductive Global (R : Type) where
  | msg    (i : Interaction R) (k : Global R)        -- one message, then k
  | act    (role : R) (label : String) (k : Global R) -- an event at one role
  | choice (branches : List (Global R))               -- one role decides
  | sessions (body : Global R) (k : Global R)         -- any number of copies
  | par    (branches : List (Global R)) (k : Global R)
  | join   (need : Nat) (branches : List (Global R)) (k : Global R)
  | loop   (name : String) (body : Global R)
  | var    (name : String)
  | done
```

A choice does not name the role that decides. Scribble writes `choice at Shop`, but the name can only be right one way: the decider is the role that starts every branch, by sending its first message or by an event at that role alone. So LeanFM reads it off the branches instead, and there is one way to write a choice. An empty branch, `or { }`, is that role choosing to do nothing. The branches must begin differently. If Alice's message started one branch and Bob's the other, no role would decide; both could act at once and the system would leave the protocol, and the checker reports the race, naming who starts what. This is the non-local-choice (realizability) problem of multiparty protocols. In session-type terms the choice must be *directed*: an internal choice of one role, observed by the others as an external choice on their queues.

A second condition goes with it: every role whose later behaviour differs between branches must receive a message that tells it which branch was taken. Projection checks it (below), with one relaxation that matches how servers are built: a role whose part of the session has not started yet owes nothing to a branch that never contacts it. A first attempt at TFTP that puts the sender's timer beside the acknowledgement as a choice has no decider at all:

```
protocol TransferWithTimer (Sender, Receiver) {
  rec Block {
    d: Data from Sender to Receiver;
    choice {
      Timeout at Sender;
      continue Block;
    } or {
      Ack from Receiver to Sender cause d;
      continue Block;
    }
  }
}
```

```
no single role decides a choice, so it is a race: Sender starts Timeout; Receiver
    starts Ack
```

The sender's timer starts one branch and the receiver's `Ack` the other, and nothing stops both from happening at once. The fix, in Chapter 6, moves the timer out of the type and into the channel's physics.

Simple finite conversations can compile to regular languages. Nested request structure motivates context-free productions. Correlation, freshness, uniqueness, and authorization typically require parameter flow or guarded productions.

5.2 A BNF for composable interactions

The syntax, independently of Lean's own grammar:

```
<protocol>  ::= "protocol" <Name> "(" <Role> ("," <Role>)* ")"
                "{" <stmt>* "}"

<stmt>      ::= [<id> ":"] <Msg> "from" <Role> "to" <Role>
                  ["cause" <id>] ["where" <term>] ";"
              | <Event> "at" <Role> ";"
              | "choice" "{" <stmt>* "}"
```

```
    ("or" "{" <stmt>* "}")*
| "par" "{" <stmt>* "}" ("and" "{" <stmt>* "}")*
| "join" <natural> "of" "{" <stmt>* "}"
    ("and" "{" <stmt>* "}")*
| "rec" <Name> "{" <stmt>* "}"
| "continue" <Name> ";"
| "sessions" "{" <stmt>* "}"
| "include" <term> ";"
```

Sequence is one statement after another; an empty branch, `or { }`, is the empty conversation. `sessions` runs any number of independent copies of its body beside what follows, each a session of its own, as a server runs one handler per request. `<Msg>` and `<Event>` name Lean types; `<term>` is a Lean expression: a `Bool` for `where`, a protocol applied to roles for `include`. Guards over the payloads of named messages supply the context-sensitive correlation that plain context-free syntax does not express by itself.

A threshold join is explicit rather than inferred from the number of observed predecessors. Here a primary writes to three replicas and answers its client after any two have stored the write, and the first protocol is included in the second, each written as if it were the whole thing:

```
protocol Replicate (P, A, B, C) {
  join 2 of {
    wa: Write from P to A; Stored from A to P cause wa;
  } and {
    wb: Write from P to B; Stored from B to P cause wb;
  } and {
    wc: Write from P to C; Stored from C to P cause wc;
  }
}

/-- A production that includes another, each written as if it were the whole protocol.
    -/
protocol ReplicatedOrder (Client, Primary, A, B, C) {
  o: Order from Client to Primary;
  include Replicate Primary A B C;
  Done from Primary to Client cause o;
}
```

The continuation becomes enabled after any two of the three branches complete. Its first emitted event records `joinRequired=2`, `joinTotal=3`, and exactly two selected branch-terminal IDs. Those IDs also occur in the event's ordinary `prior` list, which may contain other backpointers such as the task start. The threshold declaration says why two predecessors suffice; the selected IDs say which two satisfied this instance. Unselected branches are not implicitly canceled and may still emit events. Cancellation, if required, must be an explicit message or production.

The OAuth example decomposes into independently readable interactions. These generated views show the two complete branches without making a message arrow look like logical implication:

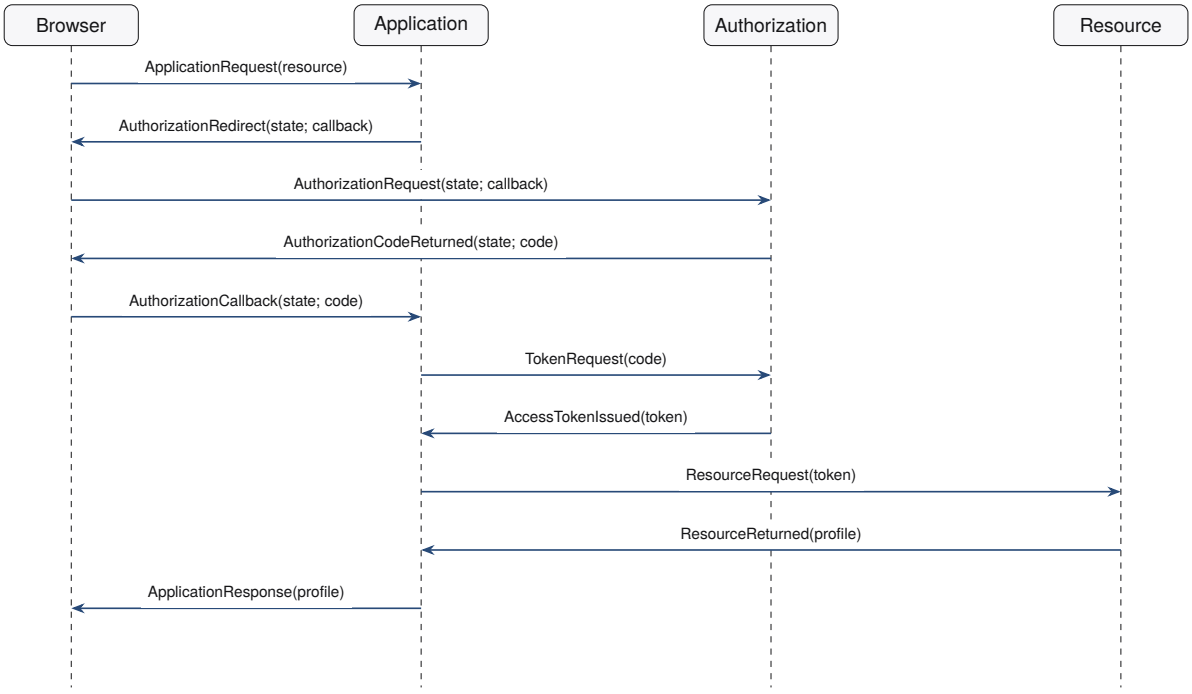


Figure 5.1: Granted OAuth interaction. The resource exchange occurs only after the authorization-code branch succeeds.

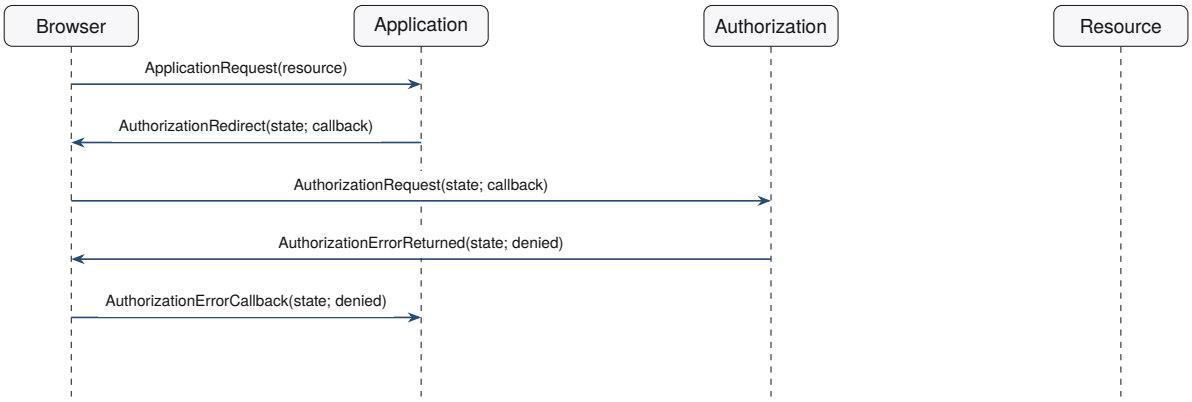


Figure 5.2: Denied OAuth interaction. No token or resource request follows the error.

Chapter 10 gives the complete production definitions, nested alternatives and guarded exchange; these terminal-level diagrams isolate the granted and denied outcomes for comparison.

5.3 Parameters carry information

A protocol should bind values at one message and reuse them later. A message's name makes its payload available to every later guard:

```
r: Redirect  from Server to Browser;
a: Authorize from Browser to Provider where a.state = r.state;
c: Callback  from Provider to Server cause a where c.state = r.state;
```

The repeated `state` is not decorative. It states the information-flow relationship that distinguishes a matching callback from an unrelated one. Lean's types, structures and ordinary expressions are the surface language; the library supplies composition, the checks, and the semantics.

5.4 Projection: from the global type to each role

A participant's implementation need not know the global execution state. The global type states the contract, and projection derives each role's *local type*: the same protocol with every message the role neither sends nor receives erased, and every remaining message rewritten as a send or a receive. A choice the role makes becomes a `select`; a choice another role makes becomes an `offer`, resolved by what arrives. LeanFM computes it (`Global.project`); for `Purchase`:

Client

```
send Order to Shop;
offer {
  receive Quote from Shop;
  send Pay to Bank;
  receive Shipped from Shop;
} or {
  receive SoldOut from Shop;
}
```

Shop

```
receive Order from Client;
select {
  send Quote to Client;
  receive Paid from Bank;
  send Shipped to Client;
} or {
  send SoldOut to Client;
}
```

Bank

```
offer {
  receive Pay from Client;
  send Paid to Shop;
} or {
```

```
}
```

The bank never learns why it was not paid, and its implementation should not depend on it. Projection is defined only for a well-formed global type: where a choice is erased, the branches a role still sees must be the same, or must start with different messages that role receives, or the role's part must not have started yet, which is the bank's empty `or { }`. Otherwise projection fails with the reason, as it did for the hold placed before the choice and for the timer above. A local type is a production of its own: it is the thread an actor runs for one session. Proving that projection and composition preserve the global type's properties remains a research obligation; what LeanFM checks today is that every projection exists.

5.5 Declared physics

A global type says which message may follow which. It does not say what happens to a message between its send and its receive. LeanFM requires that to be declared beside the protocol, as the *physics* of each channel. In `LeanFM.Session.Physics` a channel is a structure whose defaults are the reliable channel, so a declaration names only what differs from a preset:

```
protocol Transfer (Sender, Receiver) {
  rec Block {
    d: Data from Sender to Receiver;
    choice {
      Ack from Receiver to Sender cause d;
      continue Block;
    } or {
      LastAck from Receiver to Sender cause d;
    }
  }
}

def tftpPhysics : Physics Role :=
  { channels := [ (.sender, .receiver, { udp 2 with duplication := false })
                , (.receiver, .sender, { udp 2 with duplication := false }) ]
    timers := [{ role := .sender }] }
```

The checker then compares what each channel surfaces with what the protocol says it handles. For TFTP the acknowledgement channel can reorder, and the requirements do not say that is handled:

```
channel Receiver -> Sender surfaces reorder, which its protocol does not handle
```

Like the protocol, the declaration is ordinary Lean data. The six axes of Chapter 1 are its fields, and a preset is a value of the structure that a declaration updates. This is `LeanFM.Session.Physics`, abridged:

```
inductive WhenFull   | block | reject | dropNewest | dropOldest
inductive Sequencing | fifo | any

structure Sharding where
  key        : String
  partitions : Nat := 3
```

```
/-- Every field defaults to the reliable channel. -/
structure Channel where
  capacity    : Nat := 16
  whenFull    : WhenFull := .block
  order       : Sequencing := .fifo      -- within one partition
  sharding    : Option Sharding := none
  loss        : Bool := false
  duplication : Bool := false
  masked      : List Condition := []     -- hidden by a lower layer's retries
  extra       : List Condition := []     -- surfaced by layers a library adds

def udp (capacity : Nat := 2) : Channel :=
  { capacity, whenFull := .dropNewest, order := .any,
    loss := true, duplication := true }
```

Per-key ordering is not a separate value: it is `fifo` ordering on a sharded channel, where each partition is its own FIFO queue.

The checked system is then the product of the roles' local types and their channels. Each field contributes, or removes, transitions of the product:

- `capacity` and `whenFull`: a send to a full queue blocks the sender, delivers a refusal message back to it, or discards a message;
- `order`: under `fifo` only the head of a queue may be delivered; under `any` any queued message may be;
- `sharding`: the channel is `partitions` queues, each with the declared capacity, and a send goes to the queue its key selects; for checking, keys are abstracted to as many distinct values as the protocol's messages can tell apart;
- `loss`: a transition that removes any queued message;
- `duplication`: a transition that delivers a queued message and leaves it queued.

A timer contributes a transition that may fire whenever its role is waiting. Every one of these is a choice the checker explores, so each `may` multiplies the state space; the presets keep a declaration honest without making it long. Chapter 6 builds this product for TFTP and shows a global type that is well formed in the classic sense, and whose original implementation duplicates every block once a single timer fires early.

The rest of this chapter defines the default physics, the one that LeanFM's actor runtime assumes when nothing more is declared: bounded, reliable, blocking queues.

5.6 Local types run on actors with bounded queues

Each role is played by an actor, and one actor may take part in many sessions at once. An actor has separate positive finite-capacity inbound and outbound message queues. Queue capacity is part of the observable execution model: producing a message may block when the outbound queue is full, and transport may block when the destination's inbound queue is full. Sending is therefore distinct from delivery and from the destination actor consuming the message.

Messages carry a session identifier, naming the instance of the global type they belong to, as well as a task identifier. Their pair is the correlation key for actor-local work:

```
structure WorkKey where
  session : SessionId
  task    : TaskId

structure ActorRuntime where
  inboundCapacity  : Nat
  outboundCapacity : Nat
  inbound          : List Envelope
  outbound         : List Envelope
  inFlight         : List (Prod WorkKey GrammarResidual)
  inboundBounded   : inbound.length <= inboundCapacity
  outboundBounded  : outbound.length <= outboundCapacity
```

An actor may have zero or many distinct `WorkKey`s in flight. Consuming an inbound envelope selects or creates the residual associated with that envelope's `(session, task)` pair; it does not replace one actor-wide "current task." Messages from different keys may interleave while the grammar preserves the order required within each conversation.

The queues have Go-channel-like operational semantics. Capacity is enforced when an operation is attempted, not checked only after constructing a new state:

```
send(channel, message) =
  if channel.length < channel.capacity
  then sent(channel ++ [message])
  else blocked

receive(channel) =
  if channel = message :: rest
  then received(message, rest)
  else blocked

tryReceive(channel) =
  if channel = message :: rest
  then some(message, rest)
  else none
```

A blocked operation makes no progress and leaves the channel unchanged. A blocking actor may run again when the required space or data becomes available. `tryReceive` is a different operation: `none` reports that the channel is empty now and lets the actor continue. It does not put the actor to sleep, and it must not be confused with receiving a valid optional payload.

The resulting transitions are explicit:

- an actor step appends a produced envelope to its outbound queue;
- a transport step moves the outbound head to the destination's inbound queue when both operations are within capacity;
- a receive step removes an inbound envelope and advances the matching `(session, task)` residual;

- sending to a full outbound queue blocks the sender without enqueueing;
- transport to a full inbound queue blocks without removing the outbound head;
- blocking receive on an empty inbound queue blocks, while nonblocking `tryReceive` returns `none` and permits later work.

Queue order and predecessor order are not identical. FIFO determines which queued envelope is available next, while `priorIds` records the event-order graph, including forks and joins across actors. Incomparable branches may commute; the pointer relation does not itself claim that one event caused another.

Chapter 6

Declared Physics: TFTP and the Sorcerer's Apprentice

A protocol grammar says which messages may follow which. It does not say what happens to a message after it is sent: whether it waits in a queue, how large that queue is, whether the network may lose it, deliver it twice, or deliver it late. This chapter argues that those facts are part of what a protocol means, and that a grammar is not finished until they are declared. The example is the Trivial File Transfer Protocol (TFTP) [23, 24] and its best-known defect, the *Sorcerer's Apprentice* bug, which RFC 1123 named and fixed [3].

This is the book's central claim in its smallest form. The same lock-step grammar gets two opposite verdicts. Read as a multiparty session type [12], the original sender and the fixed sender are the same well-formed protocol. With the physics declared, LeanFM finds a walk in which one early timer makes every later block travel twice for the original sender, and proves that no such walk exists for the fixed one.

Status: the verdicts below are real `check_ctl` results. The protocol and its physics are checked Lean, in `SessionExample.lean`. The product of the roles and their queues, which the verdicts are about, is still built by a small script, `video/tftp_model.mjs` in the companion video repository, and handed to the checker as an ordinary finite task model; deriving it from the Lean declarations is future work.

6.1 The grammar

TFTP moves a file in numbered blocks. The sender sends `DATA` n; the receiver answers `ACK` n; the sender then sends `DATA` $n{+}1$. A block shorter than the full size ends the transfer.

In LeanFM's session-type syntax, with the physics beside it:

```
protocol Transfer (Sender, Receiver) {
  rec Block {
    d: Data from Sender to Receiver;
    choice {
      Ack from Receiver to Sender cause d;
      continue Block;
    } or {
      LastAck from Receiver to Sender cause d;
    }
```

```
  }
}

def tftpPhysics : Physics Role :=
  { channels := [ (.sender, .receiver, { udp 2 with duplication := false })
                , (.receiver, .sender, { udp 2 with duplication := false }) ]
    timers := [{ role := .sender }] }
```

Without a timer this is a textbook global type. Each choice has exactly one chooser, the receiver, which answers `Ack` for a full block and `LastAck` for a short one, and the sender learns which from what arrives. Projection gives each role a local type, and the session-type theory guarantees that the projected roles never deadlock and never receive a message they did not expect.

A real network loses packets, so the sender also needs a timer: when no `ACK` arrives in time, it sends the same block again. That retransmission is the whole engineering content of TFTP, and the session-type reading has no place to put it, because it assumes every message sent is eventually received exactly once and in order.

6.2 The physics

LeanFM states the physics next to the protocol instead of assuming it away: `tftpPhysics` above. Each channel is the `udp` preset with one difference written out. On `udp` a full buffer drops, and queued packets may be delivered in any order or dropped. The timer may fire early, because a late `Ack` and a lost one look the same to the sender.

Each direction holds at most two packets. The network may deliver them in either order, or drop either one. The sender's timer may fire at any time while a block is unacknowledged, because a slow answer cannot be told apart from a lost one. The network is declared never to duplicate a packet: the bug needs no help from it, and every duplicate below is made by the sender. The model has three blocks.

The two senders differ in one line, the reaction to an `ACK` that arrives again for a block that was already acknowledged:

Original (RFC 783).
Every `ACK` n, even a repeated one, makes the sender send `DATA` $n{+}1$.

Fixed (RFC 1123).
A repeated `ACK` is ignored. A block is sent again only when its own timer fires.

The checked machine is the product of the sender, the receiver, and both queues. A state records the sender's current block, the block the receiver expects next, and the contents of the two queues. Each sender's product has 201 reachable states and 891 transitions. Its events are `timeout`, `drop`, `data` (a new block delivered), `dup`k (block k delivered again, which the receiver acknowledges again), `ack` (the first `ACK` for the current block), `dupack` (an `ACK` for an earlier block), and `done`.

6.3 The question

Suppose the timer fires early exactly once, at the start, so `DATA` 1 is in flight twice. From then on nothing is lost and the timer never fires again. Can block 3 still arrive twice? Asked from the state after that first `timeout`, the question is one strong existential until:

$$E\,[\,\neg(\mathit{timeout} \vee \mathit{drop})\;U\;\mathit{dup}_3\,]$$

Read this as: "there is a path on which no `timeout` and no `drop` occurs until block 3 is delivered a second time." Here E is "on some path," U is "until," $\neg$ is "not," and $\vee$ is "or."

For the original sender LeanFM proves it and returns the witness walk in table 6.1. The queue columns show the packets waiting after each step, read directly from the states on LeanFM's path.

step	event	meaning	DATA queue	ACK queue
1	`timeout`	timer fires early; DATA 1 sent again	1, 1	–
2	`data`	DATA 1 delivered	1	1
3	`dup1`	DATA 1 delivered again; ACK 1 sent again	–	1, 1
4	`ack`	ACK 1 received; DATA 2 sent	2	1
5	`data`	DATA 2 delivered	–	1, 2
6	`dupack`	ACK 1 received again; DATA 2 sent again	2	2
7	`dup2`	DATA 2 delivered again; ACK 2 sent again	–	2, 2
8	`ack`	ACK 2 received; DATA 3 sent	3	2
9	`data`	DATA 3 delivered	–	2, 3
10	`dupack`	ACK 2 received again; DATA 3 sent again	3	3
11	`dup3`	DATA 3 delivered again; ACK 3 sent again	–	3, 3

Table 6.1: LeanFM's witness for the original sender: one early timer, then no loss and no further timeouts, and every later block is sent and delivered twice.

The pattern repeats with period four: a doubled block brings a doubled `ACK`; the first copy sends the next block, and the late second copy sends it again. Nothing in the walk is a failure. Every packet arrives. The duplication is self-sustaining, which is why the bug was named after the brooms that keep multiplying, and why it doubled the traffic of real transfers for the rest of their length.

For the fixed sender the same formula is disproved, and the evidence is exhaustive: LeanFM evaluated it over all 509 reachable nodes of the checker's search and found no such path. The repeated `ACK` 1 is ignored, the extra copy of block 1 is the last extra packet, and the duplication dies out on its own.

6.4 Liveness needs fairness

Two more questions are asked of both senders.

$$AG\,EF\;\mathit{terminal}\ \text{holds}, \qquad AF\;\mathit{terminal}\ \text{fails}.$$

Read this as: "from every reachable state, some continuation finishes the transfer" is proved, and "every path finishes the transfer" is disproved. For the second, LeanFM returns a lasso: the timer fires, putting a second copy of block 1 into the queue, and then fires again forever. The queue is full, so each further copy is dropped, and the network never delivers anything.

No protocol can force a network to deliver. A promise that the transfer *will* finish therefore needs an explicit assumption, such as "a packet sent infinitely often is delivered infinitely often." That is a

fairness requirement on the channel, and it belongs in the requirements, stated out loud, not hidden in the meaning of an arrow.

6.5 What the classic session-type reading leaves out

sender	as a session type	with the physics declared
original (RFC 783)	well-formed lock step	one early timer doubles every later block
fixed (RFC 1123)	the same well-formed lock step	no cascade; checked over all 509 nodes

Table 6.2: The same grammar, two meanings.

The grammar did not change between the two rows of table 6.2; only its meaning did. Session-type theory adds asynchrony with unbounded, reliable, order-preserving queues [12]. Under those assumptions a retransmission timer is unnecessary and the Sorcerer's Apprentice cannot be expressed, let alone detected. LeanFM's position follows from the actor view of a protocol:

- Every choice is an internal choice of one actor, made only from that actor's own history: what it sent, and what arrived in its queue (`prev` and `cause`).
- Every queue has a declared capacity, and a declared policy for when it is full.
- Every channel declares what the network may do: drop, reorder, duplicate, delay.
- Liveness claims carry their fairness assumptions as named requirements.

UDP protocols are the natural test bed, because they make the physics impossible to ignore. TFTP is the smallest one with a famous bug, and the bug is invisible until the queues and the early timer are part of the model.

Chapter 7

The Scenario Authoring Language

Proposed notation: this chapter develops a concrete surface language using ordinary Lean `def`, function parameters, `let`, typed values, and `do` notation, with library operations for scenarios, endpoints, choices, probability, and rendering. Names may change as semantics are implemented. The worked MDP later in this chapter is a separate **finite-model check**; its figures are **generated analyses** from the named Lean source.

7.1 Proposed scenario notation

A scenario is a named fragment of a global type: a grammar whose terminals are observable messages, each with a sending and a receiving role. This proposed syntax uses `from_` and `to_` (avoiding Lean's `from` syntax) to set the endpoints copied into each event; an `emit` without both endpoints is invalid. Ordinary `let x := e` evaluates a Lean expression, while `let x <- declare T "x"` introduces an immutable typed symbolic value bound by trace generation or matching. Reusing a binding expresses identity, while receiving an event binds its fields for later messages. `emit` and `receive` denote the same grammar-terminal kind; they differ only in participant projection.

```
def requestQuote (customer shop : Actor) : Scenario :=
  scenario "request a quote" do
    let requestId <- declare RequestId "requestId"
    let quoteId <- declare QuoteId "quoteId"
    let amount <- declare Money "amount"
    from_ customer; to_ shop
    emit "quote requested" [field "requestId" requestId]
    from_ shop; to_ customer
    emit "quote offered"
      [field "requestId" requestId, field "quoteId" quoteId,
       field "amount" amount]
```

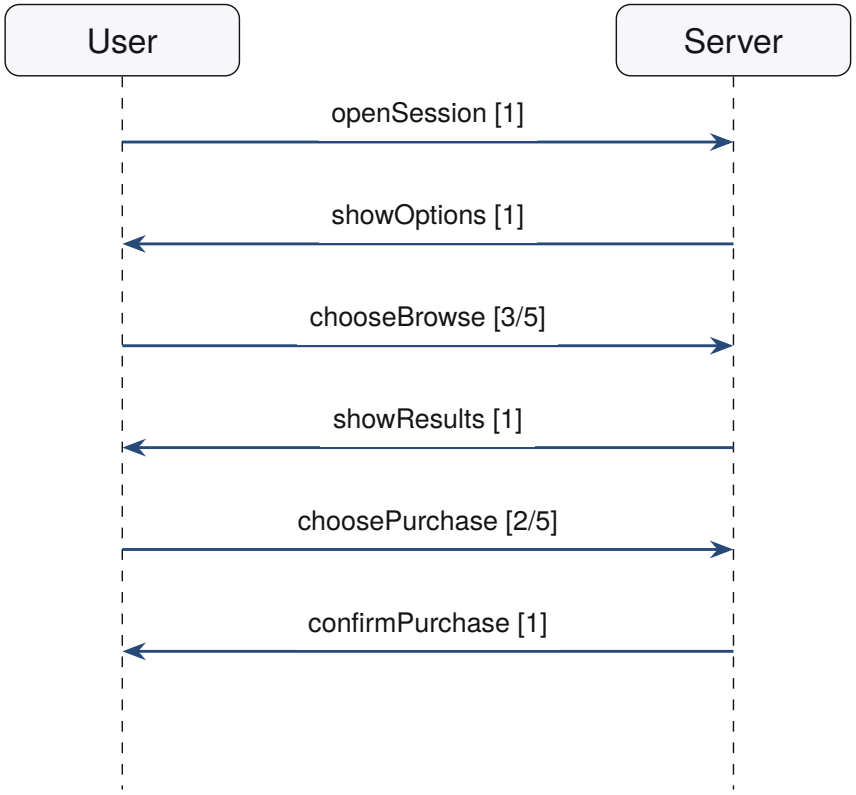
Early requirements can use stable semantic message names and typed fields without committing to a protobuf schema; these suffice for ordering, correlation and temporal properties. A later refinement maps those fields to a concrete alphabet; its obligation is language inclusion after representation details are erased. Serialization, retries and internal state may be chosen there, but externally visible behavior forbidden by the abstract grammar may not be added.

Named inclusions represent reusable productions rather than textual copying. They can export values for later events; a compiler must resolve references, check exports, reject cycles and build

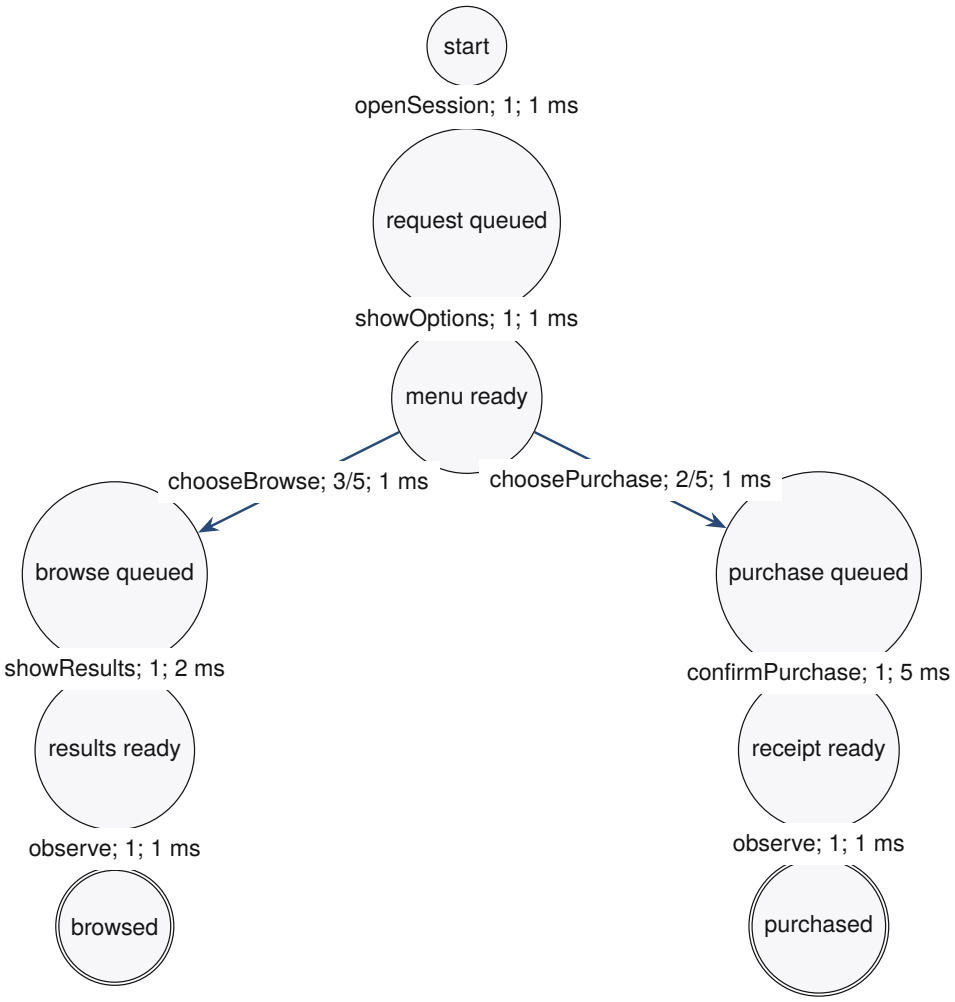
the grammar. For example, a quote outcome can choose between named accepted and declined branches. Keep those branches independently renderable rather than combining unrelated concerns in one story. No quote diagram is generated yet: the event-derived diagram contract is in Chapter 3, with generated interaction examples in Chapter 5.

`oneOf` states nondeterminism without claiming branch frequencies; `weightedOneOf` supplies a distribution when one is supported. Chapter 5 assigns agency to the first distinguishing terminal, and deterministic service transitions carry incoming probability mass forward. Requirements may begin with unresolved choices, symbolic weights or ranges. Refinement must resolve these claims and preserve every abstractly legal branch, including one assigned zero probability by the current implementation.

7.2 Worked MDP: a stochastic user and deterministic server



Weights and probability-one server edges come from the checked `Choice` values.



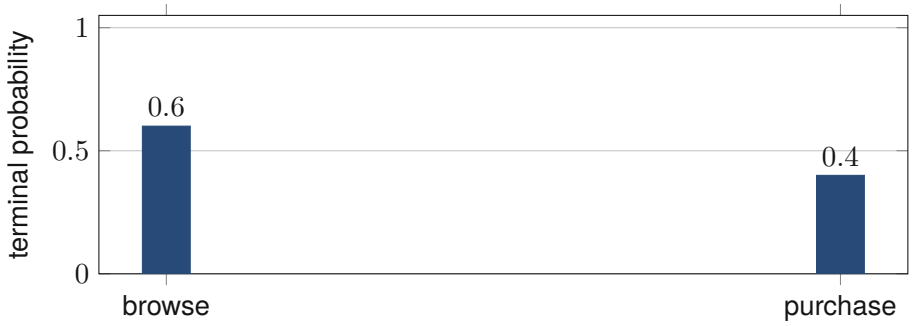
The branch weights are relative integers. Their sum supplies the denominator:

$$\frac{3}{5} + \frac{2}{5} = 1.$$

This MDP example writes weights inside the model and the MDP checker rejects an empty chance set and zero-weight branches, normalizing positive weights by their total. That is a separate mechanism from the probability overlays of Chapter 13, where the task core carries no numbers, weights are exact fractions that must already sum to one, and a branch may be given probability zero while staying legal, as the refinement rule above requires. New work should prefer overlays.

Assign reward 1 to browsing and reward 8 to purchasing. The terminal paths and their durations give

$$\mathbb{E}[R] = \frac{3}{5}(1) + \frac{2}{5}(8) = \frac{19}{5}, \qquad \mathbb{E}[T] = \frac{3}{5}(6) + \frac{2}{5}(9) = \frac{36}{5}\ \text{ms} = 7.2\ \text{ms}.$$



`StochasticUserMDPExample.lean` checks the distribution, deterministic server states, terminal paths, mass conservation and both expectations before generating these views. The present evaluator expands a finite number of steps under a selected policy. It does not yet compute an optimal policy, solve an unbounded fixed point, or take minimum and maximum values over unresolved schedulers. Those are distinct extensions; this example establishes that exact probability propagation already works for a bounded policy. Under this fixed policy, the example is a finite Markov chain: its only branching is the user's specified chance distribution, not a policy choice or unresolved scheduler.

7.3 Actor specifications and instances

An actor name in a grammar denotes a reusable specification, not necessarily one process. A finite population gives that specification concrete identities:

```
actor Client
  |>.instances "client-" 20 (sticky "session")

actor Server
  |>.instances "server-" 2 (shardHash "tenant_id")
```

Generated scenarios resolve grammar roles to concrete instances such as `client-7` and `server-2`. Each instance inherits its actor's behavior and resource bounds and follows the queue and in-flight `(session,task)` semantics of Chapter 5. A population is homogeneous: every member implements the same actor specification, so a cluster cannot mix gateways, databases, and workers merely because they share a deployment boundary.

Each cluster declares message routing: `sticky` keeps the value of an observable affinity field on the same member, `roundRobin` cycles through members in declared order, and `shardHash` hashes an observable shard key. Routing changes legal per-instance histories, queue pressure, hot spots, failure impact, and replay. Kafka and DynamoDB partition keys and MongoDB shard keys motivate shard hashing, but each product has additional replication and routing semantics. Explicit finite populations keep queue, memory, probability, and temporal checks bounded.

7.4 Non-functional requirements: rates and scalability

Using the boundary-event model from Chapter 2, suppose every client repeatedly creates a fresh server task. Each completed unit records a start, a terminal that backpoints to it, and a nonnegative amount of work. For illustration, let one moved byte be one work unit and timestamps be milliseconds; an exhaustive observation list supports two different rates.

The word *load* is too ambiguous to serve as a variable. Let N be the declared upper limit on the number of active clients in the study, and let the parameter n be the active-client population for one observation, with $1 \leq n \leq N$. Separately, let λ_o be offered work per unit time, λ_a be work admitted per unit time, and X be work completed per unit time. Offering work, accepting it into a queue, and completing it are three different observable boundaries. The client-experienced, properly weighted rate is

$$X_{\text{client}}(n) = \frac{\sum_i \text{work}_i}{\sum_i (e_i.timeAt - s_i.timeAt)}.$$

Long requests contribute in proportion to their duration; this is not an unweighted mean of per-request ratios.

The aggregate server throughput is instead

$$X_{\text{server}}(n) = \frac{\sum_i \text{work}_i}{\max_i(e_i.timeAt) - \min_i(s_i.timeAt)}.$$

Its denominator is the wall-clock interval from the earliest start to latest end, so overlapping client waits are counted once.

For an observation window of duration T, the boundary rates and backlog obey

$$\lambda_o = \frac{W_o}{T}, \qquad \lambda_a = \frac{W_a}{T}, \qquad X = \frac{W_c}{T}, \qquad B_1 = B_0 + W_a - W_c.$$

The backlog identity assumes a conserved work unit; record cancellations, expiry, and failures as terminal dispositions when they remove work. If $\lambda_o > X$, some offered work is rejected, remains queued or in flight, or ends in another recorded disposition; admitted work can also outpace completion while latency and backlog grow. Window throughput differs from the client- and server-task rates above. Fit the Universal Scalability Law against active-client population, not offered work rate λ_o unless the experiment defines an explicit mapping:

$$X(n) = \frac{\gamma n}{1 + \alpha(n-1) + \beta n(n-1)}, \qquad 1 \leq n \leq N.$$

Gunther's *Guerrilla Capacity Planning* develops the USL as a practical capacity-planning tool for exposing contention, coherency, saturation, and retrograde scaling before deployment supplies those facts [9].

Record per observation its n, selected throughput and event boundary, λ_o, λ_a, window throughput X, latency, backlog, terminal outcomes, server count, and window boundaries. Fit parameters separately and only on exhaustive windows, avoiding bias from omitted or unfinished work.

7.4.1 The three USL regimes and capacity

The scale parameter γ is analogous to clock speed: it is the single-client service-rate scale and multiplies the entire curve vertically. It is not literally a processor clock, because an observed unit of work may include networks, storage, and several actors. Changing γ does not move the population at which the dimensionless USL curve peaks.

The two overhead parameters have distinct effects:

$$\begin{aligned}
\alpha = 0,\ \beta = 0: &\quad X(n) = \gamma n, && \text{ideal linear scaling,}\\
\beta = 0: &\quad X(n) = \frac{\gamma n}{1 + \alpha(n-1)}, && \text{contention-limited saturation,}\\
\beta > 0: &\quad X(n) = \frac{\gamma n}{1 + \alpha(n-1) + \beta n(n-1)}, && \text{coherency-limited retrograde scaling.}
\end{aligned}$$

For $0 \le \alpha < 1$ and $\beta > 0$, differentiation gives the finite capacity population

$$n_* = \sqrt{\frac{1-\alpha}{\beta}}.$$

This fitted peak need not equal the study limit N.

The observed integer capacity is one of the adjacent integer populations, whichever has the larger measured or fitted throughput. When $\beta = 0$ and $\alpha < 1$, the curve increases toward the asymptote γ/α but has no finite USL peak. When both parameters are zero, the mathematical model has no internal capacity limit; a real finite system must eventually reveal an omitted bottleneck or cease to fit that ideal case.

Figure 7.1 uses $\alpha = 0.1$ and, in its third panel, $\beta = 0.005$, marking the fitted peak $n_* \simeq 13.42$.

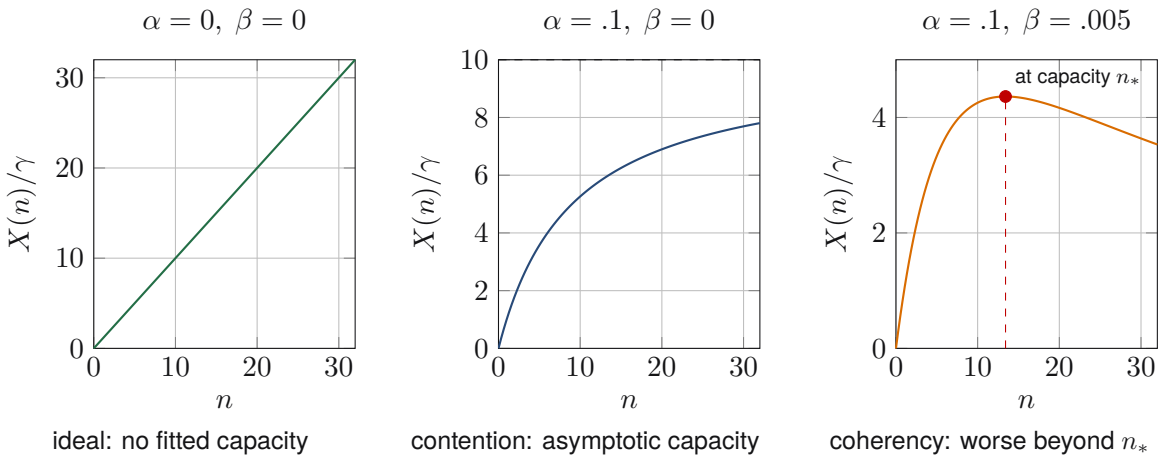


Figure 7.1: Universal Scalability Law regimes. Contention alone saturates; coherency creates a finite peak and declining throughput beyond capacity.

Population capacity is not the only overload boundary. For a fixed deployment with sustainable service rate μ, offered load λ is at capacity when $\lambda = \mu$. Below capacity, a simple queueing illustration has response time proportional to $1/(\mu - \lambda)$ and work in the system (waiting plus in service) proportional to $\lambda/(\mu - \lambda)$. At and above capacity these steady-state quantities are unbounded: completed throughput is capped by μ, while unfinished work, latency, queued bytes, and memory pressure accumulate. A finite queue converts that growth into blocking or rejection; exhausting the memory budget is a fatal terminal event rather than another slow sample.

For the queueing foundation behind this illustration—including workload characterization, service demand, utilization, response time, queueing networks, and cost/performance tradeoffs—see Menascé, Almeida, and Dowdy's *Performance by Design* [18]. USL and a single queue answer

different questions and belong together: USL summarizes scaling across populations, while queueing models explain how offered work waits for finite service centers. Neither rescues a requirements model whose demand, service rates, queue bounds, observation window, and overload behavior were never stated.

These examples use the positive finite per-instance inbound/outbound capacities and full/empty blocking semantics defined in Chapter 5.

A queue also has an idempotent `close()` operation. Closing forbids every later push. A sender already sleeping for capacity wakes with a closed rejection and does not mutate the queue or message. Values accepted before close remain available and are received exactly once in order. Receivers report closed only after those buffered values have drained; closing an already empty queue wakes blocked receivers immediately. Consequently, queue iteration has a precise termination condition: the queue is closed and empty. Empty while still open means “wait,” not “end of stream.”

Current costs and resource profile (not optimized). For C queued items, the list-backed channel sends in $O(C)$, receives in $O(1)$, and stores $O(C)$; the close semantics above are not yet implemented. Two-list or ring-buffer channels can make send and receive amortized $O(1)$ at the same capacity.

With H task events and R predecessor references, the current attempt reducer appends starts and linearly searches for terminal backpointers, costing $O(H^2)$ time and $O(H)$ retained attempts in the worst case. An indexed `map[Event][]Event` successor structure plus an open-attempt hash table targets expected $O(H+R)$ construction time and space; hashing is not a worst-case guarantee.

For N declarations and cross-references and grammar size G, repeated list membership and duplicate checks make validation $O(N^2)$ time with $O(N)$ auxiliary data. Grammar traversal is $O(G)$, but repeated concatenation can make skewed trees $O(G^2)$; indexed names and accumulator walks are optimization targets. Enumerating every path is exponential (b^d for branching factor b and depth d) only when all paths are materialized.

The initial ChatUI profile gives browser/session actors capacities 32 inbound and outbound, with at most 4 in flight; the server has capacities 256 and 32 in flight; provider workers have capacities 64 and 16 in flight. Per-instance memory budgets are 64, 512, and 256 MiB, respectively. These replaceable starting values are not measured production claims.

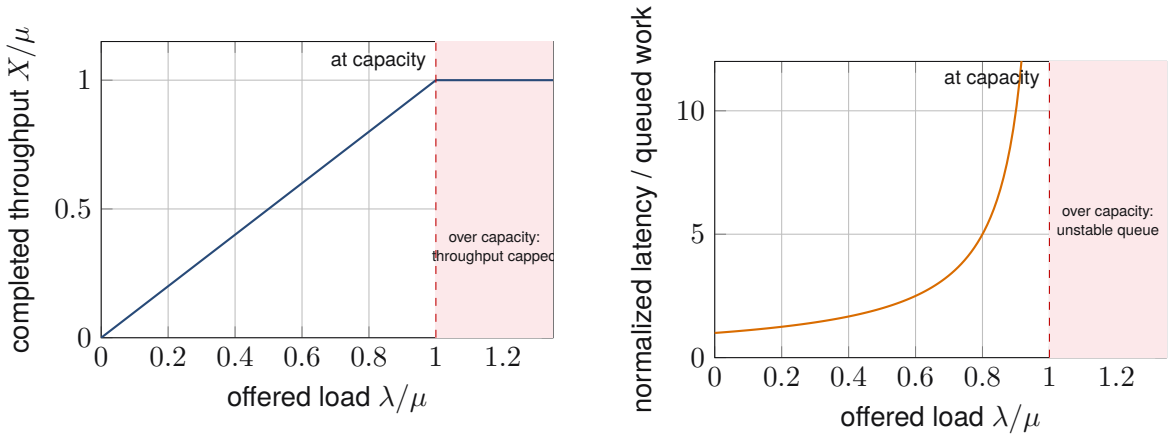


Figure 7.2: Offered-load capacity. Beyond $\lambda/\mu = 1$, throughput cannot increase but queued work and latency continue to worsen until backpressure, rejection, an outage, or fatal memory exhaustion intervenes.

Use the models before implementation to declare demand, sustainable capacity, latency and queue bounds, rejected-work dispositions, replica affordability, and fatal failure modes.

7.5 Outages, recovery, and replica populations

Reliability assumptions belong to actor specifications and are inherited by their instances. A contract may set an outage probability and an expected mean time to repair (MTTR):

```
actor Server
  |>.instances "server-" 2 (roundRobin)
  |>.outageProbability (20 / 10000)
  |>.meanTimeToRepair 45.seconds
```

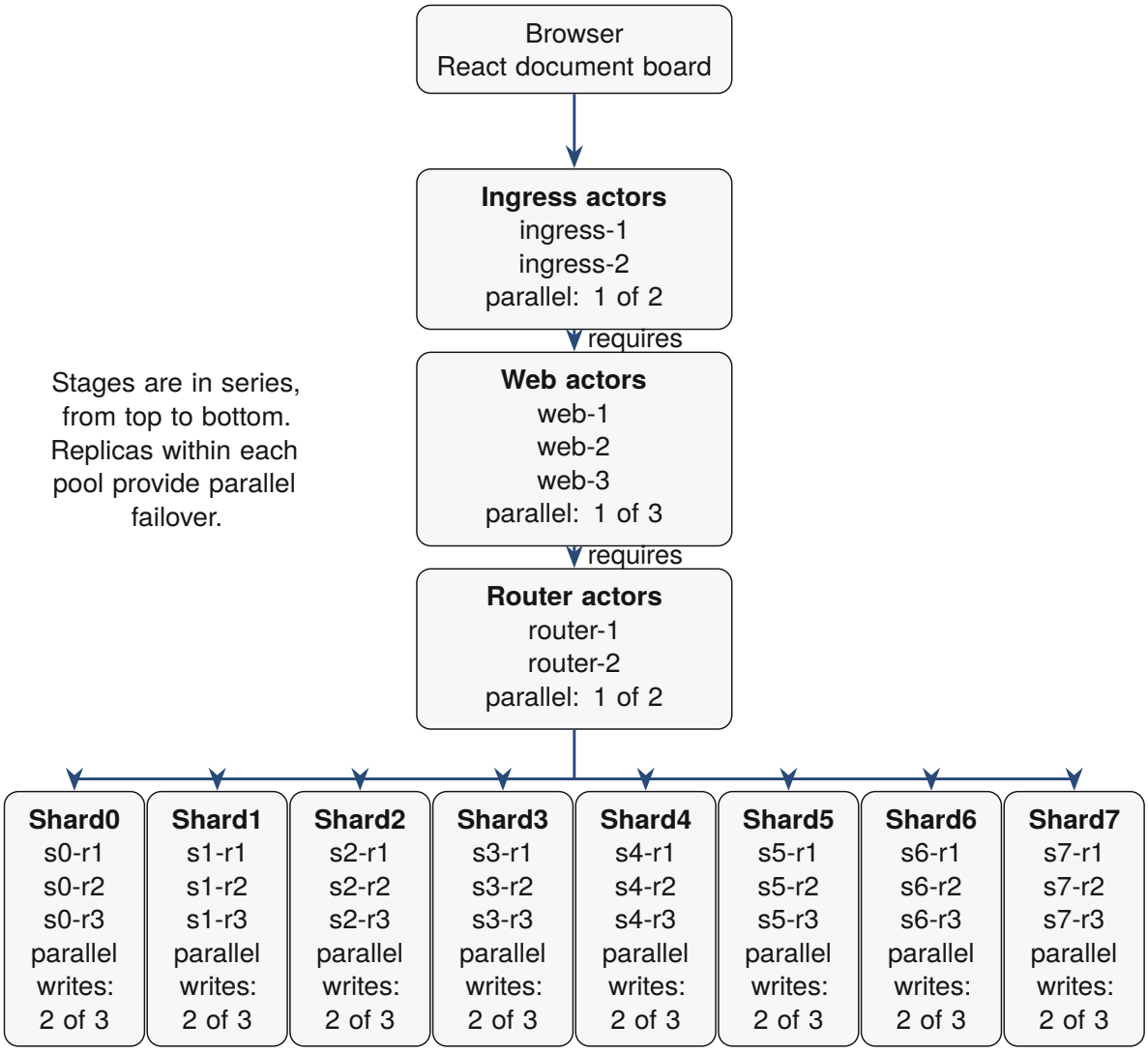
These are specified assumptions. A failed actor does not reliably announce its own failure: there is no ordinary `Unavailable` message to receive. Measurements instead come from evidence visible to an observer. Expected heartbeats stop arriving; an inbound queue ceases to drain; or messages expected from the actor's outbound side cease to appear. Each detector must declare its heartbeat or progress expectation, timeout, clock, observer, and attribution boundary. Silence without such an expectation is not evidence of failure.

The detector may emit derived observations such as `AvailabilityLost` and `AvailabilityRestored`, but these are observer conclusions, not protocol messages sent by the failed actor. A loss observation retains the last known healthy time, the deadline that was missed, the detection time, and the evidence stream. Restoration retains the first new heartbeat or queue/message progress that justifies it and backpoints to the loss observation. The true failure time is generally bounded between the last healthy observation and detection rather than known exactly. If the declared accounting policy uses derived boundary times u_i and r_i, then for R replicas observed from t_0 through t_1, the estimated outage fraction and repair duration are

$$\widehat{O} = \frac{\sum_i (r_i.timeAt - u_i.timeAt)}{R(t_1 - t_0)}, \qquad \widehat{MTTR} = \frac{\sum_i (r_i.timeAt - u_i.timeAt)}{|incidents|}.$$

Availability over the same complete window is $1 - \widehat{O}$. Reports must identify each u_i boundary (last healthy, missed deadline, or detection), since it changes the outage and MTTR estimates. In Kubernetes-style experiments, pods may be actor instances and restarts recovery events; compare configured outage probability and MTTR with observations rather than replacing them.

7.5.1 Redundant, load-balanced runtimes and blast radius



Shards are parallel data partitions with separate availability gates.
Each shard has parallel replicas; writes need a primary and a threshold quorum.
A targeted write selects one shard, not all shards.

Actor topology for a targeted document write. Generated from `RuntimeReliabilityDiagram.lean`, with provenance in `runtime-actor-network-provenance.txt`. Arrows denote required provider paths, not logical implication or message sequence.

Request path. A write needs ingress, web, router, and its selected shard; a multi-shard operation needs every shard its semantics require. Adding required series stages cannot improve availability; independent stage availabilities multiply. Kubernetes schedules and restarts actors but does not supply the application's quorum or consistency protocol.

Replica pools. Proxy and web actors are parallel alternatives within series stages; each data shard has its own parallel replica gate. A usable alternative may improve pool availability, but shared

failure domains and higher quorum thresholds can erase that benefit. Shards are partitions, not interchangeable failover copies.

Stateless pool. Zero ready instances makes new requests unavailable. Otherwise compare offered work with sustainable capacity under the latency bound: overload may trigger rejection, queue growth, or deadline misses even while heartbeats continue. Round-robin request counts need not balance unequal-cost work.

Replicated shard. Majority-acknowledged writes require a current primary and a communicating quorum of eligible configured members; one external observer's heartbeats do not establish mutual reachability. A restarted member is not ready until it catches up.

Read and shared dependencies. Declare read consistency separately: a lost write quorum may still permit local stale reads, but a surviving copy does not prove linearizability. Ingress or routing failure can affect all shards; metadata failure depends on the operation and cached routing, so does not necessarily disable every read or write.

Sharding. Additional partitions can increase capacity when work distributes well; hot keys, shared dependencies, and cross-shard operations limit the gain. Replication protects each partition, not other shards.

Evaluate these dependency gates against timestamped observations and declared uptime/MTTR assumptions; availability is operation-specific, not a total replica count.

Use detector-derived $u_i(t) \in \{0, 1\}$ values and declared boundaries to combine instances through each operation's series, parallel, or threshold gate. For a series gate, every member must be up:

$$u_{\text{series}}(t) = \prod_i u_i(t).$$

For a parallel gate, at least one member must be up:

$$u_{\text{parallel}}(t) = 1 - \prod_i (1 - u_i(t)).$$

More generally, a replicated service declares a threshold k and is available when $\sum_i u_i(t) \geq k$. Its measured availability is the fraction of the observation window for which that predicate is true. These formulas operate on synchronized observed indicators, so simultaneous and correlated failures are preserved. Products such as $\prod_i A_i$ or $1 - \prod_i (1 - A_i)$ may estimate expected availability from marginal availabilities only under an explicit independence assumption; they must not replace the interval calculation when joint outage observations exist.

A dependency edge points from a consumer service or task to the provider gate it requires. At each outage boundary, failure propagation repeatedly marks a consumer unavailable when one of its series requirements is unavailable or when a parallel/threshold requirement has too few surviving providers. The fixed point for an initiating failure set F is $C(F)$, its unavailable- operation set and structural blast radius. Retain the propagation paths, initiating set F, affected shards, request classes, tenants, workload, affected tasks, client-visible outcomes, and recovery order. A degraded read path and an unavailable write path are distinct outcomes. This makes questions such as "what happens if the database and one authorization replica fail together?" answerable without pretending that all

outages are independent. Minimal cut sets identify the smallest initiating sets that disable each operation.

The standard reliability output includes per-instance and series/parallel/threshold service uptime, a time-aligned outage chart, and the dependency graph overlaid with observed or proposed cascades. Named service-down scenarios remain replayable when unobserved and must be labeled predictions, separately from measured uptime.

Calculate impact before estimating its frequency. Uptime estimates help assign frequencies or probabilities to these scenarios; MTTR estimates help assign recovery durations. Neither marginal uptime nor mean repair time determines a cascade by itself. The dependency graph determines which operations are exposed, while correlated failure and repair models determine how often and how long. If scenarios F_j form a mutually exclusive, exhaustive partition of a declared snapshot model, with probabilities π_j, then the expected number of unavailable operations is

$$\mathbb{E}[B] = \sum_j \pi_j \, |C(F_j)|.$$

A count weights every operation equally; a report may instead use an explicitly declared demand or business-impact weight. Overlapping outage scenarios cannot simply be added as if they were disjoint.

For a time-aligned trace, let $C(t)$ be the unavailable-operation set at time t, and let $\lambda_o(t)$ be offered requests per second for operation o. The offered-request exposure over window $[t_0, t_1]$ is

$$L = \int_{t_0}^{t_1} \sum_{o \in C(t)} \lambda_o(t) \, dt.$$

This exposure is measured in requests, not necessarily lost work: queues, retries, deadlines, and recovery determine which requests ultimately fail or complete late. For overload, measure unmet work and deadline violations separately rather than counting every request as unavailable.

Repair the operation, not just the process. Actor MTTR is not automatically service MTTR. Service recovery may include failure detection, replacement scheduling, volume attachment, election, replication catch-up, routing refresh, and backlog draining. These stages can overlap, so compose their event dependencies rather than blindly adding their means. The end boundary is restored operation readiness or the declared latency target, not merely receipt of the first heartbeat. Open incidents at the end of the window remain censored observations, not zero-duration repairs.

Calibrated forecasts require a declared failure/repair process, correlated domains, placement, workload and recovery policy. Keep those assumptions separate from observations; report affected operations, request exposure, deadline outcomes and recovery sequence, with a source and reducer for each forecast. Existing `DesiredOutput` entries establish provenance. This is an analysis contract, not a claim that a general Kubernetes deployment or forecast engine is implemented.

The PaxosKV example project's derived view `renders/quorum-shutdown-network.svg` gives a concrete majority-service example. A Kubernetes-like runtime manages a parallel pool of three stateless web actors; the one-of-three web pool is in series with a two-of-three database-replica gate. Each web actor has specified uptime $9999/10000$; each replica has specified uptime $999/1000$. If replica outages were independent and each replica had uptime p, the quorum's assumed uptime would be

$$A_{\text{2-of-3}} = 3p^2(1-p) + p^3.$$

This independent-marginal baseline excludes correlated shutdowns.

Stopping `kv-1` leaves two voters; stopping `kv-2` next leaves only `kv-3` in that shard, below quorum, and makes the dependent web service unavailable even if other shards remain healthy. Planned shutdowns, shared power, network partitions and common automation therefore belong in named joint scenarios, not independent uptime factors. Adding stateless web instances cannot restore the failed database quorum and may only accumulate waiting requests.

Work waiting on an instance inferred unavailable remains stalled, is rejected, or ends in an explicit timeout/cancellation outcome according to the requirement. An ordinary send or receive attributed to that instance after a detector's loss boundary is evidence that either recovery occurred or the detector/attribution model was wrong; it is not silently discarded as an invalid trace. Thus an outage commonly falsifies a nominal liveness promise (for example, "every accepted request eventually receives a response") even when safety remains intact.

The nominal protocol language and its failure extension must therefore be distinguished. The nominal language describes ordinary successful and rejected conversations. The failure extension adds heartbeat expectations, queue and message-progress observations, derived availability boundaries, stalled or censored work, explicit failure outcomes, and legal resume/retry behavior. Cascade analysis suppresses modeled progress for every instance inferred unavailable at the dependency fixed point, then reports which nominal safety and progress properties remain true, become false, or cannot be decided before the observation window closes.

The same dependency graph supports denial-of-service analysis. A PageRank-like centrality calculation, with dependency edges directed toward the consumers whose operation they enable, highlights nodes whose failure influence is reinforced by other important services. Edge weights may incorporate observed message traffic, affected task value, or measured cascade frequency, but the chosen weighting is part of the requirement and must be shown in the report. Centrality is a prioritization heuristic, not an availability proof: ordinary PageRank can miss a low-traffic series dependency, a cut vertex, or the exact number of replica failures that crosses a quorum threshold. The denial-of- service report should therefore place the centrality ranking beside single-node and named multi-node cascade simulations, minimal service cut sets, lost task capacity, and client-visible outcomes; today only the checklist item and a `Characterization.lean` plan exist. Such a report would answer both "which node looks important?" and the stronger question "what actually stops if an attacker makes this set unavailable?"

7.6 Infrastructure cost and demand-constrained supply

Hardware accrues per provisioned hour; licenses, managed services, and operations may cost per available hour. Neither is a task-event cost to multiply by request count:

```
cost Server
  |>.hardwarePool "general-purpose"
  |>.hardwareCostPerProvisionedHour 12 USD.cent
  |>.serviceCostPerAvailableHour 3 USD.cent
  |>.sustainableCapacityPerHour 3600000 bytes
```

For R replicas, provisioned hours H_r, available hours A_r, hardware rate c_h, and service rate c_s, the accounting-period infrastructure cost is

$$C(R) = \sum_{r=1}^{R} (H_r c_h + A_r c_s).$$

Supply planning is a constrained optimization, not "pick the cheapest server." Given per-replica capacity μ in work units per hour, required availability a, and work rate D within the declared latency boundary, feasibility requires

$$R\mu a \geq D.$$

This is only a necessary capacity condition: routing imbalance, queues, or latency can still make the observed system infeasible.

Route total offered rate λ_o across R servers; let $\lambda_{o,r}$ be server r's offered share:

$$\lambda_o = \sum_{r=1}^{R} \lambda_{o,r}, \qquad \lambda_{o,r} = \frac{\lambda_o}{R} \text{ only under ideal balance.}$$

Under ideal balance, nominal demand is D/R; verify observed instance loads and effective supply. Each replica incurs hardware cost while provisioned and service cost while healthy; compare the hourly total with the optimizer's result.

Executable code and finite-model tests: `optimalSupply?` in `LeanFM/Artifacts.lean` searches replica counts 1 through `maxReplicas`, rejects availability-adjusted capacity below demand, and selects the least expected hourly cost. `LeanFM/ArtifactsTests.lean` checks one feasible optimum and one infeasible demand. This bounded function does not compute observed routing imbalance, latency, or workload-window results; the full supply-planning XY output above remains a target, not generated evidence. Shard hashing can create a hot shard despite aggregate spare capacity; sticky routing and round-robin make different balance assumptions. Product replication, reserved pricing, autoscaling delay, and nonlinear tiers require explicit constraints.

Reuse the observation window and completion boundary above, adding hourly cost and per-server utilization or imbalance. Count only demand completed within the declared outcome and latency boundary; unbounded supply, backlog, cost, or latency does not guarantee finite-deadline service.

7.7 Specified dwell time

A `dwell` constrains modeled duration for a step or named production; observed event-pair durations are defined in Chapter 2:

```
emit "token requested" [field "code" code]
  |>.dwell 3.milliseconds

include exchangeAndFetch
  |>.dwell 12.milliseconds
```

Step dwell d advances the model clock from t to $t + d$ before timestamping its completion event. Production dwell constrains entry-to-exit time, not each contained event. Use exact values, bounded ranges, or weighted outcomes when supported by the timing information.

Dwell can drive modeled latency, throughput, timeout, and queue-pressure predictions, but does not create event order: `priorIds` supplies predecessors, while the grammar states which sequences and dependencies are legal.

7.8 Composing requirements means intersecting constraints

Scenario composition reuses the grammar chapter's operations for sequencing, alternatives, independent interleavings, repetition, and context-sensitive filtering (Chapter 5). Requirements add a different operation: `requireAll` intersects the generated trace language with independent constraints instead of adding message sequences.

```
def quoteSystem : SystemSpec :=
  generatedBy quoteOutcome
  |> requireAll
      [ correlate "offered quote matches request"
          (sameField "requestId")
      , forbid "confirmation without acceptance"
          (event "order confirmed" && notPreviously "quote accepted")
      , ctl "every accepted quote is answered"
          (AG (accepted --> AF (confirmed || rejected)))
      , timing "answer within service objective"
          (accepted --> within 2.seconds (confirmed || rejected))
      ]
```

Consistency requires a nonempty intersection. If it is empty, tooling should report a minimal conflicting requirement set and the shortest available counterexample prefix.

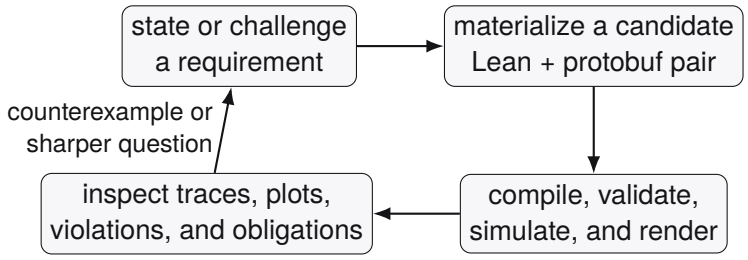
Chapter 8

Arguing Requirements through the Lean Web Server

This chapter documents the staging server that implements the workbench, its project isolation, and the three-layer workflow described in the Preface. The evidence labels in Chapter 1 distinguish its executable paths from proposed and future interfaces; each named check retains its stated scope.

8.1 The argument loop

What the user and the agent argue about is a global type: its roles, its messages, its choices and the physics of its channels. The interface resembles ChatGPT because natural language is convenient for finding omissions. The loop, however, has a formal boundary:



The human contribution is often adversarial. “The upload completes” should prompt: what observable event constitutes completion, who observes it, and does it mean receipt, durable storage, or policy acceptance? “The service handles load” should prompt: how many messages fit in each queue, how many `(session, task)` pairs may be active, and what happens at the bound? The assistant’s job is to turn each answer into structure, not merely agreeable prose.

A productive exchange might proceed as follows.

> **Person:** A submitted upload eventually completes.
> **Assistant:** I will treat an `uploadCompleted` response as completion.
> **Person:** No. Completion requires the storage actor’s acknowledgment, and the client must be able to distinguish rejection from timeout. Both queues are finite.
> **Assistant:** Then the grammar has accepted, rejected, and timed-out alternatives; acceptance contains a storage acknowledgment before the response. I will add queue capacities and a

bounded-liveness obligation.
Person: Also show the behavior with many sessions in flight, estimate the completion distribution from an event stream, and fail the requirement if queued bytes exceed the memory budget.

Every turn should change a named assumption, artifact, check, or open obligation. If it changes none of them, it has not improved the specification.

8.2 The server is a staging environment

`lake exe leanfm-server` serves the workbench on loopback. Every project owns one exclusive root (hosted layout: Section 8.9; portable contract: Chapter 19). Project-relative and canonical absolute paths are accepted only when their resolved targets remain beneath that root; traversal and escaping symlinks fail closed.

The three specifications have the separate roles defined in Section 19.4. HTTP, in-process channels, TCP and other adapters are implementation choices; validation, views, metrics, reports and OpenAPI project the same model. A language model may propose edits, but compilation and validation remain deterministic: prose establishes no proof.

8.2.1 The code that creates the TCP listener

An address value does not open a port. The running server performs four distinct operations: create a socket, bind it to an address, put it into listening mode, and repeatedly accept clients. This is the relevant executable pattern from `Server.lean`:

```
import Std.Async.TCP

open Std
open Std.Net

abbrev TcpServer := Std.Async.TCP.Socket.Server

def bindAddr : SocketAddress :=
  SocketAddress.v4
    { addr := IPv4Addr.ofParts 127 0 0 1
    , port := 8080
    }

partial def serveLoop (server : TcpServer) : IO Unit := do
  let client ← (server.accept).block
  let _ ← IO.asTask (handleClient client) Task.Priority.dedicated
  serveLoop server

def main : IO Unit := do
  let server ← Std.Async.TCP.Socket.Server.mk
  server.bind bindAddr
  server.listen 128
  IO.println "LeanFM server listening on http://127.0.0.1:8080"
  serveLoop server
```

Read the final definition as follows: `Server.mk` creates the TCP socket; `bind` attaches it to loopback port 8080; `listen 128` changes it into a listener with a pending-connection backlog of 128; and `serveLoop` calls `accept` for each connection. Only after `listen` succeeds is there a TCP listener. HTTP is the application protocol implemented by `handleClient`; there is no separate HTTP socket. The actual handler catches per-request errors before returning to the accept loop. This shortened excerpt leaves that response logic out so the listener lifecycle remains visible.

8.3 Using the agent in Codex

Codex can drive the complete bootstrap. Open the LeanFM repository as its workspace, or run `codex` from the repository root. The required book, live language references, source tree, and workflow from self-hosting through review and generation are defined in Chapter 19.

8.4 MCP: the agent-facing entrance

The browser UI is for stakeholders; the Model Context Protocol endpoint is for an LLM agent. LeanFM serves stateless JSON-RPC requests at `POST /mcp`. On initialization, the server tells the agent that the book is durable project context—effectively a maintained section of the system prompt—rather than optional background reading. The agent can enumerate and read fixed resource URIs for `book/main.pdf`, its dark rendering, the canonical requirements, protobuf and implementation files, and both live language references.

This interaction is generated from the implemented MCP dispatch surface:

The first surface is intentionally read-only. It provides resource and prompt discovery, the requirements interrogation, the code-generation prompt, the logic tools, and canonical artifact validation. It exposes no arbitrary pathname and changes nothing. Review and acceptance stay with the stakeholder and happen in git; MCP transports context and evidence.

The bundled server listens only on loopback. A deployment that exposes MCP to a network must add authentication and authorization at the transport or reverse proxy before treating the resource contents as safe to publish. Statelessness does not make project data public, and a future writable MCP surface must bind every request to a user, project and revision.

A local Codex client can register the running endpoint and inspect the result:

```
codex mcp add leanfm --url http://127.0.0.1:8080/mcp
codex mcp list
```

The MCP interface does not replace the book; it makes the book, specifications and live checker discoverable to an agent client.

8.5 Consulting MCP about logic

An agent that writes LeanFM requirements makes three kinds of logical mistake more often than any other. It writes a temporal formula whose reading differs from the sentence it was asked to formalize. It invents a sequence of events that no grammar accepts, or whose causes run backwards, and then draws a sequence diagram from it. And it states a requirement over something no observer can see—“the server knows the password”, “the cache is consistent”—so the requirement can never be

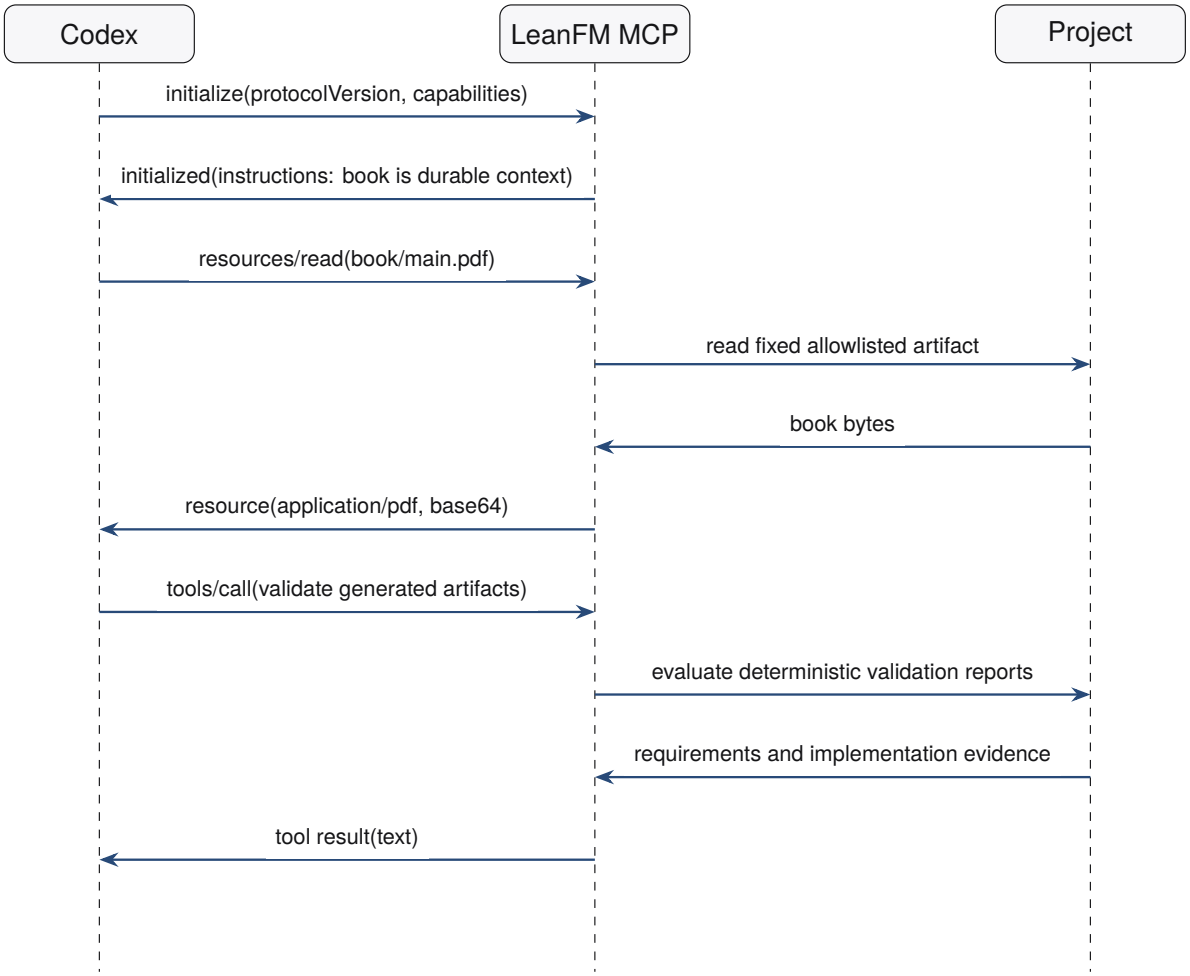


Figure 8.1: An agent obtains durable project context and deterministic evidence through MCP. Source: `LeanFM.MCP`; reducer and provenance: `book/generated/mcp-interaction-provenance.txt`.

tested against an event trace. The `selfhost.mcp.logic` clauses, implemented by `LeanFM.Logic` and exposed by the MCP endpoint, let the agent ask LeanFM before it commits to any of these, instead of finding out in review.

The consultation is a separate task in the self-hosting requirements. Its grammar is one request answered by exactly one of two outcomes:

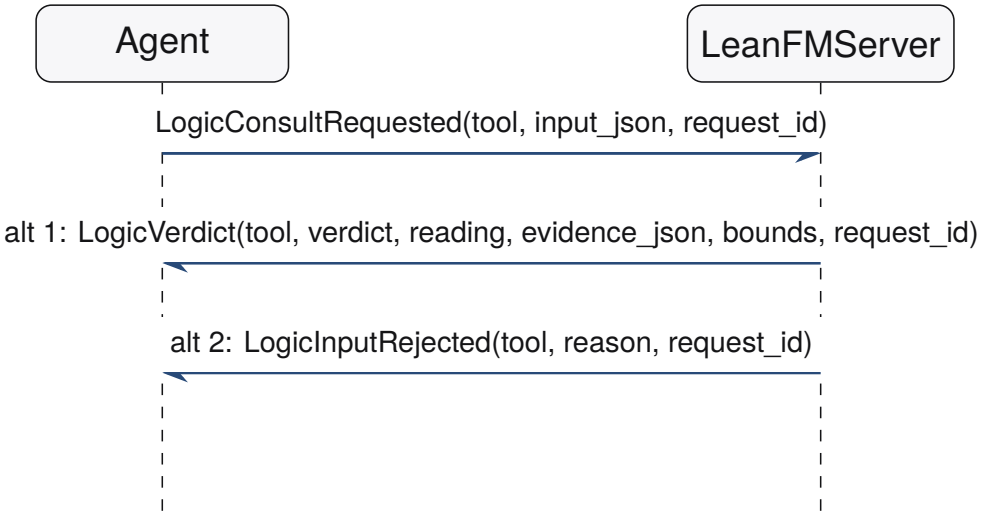


Figure 8.2: An agent consults LeanFM about a formula, trace or requirement. Every consultation ends in a verdict carrying its evidence or a rejection naming the input fault. Source: `logicGrammar` in the self-hosting requirements; provenance: `book/generated/mcp-logic-interaction-provenance.txt`.

8.5.1 The tools

`leanfm://language/logic-reference` A resource stating the JSON formula grammar, the meaning of every operator, the terminal-state convention, and worked examples. An agent reads it once per session.

`explain_formula` Returns the formula read back in English, its negation and duals, whether it is a state or path formula, typing errors, and lints. A vacuous implication, an `EF` where the sentence said "always", or a plain implication where `stronglyImplies` was meant are each reported with the suggested alternative.

`check_ctl` Evaluates a formula against a canonical task or an inline draft task. The atoms are the observable ones: `at(state)`, `after(message)` and `terminal`. The result is one of five verdicts and always carries evidence: a witness path for a satisfied existential, or a counterexample path for a refuted universal or a violated `never`. A draft may also give its transitions symbolic terms and declare what some actors know at the start, and whether an actor sees every channel. Then `knows(actor, term)` is an atom too: a state of the model is the task state together with what each declared actor holds, so two paths that reach one state knowing different things stay apart, and a requirement such as "never (Bob has finished with

Alice and the network knows Bob's nonce)" is checked directly, its counterexample listing what each actor holds at every step.

`check_trace` Accepts event envelopes with their `id`, `prevIds` and `causeIds`. It checks identifiers, cycles, cause ordering, the `src` and `dst` declared for each message, and grammar acceptance per task. When the trace is admissible it returns the generated interaction diagram and causal graph—the agent designs a sequence diagram by writing events, not by drawing.

`classify_requirement` Classifies each atom of a proposed requirement as observable, derived, or implementation-only; says which artifact should hold it; and, for an implementation-only atom, proposes the events that would make it testable.

`knows`
takes event envelopes whose payloads are symbolic terms (atoms, keys, key pairs, pairs, encryption and hashes), an actor and an event, and returns what that actor can derive at that event: every message it sent or received in the event's causal past, taken apart until nothing new appears, each piece with the event that first gave it. With a query term it answers `proved` with the derivation, or `disproved`. Events outside the causal past never count, so a leak that has not reached the actor yet does not make it know anything. The decision is the one in `LeanFM.Session.Knowledge`, proved sound (Section 17.5).

`check_protocol`
takes a security protocol as data: each honest role a short script of choices, sends, receives and claims over message patterns, an attacker with what it knows at the start, and one formula. Every channel belongs to the attacker: what an honest role sends, the attacker learns; what an honest role receives is any message of the expected shape the attacker can build. The state space is generated from that declaration and decided by `check_ctl`, where `claimed` atoms ("Bob has committed to Alice") sit beside `knows` atoms, and the answer lists the messages that crossed the wire on the returned path, in the evidence's own terms: a send names its sender, its addressee and the attacker it went *via*; a delivery names the attacker it came from. Needham–Schroeder declared this way gives back Lowe's attack (Section 17.8). Each role may run several sessions at once, each with its own fresh values, and each session runs once; the answer says so. An optional defender stands in front of named roles, dropping or rewriting what it can see, and seeing inside a ciphertext only with a key it holds (Section 17.8.2).

`check_reassembly`
asks whether a defender scanning a stream for a signature and the host behind it can be made to disagree. The attacker chooses every segment, and each side keeps either the first or the last bytes of an overlap. The answer is a verdict over every segmentation and, for an evasion, the segments that make it (Section 17.8.3).

The modal operators themselves (K_a over indistinguishable histories, O, P) are still reported as `unsupported` inside a CTL formula, so that an agent is told plainly rather than given a guessed answer; `knows` decides the state predicate the security examples need.

8.5.2 Formulas as data

A formula travels as JSON. An agent asking whether every consultation ends would send:

```
{ "task": "mcp_logic_consult",
  "formula": { "AG": { "implies": [
      { "after": "LogicConsultRequested" },
      { "AF": "terminal" } ] } } }
```

The formula is parsed into a closed syntax tree and evaluated by LeanFM. It is never compiled, interpreted as Lean, or passed to a shell. The specification records this as the obligation *Submitted logic never runs as code*; it is tested rather than proved, and its control is that the parser has no constructor that names code.

8.5.3 Five verdicts

`proved` The formula holds on every reachable state within the stated bounds, with a witness where the formula is existential. In the terms of Chapter 1 this is a *finite-model check*, not a Lean proof.

`disproved` A counterexample path is returned.

`unknown` The model is too weak to decide: a derived atom, a predicate text, or a message field value, none of which a task's state machine carries.

`unsupported` The operator is outside the implemented logic; the answer names it.

`resource-limit` The reachable model exceeded its node bound (4096 nodes). The tested obligation *Exhausted search never proves* forbids reporting this as `proved`.

8.5.4 Requirements without observable behavior

A requirement over hidden state is not wrong, but it is incomplete. The classifier makes the incompleteness explicit instead of letting it pass. An *observable* atom is decided by the event trace alone. A *derived* atom is computed from the trace by a declared reducer—a balance, a count, the set of values an actor could have read. An *implementation-only* atom refers to state that no event reveals. The tested obligation *Hidden predicates never pass as observable* keeps the third kind out of conformance tests; the classifier then proposes events, such as a `CacheAudited` or `KeyRotated` message with the fields that expose the predicate, that would move the atom into one of the first two kinds. The requirement is then rewritten over those events, and `check_ctl` can decide it.

8.5.5 The design loop

Used together, the tools give an agent a loop that converges on a specification a reviewer can accept:

1. Classify each sentence of the requirement; turn implementation-only atoms into proposed events.
2. Write example traces using those events and check them with `check_trace`; read the returned interaction diagram.
3. Formalize each positive and negative property, read it back with `explain_formula`, and decide it with `check_ctl`.

4. Treat every counterexample as either a defect in the grammar or a defect in the formula, and say which before revising.

A `proved` verdict is evidence for the stakeholder, who still decides what to commit. Malformed or oversized input—an unknown operator, a bare `G` without its path quantifier, a misspelled state or message, or a formula beyond 256 nodes—is rejected with the reason rather than answered, so a typo is never silently read as *false*.

8.5.6 Real answers

The following requests and responses are produced by running `LeanFM.Logic` while the book is built, so they cannot drift from the implementation. The bounds block is omitted for space.

A never-claim that fails (`check_ctl`)

```
{"task": "mcp_logic_consult",
 "formula": {"never": {"after": "LogicInputRejected"}}}
```

returns

```
{"verdict": "disproved",
 "tool": "check_ctl",
 "task": "mcp_logic_consult",
 "reading":
 "on every path, never LogicInputRejected was just delivered",
 "reachableNodes": 4,
 "mode": "never",
 "formula": {"never": {"after": "LogicInputRejected"}},
 "evidence":
 {"path":
  [{"state": "drafting", "message": null},
   {"state": "interpreting", "message": "LogicConsultRequested"},
   {"state": "rejected", "message": "LogicInputRejected"}],
  "kind": "counterexample"},
 "deadlocks": []}
```

A requirement over hidden state (`classify_requirement`)

```
{"task": "mcp_logic_consult", "predicates": ["the cache is consistent"]}
```

returns

```
{"verdict": "classified",
 "tool": "classify_requirement",
 "reading":
 "some atoms are implementation-only; restate them over proposed events before the
    requirements gate",
 "atoms":
 [{"restated":
   {"AG":
    {"implies":
     [{"after": "CacheConsistentObserved"},
      {"field":
```

```
      {"value": "true",
       "name": "holds",
       "message": "CacheConsistentObserved"}]}},
  "reason":
  "no declared message or state reveals this; no event trace can test it",
  "proposedEvent":
  {"to": "an observer that can check it",
   "message": "CacheConsistentObserved",
   "from": "the actor that owns this state",
   "fields": ["subject", "holds", "observed_at"]},
  "class": "implementation-only",
  "atom": "the cache is consistent",
  "advice":
  "move it to Implementation.lean as an internal invariant, or restate it over an
   event that exposes it"}]}
```


Designing a sequence diagram by writing events (`check_trace`)


```
{"events":
 [{"task": "mcp_logic_consult",
   "src": "Agent",
   "message": "LogicConsultRequested",
   "id": "e1",
   "dst": "LeanFMServer"},
  {"task": "mcp_logic_consult",
   "src": "LeanFMServer",
   "prevIds": ["e1"],
   "message": "LogicVerdict",
   "id": "e2",
   "dst": "Agent",
   "causeIds": ["e1"]}]}
```


returns


```
{"verdict": "admissible",
 "tool": "check_trace",
 "reading": "2 events over 1 task(s); 0 concurrent pair(s)",
 "issues": [],
 "concurrentCount": 0,
 "concurrent": [],
 "acceptance":
 [{"task": "mcp_logic_consult",
   "states": ["answered"],
   "reachedTerminal": true,
   "accepted": true}]}
```


with the generated Mermaid sequence diagram

```
sequenceDiagram
    participant Agent
    participant LeanFMServer
    Agent->>LeanFMServer: e1 LogicConsultRequested
    LeanFMServer->>Agent: e2 LogicVerdict
```

Who can derive what (`knows`)

```
{"query": {"atom": "nb"},
 "initial": [{"sk": "I"}],
 "events":
 [{"term":
   {"enc":
    {"key": {"pk": "A"},
     "body": {"tuple": [{"atom": "na"}, {"atom": "nb"}]}}},
   "src": "Bob",
   "id": "e1",
   "dst": "Ivo"},
  {"term": {"enc": {"key": {"pk": "I"}, "body": {"atom": "nb"}}},
   "src": "Alice",
   "id": "e2",
   "dst": "Ivo",
   "causeIds": ["e1"]}],
 "at": "e2",
 "actor": "Ivo"}
```

returns

```
{"visible": ["e1", "e2"],
 "verdict": "proved",
 "tool": "knows",
 "reading": "Ivo can derive nb at e2",
 "query": "nb",
 "holds":
 [{"term": "sk(I)", "how": "known at the start", "from": "initial"},
  {"term": "{na, nb}pk(A)",
   "how": "delivered in e1 (Bob to Ivo)",
   "from": "e1"},
  {"term": "{nb}pk(I)",
   "how": "delivered in e2 (Alice to Ivo)",
   "from": "e2"},
  {"term": "nb", "how": "opened {nb}pk(I) with sk(I)", "from": "e2"}],
 "evidence":
 {"kind": "derivation",
  "derivation":
  {"term": "nb", "how": "opened {nb}pk(I) with sk(I)", "held": "e2"}},
 "causalPast": ["e1", "e2"],
 "at": "e2",
 "actor": "Ivo"}
```

The counterexample is the shortest path to the forbidden event. The classifier refuses to call the cache predicate observable and proposes the event that would make it so. The trace is accepted by the task's transitions, and the sequence diagram is a projection of the events the agent wrote.

8.6 A tour of the LeanFM chat

Select a project in the left rail, then **Chat**. Its mounted workspace contains Project memory, Chat, Book, Grammar, Tasks and Source. Switching project or phase changes visibility while preserving

drafts, selections, scroll positions, queues, timers, previews and fetched results. Active work continues while hidden. Each message has a timestamp; the header identifies the billed provider, model and non-secret key fingerprint, with response token usage shown per turn.

Chat accepts ordinary language, images, rough pseudocode and protocol questions before Book, Grammar, Tasks or Source are settled. A response may revise the book, ask a focused blocking question, derive a requirements view or create a scratch explanation. For each turn the model receives the selected project's complete editable book and canonical requirements and implementation specifications, then a bounded recent transcript. Neither LeanFM's own book nor another project supplies fallback context. This is one contextual conversation, without a hidden mode selector.

The ordinary outcome is a decision and a checked artifact, with no clarification question. The agent reads prior answers and durable choices, selects reasonable reversible defaults within the recorded requirements and authorization, and records those decisions. It briefly lists only choices the stakeholder is likely to challenge. A question must name a consequential unresolved effect on scope, observable behavior, compatibility, security/privacy, irreversible data or material spending, and explain why a recorded decision or authorized default cannot resolve it. It never asks for an approval: review happens in git.

A readiness review can report ready without inventing questions. Compiler syntax, formatting and routine check failures use the bounded diagnostic and recovery path below. They do not consume a stakeholder clarification turn. The agent retains the exit code, narrows the correction and verifies progress; an unchanged failure class cannot authorize another paid retry.

8.6.1 Progress, cancellation and the Operational trace

Every phase shows an **Agent running** banner with the active prompt's captured model, incrementing elapsed time, latest real stage and cooperative **Cancel** control. Status, Project memory, Source and the Operational trace are polled every two seconds; transient failures retain the last successful value and retry. The changing artifacts remain inspectable.

The Operational trace is appended before the next agent step. It records each bounded workspace action as started, completed, failed or rejected. A proposed write names its normalized project-relative destination, content kind and character count before validation, so a path failure says what LeanFM was trying to write. Failures are highlighted in the collapsed banner summary and the accumulated trace may be expanded before the turn completes. Cancellation is checked at provider and workspace-tool boundaries: no new mutation begins after the request is observed, while already completed durable edits remain truthful project state.

Prompts typed during a turn enter a visible removable project-local FIFO and execute exactly once in order. Each queued prompt captures the model visible at the time it is enqueued. In the composer, focus order is prompt editor, **Send**, then model selector. Thus **Tab**, **Enter** commits a typed prompt with the already visible model rather than opening the selector.

Structured clarification is also observable: a returned question logs its stable ID, and a composer submission while that question is pending logs that it bypassed the answer form. Both events appear in the turn trace and the project diagnostic journal. They retain the question ID and bounded text lengths, never the operator's prompt or answer text.

8.6.2 A private fault barrier and actionable errors

The visible trace is for stakeholder decisions; a separate bounded private engineering journal supports diagnosis across navigation, server restarts and discarded chat history. Each project stores it at `logs/runtime-diagnostics.jsonl`. It contains sanitized stage transitions, tool destinations, compiler diagnostics, failed required artifacts, warning results, terminal provider failures, authorized project HTTP errors and correlated unhandled server exceptions. A later agent request receives the twenty newest unresolved entries before choosing a recovery strategy. Verified matching writes and book builds remove superseded failure clusters and leave one compact resolution, so the journal remains memory rather than an ever-growing graveyard.

The React client is inside the same fault boundary. A React error boundary, global JavaScript exception listener, unhandled-promise listener, API wrapper, and explicit Mermaid and LaTeX renderer catches send an authenticated report back to the server. A report names the operation, error type and message, sanitized JavaScript stack, component stack when available, page, time and active project. The reporting request uses a recursion guard, bounded fields and at most twenty reports per browser minute. Project authorization is checked again on the server. Prompts, form and workspace-write bodies, credentials, cookies, arbitrary provider responses, URL queries and cross-project data are never diagnostic payloads.

Every error shown to a person still has a separate obligation: state what happened, recommend a concrete next action, provide a stable code and offer a bounded diagnostic safe to paste to Codex. An unknown proxy response such as “527 Something went wrong” is insufficient. The private journal normally means the person need not paste anything: LeanFM or Codex can inspect the project root, repair the cause and continue the original task.

Provider credit/spend/quota rejections, rate limits, incomplete output and local session safety limits receive distinct diagnostics. Response length establishes no credit balance. An incomplete Responses result retains reported usage but cannot execute returned tools or update the book. A malformed reply envelope never exposes its hidden book source as prose. One complete, unambiguous JSON object surrounded by prose or fences, and illegal JSON escapes before LaTeX math delimiters, can be normalized locally before ordinary validation and the book build. Arrays, multiple proposed objects and truncated objects are not salvaged. Other format failures retain the parser offset, character count, leading-token category and complete-object flag, not arbitrary response text. None triggers a paid retry.

Every provider round records its input, cached-input, output and cumulative usage, request and answer character counts, outcome and tool count before the next paid call. The session budget is checked between tool rounds and before bounded recovery or repair, not just when a chat turn begins. Reported usage is charged once, including failed validation. This is an observed-token brake, not a hard monetary cap: an in-flight request can exceed the remaining budget, and an unanswered request’s usage is unknown. Provider spending controls remain necessary. Saved API keys remain password-encrypted on disk across server restarts; only unlocked sessions disappear. Blank login key fields preserve those saved keys and never display them.

Repeated failure is bounded at the point of expense. Within an active turn, a second tool call with the same name and exact arguments after an error is not executed; it terminates that strategy before a third paid attempt. Provider timeouts are never automatically retried because a timed-out request may have consumed tokens or completed an earlier mutation. The agent instead reads the diagnostic, changes strategy, exhausts safe reversible alternatives, validates the result and resumes the requested task. Only missing authority, a destructive choice or an unavailable dependency should

return control to the user.

8.6.3 Requirements properties and implementation provenance

The Tasks page separates four views of one selected task: interaction productions, residual states, temporal properties and guarded constraints. A property is an observable requirement originating in `Requirements.lean`; it is not silently invented by `Implementation.lean`. The UI renders its actual CTL wrapper rather than labeling prose as CTL: `always` becomes $AG\,p$, `eventually` becomes $AF\,p$, `never` becomes $AG\,\neg p$, and `preparedFor` becomes $EF\,p$. A `where_` trace predicate is displayed with its stable identifier under the invariant $AG\,p$. Each card identifies its requirements origin.

The implementation mapping follows Section 19.4: its answers realize the specified behavior and cite their requirements. It cannot silently introduce a property or change the wire contract.

The assistant owns routine formal-language and implementation decisions. The interrogation checklist is an internal completeness check, not a questionnaire for the stakeholder. Choose, record and validate reversible defaults without interrupting the user. A question needs a major consequence that is hard to reverse and cannot be resolved within existing authorization. There is no approval step: the stakeholder reviews the resulting diff and commits it. Successful intermediate checks are progress toward the original goal. The assistant selects the next reasonable reversible step and continues through client-driven batches without asking whether to continue. The user can Cancel. Completion compares real artifacts and behavior with the accepted goal, including required negative behavior and conformance evidence; source existence or passing self-authored tests alone cannot establish that result. ChatUI obtains the language references and implementation guidance through LeanFM's read-only MCP dispatcher. The selected project's sandboxed compiler still checks its own canonical files; the host's MCP self-validation cannot substitute for a foreign project's evidence. The full-book MCP resource uses iterative base64 encoding so a large central PDF cannot exhaust the server's call stack.

Provider-supported web search can resolve public implementation questions without another stakeholder turn. Source links survive client batch boundaries and remain clickable. External pages supply evidence, never authority to change project boundaries, acceptance, secrets or spend controls. Adding a provider key through the signed-in account editor preserves the current login, usage, expiry and existing provider work. Replacing an existing secret retains the session-revocation policy. Blank key fields preserve their stored values; account passwords and keys are never returned to the browser.

Application delivery ends with a working `source/Makefile` launch target, not just a set of files. `make run` uses a bounded loopback port separate from LeanFM. The project retains `spec/runtime.json` and `spec/ProjectMemory.md`: the original goal, reversible defaults, known failure remedies, remaining gaps and PR base/review context survive erased chat. The `run` workspace check launches the actual command in a private network and checks readiness and its application page. It has no host credentials or host-network access, writes only project cache, and stops only its own process group. The host records protected source-bound evidence; changed source makes that evidence stale. A failed launch returns its concrete diagnostic to the same client-driven task. Passing the launch check establishes these bounded observations, not complete semantic refinement of the accepted specification.

An existing username and its correct password can update retained API keys. Login and account settings also offer an optional password change. The current password must verify and both new-password fields must match. A single atomic credential envelope contains the new verifier and

re-encrypted provider bundle, so an interrupted write cannot split the two. Old sessions are revoked; a credential revision checked at authentication and tool boundaries also rejects stale sessions in another service process sharing the vault. Additive provider saves preserve that revision. A legacy verifier file remains a compatibility mirror; it cannot override the envelope. The `?keys=1` link opens the secure account editor and submits no credential request by itself.

8.6.4 How a chat turn becomes a durable edit

Guided chat selects the artifact layer required by the request. Specification work uses project-confined workspace tools to update the canonical Lean and protobuf files; it is not forced to rewrite the whole book. The `leanfm_reference` tool returns the same live language and workflow references exposed through MCP. The `leanfm_check` tool compiles the project's named typed requirements and optional implementation mapping, validates their structure and protobuf declarations, and regenerates `spec/tasks.json` from that same requirements value. Compiler execution has no network, credentials or writable project source: only project-local scratch is writable. Failed validation preserves the last valid views. The generated catalog retains canonical input digests and reducer provenance; neither a check nor a PDF build substitutes for the stakeholder's review. A turn changing canonical specifications cannot report SUCCESS without a successful check of its final bytes.

The requirements reference includes a complete compilable request/response example with matching protobuf, task states, grammar, processes, properties, negative obligations and generated-view provenance. A reduced scope still needs these collections. In particular, `markdown` contains typed `RequirementMarkdown` records, not plain strings. The implementation reference likewise includes a compiled mapping for that example. `MessageCodegen.wireType` is a string naming the generated protobuf type. `ImplementationSpec.channels` is one `ChannelCodegen` record with explicit send, receive, drain and close semantics; capacities remain in the requirements' actor resource contracts. If the model tries to finish after an unchecked specification edit, the server returns the concrete compiler or structural diagnostic within the same native provider conversation. The model can correct the files and recheck without a second user message. This continuation retains the original cancellation, session, usage accounting and request budget. An unchanged candidate and diagnostic stop further paid work; an unavailable compiler stops immediately. A repaired current check can report success while retaining earlier failures in the operational trace.

When the turn revises the book, the provider's structured reply includes the complete proposed `book/main.tex`. The server enforces root and size boundaries, writes atomically and builds in the isolated book worker. Only a successful build commits the revision and sets `book_updated=true`. If compilation fails, the first bounded compiler diagnostic becomes visible before repair begins and the exact previous source is restored after every candidate. At most three repair requests may run, each with a short deadline. Each request receives only the compiler diagnostic and a numbered seventeen-line source excerpt, asks for one replacement line with at most 2000 output tokens, and is rejected if it targets another line or makes no change. A successful candidate is rebuilt before commit; exhaustion produces WARN and leaves the last valid book authoritative. Deterministic syntax mistakes such as literal ampersands and underscores should be corrected directly when their intended prose meaning is unambiguous rather than rediscovered through repeated paid model calls.

The same rule becomes stricter at the implementation boundary. Once the specification checks and the security critique is done, an implementation request writes real buildable files, generated bindings, and tests beneath the selected project's `source/` directory by using the project-confined workspace tools. The Source phase reads those files directly. A provider response that says

"implemented," or pastes a Go listing into chat, cannot be SUCCESS while `source/` is empty. When a material implementation decision is missing, the valid alternative is a focused selectable question naming that blocker—not an invented implementation and not another generic plan.

This instantiates Chapter 14: substantive turns can create validated artifacts, while malformed, oversized, cross-project, unbuildable or merely narrated artifacts cannot be reported as committed. The deterministic HTTP test checks a compiled book update, an actual `source/main.go` write and rejection of an implementation claim with empty Source.

8.6.5 Example: ask to see MCP move

The following is a useful prompt because it asks for both an explanation and a specific interaction technique. It is reproduced as entered; spelling does not need to be polished before the model can act.

```
i want to understand what MCP looks like.  make me an animation that i
can step forwrd or back in, showing messages passing back and forth in
dynamic interaction diagrams.  visualized right here in the chat ui
```

Here *MCP* means the Model Context Protocol, not MQTT and not a generic message broker. A faithful first tour uses a host or MCP client and an MCP server as lifelines. When the selected example invokes a real downstream tool, that tool may be a third lifeline; otherwise it is omitted. A representative trace is: the client sends JSON-RPC `initialize` with its version and capabilities; the server replies with its supported version, capabilities and identity; the client sends `notifications/initialized`, discovers tools with `tools/list`, then sends `tools/call` with a tool name and arguments and receives its result or error correlated by JSON-RPC ID.

This is the same kind of interaction shown durably in figure 8.1, but projected into a small review instrument. The inline view should show the participating lifelines, one selected message in motion, its method or result label, and a detail panel containing the request ID, direct cause, arguments or result summary. **Forward** completes the next event; **Back** restores the immediately preceding prefix; **Play/Pause** advances or suspends automatic stepping; **Restart** returns to the empty prefix; and a scrubber may select any event boundary directly. Keyboard left and right arrows are convenient equivalents when focus is inside the animation.

Stepping backward does not send an inverse protocol message. It changes which prefix of the same immutable example trace is visible. At step k, the canvas shows exactly events $e_1, \ldots, e_k$; advancing exposes e_{k+1} and stepping back hides e_k. Request and response remain correlated by their JSON-RPC identifier, while LeanFM's richer event representation can separately retain a direct causal pointer and predecessor links. Thus the animation may explain causality without falsely claiming that temporal adjacency alone pairs a response with a request.

8.6.6 What should appear in the response

The chat response should contain a short explanation followed by the working animation itself. Its HTML and JavaScript are transport material, not the answer: they must not fill the message as pages of source code. LeanFM expects an actual `leanfm-canvas` fenced block and removes that block from the prose before placing its canvas and self-contained script in an isolated frame. The frame has no network access, cannot navigate the parent, and may load no external library or image. Stop and Restart controls outside the frame allow the stakeholder to halt an expensive or confusing

animation. Unless the user specifically requests a text color, canvas labels, legends, and annotations use a contrasting foreground: light text on dark drawn backgrounds and dark text on light drawn backgrounds. Choose from the canvas drawing's actual background rather than the surrounding page theme.

There is one bounded repair for a common provider formatting error. If the model literally writes "fenced leanfm-canvas" and immediately follows it with exactly one canvas element and one script element, the client recovers that pair into the same sandbox instead of displaying it as noise. This exception does not turn chat into a general HTML renderer: unmarked or arbitrary HTML is escaped and remains inert text. A response that shows the raw program instead of an animation is a rendering failure, not SUCCESS. Resend preserves the original user request while allowing the corrected renderer and prompt contract to try the turn again.

The animation is initially a scratch explanation. It answers "what would this interaction look like?" quickly, but it is not yet a generated view of the requirements. If the discussion changes the project design, retain the originating prompt, bind the selected messages to a named scenario, update the typed grammar and regenerate the interaction diagram from that specification. Only that provenance-bearing derived view can serve as evidence when the requirements are reviewed. If stakeholders instead want to share the exact motion, promote the chosen canvas revision to a project-owned GIF or MP4 with fixed frame timing and serve that media inline.

Chat's **Attach image** control accepts PNG, JPEG, GIF and static SVG files, shows a removable preview, and retains the selected images with the queued prompt and later Resend. The original and a digest manifest remain below the owning project's `artifacts/attachments/` directory. The provider receives normalized PNG pixels through its native visual-input protocol; a GIF is explicitly represented by its first frame. The current implementation bounds each original to 2 MiB and each source to 16 megapixels, and sends at most four current or recently referenced images per turn. External SVG resources and executable content are rejected. These controls realize the existing multimodal requirement; their negative-test mappings are recorded in `evidence/chat-images/security.md`.

Task interaction renderers must receive the owning specification along with the task. Looking up a project's task in LeanFM's host ChatUI grammar yields a misleading missing-grammar placeholder even when the project grammar is valid. The serializer and renderer now share that explicit specification, and regression evidence includes a foreign task whose ID collides with ChatUI.

Every still image displayed in Chat retains a reference that can leave the screen. An adjacent keyboard-accessible **Copy reference** control copies a project image as its stable project-relative artifact path. For a public-web image it copies Markdown containing the displayed HTTPS image URL followed by the visible source-page URL. It never copies image bytes, a data or ephemeral object URL, the host filesystem path, credentials or unrelated response text. This makes "use the image above" a durable artifact reference that can be pasted into a later Chat turn, the book or Codex even after conversation state has been discarded. Clipboard failures cross the same client fault barrier as other JavaScript errors.

8.6.7 Screenshot-first documentation

Chapter 9 provides the detailed user walkthrough and named captures of the production React DOM. Its local deterministic provider and isolated scenario runner stop at declared boundaries, including prompt submission, response rendering, the accepted specification views, Source, and a verified persistent app launch. Each caption identifies the control state and the claim demonstrated.

A screenshot is useful evidence only when it is reproducible. Its manifest retains the project

and requirements revision, scenario and visible event IDs, viewport, font scale, theme, browser version, capture time, reducer and content digest. The image receives meaningful PDF alternative text. Headless capture runs with a disposable account and deterministic data, without a provider key, external network or arbitrary target URL. A missing or stale required capture fails the book build instead of silently leaving a blank figure.

The PNGs, accessibility snapshots, keyboard traces and manifest are retained in `book/assets/ui-tour/`. `make ui-snapshots` reproduces them, and the book build rejects missing, altered or stale required evidence. Formal definitions and security boundaries remain in text where an image cannot express them.

8.7 Local credentials and visible billing identity

The LeanFM login is not a provider login. One account password unlocks the user's OpenAI and Anthropic keys together. The `API keys` form adds or replaces either key; blank inputs preserve stored keys. Provider labels and keys are authenticated together inside a password-encrypted bundle, never returned to the browser or stored in plaintext. Existing single-key accounts migrate when keys change. Updates revoke older sessions; an already-running provider request may finish, but its returned tools cannot run afterward. An optional local GoServer example, when present under `projects/`, appears under one canonical `GoServer` identity.

Account records are runtime state, not source. By default the salted password verifier and password-encrypted provider keys live in the private vault `.leanfm/accounts/`, which Git ignores; sessions are held by the server. Example projects contain books, specifications, source, tests, and deliberately selected media, but never user logins or credentials.

The model selector combines models discovered for the unlocked providers. Its server-owned catalog binds each selection to one provider and the exact account-owned key; an unknown model or missing key fails without a paid call or a fallback to another provider. OpenAI Responses and Anthropic Messages use the same confined workspace tools, retry brakes, and book recovery. The localhost HTTP interface waits for the complete byte-counted request before dispatch. Its framing contract accepts UTF-8 with one Content-Length, bounds headers to 32 KiB and bodies to 8 MiB, and expires incomplete requests after 15 seconds. Ambiguous lengths, transfer encodings, and pipelined extra bytes are rejected without provider calls or mutations.

An authenticated header continuously identifies the provider, model, credential reference, and a one-way fingerprint of the exact key used for the next request. It also shows provider-reported session tokens used and tokens remaining in a finite local safety budget; exhaustion stops another request. That cross-provider token count is not a credit or dollar balance, because a model response does not expose such a balance. The UI says so explicitly. A provider project spend cap remains the authoritative protection against monetary overrun, while per-response token counts remain attached to the request that incurred them.

8.8 Cursor as an agent client

A Cursor subscription connects to LeanFM as an agent client, not as a model-provider API key. A project may commit `.cursor/mcp.json` with the local URL `http://127.0.0.1:8080/mcp`. Cursor Agent opens the project root, reads `AGENTS.md`, and discovers LeanFM's book, canonical artifacts,

language references, prompts and validation tools through MCP. Cursor continues to own its subscription authentication and billing; LeanFM receives no Cursor login secret. The current MCP surface remains read-only, so Cursor edits files using its ordinary project tools; the user reviews and commits them with git.

For the inverse arrangement—the LeanFM browser driving a Cursor-backed conversation—Cursor CLI exposes the Agent Client Protocol over standard input and output. A `CursorACP` adapter can initialize and authenticate the local agent, create or load one session for each LeanFM project conversation, submit prompts, stream updates, answer explicit permission requests and cancel work. This is an implementation direction, not a claim about the running server. The adapter must isolate user, project, conversation, process and request identity; bound its queues; preserve tool and permission evidence; and terminate the process only after its queue is closed and drained. Cursor's authentication token is never imported into LeanFM's provider-key store.

8.9 One project, one directory

8.9.1 Shared workspace and fresh start

The requirements and separate implementation mapping for the fresh start were reviewed on October 23, 2026. Ordinary projects live directly below `projects/<readable-project-name>` and have no user owner. The stakeholder has confirmed that the workspace opens before login; login is required for LLM calls and API-key management. Shared project contents include their books, specifications, source, artifacts and conversations. Password verifiers and encrypted API keys belong in the private application vault `.leanfm/accounts/`; they never become project files or model input.

The typed component is `LeanFM/LLMGenerated/SharedProjects.lean`, exported by the canonical requirements and composed into the self-hosting product. The separate realization is in the canonical `LeanFM/LLMGenerated/Implementation.lean`. Running `lake env lean -run ReviewSharedProjects.lean` validates both candidates and derives the review, task views and provenance below `reports/shared-projects/`, where the historical review record `reports/shared-projects/acceptance.json` also remains. The explicitly requested reset discarded five ordinary projects and the empty legacy scaffold, verified the preserved credential bytes and permissions, and removed `./data`. It preserved LeanFM's own source, book and specifications. The reset report is `reports/shared-projects/reset-result.json`.

`LEANFM_WORKSPACE_ROOT` selects the exclusive workspace directory and defaults to the server's current directory. Ordinary roots are discovered from their own validated manifests below `projects/`, independently of any credential account. A project contains its own book, canonical specifications, source, tests, media, conversation, evidence and caches. Deleting its root removes that project's state. The credential vault is excluded from every project capability. Foreign browser origins and unapproved Host values cannot mutate the loopback workspace; a conflicting project writer is rejected before mutation or provider work. Queued prompts retain their initiating credential identity and are held if that identity changes.

Directory labels come from project names, such as `projects/GoServer`, rather than opaque hashes. Spaces and punctuation become safe separators; collisions receive a short numeric suffix. Atomic directory reservation never adopts or overwrites an occupied root. The manifest's stable project identifier remains independent of the directory label, so relocating an existing root preserves its ChatUI URLs, conversation and checked revisions. Discovery rejects duplicate identities instead of choosing an arbitrary root.

Project memory shows, for the requirements and the implementation mapping, whether each passes its canonical check and whether it is committed in git. There is no approval control: review and acceptance are git commits, and LeanFM blocks no edit, source write or launch on an approval. Model tools cannot write checker evidence. The current status is supplied to subsequent model turns, and committed decisions survive a cleared chat. Project-authored checks and media generators execute without network or host credentials and have only bounded project-local cache or scratch write access.

8.9.2 Credential accounts and project roots

The portable root and containment rules are in Chapter 19; this section records the hosted-runtime mapping. `LeanFM.ChatUI.Store.createProject` creates shared roots under `projects/<readable-project-name` writes a runtime `manifest.json`, and provisions the book, `spec/`, source, tests, artifacts, renders, events, reports, logs, cache, and conversations beneath that root. The manifest records all canonical project-relative paths. Portable `project.json` roots are also discovered when their identity and configured paths validate; they are ordinary editable projects. The removed examples are not inputs to provisioning or validation.

8.10 What the Lean artifact must settle

The Lean artifacts are the semantic source for actors, framed messages, process projections, grammars, properties, proof obligations, and desired outputs. Canonical definitions are in Chapter 2 (event fields, correlation, predecessor order, causation, and elapsed time), Chapter 5 (legal conversations, actor projections, and bounded queue semantics), and Chapter 7 (choices, dwell, rates, scalability, reliability, and infrastructure cost).

One cross-cutting obligation deserves emphasis: message-count capacity alone does not bound memory when payload sizes vary. Derive queued bytes from encoded envelopes and declare a bounded footprint for active task state. A representative requirement is

$$\mathrm{AG}\big(\mathrm{queuedBytes}(a) + \mathrm{inFlightBytes}(a) \leq \mathrm{budget}(a)\big).$$

Here AG means every position on every permitted future, and the repeated a fixes the actor being measured. This is a modeling obligation, not a claim that the current validator proves the heap usage of an arbitrary implementation.

8.11 Interrogate, characterize, synthesize, replay

8.11.1 Characterization readiness

The bootstrap contract defines the opening question and durable project-root handoff (Chapter 19). Characterization tests whether desired claims are identifiable from evidence: scenario and task boundaries, terminal-to-start links, actor populations, clocks and units, observation windows, resource bounds, failure evidence, and probability provenance. Missing inputs remain indeterminate, not invented.

One event stream should produce the standard characterization. Its canonical inventory is `outOfBoxArtifacts` in `LeanFM/Characterization.lean`; `baselineDerivedProperties` records

each property's required fields, reducer, unit, visualization and boundary. A domain retains, replaces, extends or marks each applicable property indeterminate.

Specified distributions may drive predictions or generation, but are never presented as measurements; event-stream estimators are detailed in Chapter 2.

For generative similarity, reducers map the source stream to a named metric vector, with each histogram bin represented as an exact ratio. A candidate is serialized, decoded and reduced identically; it passes only when every required metric is within its declared relative tolerance. "Similar" therefore means equivalent under this finite characterization, not visually or textually similar. The grammar, invariants and byte language still constrain legal traces.

8.12 The protobuf artifact is part of the requirement

The parallel protobuf schema answers a question that a state diagram cannot: what bytes can cross a component boundary? Atomic payload messages and a `RequirementEnvelope.oneof` discriminator connect a grammar terminal to its wire representation. Validation checks, among other things, that field tags are nonzero and unique and that framing is explicit.

This pairing exposes useful disagreements. A Lean event may require a `priorIds` list while the wire schema provides only one predecessor. A latency property may depend on a clock that is never transmitted or recorded. A memory budget may be stated in bytes while an unbounded protobuf field remains. These are requirements defects, not serialization trivia.

The exact wire size is also an input to resource analysis:

```
queuedBytes actor observation :=
  (observation.inbound actor).sum encodedSize
  + (observation.outbound actor).sum encodedSize

memoryPressure actor observation :=
  queuedBytes actor observation
  + inFlightFootprint actor observation.activeWork
```

The first term can be computed from concrete envelopes. The second requires a declared abstraction of implementation state and must remain visibly labeled as such.

8.13 A ladder of checks

No single green check answers every kind of requirement. The workbench should present layered evidence: Lean/protobuf check well-formed syntax; structural validation checks actors, endpoints, terminals, framing, process coverage and proof obligations; grammar/state-machine checks cover ordering and temporal checks cover safety, reachability and bounded liveness; probability/dwell reducers predict outcomes and event-stream reducers estimate them; queue and byte-budget checks model capacity, backpressure, concurrency and memory; generated diagrams, state machines, plots and reports support review.

Use the evidence labels defined in Chapter 1. Predictions are not measurements, incomplete windows censor tasks, frequencies do not prove specified probabilities, and bounded traces do not prove unbounded liveness. Reports label each result with one of those evidence kinds, such as finite-model check, test, generated analysis or Lean proof, and say when a number is assumed.

8.14 How to argue well

The most useful prompts attack boundaries rather than ask for polish:

- "Which event and grammar branch make this claim true?"
- "With two concurrent sessions and tasks, what blocks on a full queue and which events correlate?"
- "Which random choice belongs to the user, and how does it change the downstream completion and latency distributions?"
- "Derive the plot from the event stream, including in-flight tasks."
- "Show the corresponding protobuf payload and its encoded-size bound."
- "Which evidence label applies: Lean proof, finite-model check, test, generated analysis, or assumption?"

Use the project-root, derived-view, event-boundary and completion contracts in Chapters 19, 3, 2 and 14. Conversation discovers the specification; durable artifacts retain it.

8.15 Client-driven bounded work

The resumable-work component is recorded in `LeanFM/LLMGenerated/ResumableWork.lean`; its separate implementation mapping is beside it, and historical review records are in `reports/resumable-work/`. One user task crosses bounded provider batches through server-issued project checkpoints. The client continues the same task, while later prompts wait and Cancel stays available. A batch boundary is progress, never a total step limit. Checkpoint consumption and cached receipts prevent replay; caller, model, credential, token accounting and failure history survive the boundary. Closing the client starts no further batch. Ambiguous provider failures pause for reconciliation rather than an automatic paid retry. Oversized provider context restarts from durable project files and verified tool receipts. The generated interaction and state views retain their prompt, reducer and boundary in the typed component. Tests use local provider fixtures, the real compiler and production Chrome; they do not prove generated applications correct.

8.15.1 Persistent application launch

The October 25 request is to launch the application being built and post a working link. The current `run` check starts a private smoke test and stops it after observing readiness; its receipt is not a persistent preview. `LeanFM/LLMGenerated/ApplicationLaunch.lean` defines separate start, live-status and stop tasks, exposed by the canonical requirements as `applicationLaunchProposal`. The selected project should show Start/Restart, Stop and Open app, and a requested chat launch should return the verified browser-reachable link. Only a ready owned instance at the current source revision may be described as running. Generated preview code must not inherit the engineering UI's credentials or origin authority. The separate implementation mapping realizes runtime and preview isolation, finite admission, process ownership and cleanup. Review records, negative controls, and

native browser and protobuf validation evidence live in `reports/application-launch-review/`. Chapter 9 shows the actual controls and served preview.

Every generated HTTP application exposes the same `source/Makefile` contract: `build`, `run`, `stop`, `test`, `check`, `clean` and `serve`. The `run` target stays in the foreground for supervision; `serve` aliases it. The `stop` target may stop only its owned application. LeanFM invokes these named targets uniformly and reports missing targets explicitly; it never substitutes an arbitrary per-project shell command or claims an unsupported check passed.

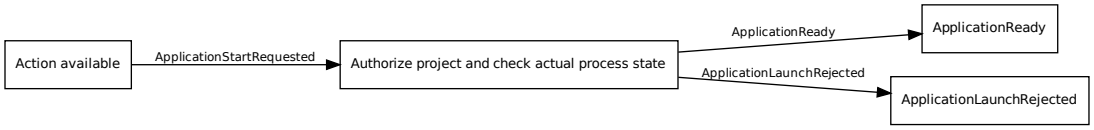


Figure 8.3: Persistent-launch request states generated from the accepted application-launch specification.

Chapter 9

Using the LeanFM Chat UI

LeanFM's Chat UI takes a project from a problem statement to a maintained book, checked requirements, a separately reviewed implementation mapping, source files, and a running application. Work in the selected project's Chat pane; use the other panes to inspect what the conversation actually produced. A reply, a diagram, and a successful PDF build each answer different questions. There is no approval button: you review what LeanFM produced and commit it with git, exactly as you would any other change.

This walkthrough uses the production interface with a disposable GoServer project and a local test provider. The project contains real checked specifications and runnable code. The displayed model name and token readings are fixture data, not provider recommendations, prices, or performance results. The screenshots contain no live account credentials or private conversations. Their keyboard traces, accessibility snapshots, input revisions, and capture boundaries are retained beside the images in `book/assets/ui-tour/`.

9.1 Open the service and unlock your account

Start LeanFM with `make serve` from its repository root, then open `http://127.0.0.1:8080/`. Use the served LeanFM address when your browser is on another machine. The application being developed has a separate runtime; opening the LeanFM page does not itself start that application.

Select **Sign in** in the account controls. For an existing account, enter your LeanFM username and password. This is the password that unlocks your locally retained, encrypted provider keys. It is not your OpenAI or Anthropic account password. After a LeanFM restart, sign in again to unlock the saved keys; empty optional key fields do not mean those keys have disappeared.

If you have no LeanFM account, choose **Create account**. Supply a username, a password of at least eight characters, and at least one provider API key. The form accepts OpenAI and Anthropic keys together. Put keys in these password fields, never in a chat message, screenshot, source file, or project book. Registration verifies the supplied provider keys before retaining them.

Once signed in, **API keys** opens the account's key editor. Re-enter the LeanFM password to add or replace a key. Blank fields preserve retained keys. Adding a second provider makes its models available without requiring a new project. Replacing a key can invalidate older unlocked sessions. The optional password-change fields require the current password and matching new values; the service re-encrypts the retained keys and ends older sessions.

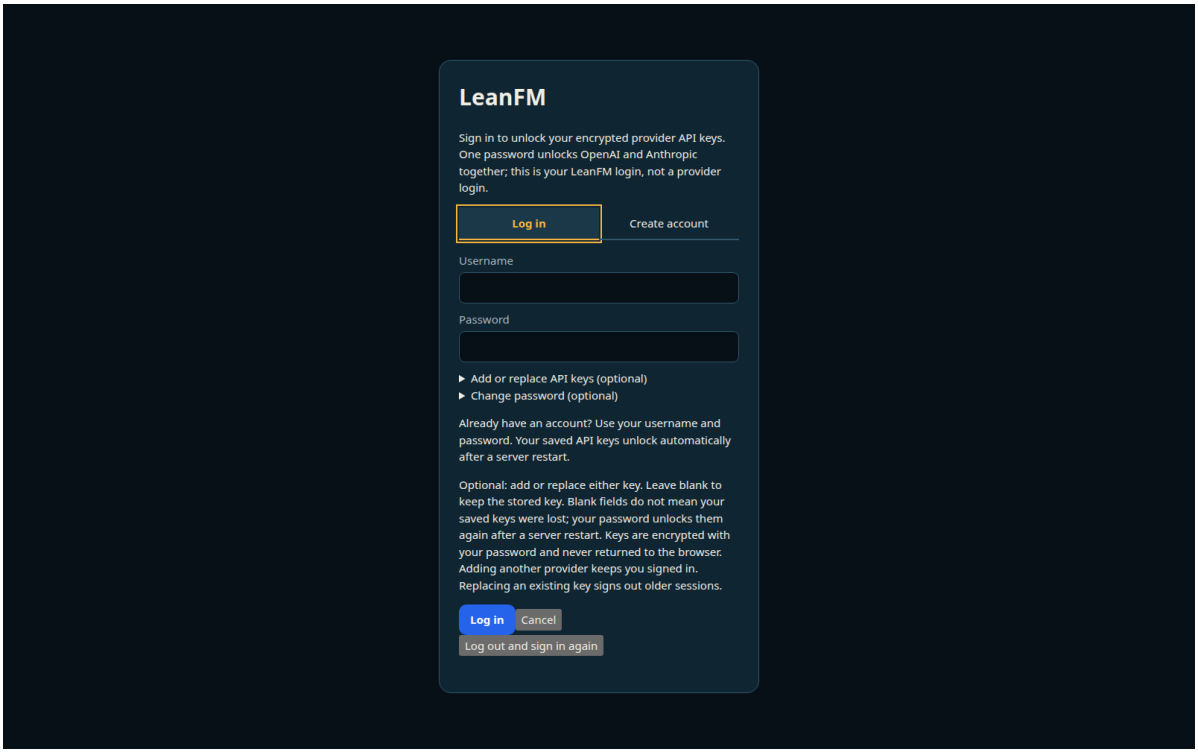


Figure 9.1: Sign in unlocks the keys already retained for the LeanFM account.

Capture boundary: before LoginSubmitted. Both credential fields are empty.

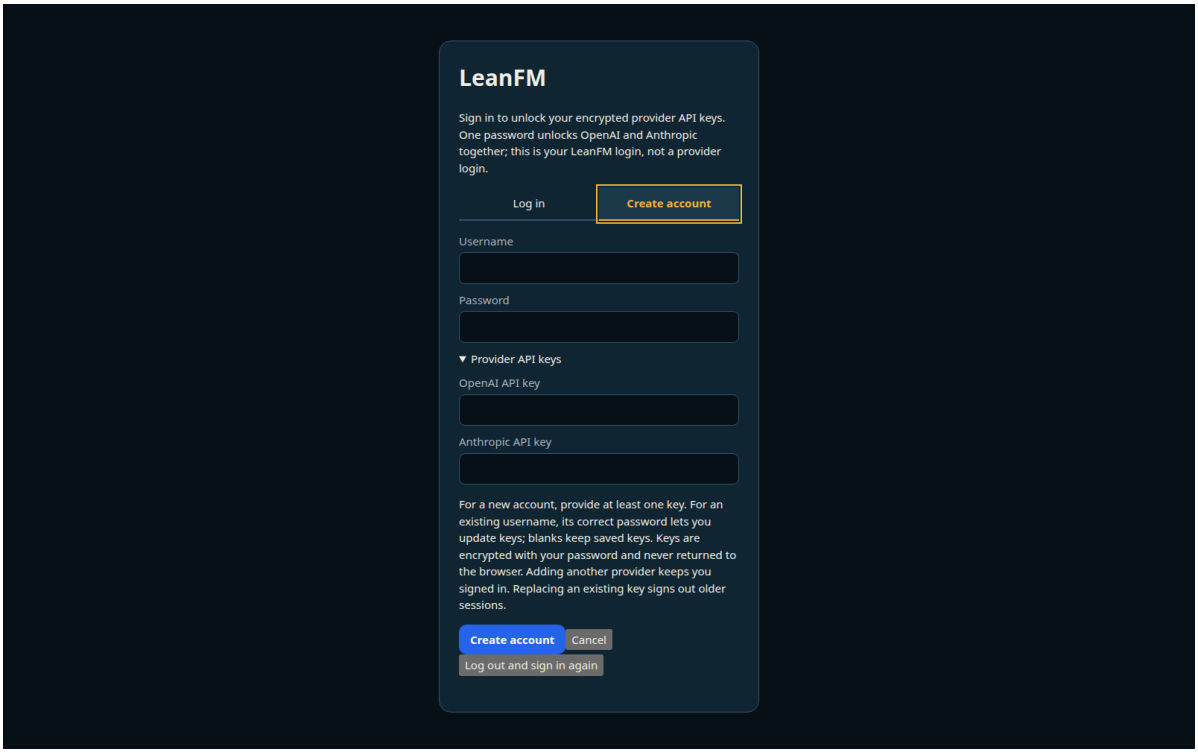


Figure 9.2: The registration tab accepts one or both provider keys.

Capture boundary: before AccountCreateSubmitted. No registration or paid request is submitted for this image.

9.2 Choose the project before writing the prompt

The left sidebar selects a project. Enter a descriptive name in **New project name**, then choose **Add project** to create a separate directory with its own identity, initial book, and specification paths. Select the project and confirm its name in the workspace header before sending a prompt. GoServer is an example project when present; it is not a required dependency of creating another project.

The project owns its book, specifications, source, tests, generated views, conversation, and evidence. Changing the selected tab changes the project boundary. An active agent turn can continue while you inspect another project; return to its tab to see its result. Delete a project only after checking the name and the confirmation: deletion removes that project's complete directory and stops its owned application.

The horizontal navigation groups the work by purpose:

Chat
: Describe the problem, answer consequential questions, and request concrete changes or checks.

Book
: Read or edit the durable design argument and preview its PDFs.

Grammar and Tasks
: Inspect the current project's generated behavioral views rather than relying on an illustration invented in a reply.

Project memory
: Review canonical artifact files and whether each is checked and committed.

Source
: Inspect the implementation files and tests actually written.

The sidebar's text-size controls change the interface scale. Project and phase navigation preserve the mounted workspace state, including drafts and scroll positions. Use `Tab` and `Shift+Tab` to move through controls and `Enter` or `Space` to activate a focused button. A focused input accepts ordinary typing; the multiline Chat composer has a separate Send button.

9.3 Send a useful prompt and understand the queue

Begin with the problem and an observable outcome. For example:

> People should be able to create named chat rooms, open a room, and post named game challenges inside that room. Keep each room's conversation separate from each challenge's conversation.

Add a concrete failure case when it matters: another room must not receive the message; a retry must not create a second challenge; an invalid action must show an error. LeanFM uses the project files to resolve ordinary implementation details. A consequential ambiguity can still require your answer.

Select the model for the new prompt in **Model**. The header identifies its provider and a fingerprint of the credential that will be used. The selection is captured when a prompt enters the queue;

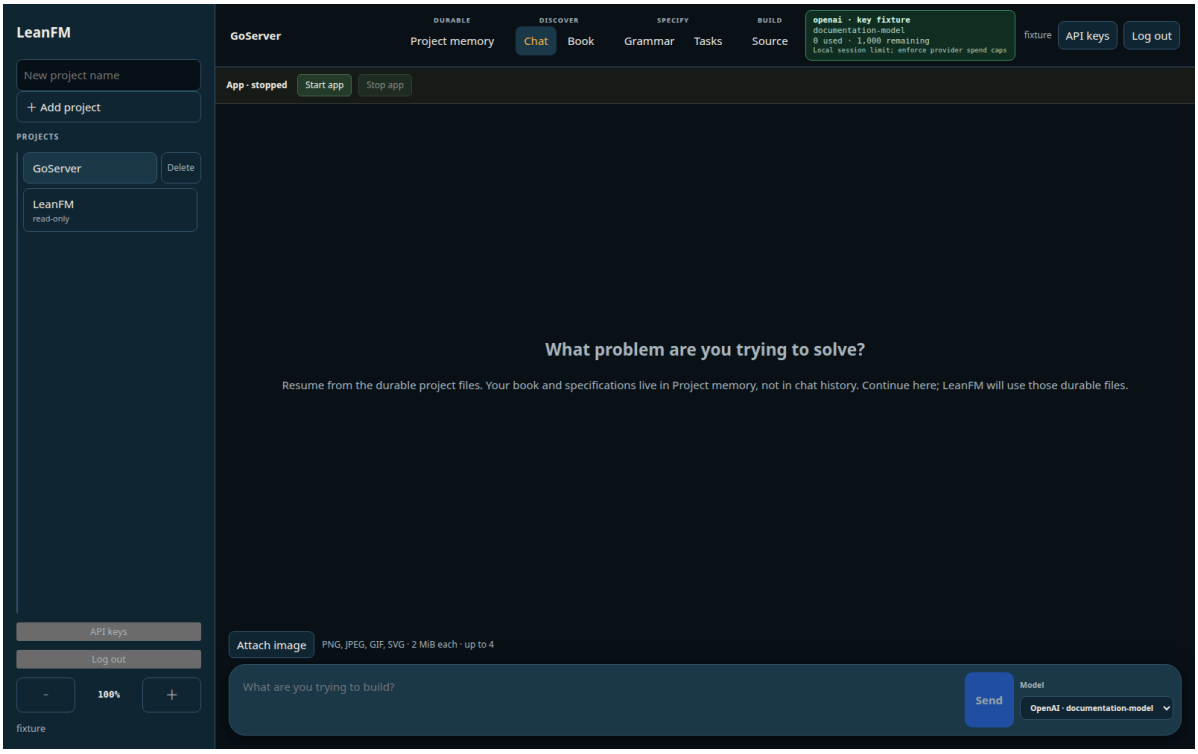


Figure 9.3: Start in the selected project's Chat pane.

Capture boundary: project opened, before ChatPromptSubmitted. The model belongs to the local documentation provider.

changing the selector later does not change a prompt already queued. LeanFM does not silently replace an unavailable provider or key with another one.

Write in the composer and activate **Send**. You may keep typing and send another prompt while the first is active. The visible queue runs in order, one prompt at a time, so a later prompt receives the results of earlier work. Use a queued item's **Remove** button to discard it before execution. Sending an extra prompt does not interrupt the active one.

Attach image accepts PNG, JPEG, GIF, and static SVG, up to 2 MiB per original and four selected attachments. A preview appears before submission; remove an attachment there if it is unintended. GIF input is represented to the model by its first frame. You can also submit an image without text. **Copy reference** on a displayed image retains its stable reference for a later prompt; copying the reference does not copy a credential or image bytes.

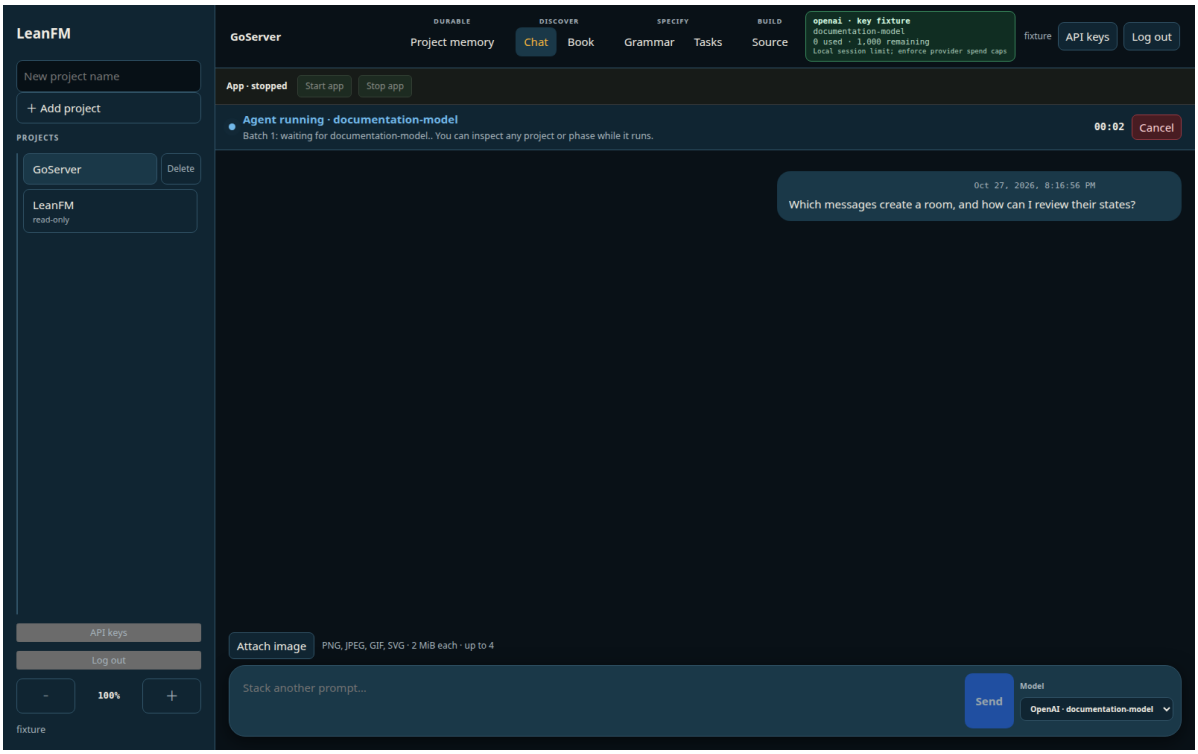


Figure 9.4: The active turn remains visible while the agent works.

Capture boundary: ChatPromptSubmitted and ModelRequestIssued, before ModelResponseReturned. The local test provider deliberately holds the response for this capture.

The running banner shows the selected model, elapsed time, and latest known stage. Expand **Operational trace** to inspect real tool destinations, checks, and failures. You may switch to Book, Tasks, Project memory, or Source while the turn continues. The live trace is more useful than inferring progress from the clock alone.

Cancel requests cancellation at a safe boundary. A provider request or tool already in progress can finish; cancellation prevents subsequent work after the server observes it. Wait for the active request to return before assuming the task has stopped. A saved multi-batch task can expose **Check progress**, **Resume task**, or **Restart from current files**; these controls distinguish continuation from a fresh attempt.

9.4 Read the result before retrying

A completed assistant message retains its time, model, response duration, and reported usage when applicable. The header's remaining-token figure is a local session safety allowance. It is not the provider's monetary credit balance. A native application launch uses the runtime controls and reports no provider token charge of its own.

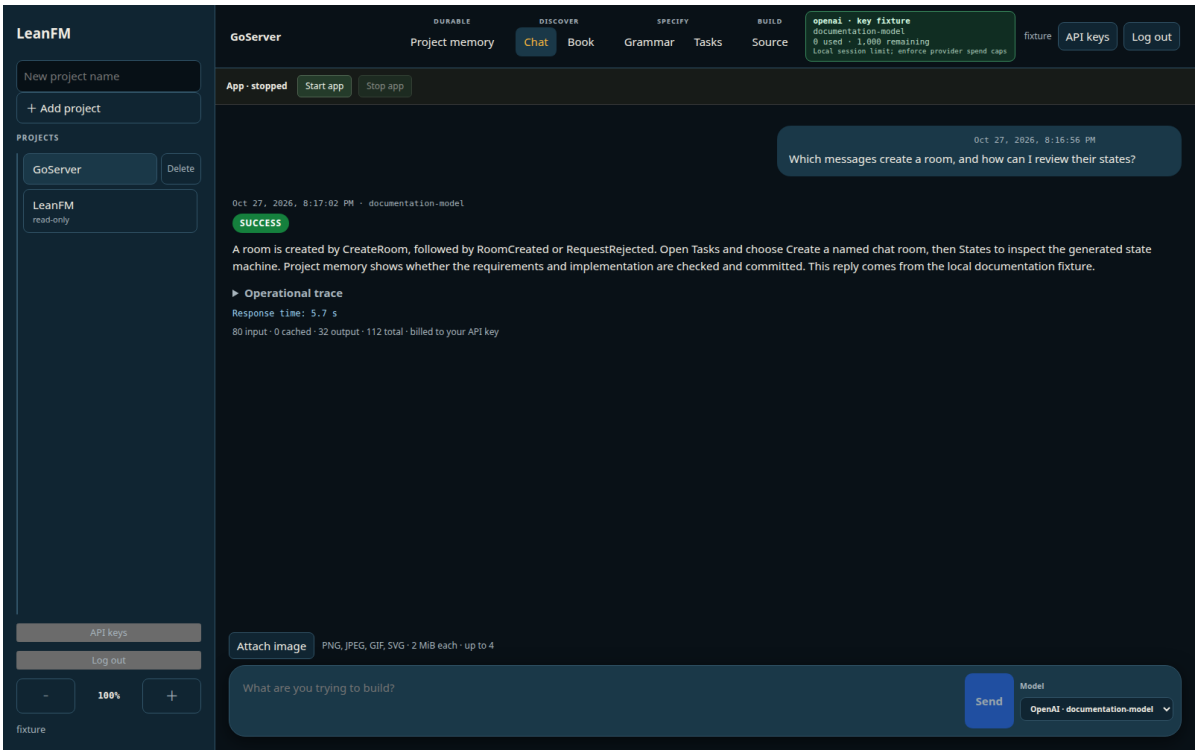


Figure 9.5: Review the answer and its evidence before taking the next action.

Capture boundary: after ChatResponseRendered. The fixture reply explains existing checked behavior; it does not generate implementation files.

Resend queues the selected user prompt again, retaining its attachment references. It can cause a new provider request. Use it after understanding the failure or changing the relevant input; it is not a promise that an uncertain earlier request had no effect. **Delete** removes that conversation message. It does not undo a committed source change or remove the book.

When a structured question appears, choose or type an answer in its form and submit it. The answer closes the identified question and becomes a later queued turn. Acceptance is a git commit: a successful compiler check or a model saying "done" tells you the work is ready to review, not that you have reviewed it.

An error includes a concrete diagnostic and, where available, an expanded **Diagnostic for Codex** with **Copy diagnostic**. Preserve that diagnostic when reporting a problem. For a provider low-credit rejection, fix that provider's billing condition or explicitly choose an available model and credential before resending. LeanFM does not make an automatic paid retry for that rejection. A local token-limit error, rate limit, incomplete response, compiler failure, and network failure need different remedies; the operational trace identifies which boundary failed.

9.5 Keep the book and inspect the generated behavior

Open **Book** to see the editable source beside its rendered PDF. **Save source** saves the text; **Build & preview** saves and builds the reading copies. The status line distinguishes a saved source from a built PDF. **Light PDF** and **Dark PDF** select equivalent renderings of the same source. **Maximize PDF** expands the reading pane, and **Restore split** returns to source and preview. The divider resizes the two panes.

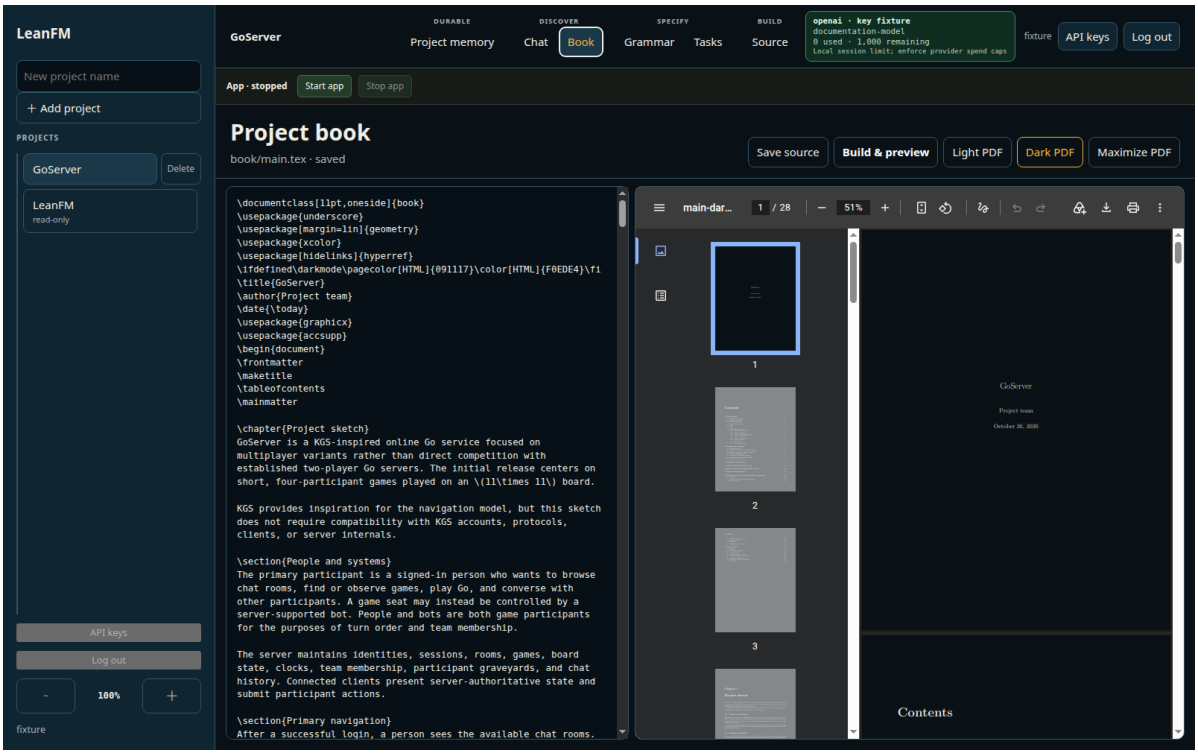


Figure 9.6: The Book pane keeps editable source and the rendered reading copy together.

Capture boundary: saved source and an existing built PDF opened. This capture does not edit the book or accept a changed revision.

The book records decisions and their explanation. **Tasks** opens the views derived from the current typed specification. Select a task in the left task list, then choose **Productions**, **States**, **Properties**, or **where_**. Productions shows its request and result alternatives; States shows its task state machine. Properties and where_ display the temporal requirements and contextual constraints. A blank task set or missing grammar is a specification gap, not evidence that the behavior exists.

Grammar provides the nested production structure. Expand the relevant production when you need to inspect composition and alternatives. Use Tasks for a focused behavioral view and Grammar for its structure. Neither pane should substitute another project's grammar or a separately drawn persuasive picture for the selected project's checked model.

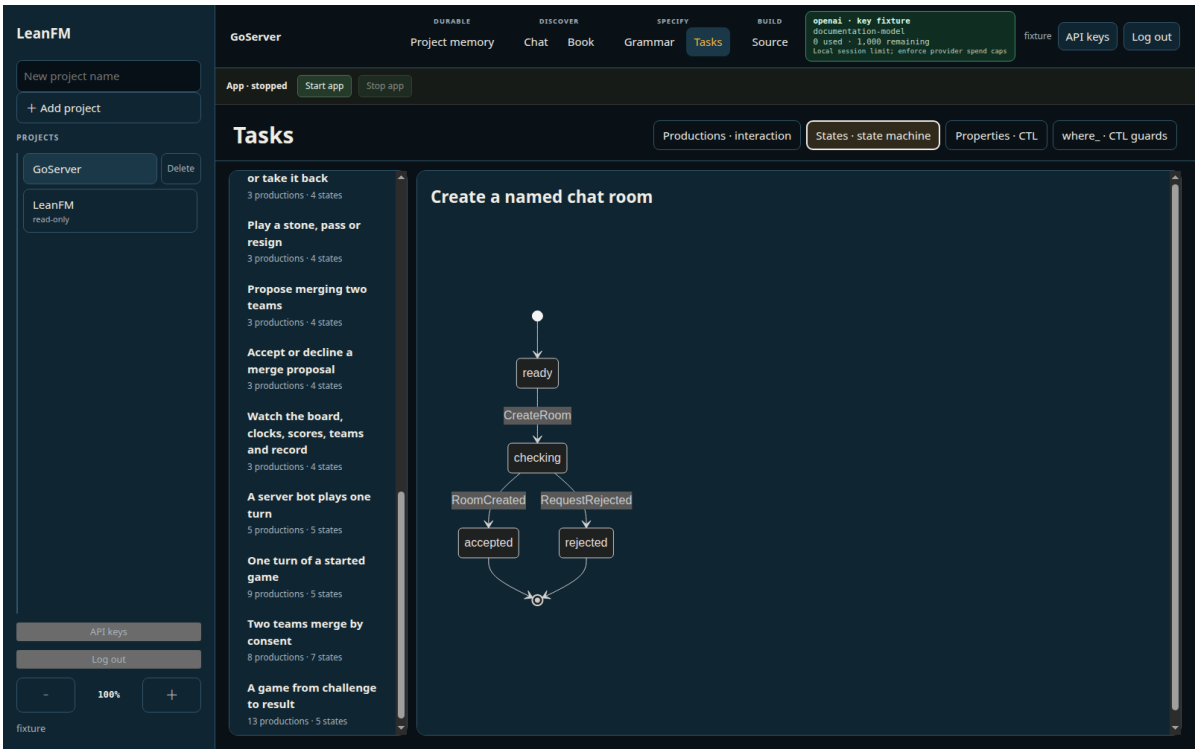


Figure 9.7: The room-creation state machine comes from the checked project specification.

Capture boundary: the current create_room task rendered from its checked typed value. The companion requirement and task-catalog snapshots are retained with the capture.

9.6 Review the specification, then inspect the implementation

Open **Project memory** to read the book, requirements, protobuf schema, implementation mapping, and implementation-source summary. Each file shows whether it is absent, still bootstrap material, or current, and the review cards above them show whether the requirements and implementation mapping pass the canonical check and are committed in git. A file's presence alone does not establish that its content has passed the check.

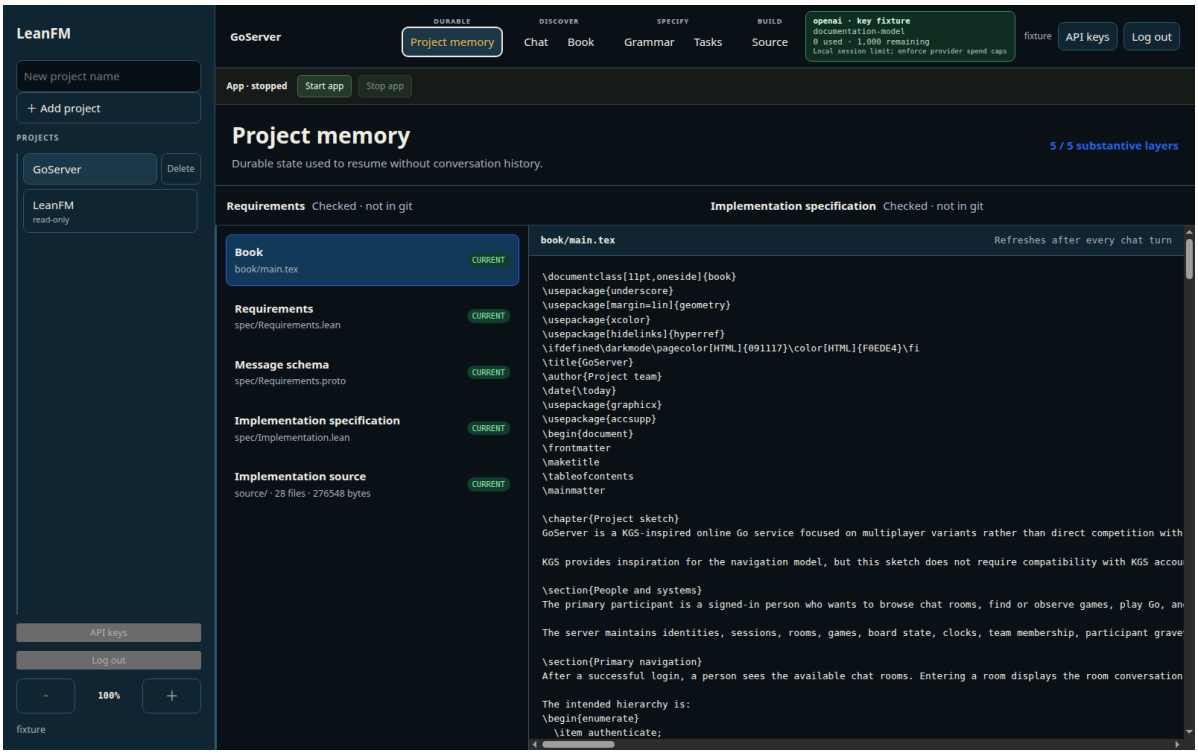


Figure 9.8: Project memory makes the durable files and their check status inspectable.
Capture boundary: both checked specification revisions loaded.

Review the requirements first: task messages, outcomes, error behavior, boundaries, and generated views. Then review the separate implementation specification and its traceability: how the chosen runtime, files, transports, resource bounds, security controls, and tests realize those requirements. When a revision says what you mean, commit it with git; when it does not, say so in Chat. Nothing waits for an approval click, so you can ask Chat to implement the actual source at any time, and LeanFM will write and check whatever specification that needs first. The committed artifacts and existing source remain the inputs; deleted conversation history must not become a missing design dependency.

Use **Source** to inspect the changed files and tests. Request edits and verification through Chat when needed; the Source pane is an inspection view. Read the reported checks and their limitations. A successful startup confirms that the app can start and answer its probes; it does not prove every game rule or security property. GoServer's present slice provides rooms, named challenges, and separate conversations. Playable boards and the larger Go rules remain deferred.

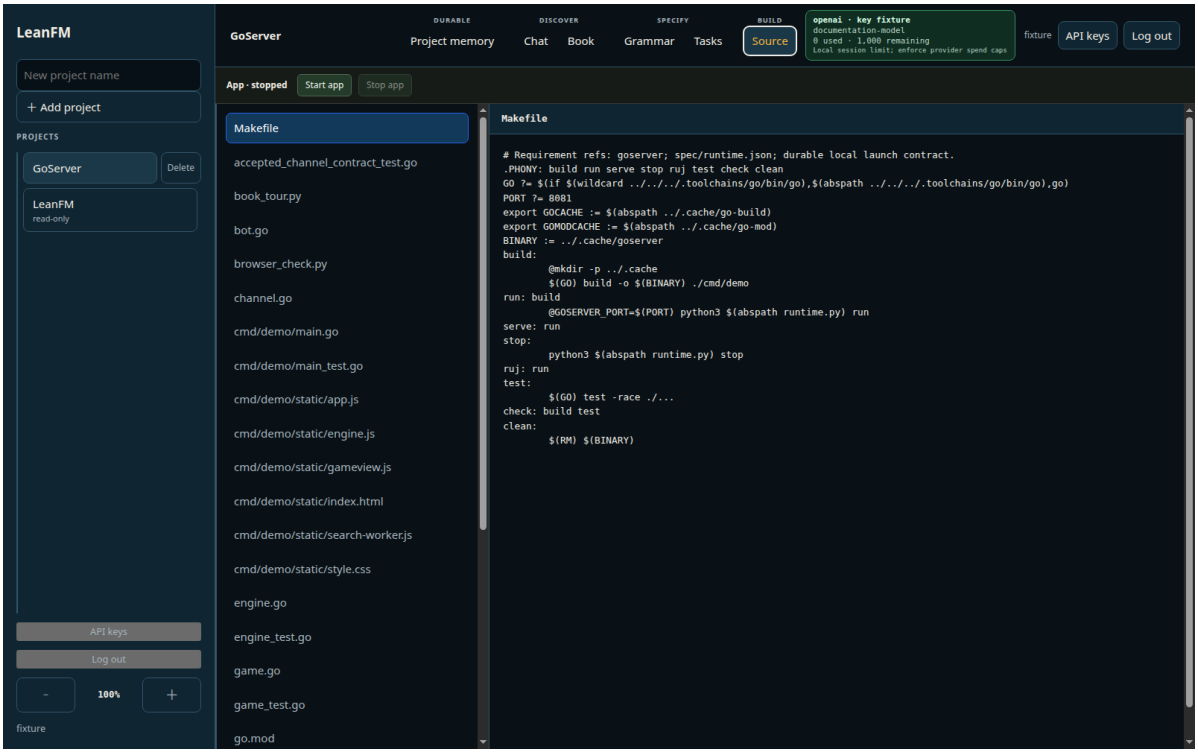


Figure 9.9: Source displays the implementation files actually present in the project.

Capture boundary: the implementation's actual source tree opened.

9.7 Start, open, and stop the app

Use **Start app** in the selected project's application bar once its source is runnable. LeanFM uses the fixed application contract, waits for actual readiness and a page response, and keeps the owned process running. The bar changes to **running**, **Open app** becomes a link, and Chat receives the same verified launch link. These native controls do not require a model request to perform the launch.

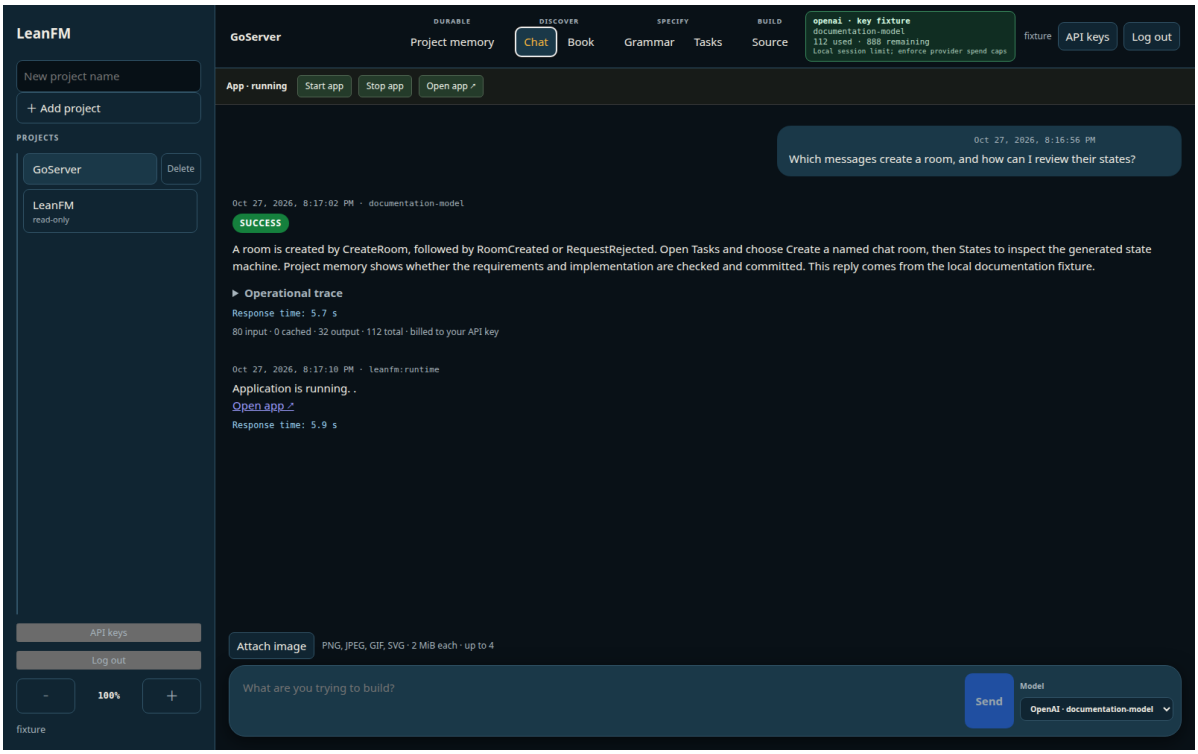


Figure 9.10: A verified running instance produces Open app in both the bar and Chat.

Capture boundary: ApplicationReady after real startup and page probes. The link names the served preview, not a private loopback endpoint.

Open app opens the served preview. The app runs inside its own filesystem, process, network, and browser-authority boundary. For a remote LeanFM browser, use this link rather than guessing a server-local port. Apps need relative or explicitly prefix-compatible asset and API URLs for the preview; the runtime reports an unsupported contract instead of publishing a link that cannot work.

If source changes, the runtime becomes **stale**. Use **Restart app** to run the new revision. Repeated Start on a current running revision reuses its instance. **Stop app** ends only the selected project's owned instance and removes its ready link. Other projects and unrelated processes are left alone. The separate terminating launch check is validation evidence; its old receipt is not a persistent Open app link.

All HTTP applications expose the same targets from `source/`:

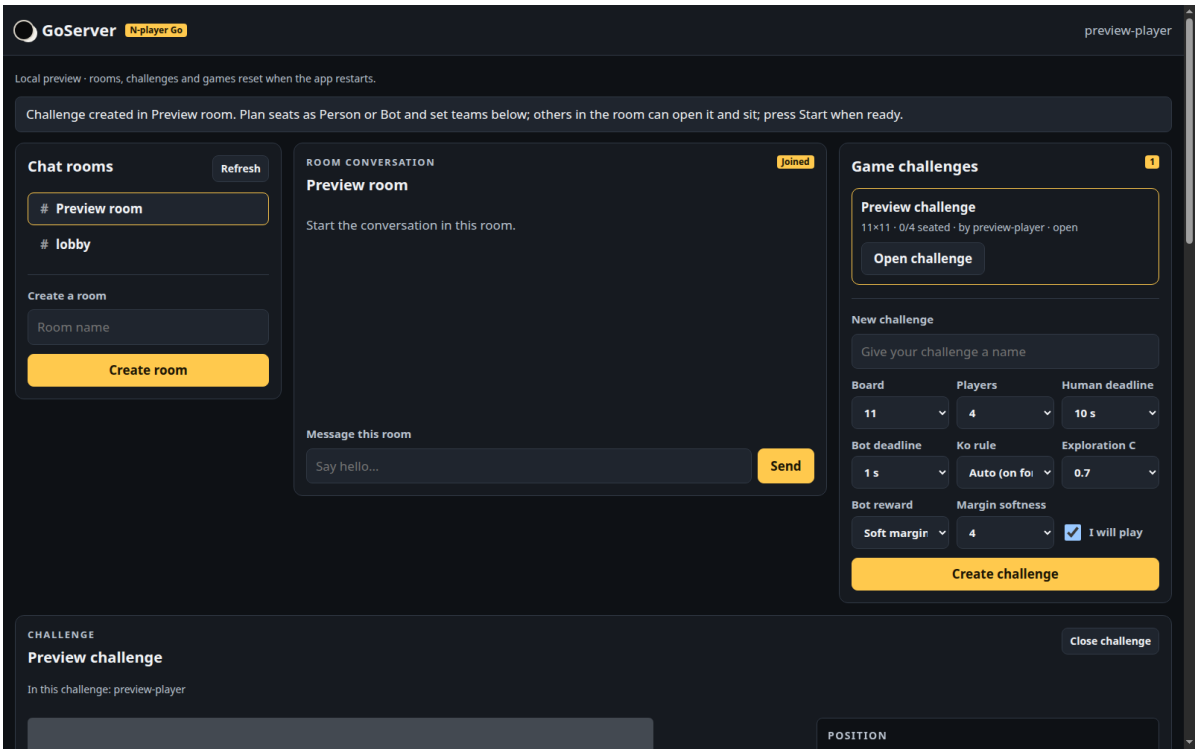


Figure 9.11: The running GoServer preview supports real room and challenge operations.

Capture boundary: RoomCreated, GameCreated, and GameChat observed through the real preview route. The app remains a local in-memory inspection slice.

Target	Purpose
`make build`	Build the application.
`make run`	Run in the foreground for supervision.
`make serve`	Alias for `run`.
`make stop`	Stop only this project's owned app.
`make test`	Execute the application's tests.
`make check`	Run its proportional verification.
`make clean`	Remove its disposable build outputs.

Missing targets, invalid launch profiles, capacity limits, and failed readiness produce explicit errors. Review the app status and diagnostic before retrying; an occupied listener is not stopped to make room for the new app.

9.8 Reproduce and review the screenshots

Run `make ui-snapshots` from the LeanFM repository after building the current server and UI. The runner uses the real production bundle, a disposable project replica, an isolated network, and a deterministic local provider. It drives named controls by keyboard and records the focused control, accessible roles and states, visible message IDs, viewport, browser version, source digests, and exact capture boundary. No live provider key or production conversation is required. The GoServer example is needed to regenerate this particular tour; the checked-in capture inputs and images allow the book to build independently of that optional project directory.

Each image has a neighboring `.accessibility.json` and `.keyboard.json` file. `book/assets/ui-tour/manifest.json` binds them to the prompt and source revisions. The book's preflight check rejects missing, altered, or stale required captures. Alternative text in both PDF editions describes the control state, so a screenshot is never the only account of the interaction.

Chapter 10

Worked Example: OAuth Authorization Code Flow

This chapter is a behavioural sketch inspired by the OAuth authorization-code flow. It is not a replacement for the OAuth specification and deliberately begins above HTTP and protobuf. Its purpose is to exercise scenario decomposition, parameter flow, visible evidence, alternatives, requirements, and rendering. In session-type terms it is a global type with five roles, a choice directed by the authorization service (grant or deny), and parameters such as `state` and the authorization code that must flow from one message to a later one. Its channels are HTTP calls and browser redirects; their physics (a redirect that is never followed, a callback replayed by an attacker) is where the security requirements live.

The `scenario`, `declare`, `from_/to_/emit`, `weightedOneOf` and theorem listings in this chapter are *proposed notation*: they are not compiled, and the definitions they name do not exist in the LeanFM library yet. The generated figures are real.

The flow is also deliberately simplified. The authorization request omits `client_id` and `response_type`; the token request carries only the code, without `redirect_uri`, client authentication or PKCE, all of which RFC 6749 and current OAuth security guidance require. The knowledge claims below hold for this simplified flow only; a deployable model must add those fields and re-check them.

10.1 Actors, values, and observable events

The example has four participants: a person using a browser, an application, an authorization service, and a resource service. The browser is modelled separately from the person because redirects are observable messages between software participants.

```
inductive Actor where
  | user | browser | app | authorizationServer | resourceServer

opaque State       : Type
opaque AuthCode    : Type
opaque AccessToken : Type
opaque Resource    : Type
opaque Profile     : Type
opaque Password    : Type
opaque Bytes       : Type
```

Every terminal elaborates to the common event envelope. For example, the vague message “authorization redirected” becomes conceptually:

```
{ timeAt := t2, id := e2, priorIds := [e1]
, session := browserSession, task := oauthAuthorizationTask
, from := Actor.app, to := Actor.browser
, kind := "authorization redirected"
, fields := [field "state" state, field "redirectUri" callback]
}
```

Authentication detail used by Production 2

Before the redirect returns, the authorization service may authenticate the person. OAuth does not prescribe the authentication mechanism; we include a password branch to exercise knowledge flow without pretending that the password is an OAuth protocol parameter.

```
def authenticateAtAuthorizationServer (password : Password) : Scenario :=
  scenario "authenticate resource owner" do
    from_ .browser
    to_ .authorizationServer
    emitBytes "credentials submitted"
      (protectFor .authorizationServer
        (encode [field "password" password]))

    from_ .authorizationServer
    to_ .browser
    oneOf "authentication result"
      [ emitBytes "authentication accepted" (encode [tag 1])
      , emitBytes "authentication rejected" (encode [tag 0])
      ]
```

The wire bytes are specified even though the transport is not. The abstract `protectFor` constructor promises confidentiality to the named endpoint. A later refinement may realize that promise with TLS, message-level encryption, or a local trusted channel. Merely placing the password inside a field would not support the intended knowledge set: without protection, a network observer would also derive the password from the bytes.

10.2 Production 1: application request and redirect

```
def applicationRequestAndRedirect (resource : Resource) : Scenario :=
  scenario "application request and redirect" do
    let state <- declare State "state"
    let callback := Uri.https "app.example" "/oauth/callback"

    from_ .browser
    to_ .app
    emit "application request" [field "resource" resource]

    from_ .app
```

```
    to_ .browser
    emit "authorization redirected"
      [field "state" state, field "redirectUri" callback]

    export "state" state
    export "redirectUri" callback
    export "resource" resource
```

This production treats the application request and its redirect as one named unit. It establishes the correlation value `state` and exports it to the next production. Nothing here says how the application stores it.

10.3 Production 2: authorize, then grant or deny

```
def authorizationGranted : Scenario :=
  scenario "authorization granted" do
    use state <- imported State "state"
    let code <- declare AuthCode "code"

    from_ .authorizationServer
    to_ .browser
    emit "authorization code returned"
      [field "state" state, field "code" code]

    from_ .browser
    to_ .app
    emit "authorization callback"
      [field "state" state, field "code" code]
    export "code" code

def authorizationDenied : Scenario :=
  scenario "authorization denied" do
    use state <- imported State "state"
    from_ .authorizationServer
    to_ .browser
    emit "authorization error returned"
      [field "state" state, field "reason" "access_denied"]
    from_ .browser
    to_ .app
    emit "authorization error callback"
      [field "state" state, field "reason" "access_denied"]

def authorizeAndDecide (password : Password) : Scenario :=
  scenario "authorize and decide" do
    use state       <- imported State "state"
    use redirectUri <- imported Uri "redirectUri"

    from_ .browser
    to_ .authorizationServer
    emit "authorization requested"
```

```
      [field "state" state, field "redirectUri" redirectUri]

    include (authenticateAtAuthorizationServer password)
    oneOf "authorization decision"
      [ include authorizationGranted
      , include authorizationDenied
      ]
```

The authorization request and its two possible continuations form one production. At a more abstract rendering level, the entire production can be shown as one node; expanding it reveals authentication and the explicitly bracketed choice. The abstract contract makes both decisions possible. A product model may later attach an estimated or measured probability distribution without changing the set of legal traces.

10.4 Scenario 3: exchange the code and use the token

```
def exchangeAndFetch : Scenario :=
  scenario "exchange code and fetch resource" do
    use code     <- imported AuthCode "code"
    use resource <- imported Resource "resource"
    let token    <- declare AccessToken "accessToken"
    let profile  <- declare Profile "profile"

    from_ .app
    to_ .authorizationServer
    emit "token requested" [field "code" code]

    from_ .authorizationServer
    to_ .app
    emit "access token issued" [field "accessToken" token]

    from_ .app
    to_ .resourceServer
    emit "protected resource requested"
      [field "accessToken" token, field "resource" resource]

    from_ .resourceServer
    to_ .app
    emit "protected resource returned"
      [field "resource" resource, field "profile" profile]

    from_ .app
    to_ .browser
    emit "application response" [field "profile" profile]
```

The token itself is visible only at the endpoints where the abstract protocol says it is transmitted. The grammar makes no claim about the authorization server’s database schema or the application’s session representation.

10.5 Reading the scenarios together

```
def oauthAuthorization (resource : Resource) (password : Password) : Scenario :=
  scenario "OAuth authorization code flow" do
    include (applicationRequestAndRedirect resource)
    include (authorizeAndDecide password)
    after authorizationGranted include exchangeAndFetch

def oauthSystem (resource : Resource) (password : Password) : SystemSpec :=
  generatedBy (oauthAuthorization resource password)
  |> requireAll oauthRequirements
```

The two named productions provide the first abstraction level. A renderer may collapse either production to a single node or expand it to its messages and nested alternatives. The `after` line is a guarded nonterminal reference; the denial branch ends without exchanging a code.

10.6 The parallel protobuf alphabet

The scenario and its wire alphabet are maintained side by side. The scenario says which messages may occur, in what predecessor-order structure, and with which parameter relationships. The protobuf file assigns field numbers and concrete wire types to those same message atoms:

```
syntax = "proto3";

package leanfm.oauth;

enum Actor {
  ACTOR_UNSPECIFIED = 0;
  BROWSER = 1;
  APPLICATION = 2;
  AUTHORIZATION_SERVER = 3;
  RESOURCE_SERVER = 4;
}

message EventEnvelope {
  string event_id = 1;
  repeated string prior_ids = 2;
  string session_id = 3;
  string task_id = 4;
  uint64 clock_tick = 5;
  Actor src = 6;
  Actor dst = 7;

  oneof payload {
    ApplicationRequest application_request = 20;
    AuthorizationRedirect authorization_redirect = 21;
    AuthorizationRequest authorization_request = 22;
    CredentialsSubmitted credentials_submitted = 23;
    AuthenticationResult authentication_result = 24;
    AuthorizationCodeReturned authorization_code_returned = 25;
    AuthorizationCallback authorization_callback = 26;
    AuthorizationErrorReturned authorization_error_returned = 27;
    AuthorizationErrorCallback authorization_error_callback = 28;
    TokenRequest token_request = 29;
    AccessTokenIssued access_token_issued = 30;
    ProtectedResourceRequest protected_resource_request = 31;
    ProtectedResourceReturned protected_resource_returned = 32;
    ApplicationResponse application_response = 33;
```

```
  }
}

message ApplicationRequest {
  string resource = 1;
}

message AuthorizationRedirect {
  string state = 1;
  string redirect_uri = 2;
}

message AuthorizationRequest {
  string state = 1;
  string redirect_uri = 2;
}

message CredentialsSubmitted {
  bytes protected_credentials = 1;
}

message AuthenticationResult {
  bool accepted = 1;
}

message AuthorizationCodeReturned {
  string state = 1;
  bytes code = 2;
}

message AuthorizationCallback {
  string state = 1;
  bytes code = 2;
}

message AuthorizationErrorReturned {
  string state = 1;
  string reason = 2;
}

message AuthorizationErrorCallback {
  string state = 1;
  string reason = 2;
}

message TokenRequest {
  bytes code = 1;
}

message AccessTokenIssued {
  bytes access_token = 1;
}

message ProtectedResourceRequest {
  bytes access_token = 1;
  string resource = 2;
}

message ProtectedResourceReturned {
  string resource = 1;
  bytes profile = 2;
}

message ApplicationResponse {
  bytes profile = 1;
}
```

Each emitted scenario message maps to exactly one member of `EventEnvelope.payload`. The envelope carries the event identity, repeated `prior_ids`, `(session_id, task_id)` correlation key, monotonic clock tick, and endpoints outside the payload. The payload carries only the fields specific to that message.

Protobuf alone does not specify the protocol order, queue behavior, probability, or dwell time; those remain in the scenario. Conversely, the scenario does not invent byte tags. Validation checks that every scenario terminal has one protobuf variant, that its endpoints agree with the scenario, and that every referenced field has the expected protobuf type.

The schema determines valid protobuf encodings, but protobuf does not generally promise one canonical byte sequence. When byte-for-byte comparison is required, the refinement also fixes deterministic serialization and length-delimited message framing. An instantiated envelope can then be serialized and displayed as the actual bytes sent between the named actors.

10.7 Generate one legal trace and its artifacts

We can now run the specification in one direction. Grammar generation chooses productions and concrete field values; everything after that is a deterministic projection or reduction of the resulting events. For this run choose the authentication-accepted and authorization-granted alternatives and instantiate

```
session = "s1"             task = "oauth"
resource = "profile"       state = "x7"
redirectUri = "https://a/cb"
protectedCredentials = aa bb
code = 01 02               accessToken = 03 04
profile = 05 06
```

The production derivation is

```
oauth
=> applicationRequestAndRedirect ;
   authorizeAndDecide ;
   after(authorizationGranted, exchangeAndFetch)
=> applicationRequestAndRedirect ;
   authorizationRequested ;
   credentialsSubmitted ; authenticationAccepted ;
   authorizationGranted ; exchangeAndFetch
```

Because `authorizationGranted` was selected, the `after` guard admits `exchangeAndFetch`. Selecting the denial alternative would disable that production. Expanding every remaining nonterminal yields this terminal word:

The choice has now disappeared into ordinary terminals. Event `e5`'s payload bytes select the accepted authentication result. Event `e6`'s `authorizationCodeReturned` oneof tag, source, and bytes distinguish the grant branch from the error-returned branch. No independent "chooser" value is needed.

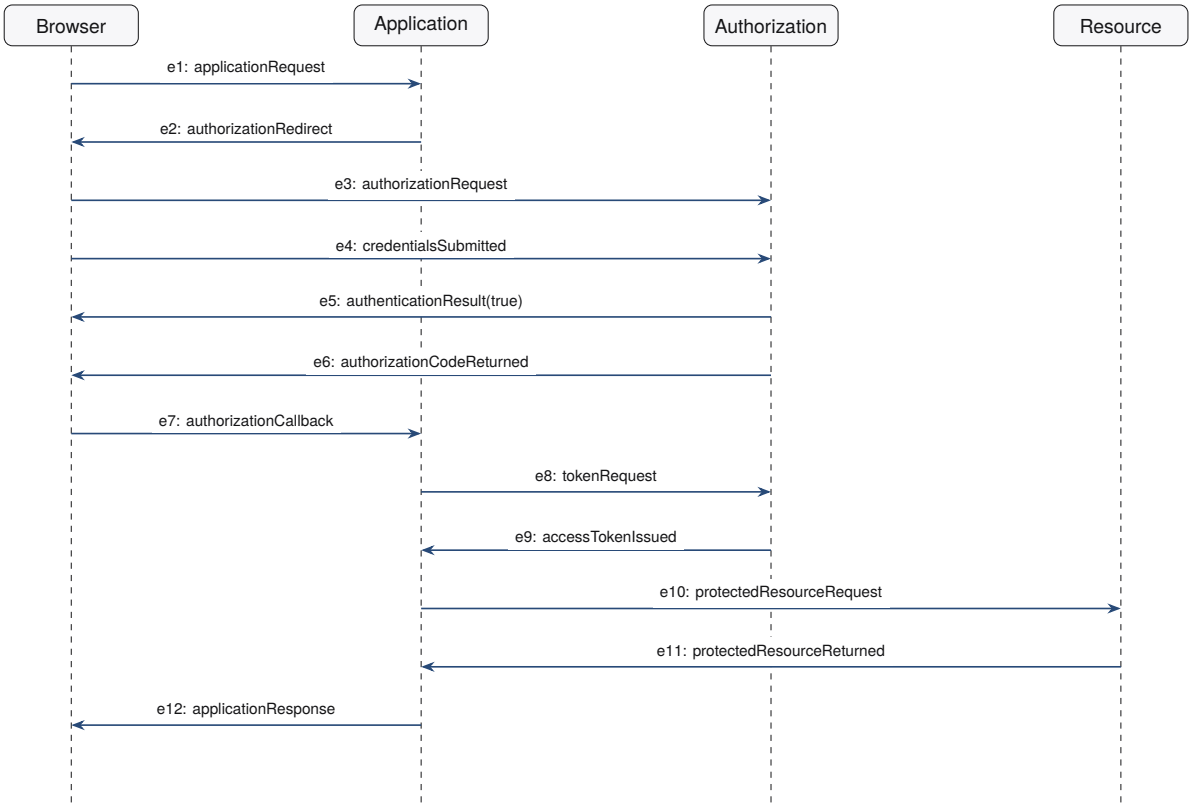


Figure 10.1: The selected terminal word as an interaction diagram. Event IDs label messages on arrows; no actor-to-actor arrow is written as implication-like text.

10.7.1 Envelope stream and concrete bytes

Assign monotonically increasing clock ticks and event identifiers. This example has a linear predecessor chain, so `priorIds(e1)` is empty and `priorIds(e`n`)` is `[e`$n-1$`]`. A fork or join would make these lists non-linear without changing the procedure. Deterministic protobuf serialization is the intended encoding; the companion fixture records one concrete stream:

```
e1   0  application_request          31  0a0265311a0273122056f6175746830013802a201090a0770726f66696c65
```

The companion fixture `examples/oauth-granted.bytes` retains all twelve envelopes (440 encoded bytes total; largest envelope 46 bytes). The excerpt above shows one complete envelope. The repository does not currently include a generator or round-trip check for this particular fixture, so it is a concrete example rather than evidence that serialization was reproduced or verified.

10.7.2 Event graph with stored backpointers

Parsing each envelope selects a payload alternative, checks its declared `src`/`dst`, and consumes one grammar terminal. For this selected trace, the useful concrete view is the event graph itself:

This graph is one selected path. The full residual state machine is derived from every legal grammar residual and also contains the rejected-authentication and denied-authorization branches; it should be rendered as a separate graph when those alternatives are under discussion.

10.7.3 Actor projections

Project each terminal onto its sender and receiver, preserving order and erasing the rest. For example, Resource receives `e10` and sends `e11`; the other local scripts follow the same endpoint filter. The example generator emits the shared interaction and predecessor diagrams, not replay scripts. The global word remains authoritative for cross-component order and parameter equality; a local projection does not reveal erased events.

10.7.4 Event order, queues, and memory

The `priorIds` fields generate a twelve-node order graph with edges $e_i \to e_{i+1}$. The interaction diagram is the same graph laid out by actor; the state-machine path lays it out by grammar residual. Neither is independently authored.

The message word does not uniquely determine a transport schedule. For a concrete queue artifact, choose the canonical schedule that performs, for each event, `produce; transport; receive` before generating the next event. With positive inbound and outbound capacities, each queue peaks at one message. The queued-byte peak for an actor is the largest encoded envelope waiting at that actor in this schedule, and the global queued-byte peak is 46. A different legal interleaving may have a larger peak and must be explored when checking the universal memory-budget property. The selected schedule is a witness, not the capacity proof.

10.7.5 Measurements, plots, and verdicts

From the clocks and correlation key, the generated task attempt is

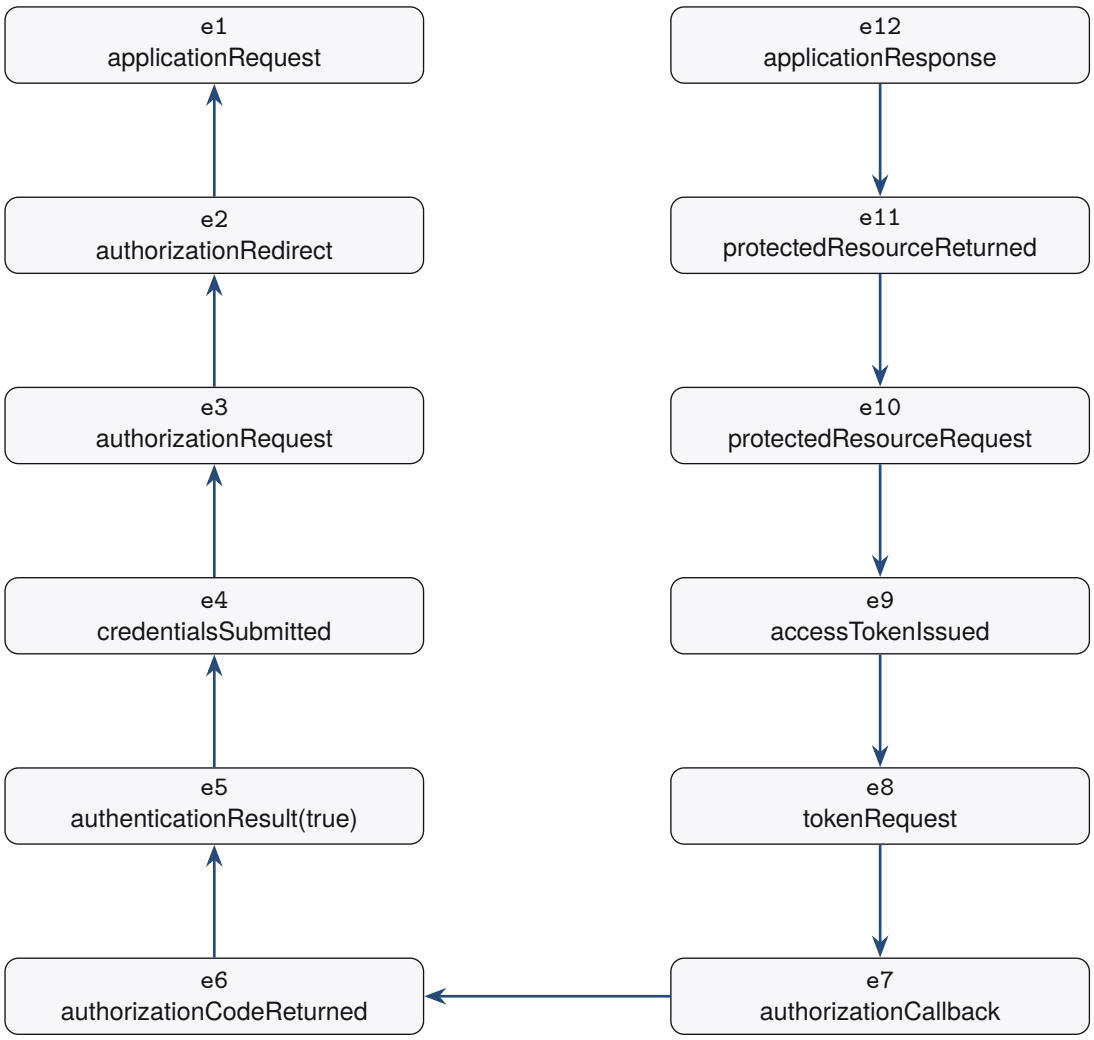


Figure 10.2: The accepted trace as an event graph. Arrows are stored backpointers: `e1` has none and each later event points to its immediate predecessor.

```
{ key = ("s1", "oauth"),
  started = { kind = applicationRequest, timeAt = 0 },
  completed = { kind = applicationResponse, timeAt = 23 },
  outcome = succeeded, latency = 23 ticks }
```

It contributes one point `(maxConcurrency=1, latency=23)` to the latency XY plot, a cumulative-byte series ending at `(clock=23, bytes=440)`, and one successful slice to the outcome chart. It does *not* establish a 100 percent success probability. The grammar-level weight predicts 0.87 mass on the grant branch; estimating that probability requires many generated or observed sessions, including incomplete ones.

Trace-local monitors accept the matching `state=x7`, the code flow `01 02`, and the fact that no exchange follows denial. CTL obligations such as “every decision reaches the application” are not proved by this trace; the checker must examine every reachable grammar residual. The complete artifact pipeline is thus

```
grammar + field values
  -> legal terminal word
  -> typed EventEnvelope stream
  -> framed protobuf bytes
  -> partial-order graph + interaction diagram
  -> residual FSM path + per-actor projections
  -> queue schedule + byte-pressure samples
  -> task attempt + probability/latency/XY reducers
  -> replay fixture + trace-local property verdicts
```

Every arrow is reproducible. Only the initial branch and value generation, and the separately stated transport schedule, add choices.

10.8 Interaction diagrams by scenario

10.9 Requirements as observable constraints

```
def oauthRequirements : List Requirement :=
  [ require "callback state matches request"
      (on "authorization callback" (previouslySame "state"
        "authorization requested"))

  , forbid "code exchange after denial"
      (previously "authorization denied" && event "token requested")

  , require "issued token came from a code"
      (on "access token issued" (previouslySame "code" "token requested"))

  , ctl "every decision reaches the application"
      (AG (decisionAtAuthorizationServer --> AF callbackAtApplication))

  , ctl "a protected result is never returned without a token request"
      (AG (resourceReturned --> previously tokenRequested))
  ]
```

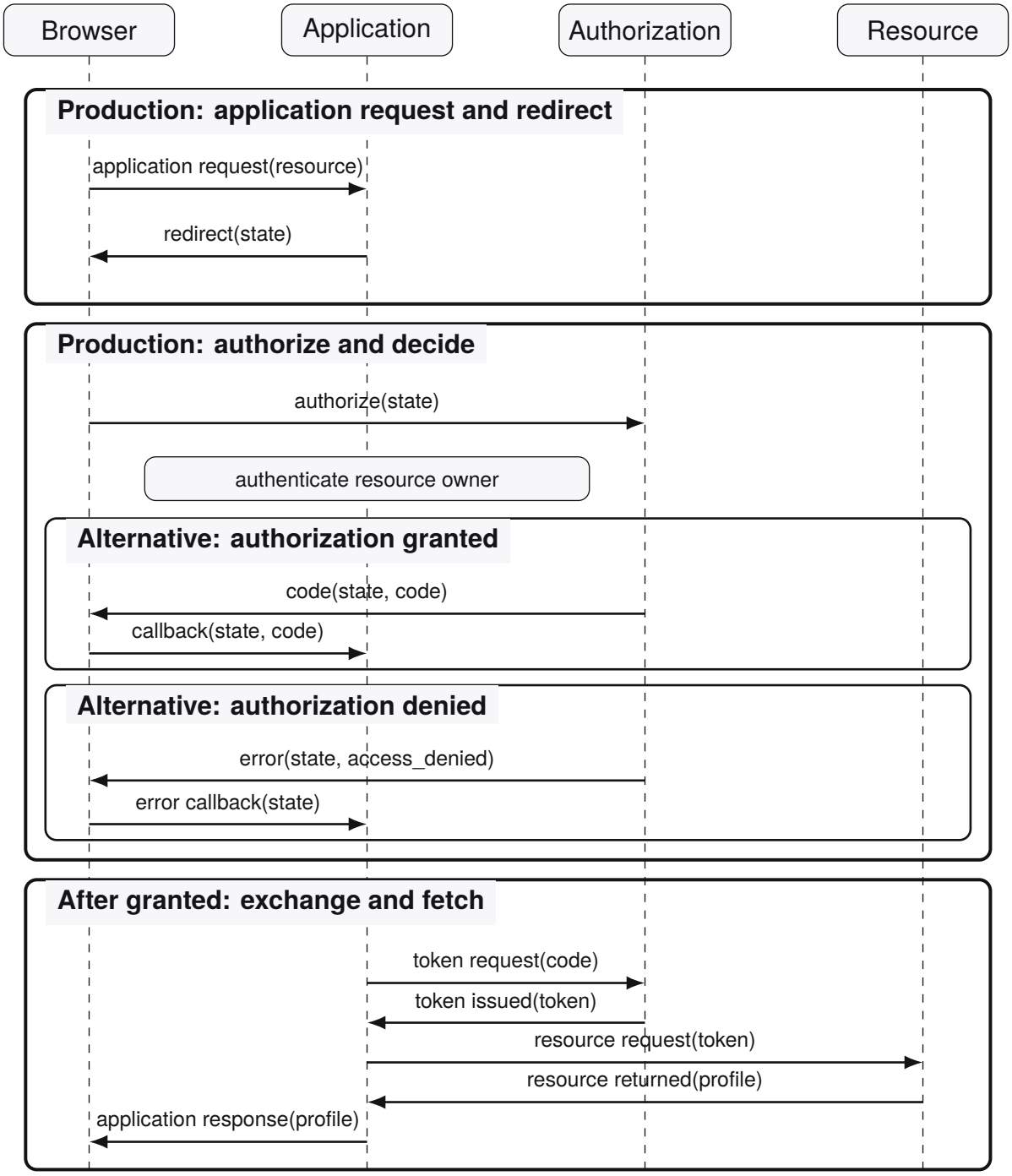


Figure 10.3: OAuth interaction view. Production frames provide collapsible abstraction boundaries. The authorization production contains its request and both alternatives; the guarded exchange production is enabled only by the granted alternative.

10.9.1 Who knows the password?

This OAuth instance applies the trace-derived knowledge closure from Section 17.5:

```
def passwordKnowers (trace : Trace WireEvent) : List Actor :=
  allActors.filter fun actor =>
    Derives (knowledgeFrom actor trace) (value password)

theorem oauth_password_knowers
    (trace : Trace WireEvent)
    (accepted : (oauthSystem resource password).Accepts trace)
    (initial : InitiallyKnown password [.browser, .authorizationServer])
    (honest : Uncompromised [.browser, .authorizationServer])
    (protection : PerfectProtection protectFor) :
    passwordKnowers trace = [.browser, .authorizationServer] := by
  exact knowledgeClosureOfProtectedField accepted initial honest protection
```

Proposed notation: neither the theorem nor the library lemma it uses exists yet.

The browser itself knows a typed password. A narrower boundary requires a trusted credential actor; compromise, logging, a visible key, or failed protection changes the result.

10.9.2 Byte-level refinement without choosing a transport

Every vague OAuth message can be assigned a canonical byte term before HTTP or protobuf is chosen:

```
refineMessage "authorization requested" where
  term := tuple
    [ tagBytes 0x01
    , namedBytes "state" (encode state)
    , namedBytes "redirectUri" (encode callback)
    ]
  bytes := encodeCanonical term

refineMessage "token requested" where
  term := tuple [tagBytes 0x02, namedBytes "code" (encode code)]
  bytes := encodeCanonical term
```

This fixes framing, tags, and value relationships at an abstract byte-algebra level. A subsequent HTTP mapping might put those bytes into form fields; a protobuf mapping might use numbered fields. Either concrete alphabet must prove that decoding and re-encoding preserve the abstract terminal and its parameter bindings.

The use of `previously` is deliberately a trace predicate in this first version. It can later receive dedicated past-time syntax without changing the event model.

10.10 Probabilities and charts

The authorization service need not be random. The randomness belongs to the user, who grants or denies consent; the browser, queues, transport, and authorization service then react deterministically. As written above, however, the authorization server emits both the grant and the error, so under

the deciding-actor rule of Chapter 5 it is the server that chooses. A faithful model adds an explicit consent event from the browser to the authorization server, `ConsentGiven` or `ConsentRefused`, whose sender makes the user's choice observable. In the current implementation branch weights live in a separate overlay (Chapter 13), not in the grammar; the proposed `weightedOneOf` below expresses the same numbers inline:

```
def consentDecision : Scenario :=
  weightedOneOf "consent decision"
    [ weighted (87 / 100) authorizationGranted
    , weighted (13 / 100) authorizationDenied ]

def oauthCharts : List ChartSpec :=
  [ pie "authorization outcomes" do
      source events; groupBy terminalOutcome; value count
  , line "end-to-end authorization latency" do
      source (correlate events by "state")
      x clock; y (elapsed "authorization requested" "application response")
  ]
```

With that consent event in place, no separate field needs to say that the user "owns" this choice: the grant and denial branches begin with different consent-event bytes, sent by the browser on the user's behalf. After those bytes select a branch, every transition may have probability one and the global model is still probabilistic. The grant branch carries probability mass 0.87 through the outbound queue, transport, the server's inbound queue, and the deterministic grant response. The denial branch similarly carries mass 0.13. Deterministic processing transports probability mass; it does not remove it.

For the simplified path with no loss or timeout,

$$P(\text{grant callback}) = 0.87 \cdot 1 \cdot 1 \cdot 1 = 0.87, \qquad P(\text{denial callback}) = 0.13.$$

Here P denotes probability; each factor of one is a deterministic enqueue, transport, or server step. Dwell affects latency, not probability mass.

Unassigned scheduling choices (such as which enabled actor runs next) are nondeterministic, while consent and measured outcomes supply probabilities. The composed system is an MDP: analysis fixes or quantifies over a scheduling policy before propagating consent mass (see Chapter 7).

For several sessions in flight, the MDP state contains every actor's bounded queues and its per-`(session, task)` residuals. One user's random event can change later queue pressure and scheduling possibilities for other sessions, so the probabilistic state is the composed system state rather than a special marker attached only to the user actor.

The percentages remain model inputs or measurements, not truths inferred from the grammar. The grammar defines which behaviours are possible; the probabilistic refinement says how mass is distributed over them.

10.11 Residual state machine

This automaton is intentionally behavioural. It does not contain browser cookies, database rows, server threads, or cryptographic library state. Those details may be essential to an implementation proof, but they are refinements of this contract, not the contract itself.

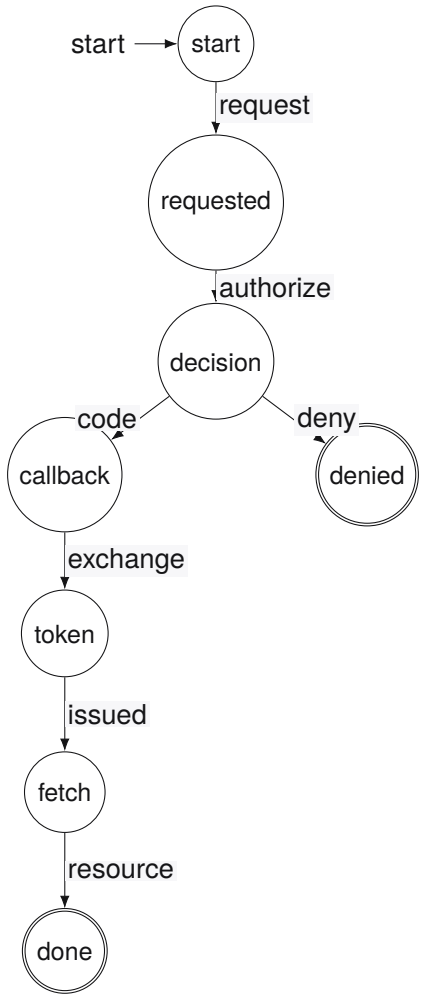


Figure 10.4: Illustrative OAuth state machine, drawn by hand. The full machine also contains the rejected-authentication branch.

Chapter 11

CTL over Generated Futures

A well-formed global type already guarantees safety, fidelity and progress for roles that follow their local types, under the assumptions of its channels. Everything else a stakeholder wants to know is a question about the system's futures: can the transfer always still finish, must it, can this message ever precede that one? The global type, together with the declared physics of its channels, generates a branching transition system. CTL speaks directly about that branching, but its vocabulary becomes clearer when placed between basic modal logic and linear temporal logic. Path quantifiers distinguish *there exists a future* from *every future*, while temporal operators distinguish the next state, eventuality, invariance, and until.

The author's modal-logic background used here was learned mostly from Kröger and Merz [13]. This citation identifies the source of that education; it is not a claim that the authors originated modal or temporal logic.

11.1 Basic modal logic

Propositional logic evaluates a formula at one world or state. Basic modal logic adds an accessibility relation R between worlds and two operators:

$$\begin{aligned} w \models \Box p \quad &\text{iff} \quad p \text{ holds at every } v \text{ such that } wRv, \\ w \models \Diamond p \quad &\text{iff} \quad p \text{ holds at some } v \text{ such that } wRv. \end{aligned}$$

Thus $\Box p$ and $\Diamond p$ read as "necessarily p" and "possibly p." Under classical negation they are dual: $\Diamond p \equiv \neg\Box\neg p$. The symbols do not fix the meaning of accessibility. Depending on the model, an accessible world may be a next computation state, a world compatible with an actor's knowledge, a normatively permitted world, or something else.

The basic normal modal logic K allows arbitrary Kripke frames. Additional frame conditions validate additional principles: reflexivity, transitivity, symmetry, and the Euclidean condition lead to familiar stronger systems. Those principles must not be imported merely because the English word "necessarily" sounds strong. LeanFM uses a directed computation-successor relation and does not silently treat it as reflexive, symmetric, or epistemic.

11.2 LTL: time along one path

Linear Temporal Logic (LTL) describes one linear path with operators including X (next), F (eventually), G (always), and U (until). Thus Fp says that p occurs at some suffix of the selected path, while Gp says that p holds at every suffix of that path.

LTL's temporal words answer *where along this path*; they do not themselves choose between one possible branch and every branch of a transition system. A model-checking convention often asks whether every initial path satisfies an LTL formula. That outer convention should not be confused with a path quantifier in the LTL syntax. For example, asking whether some legal execution eventually reaches recovery and asking whether all legal executions eventually reach recovery require different branching claims even though both contain the words "eventually recovery."

LeanFM does not currently implement LTL. It uses CTL because the requirements deliberately combine two independent axes:

- *always* or *eventually* says where along forward time a predicate must hold; and
- *necessarily* or *possibly* says whether it must hold on every permitted forward continuation or may hold on at least one.

This choice arose from trying to combine temporal logic with alethic logic while investigating whether all the familiar modal families could be combined. The construction retained by LeanFM is specifically *temporally alethic*: possible means possible now or at some point along a legal future. Ordinary language uses "always" both for universal modality and for temporal persistence. CTL keeps those coordinates visible long enough to prevent the ambiguity from disappearing inside an English gloss.

11.3 CTL: path modality combined with time

CTL pairs a path quantifier with each temporal operator. E is existential: there is at least one permitted continuation. A is universal: every permitted continuation satisfies the temporal condition. In the vocabulary of modal logic, E supplies temporal possibility and A supplies temporal necessity.

"Necessary" and "possible" are future-relative readings: they quantify over legal computations from the current observation, not timeless logical worlds. The temporal operators and path quantifiers do different work: F versus G means eventuality versus invariance along a continuation; E versus A means some versus all permitted continuations.

The four core combinations and their checking dualities are:

$EF\,p$: possibly eventually p, $\quad AF\,p \equiv \neg EG\,\neg p$: on every future, eventually p,
$EG\,p$: p always on some future, $\quad AG\,p \equiv \neg EF\,\neg p$: p holds on every future at every position.

These are exact CTL dualities. A ranges over all legal futures, while G ranges over all positions along one future; their English glosses must keep those scopes distinct. The identities support counterexample checking: test $AG\,p$ by searching for $EF\,\neg p$, and test $AF\,p$ by searching for $EG\,\neg p$. A witness refutes the universal property; absence of one establishes it only within the finite checked model.

11.3.1 Least upper bounds and greatest lower bounds

Eventuality and strong until choose the least solution of their recursive equations. Over the lattice of state sets, begin with the empty set and repeatedly add states justified by the previous approximation:

$$\mathrm{Sat}(EF\,p) = \mu Z.(P \cup \mathrm{Pre}_\exists(Z)) = \bigcup_{n\geq 0} Z_n, \qquad Z_0 = \varnothing.$$

The same construction gives existential until:

$$\mathrm{Sat}(p\ EU\ q) = \mu Z.(Q \cup (P \cap \mathrm{Pre}_\exists(Z))).$$

AF and strong AU are also least fixed points, using universal predecessor. A cycle that never reaches the target therefore adds nothing to the least solution.

Invariance and weak until make the dual choice. They begin with all candidate states and remove unsupported states, producing the greatest fixed point—the greatest lower bound of descending approximations:

$$\mathrm{Sat}(EG\,p) = \nu Z.(P \cap \mathrm{Pre}_\exists(Z)) = \bigcap_{n\geq 0} Y_n.$$

LeanFM has two evaluators. The small recursive one in `LeanFM.CTL`, used by task metrics, implements these choices with cycle-aware searches rather than by materializing every approximation set. An unresolved revisit is false for `EF`, `EU`, and strong `AU`, selecting the least fixed point. A predicate-preserving revisit is true for `EG` and weak until, selecting the greatest fixed point. `AG` is checked as $\neg EF\,\neg p$, and `AF` as $\neg EG\,\neg p$. Thus least upper bound and greatest lower bound are paired implementation choices; neither describes every CTL operator by itself. The second evaluator, in `LeanFM.Logic` and used by the MCP `check_ctl` tool, computes the approximation sets directly: it labels every reachable node with the set satisfying each subformula by least and greatest fixed-point iteration, and returns a witness or counterexample path. Tests check that the two agree.

11.3.2 Asymptotic cost

The smaller model boundary is not itself a faster model-checking algorithm. Let V be the number of reachable observations, E the successor edges, L the number of nodes in the checked CTL formula, and P the number of simple paths inspected by a search. The current Lean implementation carries a path-local `List` of visited observations. Membership in that list is linear in the current path length, and completed subgraphs are not memoized between branches. For that evaluator, a single temporal search therefore costs $O(E_P + VP)$ time, where E_P counts edge visits across the inspected paths, and $O(V + L)$ stack and path memory. P can be exponential—and in dense graphs worse than 2^V—even though the input graph contains only V states. On an acyclic tree, where each state has one path from the root, the list membership overhead alone is bounded by $O(V^2)$ and is often smaller for a shallow tree.

Read this as: “the present checker may revisit the same state once for every different path that reaches it; its cost is governed by paths, not merely by states and edges.” Formula nesting may repeat those searches because subformula results are also not cached.

The fixed-point evaluator in `LeanFM.Logic` avoids the path explosion. Each fixed point needs at most $V + 1$ iterations, each costing $O(V + E)$, so a formula of L nodes costs $O(L\,V\,(V + E))$ time; it bounds models at 4096 nodes. Worklist propagation over predecessor adjacency would reach the textbook $O(L(V + E))$; that remains an implementation target, to be accompanied by measurements on specified graph families.

No comparison with TLC follows from these bounds. LeanFM often constructs a smaller observable-event graph because it deliberately omits irrelevant private variables; a carefully abstracted TLA+ model may omit them too. A claim that LeanFM is faster than TLA+ requires the same protocol, properties, finite bounds, hardware, checker settings, and reported state/edge counts. Until that experiment exists, the book claims only a different modeling boundary, not a performance victory.

An observation is immutable once recorded. A value changes only by moving forward to a successor observation; it is never updated retroactively inside the current state. This does not mean that values are monotone—a queue may grow and later shrink—but every change occurs on a forward transition.

Formula	Reading	Typical use
$EF\,p$	possibly, eventually p	temporal possibility / witness
$AF\,p$	necessarily, eventually p	termination / response
$EG\,p$	possibly, always p	viable invariant path
$AG\,p$	necessarily, always p	safety invariant
$EX\,p$	possibly, next p	enabled successor / one-step witness
$AX\,p$	necessarily, next p	required one-step consequence
$(p\ AU\ q)$	every path keeps p until q	constrained progress
$(p\ EU\ q)$	some path keeps p until q	possible progress
$(p\ AW\ q)$	every path: p until q, or p forever	necessary weak until
$(p\ EW\ q)$	some path: p until q, or p forever	possible weak until

For example, $AF\,p$ is not merely "eventually p" and $EF\,p$ is not a weaker spelling of the same linear claim. The first requires p on every legal future generated from the current trace prefix; the second needs one legal future as a witness. Branching is part of the requirement.

Strict CTL requires every temporal operator to be governed by A or E. CTL* is a more general branching-time language in which state formulas and path formulas can be nested more freely; LeanFM does not implement CTL*. This matters when translating prose: not every arbitrary alternation of "on all paths" and "eventually" is a formula of the small CTL datatype used here.

11.4 Projecting to LTL-style F and G

Requirements often want the simpler LTL surface forms Fp and Gp. LeanFM may display or accept those forms only with an explicit path-policy boundary. On a selected linear execution, the branching coordinate has already been fixed, so AFp and EFp both project to Fp, while AGp and EGp both project to Gp. Under the common model-checking convention that an LTL formula must hold on every initial path, Fp corresponds to AFp and Gp corresponds to AGp.

Consequently, presenting frequently used AF and AG obligations as F and G is a deliberate interface reduction, not a CTL equivalence on a branching model. The durable artifact retains the exact originating CTL formula so the checker can still use existential witnesses, negation, and the correct counterexample search.

11.5 Persistence and recurrence

Nested CTL operators express two common requirements:

$$AF\,AG\,p, \qquad AG\,AF\,p.$$

Read the first, symbol by symbol: “On every path, eventually reach a state from which, on every path, p always holds.” This is eventual permanent safety: for example, eventually finish key rotation and never again accept the old key. Read the second: “On every path, at every state, on every path from that state, eventually p holds.” This is universal recurrence, often pronounced “always eventually p”: for example, service keeps returning to a ready state.

For witness searches their duals are:

$$\neg(AF\,AG\,p) \equiv EG\,EF\,\neg p, \qquad \neg(AG\,AF\,p) \equiv EF\,EG\,\neg p.$$

Read: “Failure of eventual permanent safety means some path always retains some future reaching not-p. Failure of universal recurrence means some future reaches a state with some path on which not-p holds forever.”

The corresponding existential forms are $EF\,EG\,p$ and $EG\,EF\,p$. Read: “Some path eventually reaches a state with some path keeping p forever,” and “some path always visits states from which some path eventually reaches p.” In the second formula those eventual witnesses may leave the outer path; it does not require infinitely many occurrences of p on that same path.

These forms cover useful LTL-style persistence and recurrence, not all LTL. Under universal LTL path semantics, GFp corresponds to $AGAFp$ on the usual total transition model. But $AFAGp$ is stronger than pathwise FGp: it requires eventually reaching a state where *all* continuations stay safe, not merely a separate stabilization point on each path. Terminal states require the same explicit continuation convention used by the core checker.

11.6 Knowledge, obligation, and permission

General modal logic need not have only one accessibility relation. A multimodal model can use $\Box_t/\Diamond_t$ for temporal reachability, $K_a/\widehat{K_a}$ for what actor a knows or considers possible, and a different pair for permission or obligation. Dynamic logic can index modalities by programs. Tense logics can add past-directed relations. These systems share the Kripke-style idea of evaluating modal formulas through accessibility, but their modalities are not interchangeable.

LeanFM’s CTL A and E range only over legal forward computations generated by the current model. They do not mean:

- probability one versus positive probability;
- known versus merely believed by an actor;
- obligatory versus permitted by a policy; or
- physically necessary versus logically possible.

Knowledge and policy relations are documented modelling techniques, not built-in operators. Projects may define them explicitly using the relations below. The Kerberos video [16] illustrates their use; it does not verify the protocol.

For knowledge, let W be the declared set of legal histories and $\mathrm{obs}_a(h)$ actor a's local observation of history h:

$$K_a\varphi(h) \iff \forall h' \in W.\ (\mathrm{obs}_a(h) = \mathrm{obs}_a(h') \Rightarrow \varphi(h')).$$

Read: "Actor a knows φ at history h if and only if, for every history h' in W, if a observes the same thing in h and h', then φ holds at h'." Observations must declare visible messages, private keys, clock information, and retained memory. With the current history included in W, knowledge is factive: what is known is true in that model. Belief needs a different relation; it must not inherit this guarantee accidentally.

$$\widehat{K}_a\varphi \equiv \neg K_a \neg\varphi.$$

Read: "a considers φ possible exactly when a does not know that φ is false." This is epistemic possibility, not temporal reachability.

For policy, let $\mathrm{Ideal}_a(h)$ be the declared policy-compliant futures from h, and let ψ be a condition on a whole future path:

$$O_a\psi(h) \iff \forall \pi \in \mathrm{Ideal}_a(h).\ \psi(\pi), \qquad P_a\psi(h) \iff \exists \pi \in \mathrm{Ideal}_a(h).\ \psi(\pi).$$

Read: "a is required to satisfy ψ at h exactly when every ideal future π satisfies ψ; a is permitted to satisfy ψ exactly when some ideal future satisfies ψ." Require a nonempty set of ideal futures: contradictory policies must be reported, not certified by vacuous quantification. Actual legal executions may violate policy; detecting that violation is part of the analysis. These are obligations under a named policy, not judgments of morality.

$$P_a\psi \equiv \neg O_a \neg\psi.$$

Read: "ψ is permitted exactly when its negation is not required." This duality applies only when both operators use the same declared policy relation. For example:

$$O_a F\ \mathrm{audit}(r)(h) \iff \forall \pi \in \mathrm{Ideal}_a(h).\ \exists i \geq 0.\ \mathrm{audit}(r, \pi_i).$$

Read: "a must eventually audit request r at h exactly when, for every ideal future π, there is a position i, zero or later, where r is audited." Here F is a path operator; a project implementation must distinguish state formulas from path conditions.

Temporal, epistemic, and policy quantification can then be combined without confusing their domains. A legal future may be possible, unknown to Alice, and forbidden by policy at the same time. Probability, belief, strategic ability, and program-indexed modalities remain separate design work; adding named relations does not implement every modal logic.

Past predicates remain useful for security: a ticket must have been issued before it is accepted. Compile bounded past conditions into forward monitor state where an equivalence can be established. Evaluate "previously issued" before updating the monitor with the current event. On concurrent histories, declare whether "before" means causal ancestry or a chosen linearization; screen order is not evidence. Unbounded past logic is not thereby implemented.

Symbolic uncompromised-key assumptions do not establish computational cryptographic security. General knowledge and policy checking is deferred.

11.7 Possibility means preparedness

There is a practical consequence to treating possibility as a first-class temporal mode. When a requirement declares $EF\,p$—"maybe possible p"—it does more than acknowledge that a model checker can find an accidental path to p. It says that p is a supported future for which the system must prepare.

A `possibly` obligation therefore contains three linked artifacts:

1. a legal grammar branch and witness trace reaching the condition;
2. a handling plan in `Requirements.lean`; and
3. generated implementation code whose justification cites that possibility requirement.

This is the target; today validation checks only that a handling plan is cited, not that generated code exists or covers the witness.

For example, "a bakery delivery may fail" requires a decodable `DeliveryFailed` message, a legal failure branch, and code that performs the specified refund or notification behavior. Removing every failure branch does not implement reliability; it makes the real failure trace out-of-spec.

Possibility is distinct from probability. A possible condition may have a small estimated probability, an expected distribution, or no known probability at all. Conversely, assigning positive probability to a condition makes its branch possible. The handling obligation comes from supported possibility, not from whether the estimated percentage crosses a threshold.

The separate operational obligation applies to $EG\,p$—"p always in some cases." It identifies at least one future on which p persists. The system must pay attention to the conditions that keep that future viable: monitor them, expose their loss, and, where required, control choices that would destroy the invariant path. $EG\,p$ alone does not promise that every execution preserves p; that stronger claim is $AG\,p$.

11.8 Strong implication and the Chewbacca argument

The name "Chewbacca defense" is a cultural reference to the *South Park* episode "Chef Aid," whose courtroom parody uses an irrelevant claim about Chewbacca to distract from the conclusion at issue. The reference supplies a memorable example of non sequitur; it is not a technical term originating in modal or temporal logic.

Material implication is weak in the precise sense that $p \to q$ is true whenever p is false. That is useful inside a state, but it permits a vacuous global claim when no reachable state ever satisfies the antecedent. LeanFM names the non-vacuous temporal combination

$$p \Rightarrow_s q \quad \stackrel{\text{def}}{=} \quad EF\,p \ \wedge\ AG(p \to q).$$

The first conjunct requires a legal future witnessing p. The second requires that, necessarily always, every reachable occurrence of p also satisfies q. Together they imply $EF\,q$: the p witness cannot avoid q.

The deliberately absurd test uses

$p =$ “Chewbacca lives on Endor”,
$q =$ “you must acquit”.

A Chewbacca argument cannot obtain q merely by mentioning p. To enter the requirements as strong implication, the model must admit a reachable p witness and establish $AG(p \to q)$ over every permitted continuation. If the observable grammar supplies no relation between residence and acquittal, that second obligation has no justification and is rejected. Logic cannot determine the meaning or relevance of arbitrary English propositions by itself; the useful point is that neither a missing antecedent nor a single rhetorical occurrence of it can make the formal implication pass vacuously.

In Lean the definition is direct:

```
def stronglyImplies (p q : CTL S) : CTL S :=
  CTL.and (CTL.ef p) (CTL.ag (CTL.implies p q))
```

The path quantifier is part of the until operator. We write `AU` and `EU` as binary algebraic operators, not an outer `A` or `E` wrapped around a bracketed until formula. Consequently a subexpression never changes meaning merely because another expression surrounds it:

```
def boundedUntilTerminal : CTL Observation :=
  atom withinCapacity AU atom isTerminal

def someAuthorizedContinuation : CTL Observation :=
  atom hasValidAuth EU atom isSucceeded
```

11.9 Weak until: AW and EW

Strong until requires the right operand eventually to occur. Weak until permits it never to occur, provided the left operand continues to hold. Thus

$p\ AW\ q$: on every future, p holds until q, or p remains forever,
$p\ EW\ q$: on some future, p holds until q, or p remains forever.

Read the first line as: “on every future, p holds until q, or if q never occurs, p remains forever.” Read the second with “on some future.” AW and EW are the universal-path and existential-path weak-until operators.

The boundary state satisfying q need not also satisfy p; p is required only before q. On a future where q never occurs, every state must satisfy p. This makes weak until useful for requirements such as “necessarily remain available until shutdown, while shutdown is permitted but not required.”

For existential paths, $p\,EW\,q$ is equivalent to $(p\,EU\,q) \vee EG\,p$. The analogous-looking $(p\,AU\,q) \vee AG\,p$ is *not* a correct definition of AW on a branching system. Some futures may reach q while other futures preserve p forever; neither global disjunct need hold even though every individual future satisfies weak until. LeanFM therefore implements `AW` directly.

```
def serviceMayRunUntilShutdown : CTL Observation :=
  atom available AW atom shutdown

def someWorkerCanRemainReady : CTL Observation :=
  atom ready EW atom retired
```

This convention also makes tree structure explicit. In `r && (p AU q)`, the right operand is the complete `AU` expression; there is no context-sensitive reinterpretation of `[p U q]`.

For a request event carrying identifier r, an expected response property might say that on every continuation a matching success or failure eventually occurs. Authentication might say that every successful use of a session has an earlier issuance event with the same session identifier.

11.10 Past claims

CTL is future-facing at the current state, but security and protocol claims often refer to evidence in the past: “this capability was previously issued” or “this response matches an earlier request.” Since states are trace prefixes, past facts can initially be expressed as predicates over the accumulated trace. This gives a clear baseline without prematurely committing the surface language to a particular past-time temporal logic.

There is still an open design question. Explicit past operators may make properties more readable and allow better checking algorithms. The project should compare trace predicates, CTL with past operators, and compilation of finite-history monitors into state. The criterion is maintainability of specifications as the event schema evolves.

11.11 Fairness and time bounds

AF does not by itself distinguish a participant that is perpetually denied a turn from one that violates the protocol. Fairness assumptions must therefore be named, scoped, and visible in theorem statements. Similarly, “eventually” is not “within 200 milliseconds.” Quantitative bounds belong in an explicit timed layer rather than being smuggled into timestamps and informal prose.

Chapter 12

A Legal Example: Lawful, Right, Known, Required

The knowledge-closure discussion of Section 17.5 needed two questions beyond temporal logic: *who knows which values*, and *who is permitted to hold them*. Law adds more pressure on the same machinery. Obligations have deadlines; they are triggered by what someone knew; breaking one creates another; and two codes can disagree. This chapter works one small legal example end to end so that each modality is used for something concrete. Its tables and diagram are generated by `LegalDepositExample.lean` from the event histories, not drawn by hand.

The statute below is invented, and so is the moral code. Neither describes any real jurisdiction, and nothing here is legal advice.

12.1 Three meanings of "legal"

LeanFM already uses the word *legal* for histories that the global type admits. A legal example needs two further meanings, and conflating them is the most common error in normative reasoning about software.

Term	Judged against	Operators
admissible	the protocol grammar: what can happen at all	CTL A, E, F, G over generated futures
lawful / unlawful	a declared statute's ideal futures	O_a^{Act}, P_a^{Act}
right / wrong	a declared moral code with a named author	O_a^{Code}, P_a^{Code}

In this chapter "admissible" replaces "legal" wherever the grammar is meant. An unlawful act is usually admissible: people can break the law, so $EF\ unlawful$ is normally true, and that is not a defect of the model. Statutes constrain which futures are *ideal*, not which futures are *possible*. A moral code is handled with exactly the same machinery and a different declared relation. LeanFM never decides what is right; it checks histories against a code that someone wrote down and signed, and it shows where codes disagree.

12.2 The Model Tenancy Deposit Act and a declared code

The actors are a tenant T, a landlord L, a deposit registry, the post and a court. Writing $Ideal_a^N(h)$ for the futures that code N regards as ideal for actor a after history h, the obligation and permission operators of the previous chapter are indexed by the code:

$$O_a^N\psi(h) \iff \forall \pi \in Ideal_a^N(h).\ \psi(\pi), \qquad P_a^N\psi(h) \iff \exists \pi \in Ideal_a^N(h).\ \psi(\pi).$$

Read: "Under code N, a must make ψ true when every future that N calls ideal does so, and may make ψ true when at least one does." Prohibition is obligation of the negation: $Forbidden_a^N\psi \equiv O_a^N\neg\psi$. $F_{\leq d}$ is a bounded path operator: within d days.

Act §1 (protection). Within 30 days of receiving a deposit, the landlord registers it.

$$AG\,(paid \to O_L^{Act}\,F_{\leq 30}\,protected)$$

Act §2 (return, triggered by knowledge). Once the lease has ended *and the landlord knows the tenant's forwarding address*, the landlord returns the deposit or itemises deductions within 14 days.

$$AG\,((ended \wedge knows(L, address)) \to O_L^{Act}\,F_{\leq 14}\,(returned \vee itemised))$$

Act §3 (reparation). If §2 is violated, the landlord pays the deposit plus an equal penalty.

$$AG\,(violated_{\S 2} \to O_L^{Act}\,F\ penaltyPaid)$$

Act §4 (prohibition). A deposit is never applied to rent.

$$O^{Act}\,G\,\neg offset \ \equiv\ Forbidden^{Act}\,F\ offset$$

Code (declared by the stakeholder). (a) Never charge for normal wear and tear. (b) A landlord who holds another way to reach the tenant makes a reasonable effort to use it. (c) Help a tenant in documented hardship when it costs nothing you are owed.

12.3 Knowledge from the causal past

Act §2 is conditioned on what the landlord knew. In LeanFM that is computed, not asserted. Every event carries `prevIds` and `causeIds`; an actor knows a value at an event when a message carrying it was delivered to that actor somewhere in the event's causal ancestry:

$$knows(a, v)@e \iff \exists e' \in past(e) \cup \{e\}.\ dst(e') = a \wedge v \in derive(msg(e'))$$

Read: "a knows v at event e exactly when some event in e's causal past delivered to a a message from which v can be derived." This is *possession* knowledge, the kind security protocols need. The

epistemic operator K_a of the previous chapter, which quantifies over histories that a cannot tell apart, is reserved for claims such as "the landlord could not have known" that range over alternatives.

The *lost in the post* history shows why the distinction matters. The tenant posts the address; the post never delivers it. The tenant then records that she *believes* §2 was violated. She is wrong: the landlord's causal past never contains the address, so §2 is never triggered. Belief is an event in the tenant's own history and is not factive; knowledge is computed and is. The same history is nonetheless *wrong* under Code (b): the landlord held the tenant's email from the lease (event e1 is in every causal past) and made no effort to use it.

Id	Day	From → To	Message	Detail	prevIds	causeIds
e1	0	Landlord → Tenant	`LeaseSigned`	rent 1000, deposit 1000, tenant email on file		
e2	1	Tenant → Landlord	`DepositPaid`	1000	e1	e1
e3	12	Landlord → Registry	`DepositProtected`	scheme ref 7731	e2	e2
e4	365	Tenant → Landlord	`LeaseEnded`	keys returned	e2	
e5	366	Tenant → Post	`AddressPosted`	letter to landlord	e4	
e6	400	Tenant → Tenant	`BelievesViolated`	no refund after 34 days	e5	

Figure 12.1: The *lost in the post* history. No event delivers `ForwardingAddress` to the landlord, so $knows(L, address)$ never holds and Act §2 is never triggered.

12.4 Violation and reparation

A naive deontic logic breaks on the late return. If the landlord ought to return the deposit by day 380, and ought to pay a penalty if he does not, then after day 380 he seems to be both obliged to have returned it and obliged to pay for not having done so. This is the contrary-to-duty problem, best known as Chisholm's paradox.

Ideal futures dissolve it, because they are computed *from the current history*. Before day 380 every ideal future contains a timely return. After the deadline passes, no admissible future contains one, so the ideal futures from that history are the best ones still admissible: those in which the penalty is paid. The obligation set changes as the history grows; it never becomes inconsistent. The checker therefore reports three outcomes, not two: *fulfilled*, *violated*, and *violated, then repaired*.

12.5 Lawful, right, and the four quadrants

Table 12.1 evaluates every rule in every history. Table 12.2 summarises each history under each code. All four combinations of lawful and right appear, which is the point of keeping the codes separate:

- *lawful but wrong*: an on-time return that deducts for normal wear (Code a), and the lost address the landlord could have chased (Code b);
- *violated, then repaired*: the late return, under the Act, with no moral duty arising;
- *unlawful but right*: the landlord who, on a laid-off tenant's request, applied the deposit to the final month's rent. That breaks Act §4 and fulfils Code (c).

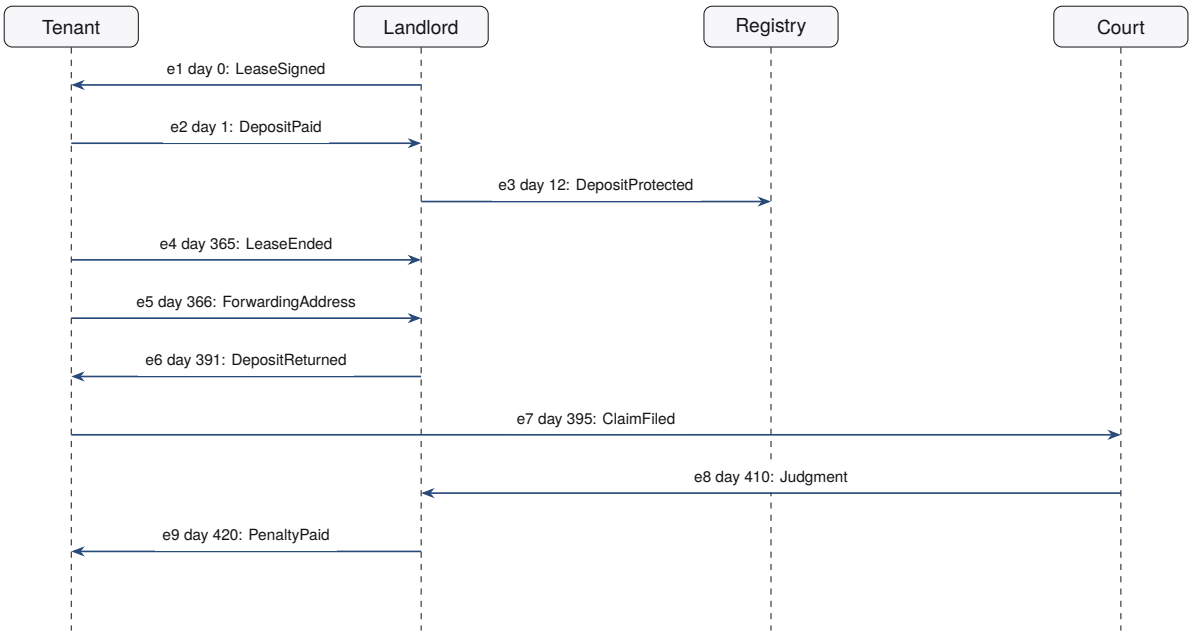


Figure 12.2: The *late return* history, rendered by `MDPDiagram.renderInteractionTeX` from its events. The address reaches the landlord on day 366, so the §2 deadline is day 380; the return on day 391 violates it, and the penalty on day 420 repairs it.

History	Act §1	Act §2	Act §3 (CTD)	Act §4	Code (a)	Code (b)	Code (c)
Itemised deduction on time	fulfilled	fulfilled	not triggered	fulfilled	violated	not triggered	not triggered
Returned late, penalty paid	fulfilled	violated, then repaired	fulfilled	fulfilled	not triggered	not triggered	not triggered
Address lost in the post	fulfilled	not triggered	not triggered	fulfilled	not triggered	violated	not triggered
Hardship: deposit used as last rent	fulfilled	fulfilled	not triggered	violated	not triggered	not triggered	fulfilled

Table 12.1: Every rule in every history, generated by `LegalDepositExample.lean`. §3 is a contrary-to-duty rule: it is triggered only on histories where §2 was violated.

The hardship history is a genuine dilemma: the Act requires $G\,\neg \mathit{offset}$ and the Code requires $F\ \mathit{offset}$, so no future is ideal under both. LeanFM reports the conflict:

$$\mathit{Conflict}(N_1, N_2, a, \psi, h) \iff O_a^{N_1}\psi(h) \wedge O_a^{N_2}\neg\psi(h).$$

It does not resolve it. Choosing a priority—law over conscience, or the reverse—is a stakeholder decision. If one is made, it belongs in the typed requirements as an explicit ordering, where a reviewer can see it.

12.6 What CTL says, and what it does not

CTL's A and E range over admissible futures, so they cannot by themselves express "ought". They still carry most of the engineering weight once the normative facts are computed as state predicates, the same technique the knowledge-closure discussion (Section 17.5) uses for knows, allows and $\mathit{requires}$:

- A deposit registry's own conduct is ordinary safety: $AG\,\neg(\mathit{receiptIssued} \wedge \neg \mathit{protected})$.

Id	Day	From → To	Message	Detail	prevIds	causeIds
e1	0	Landlord → Tenant	`LeaseSigned`	rent 1000, deposit 1000, tenant email on file		
e2	1	Tenant → Landlord	`DepositPaid`	1000	e1	e1
e3	12	Landlord → Registry	`DepositProtected`	scheme ref 7731	e2	e2
e4	365	Tenant → Landlord	`LeaseEnded`	keys returned	e2	
e5	366	Tenant → Landlord	`ForwardingAddress`	12 Elm St	e4	
e6	391	Landlord → Tenant	`DepositReturned`	1000	e5, e3	e5
e7	395	Tenant → Court	`ClaimFiled`	Act s2 deadline day 380 missed	e6	e6
e8	410	Court → Landlord	`Judgment`	pay penalty 1000 under s3	e7	e7
e9	420	Landlord → Tenant	`PenaltyPaid`	1000	e8	e8

Figure 12.3: The same history as events. `causeIds` pair each response with its cause (the return answers the address; the judgment answers the claim); `prevIds` order each actor's own events.

History	Under the Act	Under the Code	Landlord knows address from	Normative conflict
Itemised deduction on time	lawful	wrong	day 366	none
Returned late, penalty paid	violated, then repaired	no duty arose	day 366	none
Address lost in the post	lawful	wrong	never	none
Hardship: deposit used as last rent	unlawful	right	day 366	$O_L^{Act} G \neg offset$ (Act §4) and $O_L^{Code} F\ offset$ (Code c): no future is ideal under both

Table 12.2: Each history under the Act and under the declared Code, when the landlord first knew the forwarding address, and any normative conflict.

- Remedies must stay available. Following *possibility means preparedness*, a violation requires a supported path to a claim: $AG\,(violated_{\S 2} \rightarrow EF\ claimFiled)$, with a handling plan in the requirements.
- The *late return* history is simultaneously a *counterexample* for the universal claim that §2 is always met and a *witness* for $EF\ repaired$. Checker output should name both roles.
- Knowledge never exceeds permission, the security invariant, has the same shape here: a registry must not disclose the tenant's address beyond those permitted, $AG\,(knows(a, address) \rightarrow permitted^{Act}(a, address))$.

What CTL does not provide is the per-code quantification over ideal futures, bounded deadlines measured from a knowledge-triggered start, or the contrary-to-duty recomputation. The example computes those directly over each history.

12.7 Computing what is owed: Catala

The deposit Act mixes two kinds of question. *What* does the law compute from the facts—how much is returned, what deductions are allowed, what penalty applies? And *when* did duties arise, which deadlines passed, and was a violation repaired? This chapter answers only the second kind.

The first kind is the territory of Catala [19], a programming language for literate law: the statute's own text and its formalization sit side by side, and its default logic expresses the ordinary legal shape "the rule is X, except when Y, except when Z" directly. That exception structure is Catala's strength, and it is handled only crudely here by a priority between codes. Catala's project also maintains a formally specified library of date computations; legal deadlines such as "within one month" of 31 January, or counts of business days, need exactly that precision before a time-bounded verdict such as $AF_{\leq 14\,\mathrm{days}}$ can be trusted on a real statute.

The two approaches compose rather than compete. Catala computes the amounts and statuses the law prescribes; the grammar and CTL of this chapter check, over a history of observable events, who owed what by when and what happened when it was not done. A verdict remains conditional on its premises: the code that governs is chosen, not derived; open-textured terms such as "reasonable effort" or "normal wear" are declared atoms whose values a person supplies; and the facts are only as good as the event history.

12.8 Status

`LegalDepositExample.lean` is an executable book example, not a general normative checker. It hard-codes the four rules and three code clauses. Turning it into a LeanFM language feature needs a typed form for a normative code: its author; each rule's actor, trigger (a state formula that may use $knows$), deadline, path condition and reparation; and an optional priority between codes. The checker would return a status per rule per history with its witness or counterexample trace. Like every LeanFM feature, that form gets checked requirements and a checked implementation mapping before any checker code. Regenerate this chapter's tables and diagram with `lake env lean -run LegalDepositExample.lean`; the provenance record is in `book/generated/legal-provenance.txt`.

Chapter 13

Generated Examples and Probability Overlays

Most specifications explain themselves with hand-written examples: a trace someone typed, a sequence diagram someone drew. Those examples drift. This chapter takes the opposite approach. Every example below is the answer to a question asked of the specification, and every number is computed from it. Nothing in the figures was drawn by hand; regenerate them with `lake env lean -run ProbabilityOverlayExample.lean`.

A global type generates runs, and each run is an example of the conversation. The reading of the temporal operators is generative. If we generate event lists from the grammar, then

$EF\,p$ means some generated list contains p; the witness is that list, and it is the example.

$AG\,p$ means every generated list satisfies p; a counterexample is a generated list that does not.

$AF\,p$ means every generated list eventually reaches p; a counterexample is a list that loops forever without it.

13.1 The core says what can happen

A client sends a `Request`. The server answers with a `Response`, or a `Timer` fires a `Timeout` and the client retries. After the third timeout the client gives up. Retries are bounded, so the task unrolls them into states: one waiting state per attempt.

The core carries no numbers. It says only which messages can follow which, and it already answers the qualitative questions: every run ends, and giving up is possible.

13.2 The overlay says how likely and how long

A separate table, the *overlay*, annotates each transition with a branch probability and a delivery delay. This one is *declared*: it states what we expect (a 10% timeout rate, 40 ms responses, 200 ms timeouts). A *measured* overlay would have exactly the same shape, reduced from recorded traces instead of asserted, and the same questions could be asked of either. Keeping the numbers out of the grammar means a project without probabilities loses nothing, and declared and measured values can be compared on one unchanged specification.

The overlay is validated before use: every transition must be annotated, nothing else may be, and the probabilities leaving each state must sum to exactly 1. They are exact fractions and are never normalized silently.

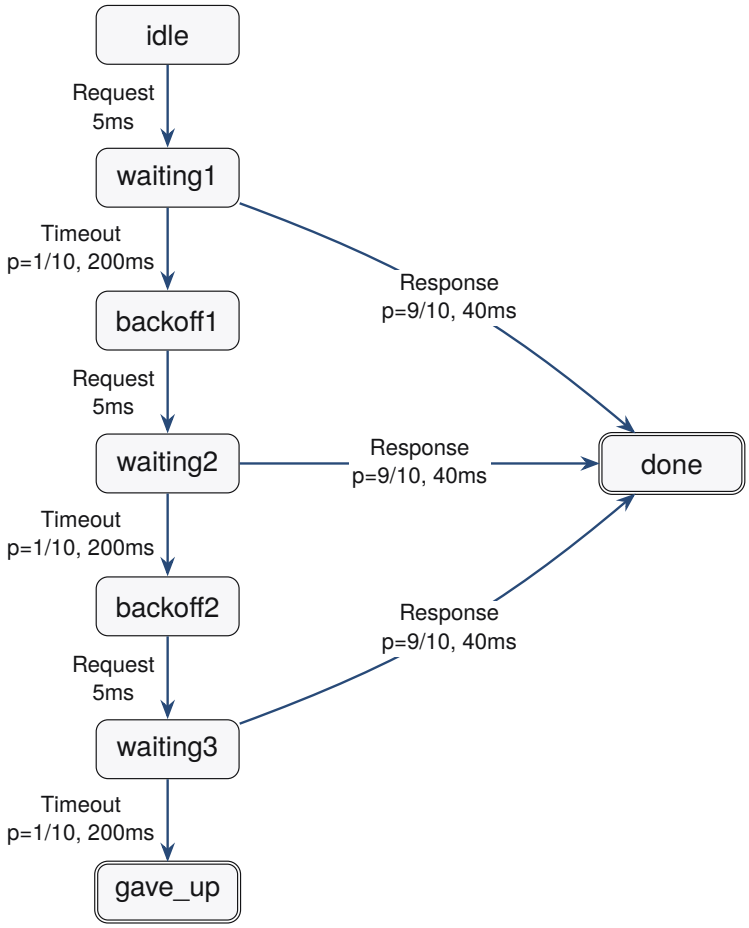


Figure 13.1: The bounded-retry task (the core) with the declared overlay drawn on its edges. Double borders mark terminal states.

Keeping probabilities outside the model is a deliberate break with tools such as PRISM [14], where branch probabilities are part of the model itself, and it follows FizzBee [1], which leaves the specification unchanged and supplies probabilities and costs in a separate performance file. The same separation lets one specification be checked under a declared overlay and a measured one.

13.3 Questions and their generated answers

Question	Formula	Answer
Does every run end? (core, no overlay)	`AF terminal`	proved
Can the client give up?	`EF gave_up`	proved; most likely witness has p=1/1000, 615 ms
How likely is success?	`P=? [F done]`	999/1000 (0.999)
Do 99% succeed within 300 ms?	`P>=99/100 [F<=300ms done]`	proved; exactly 99/100
Do 99.9% succeed within 300 ms?	`P>=999/1000 [F<=300ms done]`	disproved; most likely late run p=9/1000, 455 ms
Expected duration of a run	`E[duration]`	67.71 ms

Table 13.1: Every answer is computed from the task and the declared overlay. The first row uses the core alone.

The table reads like a service-level review. Success is overwhelmingly likely ($999/1000$). Ninety-nine percent of runs finish within 300 ms, and that bound is met *exactly*, with no margin. Tightening it to 99.9% fails, and the checker does not just say so: it produces the most likely run that misses the deadline.

13.3.1 The usual run

The most likely generated run is the one a newcomer should see first.

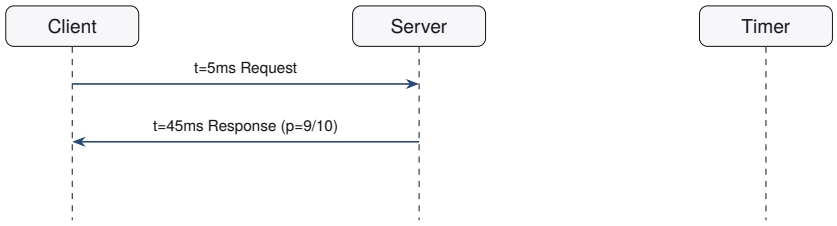


Figure 13.2: The most likely run: one request and one response, 45 ms, probability $9/10$.

13.3.2 A precedent: the client gives up

$EF\ gave_up$ holds, and its witness is a concrete case. It is rare, with probability $1/1000$, but it is possible, and this is what it looks like.

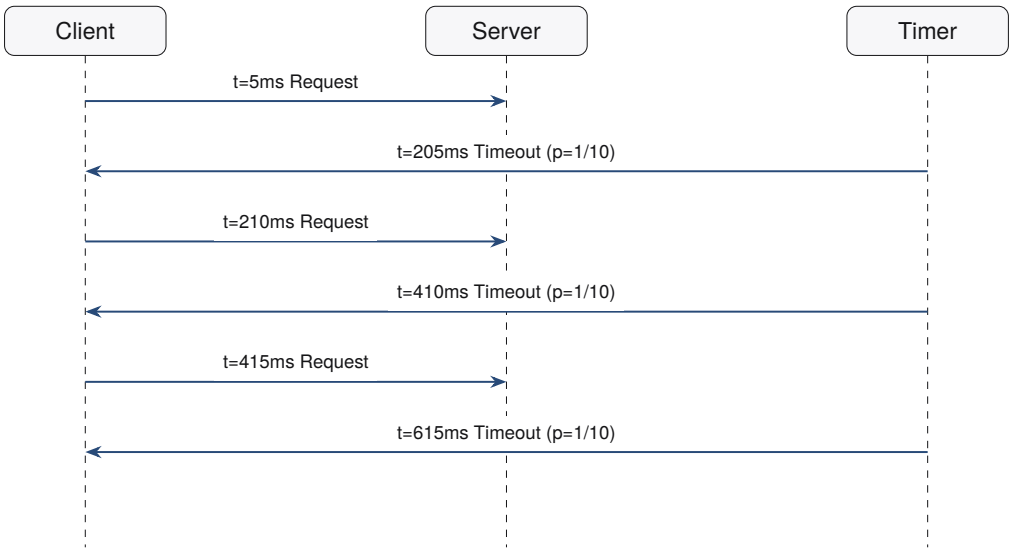


Figure 13.3: The witness for $EF\ gave_up$: three timeouts, 615 ms, probability $1/1000$.

13.3.3 A counterexample: the late success

The claim $P_{\geq 999/1000}[F_{\leq 300\,\mathrm{ms}}\ done]$ fails. Of all the runs that miss the deadline, the most likely is not the give-up run but a success on the third attempt. That is the run worth fixing first.

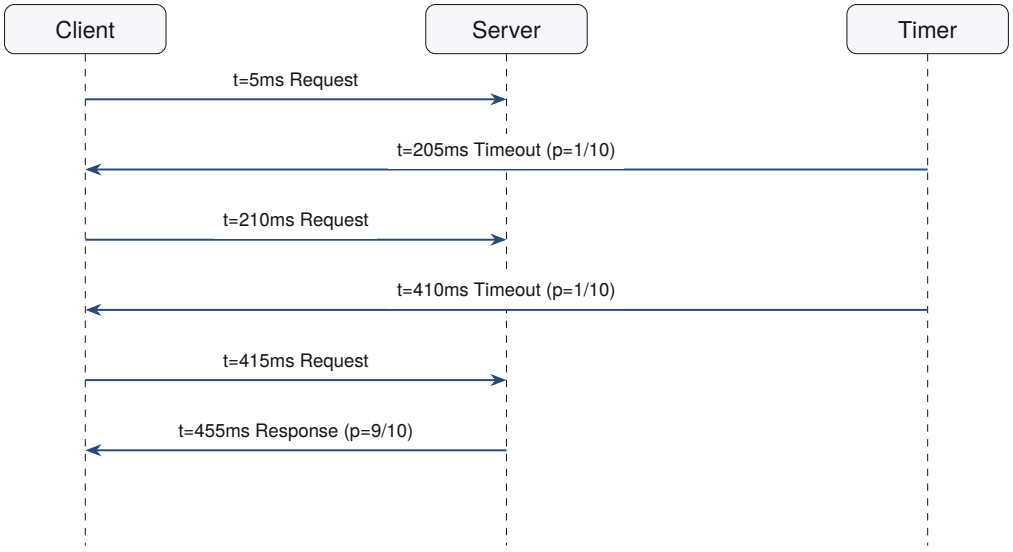


Figure 13.4: The most likely counterexample to the 99.9% / 300 ms objective: success on the third attempt at 455 ms, probability $9/1000$.

13.4 The distribution behind the numbers

The completion-time distribution is computed exactly by enumerating the generated runs; it is not sampled. The logarithmic scale shows the long tail that a mean of 67.71 ms hides.

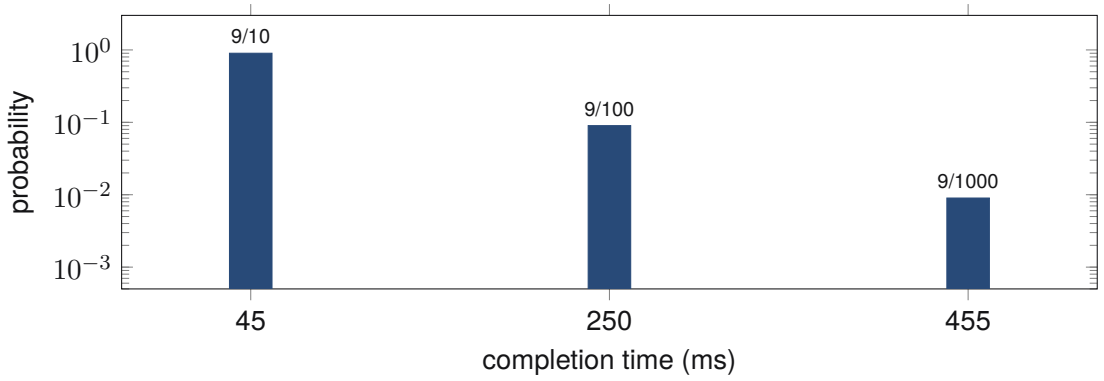


Figure 13.5: Completion times of successful runs under the declared overlay. The remaining $1/1000$ gives up at 615 ms.

13.5 What comes next

The same functions accept a measured overlay, reduced from recorded traces by counting how often each branch was taken and averaging the delay before each message. Chapter 4 does exactly that for 20,000 recorded sales, compares the result with a declared overlay, and shows why measured delays must eventually be distributions rather than means. Retry loops here are unrolled into bounded states; unbounded loops need the linear-equation solution of probabilistic model checking rather than run enumeration, and the current implementation reports them instead of truncating them.

Chapter 14

Feature Completion as Positive and Negative Proof

A request to show a message, list source, accept a transaction, render a diagram or produce an MP4 states a desirable observation. The change can also introduce stale state, disclosure, resource exhaustion, weaker authentication, corruption or success without an artifact. Completion therefore requires a paired argument:

1. a *positive invariant* states what the system must do; and
2. applicable *negative invariants* state what the system must never do.

For a feature that crosses a boundary, both kinds of statement are about the conversation: the positive invariant is a property of the global type, and the negative ones usually name a message that must never be sent, or never accepted, under the declared physics of a channel (a replayed request, a response to the wrong session, a call that arrives after its deadline).

Review before declaring completion, in proportion to the boundary crossed: button spacing rarely needs a cryptographic audit; filesystem access needs authorization and path analysis. A material risk without a mapped and tested control blocks completion.

14.1 The logical shape of a feature

Let $good_f$ mean the observable contract of feature f, and let $bad_{f,1}, \ldots, bad_{f,n}$ name reachable failures introduced or exposed by that feature. A common safety-shaped completion condition is

$$AG\, good_f \;\wedge\; \bigwedge_{i=1}^{n} AG\, \neg bad_{f,i}.$$

The first conjunct is positive: the requested behavior remains true throughout every permitted execution. Each remaining conjunct is negative: no permitted execution reaches a forbidden observation. The equivalent counterexample form

$$AG\, \neg bad \;\equiv\; \neg EF\, bad$$

is operationally useful. A negative test tries to construct the $EF\,bad$ witness: another user's project path, a stale response after a project switch, an oversized upload, a duplicate response, or a secret in a log. Finding such a witness refutes the claimed completion.

Not every feature is purely invariant. "A submitted request eventually gets a terminal response" is a liveness obligation such as $AG(submitted \rightarrow AF(complete \vee failed))$. A feature review must not replace liveness with safety or hide fairness and availability assumptions. Positive and negative requirements classify intent; the temporal operator must still match the claim.

14.2 Stability review before security review

Before editing code, identify the existing invariant and review the applicable ways an environment change could break it:

Regression. Which accepted behavior, file format, route, proof, or UI state can this change invalidate? Preserve conforming behavior and add a regression test at the narrowest durable boundary.

Failure. What happens when a dependency, renderer, compiler, provider, filesystem operation, or network request fails? Failure must become an explicit terminal observation rather than a false success or endless wait.

Concurrency. Can a late response, simultaneous session, queued prompt, or overlapping write be attributed to the wrong project or revision? Correlation identity and revision checks must survive reordering.

Resources. What bounds memory, bytes, queue occupancy, recursion, process time, media dimensions, and response size? A new feature must not silently turn a finite resource contract into an unbounded one.

Durability and recovery. Which state is authoritative after restart or partial failure? Atomic writes, rebuildable indexes, and project-local ownership make recovery inspectable.

Compatibility. Which clients, stored revisions, generated artifacts, and build environments must continue to work? Version or migrate a changed representation rather than reinterpreting old bytes silently.

Retain concrete invariants at the changed boundary, briefly recording when a dimension is non-material.

14.3 Security as forbidden observable behavior

"A user never reads a project they do not own" states protected behavior; "use authentication" only names a mechanism. Before selecting a control, identify:

1. the trust boundary crossed by the feature;
2. the asset being protected;
3. the attacker's relevant capability; and

4. the forbidden observable outcome.

Applicable forbidden behaviors include unauthorized access, cross-user or cross-project disclosure, secret persistence or logging, injection, path or symlink escape, replay, request/response confusion, privilege expansion, executable media, resource exhaustion and fail-open behavior. State testable invariants, such as “a source path absent from the authorized project inventory never returns bytes.”

Map each material negative invariant to a control—canonical-path confinement, authenticated project identity, an allowlisted inventory, a byte limit or an atomic revision comparison—and a negative test or proof. A mechanism alone does not establish the forbidden transition’s absence.

14.4 The feature-completion ledger

For each feature, retain the following compact traceability record in the typed requirements and implementation mapping:

Field	Required content
Prompt	The stakeholder request whose completion is being claimed.
Positive claim	Observable always or liveness property, with actor, scenario, boundary, and revision.
Negative claims	Applicable never or always-not properties; omitted families are explicitly judged non-material.
Controls	Concrete implementation mechanisms mapped one-to-one or many-to-one to the negative claims.
Evidence	Positive conformance tests plus negative tests, proofs, generated traces, and named checks.
Residual risk	Stated limitation the stakeholder reviews in the diff, assumption, deferred mitigation, or an explicit completion blocker.

Both sides need evidence: forbidding failures does not demonstrate the requested feature, and a successful demonstration does not rule out disclosure or corruption.

14.5 Worked example: the project Source view

Consider the request “show implementation files in the Source tab.” The positive claim is that selecting an authorized project lists its implementation files and selecting one readable UTF-8 file displays its content. Project switching clears the old selection, so a late response cannot repopulate the new project with stale content.

The feature crosses a filesystem boundary. Project source, ignored account records, repository metadata, and provider credentials are protected assets. A user can choose a URL path and can guess names that the visible tree never listed. Applicable negative invariants therefore include:

- a user never reads another user’s project source;
- a guessed ignored or metadata path never returns bytes;

- a path or symlink never escapes the authorized source root;
- a binary or oversized file never becomes an unbounded text response;
- a project switch never retains the previous project's source state; and
- an empty source directory never accompanies a claim that implementation succeeded.

Controls are an authenticated explicit project ID, canonical root resolution, exact source-inventory membership, UTF-8 reads, a finite response limit and project-keyed React state. Negative tests request another account's file and a guessed `.git/config`; both return not-found without revealing existence. The positive test creates, lists and opens a project-local file and checks its exact content.

14.6 Worked example: chat auto-scroll

Even a small convenience has stability boundaries. The positive invariant is that initial conversation load and every newly rendered user, assistant, or failure message reveal the bottom of the chat message pane. The implementation must scroll that pane rather than the entire page, must not move the independent project sidebar, and must reset with the project-keyed chat component. No new authorization boundary is crossed, so a full threat model would be noise; the project-isolation and stale-response invariants remain material.

14.7 When the result is not complete

An unobservable or untested positive claim, missing durable negative requirement, uncontrolled risk or material control without a negative test or proof leaves the ledger incomplete. So does a liveness claim missing its fairness or availability assumptions, failed or skipped validation, or a check of a substitute artifact. Report the blocker and return to the relevant layer; a stakeholder-visible residual risk must be stated where the stakeholder will review it before committing. A green build or persuasive diagram does not fill these gaps.

Chapter 15

Lean as the Rendered Language

The user-facing notation for global types, local types and channel physics should render to a conservative subset of Lean. There is no second opaque specification file: domain declarations are Lean definitions, expected behaviours are Lean values, and requirements are Lean propositions or checked queries. Rendering the proposed scenario DSL into Lean is a **future interface**; its listings are **proposed notation**. Current **executable code** is the typed LeanFM API, and a **Lean proof** claim applies only when its theorem and proof are checked in the declared build.

Throughout this book, every displayed mathematical formula is accompanied by a literal English reading that accounts for each operator, index, bound, and new glyph. The reading is part of the notation discipline: it must map back to the symbols one-to-one rather than merely paraphrase the formula's intent.

```
def checkout : Grammar CheckoutEvent :=
  request >>> (accepted <||> rejected)

def eventuallyAnswered : CTL Observation :=
  CTL.ag (CTL.implies requestPending
    (CTL.af (CTL.or requestAccepted requestRejected)))
```

Most semantic machinery should be imported from a fixed library. A system author normally supplies only:

- typed actors, identifiers, event kinds, and payloads;
- parameterised grammar definitions;
- projections from trace prefixes to observable propositions;
- named temporal requirements and proofs or decidable checks.

This boundary matters for automation. Generated Lean remains small enough to review, while library proofs and algorithms are reused rather than regenerated.

15.1 An unchecked exploratory fixture

`book/logic.lean`, reproduced completely below, is an unchecked side experiment, not part of the method. It explores a regularized assignment for $1 + 2 + 3 + \cdots$. For each finite cutoff n, its theorem

equates the triangular prefix $n(n+1)/2$ plus the defined correction `Tail_S(n)` with $-1/12$. This identity under that definition does not assert convergence of ordinary partial sums. The optional example is separate from protocol/trace semantics and excluded from `make book`; `lake env lean book/logic.lean` stops here because Mathlib is absent. Its inclusion therefore claims no successful proof check. A validity objection should identify a circular dependency, unchecked assumption or interpretation mismatch; kernel acceptance establishes only the stated theorem under its imports and axioms.

```
import Mathlib

set_option linter.style.whitespace false
set_option linter.unusedVariables false

/-!
# 1 - 1 + 1 - ... = 1/2,  1 - 2 + 3 - ... = 1/4,  1 + 2 + 3 + ... = -1/12

Everything lives in ℚ. No limits, no convergence, and no values are given:
all three totals are derived.

Partial sums:
* ‘Sum_G n‘ : 1 - 1 + 1 - 1 + ...   (n terms)
* ‘Sum_B n‘ : 1 - 2 + 3 - 4 + ...   (n terms)
* ‘Sum_S n‘ : 1 + 2 + 3 + ... + n

Three finite identities, proved for every n:
* ‘idG‘ : Sum_G (n+1) + Sum_G n       = 1
* ‘idB‘ : Sum_B (n+1) + Sum_B n       = Sum_G (n+1)
* ‘idS‘ : Sum_S (2n)  - Sum_B (2n)    = 4 * Sum_S n

The rules of the system (hypotheses, never baked into definitions):
1. A total does not depend on n:  Sum n + Tail n = constant
2. Tails obey the same three identities (the constant 1 in idG has no tail,
   so its tail version has 0 on the right)

Only these three identities are admitted. Admitting *every* finite identity
is inconsistent (e.g. the shift identity for S), so the list is explicit
and ‘tails_exist‘ proves this list is consistent.
-/

/-! ## Partial sums -/

/-- 1 - 1 + 1 - 1 + ... (n terms) -/
def Sum_G (n : ℕ) : ℚ :=
  (1 - (-1)^n) / 2

/-- 1 - 2 + 3 - 4 + ... (n terms) -/
def Sum_B (n : ℕ) : ℚ :=
  (1 - (2 * (n : ℚ) + 1) * (-1)^n) / 4

/-- 1 + 2 + ... + n -/
def Sum_S (n : ℕ) : ℚ :=
  n * (n + 1) / 2

/-! ## The finite identities -/

theorem idG (n : ℕ) :
    Sum_G (n+1) + Sum_G n = 1 := by
  unfold Sum_G
  ring

theorem idB (n : ℕ) :
    Sum_B (n+1) + Sum_B n = Sum_G (n+1) := by
  unfold Sum_B Sum_G
```

```
  push_cast
  ring

theorem idS (n : ℕ) :
    Sum_S (2*n) - Sum_B (2*n) = 4 * Sum_S n := by
  have h : ((-1 : ℚ))^(2*n) = 1 := by
    rw [pow_mul]
    norm_num
  unfold Sum_S Sum_B
  rw [h]
  push_cast
  ring

/-! ## The rules force all three totals -/

/-- Any tails obeying the rules force G = 1/2, B = 1/4, S = -1/12.
    None of these numbers appears in the hypotheses. -/
theorem totals_forced (TS TB TG : ℕ → ℚ) (s b g : ℚ)
    (hG : ∀ n, Sum_G n + TG n = g)                    -- rule 1
    (hB : ∀ n, Sum_B n + TB n = b)                    -- rule 1
    (hS : ∀ n, Sum_S n + TS n = s)                    -- rule 1
    (tG : ∀ n, TG (n+1) + TG n = 0)                   -- rule 2 for idG
    (tB : ∀ n, TB (n+1) + TB n = TG (n+1))            -- rule 2 for idB
    (tS : ∀ n, TS (2*n) - TB (2*n) = 4 * TS n) :      -- rule 2 for idS
    g = 1/2 ∧ b = 1/4 ∧ s = -1/12 := by
  have hg : g = 1/2 := by
    have i := idG 0
    have a := hG (0+1)
    have c := hG 0
    have t := tG 0
    linarith
  have hb : b = 1/4 := by
    have i := idB 0
    have a := hB (0+1)
    have c := hB 0
    have d := hG (0+1)
    have t := tB 0
    linarith
  have hs : s = -1/12 := by
    have i := idS 1
    have a := hS (2*1)
    have c := hB (2*1)
    have d := hS 1
    have t := tS 1
    linarith
  exact ⟨hg, hb, hs⟩

/-! ## The rules are consistent -/

/-- Witness tails (used only to show the rules can be met). -/
def Tail_G (n : ℕ) : ℚ := 1/2 - Sum_G n
def Tail_B (n : ℕ) : ℚ := 1/4 - Sum_B n
def Tail_S (n : ℕ) : ℚ := -1/12 - Sum_S n

theorem tails_exist : ∃ TS TB TG : ℕ → ℚ,
    (∀ n, Sum_G n + TG n = 1/2) ∧
    (∀ n, Sum_B n + TB n = 1/4) ∧
    (∀ n, Sum_S n + TS n = -1/12) ∧
    (∀ n, TG (n+1) + TG n = 0) ∧
    (∀ n, TB (n+1) + TB n = TG (n+1)) ∧
    (∀ n, TS (2*n) - TB (2*n) = 4 * TS n) :=
  ⟨Tail_S, Tail_B, Tail_G,
    fun n => by unfold Tail_G; ring,
    fun n => by unfold Tail_B; ring,
    fun n => by unfold Tail_S; ring,
    fun n => by
```

```
      have := idG n
      unfold Tail_G
      linarith,
    fun n => by
      have := idB n
      unfold Tail_B Tail_G
      linarith,
    fun n => by
      have := idS n
      unfold Tail_S Tail_B
      linarith⟩

/-! ## The tail is determined by the Sum -/

theorem tails_unique (TS TB TG : ℕ → ℚ) (s b g : ℚ)
    (hG : ∀ n, Sum_G n + TG n = g)
    (hB : ∀ n, Sum_B n + TB n = b)
    (hS : ∀ n, Sum_S n + TS n = s)
    (tG : ∀ n, TG (n+1) + TG n = 0)
    (tB : ∀ n, TB (n+1) + TB n = TG (n+1))
    (tS : ∀ n, TS (2*n) - TB (2*n) = 4 * TS n)
    (n : ℕ) :
    TS n = Tail_S n := by
  obtain ⟨_, _, hs⟩ := totals_forced TS TB TG s b g hG hB hS tG tB tS
  have := hS n
  unfold Tail_S
  linarith

/-! ## Final statement -/

theorem Goal (TS TB TG : ℕ → ℚ) (s b g : ℚ)
    (hG : ∀ n, Sum_G n + TG n = g)
    (hB : ∀ n, Sum_B n + TB n = b)
    (hS : ∀ n, Sum_S n + TS n = s)
    (tG : ∀ n, TG (n+1) + TG n = 0)
    (tB : ∀ n, TB (n+1) + TB n = TG (n+1))
    (tS : ∀ n, TS (2*n) - TB (2*n) = 4 * TS n)
    (n : ℕ) :
    -1/12 = n * (n + 1) / 2 + TS n := by
  obtain ⟨_, _, hs⟩ := totals_forced TS TB TG s b g hG hB hS tG tB tS
  have := hS n
  unfold Sum_S at this
  linarith

#print axioms Goal
```

For a LeanFM runtime judgment, define the double turnstile $\models$ relative to the declared transition model and initial state. It means “at every reachable state, the premises require at least one of these conclusions.” Write it in a rule-like form, familiar from Prolog:

$$a \wedge b \wedge \cdots \wedge w \;\models\; x \vee \cdots \vee z.$$

Its executable definition is that the following CTL formula holds at the declared initial state:

$$AG\big((a \wedge b \wedge \cdots \wedge w) \rightarrow (x \vee \cdots \vee z)\big).$$

Read: “at every reachable state, if a and b and so on through w all hold, then x or so on through z holds.” A means every continuation; G means every position along it, including the current state. This is entailment within the declared runtime model, not a redefinition of general entailment across every possible model. The model and initial state must accompany the judgment. The $\wedge$ symbols collect premises with “and”; the $\vee$ symbols collect alternative conclusions with “or.” The conclusion need not be the same in every state.

For code, represent the left side as a list of premises and the right side as a list of conclusions. A counterexample is a reachable state where every premise is true and every conclusion is false:

```
counterexample(state, premises, conclusions) :=
  premises.all(p => evaluate(p, state)) &&
  !conclusions.any(q => evaluate(q, state))

entails(successors, initialState, premises, conclusions) :=
  !anyReachable(successors, initialState,
    state => counterexample(state, premises, conclusions))
```

The evaluator and successor relation come from the declared model. A sampled trace checks only that sample, not all reachable states.

The same counterexample condition can be constructed symbolically:

$$\neg EF(a \wedge b \wedge \cdots \wedge w \wedge \neg x \wedge \cdots \wedge \neg z).$$

Read: “there is no reachable state with a and b and so on through w, and not x and so on through not z.” This is exactly the AG implication above, using $AG\,p = \neg EF\,\neg p$, read “always p is no reachable not-p.”

With a single conclusion, the analogy is the Prolog rule `x :- a, b, ..., w`: x follows when all its premises hold. The displayed disjunctive conclusion is more general than an ordinary single-head Prolog rule; $\models$ does not specify Prolog’s execution or search order. The compact notation $\Gamma \models p$, read “Gamma semantically entails p,” collects the premises into a context Γ and names the conclusion p. The distinction remains: $\Gamma \vdash p$, read “Gamma derives p,” asserts a derivation. Our runtime judgment checks that no reachable state makes all the premises true and p false. General semantic entailment uses all models of the assumptions instead; it must not be confused with this explicitly scoped CTL test.

For the proof-theoretic background behind derivations, natural deduction, sequent calculus, normalization, and cut elimination, see Mancosu, Galvan, and Zach’s *An Introduction to Proof Theory* [17].

15.2 From specification to service

Lean can host executable code, including a small web server. That makes the formal artifact part of the running project rather than a document beside it. The design target is for typed definitions to drive validation endpoints, test-trace generation, protocol schemas, visualisations, and runtime monitors; current coverage is partial, as Chapter 18 records. Extraction and integration choices must preserve the distinction between a checked behavioural model and an unverified infrastructure layer.

Chapter 16

Scope and Open Questions

The implementation backlog and audit findings live in `CRITIQUE.md`. This chapter distinguishes current evidence from open technical questions using the labels defined in Chapter 1; proposed backlog items are **future interfaces**, not implemented behavior.

Self-hosting consolidation

The canonical requirements entry point now composes the existing LeanFM ChatUI specification with a proposed revision-bound requirements-to-PR workflow. The Worker protocol remains an example, not a specification of the whole product. `make selfhost-check` validates the typed composition and regenerates `SELF_HOSTING_REVIEW.md`, including its interaction diagram and state machine. A component registry and fresh-session handoff preserve decisions without requiring conversation history.

The consolidation proposes checker outcomes, check invalidation, probability interpretations and PR evidence. Earlier proposals also cover modal indexes; built-in knowledge and policy checking is now deferred. These records are not implemented features or proofs. Error protocols, formula semantics and complete traceability remain prerequisites for a complete one-shot specification.

16.1 Proof obligations

The initial metatheory is session-type metatheory with the channel physics added. It should make the following questions precise:

- Which global types are well formed: every choice directed by one role, and every role that must learn a branch either told by a message or released by a declared timer?
- Does projection to local types preserve the global type, so that roles following their local types over the declared channels produce only runs of the global type (session fidelity)?
- Which channel physics preserve communication safety and progress, and which (loss, duplication, early timers) require the protocol itself to recover, as TFTP's retransmission does?
- Does a recorded production event stream conform to the global type, and does each event's `causeId` stay within its sender's own history?

- Under what finiteness restrictions is CTL checking decidable and executable?
- Which guards can be compiled into finite monitor state?
- When does modular composition preserve an established temporal property?
- Which fairness and timing assumptions are required by a liveness theorem?
- Which observation and policy relations justify each knowledge, permission, and obligation assertion, and which bounds make their combination with CTL executable?

Part II

Security Protocol Exercises

Chapter 17

Protocol Exercises: Roles, Attackers, and Knowledge

The author learned from Ryan et al. [22] to treat a security protocol as a set of communicating roles plus an active intruder, and to state security as properties of every trace that intruder can produce. This chapter keeps that idea and none of that book's examples. It works three small invented protocols, each chosen to put pressure on one part of the grammar method:

1. an *on-call handoff*, where three separately written roles must compose into one protocol;
2. a *sealed-bid auction*, where an attacker who cannot read a bid can still copy it; and
3. *speaker pairing*, where a code shown on a screen is visible to some actors and not to others.

Status: this chapter is a design exercise, not an account of implemented LeanFM features. Its `RoleSpec`, `WireTerm`, `composeRoles`, and attacker-deduction listings are proposed notation; the current executable model has `GrammarExpr` and finite trace/CTL checks, but no role-projection or composition engine, symbolic byte algebra, or attacker-knowledge evaluator. The checklist at the end is a set of proposed library requirements, not implemented capabilities.

17.1 Roles compose by sharing events

This is the session-type picture seen from the other side. Projection derives each role's local type from one global type; here each role is written first, and the global protocol is whatever their composition admits. The attacker is the case session types usually leave out: a role that controls the channels, and can read, drop, replay or forge what passes through them. In the vocabulary of Chapter 1, the attacker is the most hostile physics a channel can declare.

Each actor is specified separately, as a grammar over its own view of the global event alphabet. Composition is synchronization on the same observable events, not concatenation of independent scripts.

```
structure RoleSpec where
  actor    : Actor
  accepts  : Grammar (ProjectedEvent actor)
  initialKnowledge : Knowledge actor
```

```
def composeRoles (roles : List RoleSpec) : GlobalGrammar :=
  synchronize roles where
    sendReceiveAgree := sameEventId && sameBytes && oppositeEndpoints
```

For a global trace t, let $t \restriction_a$ keep only the events actor a sends or receives. The composition accepts a trace exactly when every role accepts its own view of it:

$$L(\text{composeRoles}(R_1, \ldots, R_n)) = \{t \mid \forall i,\ t \restriction_{a_i} \in L(R_i)\}.$$

Read this as: "the language L accepted by the composition of roles R_1 through R_n is the set of global traces t such that, for every role index i, trace t restricted to actor a_i belongs to the language accepted by role R_i." Here n is the number of roles, i is one role's index, the vertical bar means "such that," and $\restriction$ means "restricted to."

A message is one global event with one identity, one byte string, one sender and one recipient. Both endpoint grammars constrain that same event. An unmatched send, an unexpected receive, or disagreement about the bytes makes the composed language empty from that prefix on, and the checker reports the prefix.

A role may bind values from events it has seen and test later events against them. Those bindings are a parser environment for the trace, not a claim about implementation memory: "the code in this reply equals the code I sent" says nothing about tables, variables or retries.

17.2 A transport-neutral byte algebra

Security claims need to say which bytes expose which values, without choosing HTTP, protobuf or a cipher suite. Four symbolic constructors are enough for this chapter:

```
abbrev Bytes := ByteArray

inductive WireTerm where
  | atom    : Name -> Bytes -> WireTerm
  | tuple   : List WireTerm -> WireTerm
  | encrypt : KeyRef -> WireTerm -> WireTerm
  | hash    : WireTerm -> WireTerm

structure WireEvent where
  timeAt   : ClockTimestamp
  id       : EventId
  priorIds : List EventId
  session  : SessionId
  task     : TaskId
  from     : Actor
  to       : Actor
  channel  : Channel
  term     : WireTerm
  bytes    : Bytes
  encodedBy : bytes = encodeCanonical term
```

Encryption and hashing are symbolic constructors with a canonical encoding. Anyone may hash a term they know; nobody may invert a hash; decryption needs the key. `channel` records who can observe an event at all, which the pairing example needs. A concrete refinement must replace this idealized algebra with real transports and cryptography whose assumptions justify it.

17.3 Example 1: an on-call handoff

A pager service records who is on call for a team. Exactly one engineer is on call at any time. Olu is on call and wants to hand off to Ines. The pager must not move the duty until Ines has actually accepted it, and Olu must learn that the handoff completed. Each person shares a long-term key with the pager: K_O for Olu, K_I for Ines.

1. Olu $\rightarrow$ Pager: $\{Ines, h\}_{K_O}$, with a fresh handoff id h.
2. Pager $\rightarrow$ Ines: $\{Olu, h, c\}_{K_I}$, with a fresh challenge c.
3. Ines $\rightarrow$ Pager: $\{h, c, accept\}_{K_I}$.
4. Pager $\rightarrow$ Olu: $\{Ines, h, done\}_{K_O}$.

17.3.1 Three roles, written separately

```
def oluRole (olu ines pager : Actor) (ko : SharedKey) : RoleSpec :=
  role olu "outgoing engineer" do
    let h <- fresh Nonce "h"
    from_ olu; to_ pager
    send (encrypt ko.ref (tuple [actor ines, nonce h]))
    from_ pager; to_ olu
    receive (encrypt ko.ref (tuple [actor ines, nonce "h", atom "done"]))
      where_ field "h" == h

def inesRole (olu ines pager : Actor) (ki : SharedKey) : RoleSpec :=
  role ines "incoming engineer" do
    from_ pager; to_ ines
    receive (encrypt ki.ref (tuple [actor olu, nonce "h", nonce "c"]))
    let h <- take Nonce "h"
    let c <- take Nonce "c"
    from_ ines; to_ pager
    send (encrypt ki.ref (tuple [nonce h, nonce c, atom "accept"]))

def pagerRole (olu ines pager : Actor) (ko ki : SharedKey) : RoleSpec :=
  role pager "pager service" do
    from_ olu; to_ pager
    receive (encrypt ko.ref (tuple [actor "next", nonce "h"]))
      where_ field "next" == ines
    let h <- take Nonce "h"
    let c <- fresh Nonce "c"
    from_ pager; to_ ines
    send (encrypt ki.ref (tuple [actor olu, nonce h, nonce c]))
    from_ ines; to_ pager
```

```
    receive (encrypt ki.ref (tuple [nonce "h", nonce "c", atom "accept"]))
      where_ field "h" == h && field "c" == c
    from_ pager; to_ olu
    send (encrypt ko.ref (tuple [actor ines, nonce h, atom "done"]))
```

The `where_` clauses are the context-sensitive part. The message sequence is context-free; whether a reply is legal depends on values bound earlier in the same trace.

17.3.2 What composition catches

Suppose a later edit lets the pager send *done* as soon as it has sent the offer, before Ines replies. Olu's role still accepts that order: Olu never sees messages 2 and 3. Ines's role still accepts it too. Only the composed protocol exposes the problem, through the duty properties below:

$$AG\ exactlyOneOnCall.$$

$$onCall(Olu)\ AW\ accepted(Ines, h).$$

$$AG(toldDone(Olu, h) \rightarrow \mathbf{P}\ accepted(Ines, h)).$$

The second formula uses weak until on purpose: Ines may never answer, and then Olu simply remains on call. Strong AU would wrongly demand that every handoff eventually succeeds. The third is a past claim ($\mathbf{P}$ reads "previously"). With the faulty edit, the checker returns a counterexample trace in which Olu is told *done* with no earlier acceptance, drawn both as the global interaction diagram and as Olu's local view, in which nothing looks wrong.

The same mechanism catches disagreement about values. If the pager's grammar could forward the offer to someone other than the engineer Olu named, the `where_ field "next" == ines` guard and Ines's projection cannot both accept the same event, and the composed language is empty at that prefix.

17.4 Example 2: a sealed-bid auction

An auctioneer collects bids in two phases. While bidding is open, each bidder sends a *commitment*: the hash of the amount and a fresh random salt. After bidding closes, each bidder reveals the amount and salt, and the auctioneer accepts a reveal only if it hashes to the commitment. The highest revealed bid wins. Ana and Ben bid honestly. Ivo is also a registered bidder; he is the attacker, and he sees every message on the network.

```
def bidderRole (b auctioneer : Actor) (amount : Amount) : RoleSpec :=
  role b "bidder" do
    let s <- fresh Nonce "salt"
    from_ b; to_ auctioneer
    send (tuple [actor b, hash (tuple [value amount, nonce s])])   -- commit
    from_ auctioneer; to_ b
```

```
    receive (atom "closed")
    from_ b; to_ auctioneer
    send (tuple [actor b, value amount, nonce s])                    -- reveal

def auctioneerRole (auctioneer : Actor) (bidders : List Actor) : RoleSpec :=
  role auctioneer "auctioneer" do
    let commits <- collectWhileOpen fun b =>
      receive (tuple [actor b, ?commitment]) where_ b in bidders
    broadcast (atom "closed")
    forEach commits fun (b, commitment) =>
      receive (tuple [actor b, ?amount, ?salt])
        where_ hash (tuple [amount, salt]) == commitment
    announce (highestRevealed commits)
```

17.4.1 Secrecy holds, and the auction is still broken

Before close, Ivo has seen Ana's commitment but cannot invert the hash, so he does not know her amount. That secrecy property holds:

$$\neg\,\mathrm{knows}(\mathit{Ivo}, a_{\mathit{Ana}})\ \mathit{AU}\ \mathit{closed}.$$

The auction is still broken. Ivo does not need to know Ana's amount to bid it. He copies her commitment and sends it under his own name. That is a legal attacker move: he observed the hash term, and resending a known term is always derivable. After close he waits for Ana's reveal, which he also observes, and sends the same amount and salt under his name. The auctioneer's check passes, because the hash really does match. Ivo has tied Ana's bid without ever having known it while bidding was open.

The property that fails is about who *formed* each commitment, not about who knows what:

$$\mathit{AG}\big(\mathrm{acceptsCommit}(\mathit{Auctioneer}, b, x) \rightarrow \mathbf{P}\ \mathrm{formed}(b, x)\big).$$

The checker's search for a violation, $\mathit{EF}(\mathrm{acceptsCommit}(\ldots) \wedge \neg\mathbf{P}\,\mathrm{formed}(\ldots))$, returns the copy trace above as its witness.

17.4.2 The fix: bind the bidder into the commitment

Put the bidder's own name inside the hash: $\mathit{hash}(b, \mathit{amount}, s)$, and have the auctioneer recompute the hash with the name of the bidder who sent the reveal. Ivo's copied commitment was formed with Ana's name. To open it under his own name he would need a salt that makes $\mathit{hash}(\mathit{Ivo}, \ldots)$ equal $\mathit{hash}(\mathit{Ana}, \ldots)$, and no rule derives one. Copying is now useless.

This gives the acceptance test for an intruder model. For the original commitment, the checker must produce the copy trace as a counterexample. For the fixed commitment, it must find none. A model that cannot find the first attack is too weak to trust when it finds nothing in the second.

One limit of the symbolic model is visible here. If amounts came from a small set, Ivo could hash each candidate and compare, and the salt is what stops that. Dolev–Yao deduction has no notion of guessing, so the requirement that salts be fresh and unguessable must be written down as an explicit assumption, not left implicit.

17.5 Knowledge and agreement as theorems

The attacker is one more role. It observes the events its channels let it see, may block or replay them, and may send any term it can derive from what it has observed. Seeing ciphertext bytes does not reveal the plaintext, and seeing a hash does not reveal its input.

```
inductive Derives : Knowledge -> WireTerm -> Prop where
  | initial : term in knowledge -> Derives knowledge term
  | tuple   : (forall x in xs, Derives knowledge x) ->
              Derives knowledge (.tuple xs)
  | project : Derives knowledge (.tuple xs) -> x in xs ->
              Derives knowledge x
  | encrypt : Derives knowledge key -> Derives knowledge body ->
              Derives knowledge (.encrypt key body)
  | decrypt : Derives knowledge (.encrypt key body) ->
              Derives knowledge (inverse key) -> Derives knowledge body
  | hash    : Derives knowledge x -> Derives knowledge (.hash x)
  -- no rule recovers x from .hash x

def intruderRole (ivo : Actor) : RoleSpec :=
  adversary ivo do
    observe (channelsVisibleTo ivo)
    mayBlock
    mayReplay
    maySend fun knowledge term => Derives knowledge term
```

Knowledge is calculated for every actor from every accepted trace, not only the intended run. Security claims then become theorems over accepted traces, derived knowledge and explicit compromise assumptions. For the fixed auction:

```
theorem auction_bid_sealed_until_close
    (trace : Trace WireEvent)
    (accepted : fixedAuction.Accepts trace)
    (fresh : SaltsFreshAndUnguessable trace) :
    forall b x, beforeClose trace x ->
      not (Derives (knowledgeAt .ivo trace x) (value (amountOf b))) := by
  exact hashHidesSaltedInput accepted fresh

theorem auction_commitments_not_copyable
    (trace : Trace WireEvent)
    (accepted : fixedAuction.Accepts trace) :
    forall b c, acceptsCommit trace b c -> formedBy trace b c := by
  exact boundNameDefeatsCopy accepted
```

The two theorems are still proposed notation: the lemmas they name do not exist yet. The deduction rules do. `LeanFM.Session.Knowledge` defines them as `Derives`, with messages built from atoms, symmetric keys, key pairs, pairs, encryption and hashes; a ciphertext under a private key is a signature, opened with the public key. It also decides them for one value at a time: `analz` takes what an actor has seen apart until nothing new appears, and `synth` asks whether the value can be built from the pieces. Enumerating everything an actor could build would never end; breaking down and then checking one value does. The library proves the decision sound:

```
theorem knows_sound {K : List Msg} {v : Msg}
    (h : knows K v = true) : Derives K v
```

So when the checker says the attacker knows a value, a derivation exists. Completeness, that `knows` misses nothing `Derives` allows, is tested on examples but not yet proved, and secrecy verdicts lean on it.

17.5.1 Calculated knowledge and policy inside CTL

Knowledge and policy need not start as new modal operators. Define state predicates by reducing the event history, then use them inside ordinary CTL. `knows` says who can derive a value; `allows` and `requires` say what a declared policy permits and obliges:

```
def knows (actor : Actor) (value : Secret) (s : ProtocolState) : Bool :=
  value in knowledgeClosure actor s.events s.compromise

def allows (policy : Policy) (actor : Actor)
    (action : Action) (s : ProtocolState) : Bool :=
  policy.allows actor action s

def requires (policy : Policy) (actor : Actor)
    (action : Action) (s : ProtocolState) : Bool :=
  policy.requires actor action s
```

These functions label a state; CTL says when the labels may or must hold. Each formula below reads directly as English:

$$\neg\,\mathrm{knows}(\mathit{Auctioneer}, a_b)\ AU\ \mathit{closed}.$$

$$EF\ \mathrm{knows}(\mathit{Auctioneer}, a_b).$$

$$AG(\mathrm{knows}(x, c) \rightarrow \mathrm{allows}(\mathit{DutyPolicy}, x, \mathrm{possess}(c))).$$

$$AG(\mathit{dutyMoved}(h) \rightarrow (\mathrm{requires}(\mathit{DutyPolicy}, \mathit{Pager}, \mathrm{notify}(\mathit{Olu}, h)) \wedge AF\ \mathit{toldDone}(\mathit{Olu}, h))).$$

A universal claim is checked by searching for its violation, and the search result is the useful artifact:

$$AG(\mathrm{discloses}(a, s) \rightarrow \mathrm{allows}(P, a, \mathrm{disclose}(s))) \ \equiv\ \neg EF(\mathrm{discloses}(a, s) \wedge \neg\,\mathrm{allows}(P, a, \mathrm{disclose}(s))).$$

This state-local `knows` predicate is not the full epistemic modality $K_a\varphi$ over indistinguishable histories defined in Chapter 11. It tests whether one declared deduction closure contains a value, which suffices for many secrecy and authorization claims. Use K_a when the requirement compares histories, as the next example does.

17.6 Example 3: pairing a speaker by a code on its screen

A new speaker shows a six-digit code d on its own display. Its owner types the code into a phone, and the phone proves to the speaker that it knows d. The network is hostile; the display is not on the network.

1. Speaker $\rightarrow$ Owner, on channel *display*: d.
2. Owner $\rightarrow$ Phone, on channel *keypad*: d.
3. Phone $\rightarrow$ Speaker, on channel *network*: $p,\ hash(p, d, Phone)$, with a fresh nonce p.
4. Speaker $\rightarrow$ Phone, on channel *network*: *paired* or *rejected*.

```
receive (tuple [nonce "p", ?proof]) on network
  where_ sender != speaker
  where_ proof == hash (tuple [nonce p, value shownCode, actor sender])
  where_ failedAttempts < 3
```

Three things are new compared with the first two examples.

Channels decide what each actor observes. The display and keypad events belong to the observations of the speaker, the owner and the phone, not to Ivo's. That is what keeps d out of Ivo's knowledge closure, and it is a modeling assumption about the room, so it is stated as one. Remove it and the checker should immediately find Ivo pairing his own phone.

A counter is context, too. `failedAttempts < 3` depends on how many earlier proofs failed in this trace. Like the equality guards, it keeps the grammar context-free and moves the counting into a named predicate.

Guessing is a probability, not a deduction. Ivo cannot derive d, but he can guess it. The symbolic model says $EF\ paired(Ivo)$ is true: some future has a lucky guess. That verdict is correct and useless on its own. The requirement is a bound on how likely that future is, carried by a probability overlay: with three attempts against 10^6 codes,

$$P[F\ paired(Ivo)] \leq 3/10^6.$$

The next section explains why the number attaches to E.

Some claims compare histories. "Ivo cannot tell which code was shown" is not true or false of one state. It says that any two histories differing only in d look identical to Ivo, which is exactly K_a over indistinguishable histories. It belongs in a separate family of relational theorems, not in a single CTL state formula.

17.7 Where each modality attaches

The three examples have used every kind of claim a requirement needs: what can or must happen, how likely, by when, who knows, who may, and who must. None of them needs a new logic. Each attaches to one place in an ordinary CTL formula. A CTL operator is a path quantifier, A or E, fused with a temporal operator such as F, G or U. Probability belongs to the quantifier, deadlines belong to the temporal operator, and knowledge and policy are predicates on states.

The path quantifier E invites "how likely?" E says that *some* future satisfies the path formula. The natural refinement is to ask how much of the future does: replace "some" with a probability measured over futures. That works the same way whatever follows the E:

$$EF\,p \;\rightsquigarrow\; P[F\,p], \qquad EG\,p \;\rightsquigarrow\; P[G\,p], \qquad p\;EU\;q \;\rightsquigarrow\; P[p\;U\;q].$$

The number comes from an overlay of branch probabilities, as in the bakery chapter, and is reported beside the CTL verdict, never in place of it. The overlay cannot make $EF\,p$ false: a future with probability $3/10^6$ is still possible, and so still needs the handling that possibility demands. Nor does A become "probability one": a set of futures can have probability one and still leave out a future that A must cover, which is why Chapter 11 keeps A and E about legal futures only.

The temporal operator F invites "by when?" F says "at some later point" with no bound. A user waiting on a handoff or an auction result needs a bound, so the deadline is written on F (and, the same way, on the right-hand side of U). It means the same thing under either quantifier: $AF_{\le t}\,q$ says every future reaches q within t; $EF_{\le t}\,q$ says some future does.

A deadline under A is a promise the grammar must be able to keep. Every waiting state needs a timeout branch that ends in an explicit outcome. Without one, $AF_{\le t}$ fails, because some future waits past t. With one, the deadline is part of the guaranteed behaviour:

$$AF_{\le 15\,\mathrm{min}}\big(accepted(Ines, h) \lor expired(h)\big).$$

$$AG\Big(closed \rightarrow AF_{\le T_{reveal}}\big(revealed(b) \lor forfeited(b)\big)\Big).$$

The handoff therefore needs an *Expired* message that the pager sends to both engineers when the offer times out, and the auction needs a *Forfeited* outcome for a bidder who never reveals. Both are new legal branches, so both need decodable messages and code that handles them.

The two annotations compose. Probability on the quantifier and a deadline on F combine into the question stakeholders usually ask: how likely is it to happen in time? An owner pairing a speaker within a minute is

$$P[F_{\le 60\,\mathrm{s}}\,paired(Phone)],$$

which a measured overlay reports from real pairing attempts. Keep a guaranteed deadline ($AF_{\leq t}$, enforced by a timeout branch) separate from a usual one ($P[F_{\leq t}]$, measured). The bakery chapter shows why: a declared delay promised a 1.2 s sale while only about half the recorded sales met it.

`knows`, `allows` and `requires` complete the set. The remaining modalities are about actors rather than futures: what an actor can derive, what a policy allows it to do, and what a policy requires of it. Each is computed from the history up to a state, so each is a state predicate, and the temporal operators carry them without change. Secrecy is `knows` under AU; authorization is `knows` implying `allows` under AG; an obligation is `requires` paired with $AF_{\leq t}$ on the act that discharges it.

Question	Attaches to	Example from this chapter
can it happen? must it?	E, A	$EF\ paired(Ivo)$
how likely?	E	$P[F\ paired(Ivo)] \leq 3/10^6$
by when?	F, U	$AF_{\leq 15\,\mathrm{min}}(accepted \vee expired)$
who knows?	state: `knows`	$\neg\,\mathrm{knows}(Ivo, a_{Ana})\ AU\ closed$
who may?	state: `allows`	$AG(\mathrm{knows}(x, c) \rightarrow \mathrm{allows}(Duty, x, \ldots))$
who must?	state: `requires`	$AG(dutyMoved \rightarrow \mathrm{requires}(\ldots) \wedge AF\ toldDone)$

One claim in the chapter does not fit the table: “Ivo cannot tell which code was shown.” It compares histories rather than labeling one, so it needs the full K_a over indistinguishable histories, as noted in the pairing example.

17.8 Needham–Schroeder, checked

The public-key protocol of Needham and Schroeder (1978) has three messages:

```
protocol NS (A, B) {
  m1: Msg1 from A to B;
  m2: Msg2 from B to A cause m1 where m2.na = m1.na;
  m3: Msg3 from A to B cause m2;
}
```

Alice runs it once, with a peer she chooses: Bob, or Ivo. Bob answers once, whoever claims to be calling. Ivo is a legitimate user with his own key pair, so Alice may well choose him; he is also the network, seeing every message and sending anything he can derive.

Nobody builds the state space. Each honest role is declared as a short script over message patterns: a choice, sends, receives and claims. A variable in a received pattern binds whatever the message carries there.

```
def alice (v : Variant) : Role := { name := "Alice", steps :=
  [ .choose "peer" [.atom "B", .atom "I"]
  , .claim "running" [name "A", .var "peer"]
  , .send (.var "peer") (.enc (.pk (.var "peer")) (.tuple [.lit na, name "A"]))
  , .recv (.enc (.pk (name "A")) (match v with
      | .original => .tuple [.lit na, .var "n"]
      | .lowe => .tuple [.lit na, .var "n", .var "peer"]))
  , .send (.var "peer") (.enc (.pk (.var "peer")) (.var "n"))
```

```
  , .claim "finished" [name "A", .var "peer"] ] }

def bob (v : Variant) : Role := { name := "Bob", steps :=
  [ .recv (.enc (.pk (name "B")) (.tuple [.var "m", .var "x"]))
  , .send (.var "x") (.enc (.pk (.var "x")) (match v with
      | .original => .tuple [.var "m", .lit nb]
      | .lowe => .tuple [.var "m", .lit nb, name "B"]))
  , .recv (.enc (.pk (name "B")) (.lit nb))
  , .claim "commit" [name "B", .var "x"] ] }

def declared (v : Variant) : Protocol :=
  { attacker := "Ivo"
  , attackerKnows := [.atom "A", .atom "B", .atom "I", .pk "A", .pk "B", .pk "I", .sk
     "I", ni]
  , domain := [na, nb, ni]
  , domains := [("x", [.atom "A", .atom "B", .atom "I"])]
  , roles := [alice v, bob v]
  , flags :=
    [ ⟨"SecrecyBreach", fun w => w.claimed "commit" [.atom "B", .atom "A"] && w.knows
     nb⟩
    , ⟨"Impersonation", fun w => w.claimed "commit" [.atom "B", .atom "A"] && !w.
    claimed "running" [.atom "A", .atom "B"]⟩
    , ⟨"HonestRun", fun w => w.claimed "commit" [.atom "B", .atom "A"] && w.claimed "
    finished" [.atom "A", .atom "B"]⟩ ] }
```

`LeanFM.Session.Attack` generates the model from that declaration. Every message an honest role sends goes to the network, which is Ivo; every message an honest role receives is one Ivo can build that has the shape the role expects. That is the Dolev–Yao channel, declared once rather than written into each role. A state of the model is each role's position and bindings, Ivo's knowledge, kept taken apart so that `knows` is an ordinary label of the state, and the claims made so far.

Requirements become flags that events trip. `SecrecyBreach` trips when Bob has committed to a run with Alice and Ivo can derive Bob's nonce; `Impersonation` when Bob has committed to Alice although Alice never claimed to be running with Bob. `check_ctl` decides `never` of each, and that an honest run can complete. The last row asks secrecy again with a `knows` atom and a `claimed` atom instead of a flag, and gets the same verdicts. The same declaration, written as JSON, is what the MCP tool `check_protocol` takes, so an agent can ask it without writing Lean:

LeanFM `check_ctl`	1978 (242 states)	Lowe (222 states)
Ivo never learns the nonce Bob shares with Alice	disproved	proved
Bob never commits to Alice unless Alice ran with Bob	disproved	proved
Alice and Bob can complete an honest run	proved	proved
the same secrecy, as `never (claimed commit(B, A) and knows(Ivo, nb))`	disproved	proved

For the 1978 protocol the counterexample to secrecy is the attack Lowe found in 1995, in the checker's own words:

```
1. Alice chooses peer = I
2. Alice claims running(A, I)
3. Alice -> I: {na, A}pk(I)
```

```
4. Bob <- {na, A}pk(B)
5. Bob -> A: {na, nb}pk(A)
6. Alice <- {na, nb}pk(A)
7. Alice -> I: {nb}pk(I)
8. Bob <- {nb}pk(B)
9. Bob claims commit(B, A)
10. SecrecyBreach
```

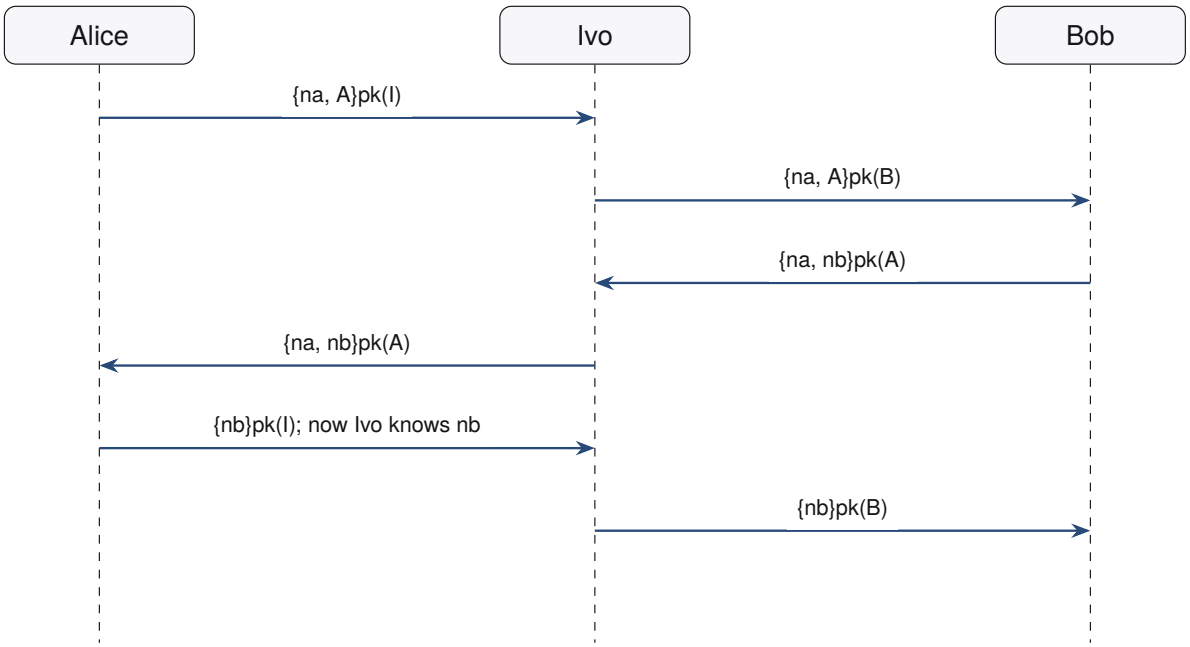


Figure 17.1: Lowe's attack as it crosses the wire: the checker's walk replayed through the generated model, which draws every message through Ivo, because Ivo is the network. The label marks where his knowledge first contains Bob's nonce.

Alice talks to Ivo, as she meant to. Ivo re-encrypts her first message for Bob, who answers "Alice" with his nonce under Alice's key. Ivo cannot read that answer, but he does not need to: he forwards it, and Alice, who believes it is Ivo's reply, decrypts it and sends Bob's nonce back to Ivo under Ivo's key. Ivo now knows the nonce, and returns it to Bob, who finishes believing he talked to Alice. Lowe's fix adds Bob's name to the second message, and Alice checks it is the peer she chose. With it both properties hold on every reachable state.

The model is bounded, and the bounds are where attacks could still hide: each session runs once, a variable binds a name or a nonce from a declared domain, never a whole ciphertext, and Ivo's domain is the names and nonces of this run. Lowe's attack needs one session of each role, which is why it appears with them. Ivo forwarding a sealed blob he cannot read still needs a larger bound. An attack that needs a second run of Bob needs only a second session, which a declaration can ask for.

17.8.1 Two sessions: reflection

Some attacks play one session of a role against another. The standard example is a challenge–response that shares one key, `kAB`, between Alice and Bob. Alice challenges with a fresh nonce; Bob answers it under the key and adds his own challenge; Alice answers his. The flag `named` adds a fix, used below:

```
/-- With ‘named‘, each answer carries the name of the party who made it. -/
def answer (named : Bool) (x : String) (who : String) : Pat :=
  .enc kAB (if named then .pair (.var x) (name who) else .var x)

def challenger (named : Bool := false) : Role := { name := "Alice", steps :=
  [ .fresh "na"
  , .claim "running" [name "A", name "B"]
  , .send (name "B") (.var "na")
  , .recv (.pair (answer named "na" "B") (.var "m"))
  , .send (name "B") (answer named "m" "A")
  , .claim "finished" [name "A", name "B"] ] }

def responder (named : Bool := false) : Role := { name := "Bob", steps :=
  [ .recv (.var "c")
  , .fresh "nb"
  , .send (name "A") (.pair (answer named "c" "B") (.var "nb"))
  , .recv (answer named "nb" "A")
  , .claim "commit" [name "B", name "A"] ] }

def reflection (bobSessions : Nat) (named : Bool := false) : Protocol :=
  let roles := (challenger named).sessions 1 ++ (responder named).sessions
    bobSessions
  { attacker := "Ivo"
  , attackerKnows := [.atom "A", .atom "B", ni]
  , domain := coverFresh roles [na, nb, ni]
  , roles
  , flags := [] }
```

A `fresh` step makes a value that no other session has and the attacker does not know: `nb` in Bob’s first session, `nb#2` in his second, which is named `Bob#2`. The requirement is agreement again: Bob never commits to Alice unless Alice started a run with him.

LeanFM `check_ctl`	one Bob (33)	two Bobs (527)	two Bobs, names inside (433)
Bob never commits to Alice unless Alice ran with Bob	proved	disproved	proved
Alice and Bob can complete an honest run	proved	proved	proved

With one session of Bob the property holds: the only party that encrypts under `kAB` on demand is Bob, and he does it before he has made the nonce that would need answering. With two sessions Ivo needs no key at all:

```
1. Bob <- ni
```

```
2. Bob makes nb
3. Bob -> A: {ni}kAB, nb
4. Bob#2 <- nb
5. Bob#2 makes nb#2
6. Bob#2 -> A: {nb}kAB, nb#2
7. Bob <- {nb}kAB
8. Bob claims commit(B, A)
```

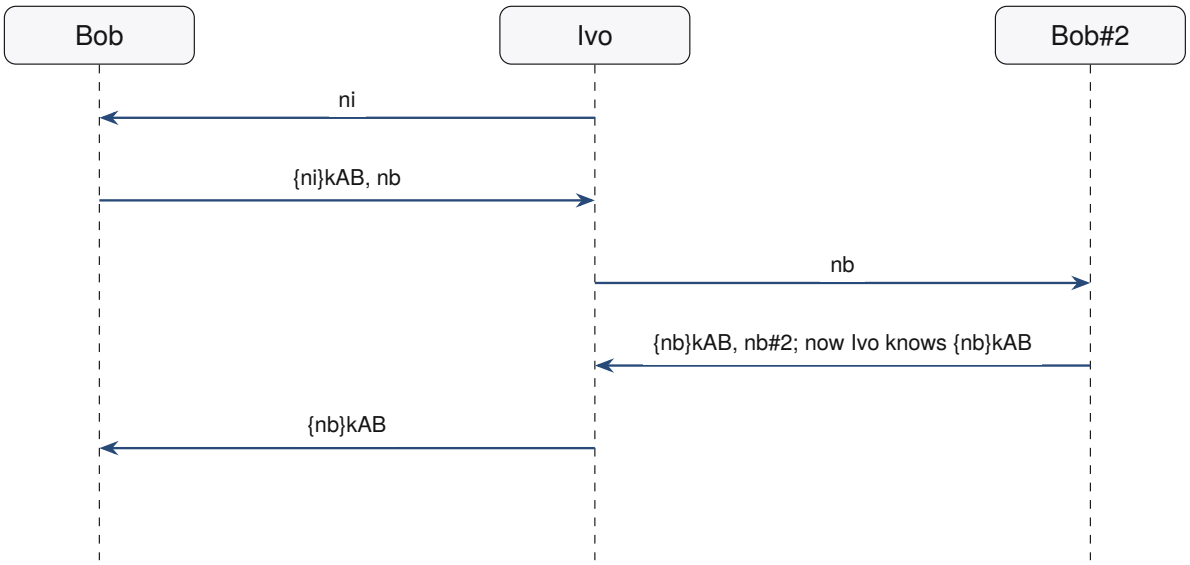


Figure 17.2: Reflection: Bob's own second session answers the challenge of his first. Alice never appears; the label marks where Ivo first holds the answer he could not compute.

Ivo opens a session with Bob as Alice and gets Bob's challenge `nb`. He opens a second session and offers `nb` as the challenge there. Bob's second session encrypts it, as it should, and Ivo returns that answer to the first session, which commits to Alice. Nothing was decrypted; the key was used as an oracle. The usual fixes put the direction or the sender's name inside the encryption, or use a separate key in each direction. With `named`, each answer carries the name of the party who made it: Bob's second session now produces `{nb, B}kAB`, and his first session waits for `{nb, A}kAB`, which only Alice makes. The third column of the table checks that fix with the same two sessions: agreement holds and an honest run still completes.

17.8.2 A defender on the wire

So far the network has belonged to the attacker alone. A real path often has a defender on it too: an intrusion-prevention system, a filtering proxy, a gateway that intercepts TLS. A declaration may put one in front of named roles. Every message delivered to them passes through it first, and it drops what matches a drop pattern, rewrites what matches a rewrite rule, and forwards the rest. What it can match is what it can see, which is `knows` again from the defender's side. It looks inside a ciphertext only when it can build the opening key. A blob it cannot open matches nothing, not even a copy of itself, because real encryption is randomised. Here Bob accepts whatever reaches him, and Guard drops `bad`:

```
def guarded (bobReceives : Json) (defender : Option Json) (formula : Json) : Json :=
  let base := json% {
    roles: [{name: "Bob", steps: [
      {recv: $(bobReceives)},
      {claim: {name: "accept", args: [{atom: "B"}, {var: "x"}]}}]}],
    attacker: {name: "Ivo", knows: [{atom: "B"}, {atom: "good"}, {atom: "bad"}, {pk:
    "B"}]},
    domain: [{atom: "good"}, {atom: "bad"}],
    formula: $(formula) }
  match defender with
  | some d => base.setObjVal! "defender" d
  | none => base

def plain := json% {var: "x"}
def sealedForBob := json% {enc: {key: {pk: "B"}, body: {var: "x"}}}
def filter := json% {name: "Guard", protects: ["Bob"], drop: [{atom: "bad"}]}
def intercepting := json% {name: "Guard", protects: ["Bob"], knows: [{sk: "B"}], drop
    : [{atom: "bad"}]}
```

Bob receives, through	Bob never accepts bad	good still arrives
no defender, plaintext	disproved	proved
filter, plaintext	proved	proved
filter, sealed for Bob	disproved	proved
intercepting filter (holds sk(B)), sealed for Bob	proved	proved

The filter stops a plaintext `bad` and lets `good` through. Sealed for Bob, `bad` goes straight past it: the counterexample shows Guard itself delivering `{bad}pk(B)`. Give Guard Bob's private key, as an intercepting gateway has, and it sees inside and drops it again. The table is the whole argument for and against interception in four rows.

17.8.3 Reassembly

Signature matching has a subtler gap. A defender scanning a TCP stream for `EVIL` has to put the stream back together the way the host behind it will, and the attacker controls segmentation. It can send any byte range, in any order, and send the same range again with different bytes. When two segments overlap, one implementation keeps the bytes that arrived first and another keeps the bytes that arrived last, and operating systems have differed in exactly this. `check_reassembly` models each position of the stream as empty, the signature's byte, or some other byte (written `.`). It applies every segment to the defender's copy and the host's copy with their own policies, and resets the stream when a blocking defender matches. The search covers every segmentation, of any length:

defender keeps	host keeps	never delivered	evasion
first	last	disproved	`.` at 0, then `EVIL` at 0
last	first	disproved	`E` at 0, then `.VIL` at 0
first	first	proved	—
last	last	proved	—

When the policies differ, two segments are enough. A first-wins defender is fed a decoy byte at offset 0, then the whole signature over it. It keeps the decoy and sees `.VIL`, while a last-wins host sees `EVIL`. The other mismatch is the mirror image. When the policies agree, no segmentation separates the two copies, and the property is proved over every reachable state. The same disagreement is what makes a host fingerprintable: send overlapping segments and see which bytes it acts on. Normalising the stream before scanning, as later intrusion-prevention systems did, is the fix: the defender forwards the bytes it scanned, so the host has no overlap left to resolve differently.

17.9 What these exercises demand from the library

The three examples produce a concrete checklist:

1. actor-local grammar definitions and trace projection;
2. synchronization by common event identity and bytes;
3. typed symbolic variables and prefix-dependent guards, including equalities and counters;
4. a transport-neutral term-to-byte relation with encryption and hashing;
5. channels, so that each actor's observations are declared;
6. explicit initial knowledge, compromise and freshness assumptions;
7. attacker closure under construction, decryption and hashing (now in `LeanFM.Session.Knowledge`, with its decision proved sound);
8. past operators for agreement claims such as "accepted only if previously formed";
9. probabilities on the E quantifier, from overlays, reported next to CTL verdicts;
10. deadlines on F and U, with timeout branches wherever A must meet them;
11. relational theorem schemas for indistinguishability; and
12. counterexamples drawn both as a global interaction diagram and as each actor's local projection.

None of this is security decoration. It tests whether separately written, context-sensitive role grammars stay modular while still expressing the information flow that real behavioural claims depend on.

Part III

Machine Reference

Chapter 18

The LeanFM Web-Server Protocol

This chapter is the protocol reference for the LeanFM web server. It is primarily machine-oriented context for an implementation agent and may be read as a maintained portion of the system prompt. The normative behavioral source is `LeanFM/ChatUI/Requirements.lean`; the byte vocabulary is `LeanFM/ChatUI/Requirements.proto`; and transport selection is `LeanFM/ChatUI/Implementation.lean`. The tables and grammars below are generated from those files. They are not a second, hand-maintained API design.

Two versions of the protocol must not be confused. The *accepted protocol* is the complete typed grammar and intended HTTP mapping. The *running adapter* is the subset currently dispatched by `LeanFM/ChatUI/Server.lean`. A route in the accepted table is not a claim that its handler already exists.

18.1 One exchange across four representations

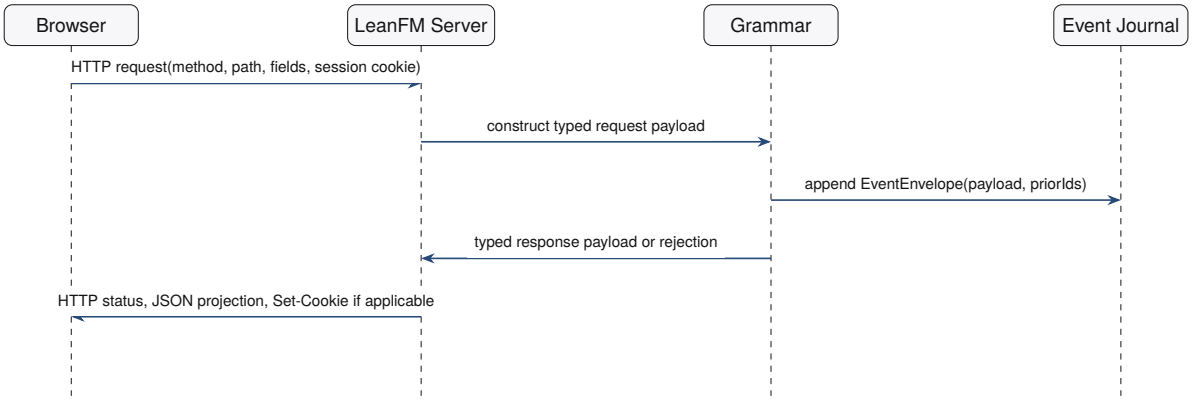


Figure 18.1: HTTP is an adapter around the typed LeanFM conversation. Source, reducer, question, and unit are recorded in `generated/web-protocol-provenance.txt`.

The figure summarizes the round trip: authenticate and decode HTTP into a grammar-permitted protobuf value, record its `EventEnvelope`, then project the selected response or rejection back to HTTP. Provider messages remain in-process, not invented HTTP calls. Protobuf fixes the typed value and stable tags; HTTP chooses the browser representation, so their bytes differ. The current

adapter uses `application/x-www-form-urlencoded` for POST requests and JSON for responses; session establishment also issues a secure cookie. The journal stores length-delimited binary `EventEnvelope`s. A future binary HTTP codec may change representation without changing message values or correlation fields. Field-level conversion follows.

18.1.1 Conservative default HTTP projection

LeanFM can generate an HTTP projection and its OpenAPI description when `Implementation.lean` does not provide an explicit binding. The task grammar supplies request/response pairing: an initiating browser-to-server message is the request, its correlated finite server-to-browser alternatives are responses, and their protobuf schemas supply the operation schemas. An explicit implementation binding always overrides the default.

The default is deliberately conservative. A field may appear in a path or query only when the implementation explicitly classifies it as a bounded, URL-safe scalar selector. Bytes, repeated fields, nested messages, secrets, large or unbounded strings, and any request whose arguments would be unnatural as a URL use `POST`; the protobuf value is carried in the request body. This prevents a convenient generator from leaking credentials, exceeding practical URL limits, or inventing ambiguous encodings. Generated encoders and decoders must preserve the protobuf value exactly.

Unless an explicit transport binding selects another codec, the default body encoder and decoder use the protobuf JSON mapping with `Content-Type: application/json`. This default is part of the generated adapter contract, not a change to the protobuf value. The server publishes the resolved OpenAPI document at `GET /openapi.yaml`; this is the default discovery endpoint for generated HTTP operations and schemas.

The generated OpenAPI operation records the resolved method and path, body content type, request schema, every grammar-permitted response schema and status, authentication requirement, correlation fields, and LeanFM requirement identifier. OpenAPI documents the transport projection; the protobuf and task grammar remain the definitions of values and legal ordering.

18.2 The conversion, field by field

For a request binding `httpRequest(method, pathTemplate)`, conversion is defined in this order:

1. Match the HTTP method exactly and split the path into decoded segments.

2. For each `{field}` in the template, take the protobuf field of that name, encode it as one path segment, and reject slashes, traversal, or a value that escapes the authenticated project root.

3. For the currently implemented POST subset, encode remaining browser input fields as UTF-8 form values. A protobuf `bytes` field is opaque bytes at the protocol boundary; a form adapter must use an explicitly declared textual encoding and must never place credentials in a URL, trace, diagnostic, or returned object.

4. Recover the authenticated user from the `leanfm_session` HttpOnly, SameSite cookie. The cookie is transport authorization, not a field silently added to every protobuf message. It authorizes model calls and use of that user's provider keys; it does not decide which project may be read, because projects are shared.

5. Validate required fields and task `where_` constraints. On success, assign the envelope metadata and the oneof payload. On failure, emit the named rejection alternative when the grammar has one; otherwise return an HTTP error without pretending a successful protocol event occurred.

For a response binding `httpResponse(RequestName)`, conversion reverses the projection. The request name selects the finite legal response set. Scalar fields become JSON strings, booleans, or numbers; repeated fields become JSON arrays; nested messages become objects; and `bytes` become an explicitly identified download or Base64 field rather than accidental UTF-8. Secrets are redacted before JSON rendering. Success commonly uses 200, creation uses 201, authentication failure uses 401 or 403, invalid input uses 400, missing owned objects use 404, revision or credential conflicts use 409, and provider failure uses 502. The named response message, not the status alone, determines the grammar transition.

Before projection, request and response are independent framed events. In that limited structural sense the stream resembles a multiplexed protocol such as HTTP/2: several exchanges may be in flight and correlation is explicit. This is not a claim that the event bytes are HTTP/2 frames. Every response envelope must set `cause_id` to the exact request event identifier within the same session and task. The HTTP adapter follows that causal backpointer to recover the exchange and emit the response on the connection that carried the request. It never pairs by adjacency, arrival order, or a process-global "current request." The separate `prior_ids` list constrains legal interleavings and is not used as a substitute for causal correlation.

HTTP projection is valid only for a causally closed exchange. Every projected request must be named by exactly one terminal response's `cause_id`, and every projected response must name one compatible request in the same session and task. Failure is still a response event: it carries an HTTP error status and points to the request. An orphan request, a response with no cause, an incompatible or cross-session cause, or two terminal responses for one request is an unchecked HTTP call and fails validation rather than becoming an underspecified adapter behavior. Protocols that deliberately permit interim or streaming responses must model those nonterminal messages explicitly and still identify one terminal response.

18.3 The running adapter

The following table is the implemented ChatUI subset. Static assets, diagnostic reports, generated source downloads, MCP, and health checks are document resources or tool protocols outside the ChatUI task grammar; they must not be mistaken for user-task protobuf events.

POST `/api/accounts`
Request: AccountCreateSubmitted. Response: AccountCreated or AccountCreateRejected. Encoding: application/x-www-form-urlencoded.

POST `/api/login`
Request: LoginSubmitted. Response: LoginAccepted or LoginRejected. Encoding: application/x-www-form-urlencoded.

GET `/api/me`
Request: authenticated session. Response: user identity JSON. Encoding: cookie only.

GET `/api/projects`
Request: ProjectListRequested. Response: ProjectListReturned. Encoding: cookie plus path.

POST `/api/projects`
Request: ProjectCreateRequested. Response: ProjectCreated. Encoding: application/x-www-form-urlencoded.

GET `/api/projects/{project_id}/conversation`
Request: project identifier. Response: conversation JSON. Encoding: path field plus cookie.

GET `/api/projects/{project_id}/tasks`
Request: project identifier. Response: task model JSON. Encoding: path field plus cookie.

GET `/api/projects/{project_id}/artifacts`
Request: ArtifactListRequested. Response: ArtifactListReturned. Encoding: path field plus cookie.

POST `/api/projects/{project_id}/chat`
Request: ChatPromptSubmitted. Response: ChatResponseRendered or ChatTurnFailed. Encoding: application/x-www-form-urlencoded.

Project routes are shared and available before login; an unknown project is 404. A valid in-memory session is required only for model calls, provider-key management and application launch, and a missing session there yields 401. The server binds to loopback by default. Network exposure requires an authenticating reverse proxy or an equivalent transport boundary.

18.4 The accepted HTTP projection

This is the complete intended browser mapping from the accepted implementation specification. Rows absent from the running-adapter table remain implementation obligations.

POST `/api/login`
Request: `LoginSubmitted`. Response: `LoginAccepted`, `LoginRejected`.

POST `/api/accounts`
Request: `AccountCreateSubmitted`. Response: `AccountCreated`, `AccountCreateRejected`.

GET `/api/projects/{project_id}/tasks/{task_id}`
Request: `TaskModelRequested`. Response: `TaskModelReturned`.

GET `/api/projects/{project_id}/tasks/{task_id}/sessions/{session_id}/events`
Request: `MessageSequenceRequested`. Response: `MessageSequenceReturned`.

GET `/api/projects/{project_id}/events/{event_id}`
Request: `ProtobufObjectRequested`. Response: `ProtobufObjectReturned`.

POST `/api/projects/{project_id}/research`
Request: `WebResearchRequested`. Response: `WebResearchReturned`, `WebResearchFailed`.

POST `/api/projects/{project_id}/markerboard/assist`
Request: `MarkerboardAssistRequested`. Response: `MarkerboardSyntaxHelpReturned`, `MarkerboardPatchPro`

POST `/api/projects/{project_id}/markerboard/assist/{artifact_id}/accept`
Request: `MarkerboardPatchAccepted`. Response: `MarkerboardDraftUpdated`.

POST `/api/projects/{project_id}/markerboard/assist/{artifact_id}/reject`
Request: `MarkerboardPatchRejected`. Response: `HTTP204(nopayload)`.

GET `/api/projects`
Request: `ProjectListRequested`. Response: `ProjectListReturned`.

POST `/api/projects`
Request: `ProjectCreateRequested`. Response: `ProjectCreated`.

GET `/api/projects/{project_id}`
Request: `ProjectOpenRequested`. Response: `ProjectOpened`, `ProblemInquiryPresented`.

DELETE `/api/projects/{project_id}`
Request: `ProjectDeleteRequested`. Response: `ProjectDeleted`.

POST `/api/projects/{project_id}/specification/problem`
Request: `ProblemStatementSubmitted`. Response: `AmbiguityQuestionPresented`, `RequirementsReviewPrese`
`ImplementationReviewPresented`.

POST `/api/projects/{project_id}/specification/ambiguities/{ambiguity_id}`
Request: `ClarificationSubmitted`. Response: `ProjectBookRevised`.

POST `/api/credentials`
Request: `ProviderCredentialSubmitted`. Response: `ProviderCredentialAccepted`, `ProviderCredentialRej`

POST `/api/projects/{project_id}/conversations/{conversation_id}/messages`
Request: `ChatPromptSubmitted`. Response: `ChatResponseRendered`, `ChatTurnFailed`.

POST `/api/projects/{project_id}/markerboard/mermaid`
Request: `MermaidDraftSaved`. Response: `MermaidPreviewRendered`.

POST `/api/projects/{project_id}/markerboard/media`
Request: `MediaUploaded`. Response: `MediaRendered`.

POST `/api/projects/{project_id}/artifacts/{artifact_id}/promotions`
Request: `ArtifactPromoted`. Response: `CandidateRequirementUpdated`.

GET `/api/projects/{project_id}/artifacts`
Request: `ArtifactListRequested`. Response: `ArtifactListReturned`.

GET `/api/projects/{project_id}/artifacts/{artifact_id}/revisions/{revision}`
Request: `ArtifactOpenRequested`. Response: `ArtifactOpened`.

Messages between `LeanFMServer` and `ModelProvider` map to the bounded in-process channels `provider.requests` and `provider.results`. They retain their protobuf message identities and request identifiers even though they do not cross HTTP. This is important for concurrency: two users can use different provider credentials because the key, session, user, project, conversation, and request identities remain request-local instead of being stored in a process-wide environment variable.

18.5 Accepted message endpoints

Every accepted event is a length-delimited `EventEnvelope`; payload tags 20 and above identify its concrete message. This generated index records each message's sender and receiver. The canonical schema that follows defines field names, tags, types, and cardinality.

`LoginSubmitted`
UserBrowser sends to LeanFMServer.

`LoginAccepted`
LeanFMServer sends to UserBrowser.

`LoginRejected`
LeanFMServer sends to UserBrowser.

`AccountCreateSubmitted`
UserBrowser sends to LeanFMServer.

`ProviderKeyCheckRequested`
LeanFMServer sends to ModelProvider.

`ProviderKeyVerified`
ModelProvider sends to LeanFMServer.

`ProviderKeyRejected`
ModelProvider sends to LeanFMServer.

`AccountCreated`
LeanFMServer sends to UserBrowser.

`AccountCreateRejected`
LeanFMServer sends to UserBrowser.

`TaskModelRequested`
UserBrowser sends to LeanFMServer.

`TaskModelReturned`
LeanFMServer sends to UserBrowser.

`MessageSequenceRequested`
UserBrowser sends to LeanFMServer.

`MessageSequenceReturned`
LeanFMServer sends to UserBrowser.

`ProtobufObjectRequested`
UserBrowser sends to LeanFMServer.

`ProtobufObjectReturned`
LeanFMServer sends to UserBrowser.

`WebResearchRequested`
UserBrowser sends to LeanFMServer.

`WebResearchDispatched`
LeanFMServer sends to ModelProvider.

`WebSearchPerformed`
ModelProvider sends to LeanFMServer.

`WebPageOpened`
ModelProvider sends to LeanFMServer.

`WebResearchReturned`
LeanFMServer sends to UserBrowser.

`WebResearchFailed`
LeanFMServer sends to UserBrowser.

`MarkerboardAssistRequested`
UserBrowser sends to LeanFMServer.

`MarkerboardSyntaxHelpReturned`
LeanFMServer sends to UserBrowser.

`MarkerboardAssistDispatched`
LeanFMServer sends to ModelProvider.

`MarkerboardAssistReturned`
ModelProvider sends to LeanFMServer.

`MarkerboardPatchProposed`
LeanFMServer sends to UserBrowser.

`MarkerboardPatchAccepted`
UserBrowser sends to LeanFMServer.

`MarkerboardPatchRejected`
UserBrowser sends to LeanFMServer.

`MarkerboardDraftUpdated`
LeanFMServer sends to UserBrowser.

`ProjectListRequested`
UserBrowser sends to LeanFMServer.

`ProjectListReturned`
LeanFMServer sends to UserBrowser.

`ProjectCreateRequested`
UserBrowser sends to LeanFMServer.

`ProjectCreated`
LeanFMServer sends to UserBrowser.

`ProjectOpenRequested`
UserBrowser sends to LeanFMServer.

`ProjectOpened`
LeanFMServer sends to UserBrowser.

`ProjectDeleteRequested`
UserBrowser sends to LeanFMServer.

`ProjectDeleted`
LeanFMServer sends to UserBrowser.

`ProblemInquiryPresented`
LeanFMServer sends to UserBrowser.

`ProblemStatementSubmitted`
UserBrowser sends to LeanFMServer.

`AmbiguityQuestionPresented`
LeanFMServer sends to UserBrowser.

`ClarificationSubmitted`
UserBrowser sends to LeanFMServer.

`ProjectBookRevised`
LeanFMServer sends to UserBrowser.

`RequirementsReviewPresented`
LeanFMServer sends to UserBrowser.

`ImplementationReviewPresented`
LeanFMServer sends to UserBrowser.

`ProviderCredentialSubmitted`
UserBrowser sends to LeanFMServer.

`ProviderCredentialAccepted`
LeanFMServer sends to UserBrowser.

`ProviderCredentialRejected`
LeanFMServer sends to UserBrowser.

`ChatPromptSubmitted`
UserBrowser sends to LeanFMServer.

`ModelRequestIssued`
LeanFMServer sends to ModelProvider.

`ModelResponseReturned`
ModelProvider sends to LeanFMServer.

`ChatResponseRendered`
LeanFMServer sends to UserBrowser.

`ChatTurnFailed`
LeanFMServer sends to UserBrowser.

`ConversationCheckpointRequested`
UserBrowser sends to LeanFMServer.

`ConversationCheckpointBlocked`
LeanFMServer sends to UserBrowser.

`ConversationCheckpointCommitted`
LeanFMServer sends to UserBrowser.

`ConversationHistoryCleared`
LeanFMServer sends to UserBrowser.

`MermaidDraftSaved`
UserBrowser sends to LeanFMServer.

`MermaidPreviewRendered`
LeanFMServer sends to UserBrowser.

`MediaUploaded`
UserBrowser sends to LeanFMServer.

`MediaRendered`
LeanFMServer sends to UserBrowser.

`ArtifactPromoted`
UserBrowser sends to LeanFMServer.

`CandidateRequirementUpdated`
LeanFMServer sends to UserBrowser.

`ArtifactListRequested`
UserBrowser sends to LeanFMServer.

`ArtifactListReturned`
LeanFMServer sends to UserBrowser.

`ArtifactOpenRequested`
UserBrowser sends to LeanFMServer.

`ArtifactOpened`
LeanFMServer sends to UserBrowser.

18.5.1 Canonical byte schema

The following complete schema is normative. It fixes envelope and payload tag numbers, repeated and nested cardinality, enum values, and field names. A reconstruction must reproduce these values rather than assigning tags in book order or replacing nested messages with opaque bytes.

```
syntax = "proto3";

package leanfm.chatui;

message EventEnvelope {
  string event_id = 1;
  repeated string prior_ids = 2;
  string session_id = 3;
  string task_id = 4;
  uint64 time_at_ms = 5;
  string src = 6;
  string dst = 7;
  // Direct causal parent for any event, including nonterminal and intermediate
  // events. Empty for a causal root. Unlike prior_ids, this does not constrain
  // otherwise legal concurrent interleavings. Renderers may use the causal
  // subtree as nested interaction boundaries.
  optional string cause_id = 8;

  // Former approval messages; review and acceptance now happen in git.
  reserved 81, 83, 84;

  oneof payload {
    LoginSubmitted login_submitted = 20;
    LoginAccepted login_accepted = 21;
    LoginRejected login_rejected = 22;
    ProjectListRequested project_list_requested = 23;
    ProjectListReturned project_list_returned = 24;
    ProjectCreateRequested project_create_requested = 25;
    ProjectCreated project_created = 26;
    ProjectOpenRequested project_open_requested = 27;
    ProjectOpened project_opened = 28;
    ProjectDeleteRequested project_delete_requested = 29;
    ProjectDeleted project_deleted = 30;
    ProviderCredentialSubmitted provider_credential_submitted = 31;
    ProviderCredentialAccepted provider_credential_accepted = 32;
    ProviderCredentialRejected provider_credential_rejected = 33;
    ChatPromptSubmitted chat_prompt_submitted = 34;
    ModelRequestIssued model_request_issued = 35;
    ModelResponseReturned model_response_returned = 36;
    ChatResponseRendered chat_response_rendered = 37;
    ChatTurnFailed chat_turn_failed = 38;
    MermaidDraftSaved mermaid_draft_saved = 39;
    MermaidPreviewRendered mermaid_preview_rendered = 40;
    MediaUploaded media_uploaded = 41;
    MediaRendered media_rendered = 42;
    ArtifactPromoted artifact_promoted = 43;
    CandidateRequirementUpdated candidate_requirement_updated = 44;
    ArtifactListRequested artifact_list_requested = 45;
    ArtifactListReturned artifact_list_returned = 46;
    ArtifactOpenRequested artifact_open_requested = 47;
    ArtifactOpened artifact_opened = 48;
    AccountCreateSubmitted account_create_submitted = 49;
    ProviderKeyCheckRequested provider_key_check_requested = 50;
    ProviderKeyVerified provider_key_verified = 51;
    ProviderKeyRejected provider_key_rejected = 52;
    AccountCreated account_created = 53;
    AccountCreateRejected account_create_rejected = 54;
    TaskModelRequested task_model_requested = 55;
    TaskModelReturned task_model_returned = 56;
    MessageSequenceRequested message_sequence_requested = 57;
    MessageSequenceReturned message_sequence_returned = 58;
    ProtobufObjectRequested protobuf_object_requested = 59;
    ProtobufObjectReturned protobuf_object_returned = 60;
    WebResearchRequested web_research_requested = 61;
    WebSearchPerformed web_search_performed = 62;
    WebPageOpened web_page_opened = 63;
    WebResearchReturned web_research_returned = 64;
    WebResearchFailed web_research_failed = 65;
    WebResearchDispatched web_research_dispatched = 66;
    MarkerboardAssistRequested markerboard_assist_requested = 67;
    MarkerboardSyntaxHelpReturned markerboard_syntax_help_returned = 68;
    MarkerboardAssistDispatched markerboard_assist_dispatched = 69;
    MarkerboardAssistReturned markerboard_assist_returned = 70;
    MarkerboardPatchProposed markerboard_patch_proposed = 71;
    MarkerboardPatchAccepted markerboard_patch_accepted = 72;
    MarkerboardPatchRejected markerboard_patch_rejected = 73;
    MarkerboardDraftUpdated markerboard_draft_updated = 74;
    ProblemInquiryPresented problem_inquiry_presented = 75;
    ProblemStatementSubmitted problem_statement_submitted = 76;
    AmbiguityQuestionPresented ambiguity_question_presented = 77;
    ClarificationSubmitted clarification_submitted = 78;
    ProjectBookRevised project_book_revised = 79;
    RequirementsReviewPresented requirements_review_presented = 80;
    ImplementationReviewPresented implementation_review_presented = 82;
    ConversationCheckpointRequested conversation_checkpoint_requested = 85;
    ConversationCheckpointBlocked conversation_checkpoint_blocked = 86;
    ConversationCheckpointCommitted conversation_checkpoint_committed = 87;
    ConversationHistoryCleared conversation_history_cleared = 88;
```

```
  }
}

enum ContentKind {
  CONTENT_KIND_UNSPECIFIED = 0;
  MARKDOWN = 1;
  LATEX = 2;
  MERMAID = 3;
  GRAPHVIZ_DOT = 4;
  IMAGE = 5;
  VIDEO = 6;
  CODE = 7;
  FILE = 8;
  LEANFM_VIEW = 9;
  URL_CITATION = 10;
}

message ChatContentPart {
  string part_id = 1;
  ContentKind kind = 2;
  string text = 3;
  string artifact_id = 4;
  uint64 artifact_revision = 5;
  string media_type = 6;
  bytes inline_content = 7;
  string alt_text = 8;
  string url = 9;
  string title = 10;
}

message LoginSubmitted {
  string username = 1;
  bytes credential = 2;
  bytes openai_api_key = 3;
  bytes anthropic_api_key = 4;
  bytes new_password = 5;
  bytes new_password_confirm = 6;
}
message LoginAccepted { string user_id = 1; string session_id = 2; }
message LoginRejected { string reason = 1; }

message ProblemInquiryPresented { string project_id = 1; string question = 2; }
message ProblemStatementSubmitted { string project_id = 1; string statement = 2; }
message AmbiguityQuestionPresented { string project_id = 1; string ambiguity_id = 2; string question = 3; repeated string affected_artifacts
       = 4; }
message ClarificationSubmitted { string project_id = 1; string ambiguity_id = 2; string answer = 3; }
message ProjectBookRevised { string project_id = 1; string book_revision = 2; repeated string decision_ids = 3; string light_pdf = 4; string
       dark_pdf = 5; }
message RequirementsReviewPresented { string project_id = 1; string requirements_revision = 2; repeated string derived_artifact_ids = 3;
      repeated string open_ambiguity_ids = 4; }
message ImplementationReviewPresented { string project_id = 1; string requirements_revision = 2; string implementation_revision = 3; string
      traceability_report = 4; }

message AccountCreateSubmitted {
  string username = 1;
  bytes login_credential = 2;
  string provider = 3;
  bytes api_key = 4;
  bytes openai_api_key = 5;
  bytes anthropic_api_key = 6;
}
message ProviderKeyCheckRequested { string provider = 1; string verification_id = 2; }
message ProviderKeyVerified { string verification_id = 1; string provider_account = 2; }
message ProviderKeyRejected { string verification_id = 1; string reason = 2; }
message AccountCreated { string user_id = 1; string credential_ref = 2; }
message AccountCreateRejected { string reason = 1; }

message TaskModelRequested { string project_id = 1; string task_id = 2; string requirements_revision = 3; }
message TaskModelReturned {
  string task_id = 1;
  string interaction_artifact_id = 2;
  string state_machine_artifact_id = 3;
  string message_sequence_artifact_id = 4;
  repeated string constraint_ids = 5;
}
message MessageSequenceRequested { string project_id = 1; string task_id = 2; string session_id = 3; }
message MessageSequenceReturned { string task_id = 1; string session_id = 2; repeated string event_ids = 3; }
message ProtobufObjectRequested { string project_id = 1; string event_id = 2; }
message ProtobufObjectReturned { string event_id = 1; string protobuf_type = 2; bytes encoded_object = 3; string redacted_json = 4; }
message WebResearchRequested { string project_id = 1; string conversation_id = 2; string request_id = 3; string question = 4; repeated
      string supplied_urls = 5; }
message WebResearchDispatched { string request_id = 1; string model = 2; string research_mode = 3; }
message WebSearchPerformed { string request_id = 1; repeated string queries = 2; }
message WebPageOpened { string request_id = 1; string url = 2; string title = 3; }
message WebResearchReturned { string request_id = 1; repeated ChatContentPart parts = 2; repeated string source_urls = 3; }
message WebResearchFailed { string request_id = 1; string reason = 2; }
message MarkerboardAssistRequested { string project_id = 1; string artifact_id = 2; uint64 base_revision = 3; string diagram_kind = 4;
      string source = 5; string instruction = 6; string mode = 7; string credential_ref = 8; }
message MarkerboardSyntaxHelpReturned { string artifact_id = 1; uint64 base_revision = 2; string help_markdown = 3; }
```

```
message MarkerboardAssistDispatched { string request_id = 1; string artifact_id = 2; uint64 base_revision = 3; string diagram_kind = 4;
      string source = 5; string instruction = 6; }
message MarkerboardAssistReturned { string request_id = 1; string proposed_source = 2; string explanation = 3; uint64 input_tokens = 4;
      uint64 output_tokens = 5; }
message MarkerboardPatchProposed { string artifact_id = 1; uint64 base_revision = 2; string proposed_source = 3; string explanation = 4; }
message MarkerboardPatchAccepted { string artifact_id = 1; uint64 base_revision = 2; string proposed_source = 3; }
message MarkerboardPatchRejected { string artifact_id = 1; uint64 base_revision = 2; }
message MarkerboardDraftUpdated { string artifact_id = 1; uint64 revision = 2; string source = 3; }

message ProjectListRequested { string user_id = 1; }
message ProjectListReturned { repeated string project_ids = 1; }
message ProjectCreateRequested { string display_name = 1; }
message ProjectCreated { string project_id = 1; string display_name = 2; }
message ProjectOpenRequested { string project_id = 1; }
message ProjectOpened { string project_id = 1; string current_requirements_revision = 2; }
message ProjectDeleteRequested { string project_id = 1; string confirmation = 2; }
message ProjectDeleted { string project_id = 1; }

message ProviderCredentialSubmitted {
  string provider = 1;
  bytes api_key = 2;
  string retention = 3;
}
message ProviderCredentialAccepted { string credential_ref = 1; string billing_source = 2; }
message ProviderCredentialRejected { string reason = 1; }

message ChatPromptSubmitted {
  string project_id = 1;
  string conversation_id = 2;
  string message_id = 3;
  repeated ChatContentPart parts = 4;
  repeated string attachment_ids = 5;
  string credential_ref = 6;
  string model = 7;
}
message ModelRequestIssued {
  string project_id = 1;
  string conversation_id = 2;
  string request_id = 3;
  string model = 4;
  string research_mode = 5;
}
message ModelResponseReturned {
  string request_id = 1;
  repeated ChatContentPart parts = 2;
  uint64 input_tokens = 3;
  uint64 cached_input_tokens = 4;
  uint64 output_tokens = 5;
  uint64 context_window_tokens = 6;
}
message ChatResponseRendered {
  string conversation_id = 1;
  string message_id = 2;
  repeated ChatContentPart parts = 3;
  repeated string artifact_ids = 4;
  uint64 input_tokens = 5;
  uint64 cached_input_tokens = 6;
  uint64 output_tokens = 7;
  uint64 context_window_tokens = 8;
  string usage_warning = 9;
  uint64 response_time_ms = 10;
}
message ChatTurnFailed { string request_id = 1; string reason = 2; }
message ConversationCheckpointRequested { string project_id = 1; string conversation_id = 2; string through_message_id = 3; }
message ConversationCheckpointBlocked { string project_id = 1; string conversation_id = 2; string checkpoint_id = 3; string reason_code = 4;
       repeated string blocking_artifact_ids = 5; }
message ConversationCheckpointCommitted { string project_id = 1; string conversation_id = 2; string through_message_id = 3; repeated string
      artifact_revisions = 4; string manifest_digest = 5; }
message ConversationHistoryCleared { string project_id = 1; string conversation_id = 2; string cleared_through_message_id = 3; uint64
      retained_message_count = 4; }

message MermaidDraftSaved {
  string project_id = 1;
  string artifact_id = 2;
  uint64 revision = 3;
  string diagram_kind = 4;
  string source = 5;
}
message MermaidPreviewRendered { string artifact_id = 1; uint64 revision = 2; bytes svg = 3; }
message MediaUploaded {
  string project_id = 1;
  string artifact_id = 2;
  string media_type = 3;
  bytes content = 4;
}
message MediaRendered { string artifact_id = 1; string media_type = 2; string content_url = 3; }
message ArtifactPromoted { string artifact_id = 1; uint64 revision = 2; string target_status = 3; }
message CandidateRequirementUpdated { string requirements_revision = 1; repeated string source_artifact_ids = 2; }
```

```
message ArtifactListRequested {
  string project_id = 1;
  string kind_filter = 2;
  string scenario_filter = 3;
  string status_filter = 4;
}
message ArtifactSummary {
  string artifact_id = 1;
  uint64 current_revision = 2;
  uint64 accepted_revision = 3;
  string status = 4;
  string validation = 5;
  string media_type = 6;
  string title = 7;
  string scenario_id = 8;
  string diagram_kind = 9;
  string updated_by = 10;
  uint64 updated_at_ms = 11;
}
message ArtifactListReturned { repeated ArtifactSummary artifacts = 1; }
message ArtifactOpenRequested { string artifact_id = 1; uint64 revision = 2; }
message ArtifactOpened {
  string artifact_id = 1;
  uint64 revision = 2;
  string status = 3;
  string media_type = 4;
  bytes content = 5;
  string provenance = 6;
}
```

Envelope metadata remain distinct. `prior_ids` stores immutable predecessor links; any `map[Event][]Event` successor index is derived, not serialized as mutable truth. `session_id` separates concurrent conversations, `task_id` selects the grammar, and `src`/`dst` must match the message schema. As defined above, `cause_id` is a separate acyclic direct-cause relation usable by roots, intermediate messages, nested task boundaries, and terminals. Its tree groups renderer boxes without ordering concurrent children.

18.6 Complete LeanFM task grammar

The following is the whole accepted ChatUI conversation grammar, rendered from the typed `TaskGrammar` values. Pronounce `A ! B : Message` as "A sends Message to B." `then` is ordered composition; `choose`/`or` is an explicit alternative; `optional` permits zero or one occurrence; `fork`/`join all` waits for every branch; and `race`/`join first` accepts the first completion and cancels the remaining branches. (A threshold join in Chapter 5 is a different construct: its unselected branches are not implicitly canceled.) A message not admitted by the selected task's current residual grammar is illegal even if its protobuf bytes decode.

18.7 Authenticate an individual user

Task: `authenticate_user`. Entry: `signed_out`; terminals: `authenticated, rejected`.

```
UserBrowser ! LeanFMServer : LoginSubmitted
then
choose
  LeanFMServer ! UserBrowser : LoginAccepted
or
  LeanFMServer ! UserBrowser : LoginRejected
```

18.8 Create an account using a user-supplied provider API key

Task: `create_account`. Entry: `registration`; terminals: `created, rejected`.

```
UserBrowser ! LeanFMServer : AccountCreateSubmitted
then
LeanFMServer ! ModelProvider : ProviderKeyCheckRequested
then
choose
  ModelProvider ! LeanFMServer : ProviderKeyVerified
  then
  LeanFMServer ! UserBrowser : AccountCreated
or
  ModelProvider ! LeanFMServer : ProviderKeyRejected
  then
  LeanFMServer ! UserBrowser : AccountCreateRejected
```

18.9 List, create, open, and delete owned projects

Task: `manage_projects`. Entry: `ready`; terminals: `listed, created, opened, deleted`.

```
choose
  UserBrowser ! LeanFMServer : ProjectListRequested
  then
  LeanFMServer ! UserBrowser : ProjectListReturned
or
  UserBrowser ! LeanFMServer : ProjectCreateRequested
  then
  LeanFMServer ! UserBrowser : ProjectCreated
or
  UserBrowser ! LeanFMServer : ProjectOpenRequested
  then
  LeanFMServer ! UserBrowser : ProjectOpened
or
  UserBrowser ! LeanFMServer : ProjectDeleteRequested
  then
  LeanFMServer ! UserBrowser : ProjectDeleted
```

18.10 Turn a fresh project conversation into checked requirements and implementation

Task: `develop_project_specification`. Entry: `project_loaded`; terminals: `ready_to_implement`.

```
LeanFMServer ! UserBrowser : ProblemInquiryPresented
then
UserBrowser ! LeanFMServer : ProblemStatementSubmitted
```

```
then
repeat
  choose
    empty
  or
    LeanFMServer ! UserBrowser : ProjectBookRevised
  then
  LeanFMServer ! UserBrowser : AmbiguityQuestionPresented
  then
  UserBrowser ! LeanFMServer : ClarificationSubmitted
then
choose
  empty
or
  LeanFMServer ! UserBrowser : ProjectBookRevised
then
LeanFMServer ! UserBrowser : RequirementsReviewPresented
then
LeanFMServer ! UserBrowser : ImplementationReviewPresented
```

18.11 Select user-funded or platform-funded model access

Task: `configure_provider`. Entry: `unconfigured`; terminals: `configured, rejected`.

```
UserBrowser ! LeanFMServer : ProviderCredentialSubmitted
then
choose
  LeanFMServer ! UserBrowser : ProviderCredentialAccepted
or
  LeanFMServer ! UserBrowser : ProviderCredentialRejected
```

18.12 Conduct one durable project-scoped requirements turn

Task: `argue_requirements`. Entry: `ready`; terminals: `rendered, failed`.

```
UserBrowser ! LeanFMServer : ChatPromptSubmitted
then
LeanFMServer ! ModelProvider : ModelRequestIssued
then
choose
  ModelProvider ! LeanFMServer : ModelResponseReturned
  then
  LeanFMServer ! UserBrowser : ChatResponseRendered
or
  LeanFMServer ! UserBrowser : ChatTurnFailed
```

18.13 Compact a project conversation into durable specifications before clearing it

Task: `checkpoint_conversation`. Entry: `retained`; terminals: `cleared, blocked`.

```
UserBrowser ! LeanFMServer : ConversationCheckpointRequested
then
choose
  LeanFMServer ! UserBrowser : ConversationCheckpointBlocked
or
  LeanFMServer ! UserBrowser : ConversationCheckpointCommitted
  then
  LeanFMServer ! UserBrowser : ConversationHistoryCleared
```

18.14 Sketch and promote multimodal requirement artifacts

Task: `markerboard`. Entry: `editing`; terminals: `candidate`.

```
choose
  UserBrowser ! LeanFMServer : MermaidDraftSaved
  then
  LeanFMServer ! UserBrowser : MermaidPreviewRendered
  then
  UserBrowser ! LeanFMServer : ArtifactPromoted
  then
  LeanFMServer ! UserBrowser : CandidateRequirementUpdated
or
  UserBrowser ! LeanFMServer : MediaUploaded
  then
  LeanFMServer ! UserBrowser : MediaRendered
  then
  UserBrowser ! LeanFMServer : ArtifactPromoted
  then
  LeanFMServer ! UserBrowser : CandidateRequirementUpdated
```

18.15 Inspect current, accepted, superseded, and stale artifacts

Task: `inspect_artifacts`. Entry: `closed`; terminals: `listed, opened`.

```
UserBrowser ! LeanFMServer : ArtifactListRequested
then
LeanFMServer ! UserBrowser : ArtifactListReturned
then
choose
  empty
or
```

```
  UserBrowser ! LeanFMServer : ArtifactOpenRequested
  then
  LeanFMServer ! UserBrowser : ArtifactOpened
```

18.16 Inspect one task's linked interaction, state machine, constraints, and protobuf events

Task: `inspect_task_model`. Entry: `closed`; terminals: `linked, sequence_visible, object_visible`.

```
UserBrowser ! LeanFMServer : TaskModelRequested
then
LeanFMServer ! UserBrowser : TaskModelReturned
then
choose
  empty
or
  UserBrowser ! LeanFMServer : MessageSequenceRequested
  then
  LeanFMServer ! UserBrowser : MessageSequenceReturned
  then
  choose
    empty
  or
    UserBrowser ! LeanFMServer : ProtobufObjectRequested
    then
    LeanFMServer ! UserBrowser : ProtobufObjectReturned
```

18.17 Search the public web, follow supplied HTTP URLs, and return clickable citations

Task: `research_web`. Entry: `ready`; terminals: `returned, failed`.

```
UserBrowser ! LeanFMServer : WebResearchRequested
then
LeanFMServer ! ModelProvider : WebResearchDispatched
then
choose
  ModelProvider ! LeanFMServer : WebSearchPerformed
  then
  ModelProvider ! LeanFMServer : WebPageOpened
  then
  LeanFMServer ! UserBrowser : WebResearchReturned
or
  LeanFMServer ! UserBrowser : WebResearchFailed
```

18.18 Recall diagram syntax or propose AI-generated detail for one markerboard revision

Task: `assist_markerboard`. Entry: `editing`; terminals: `helped, updated, rejected`.

```
UserBrowser ! LeanFMServer : MarkerboardAssistRequested
then
choose
  LeanFMServer ! UserBrowser : MarkerboardSyntaxHelpReturned
or
  LeanFMServer ! ModelProvider : MarkerboardAssistDispatched
  then
  ModelProvider ! LeanFMServer : MarkerboardAssistReturned
  then
  LeanFMServer ! UserBrowser : MarkerboardPatchProposed
  then
  choose
    UserBrowser ! LeanFMServer : MarkerboardPatchAccepted
    then
    LeanFMServer ! UserBrowser : MarkerboardDraftUpdated
  or
    UserBrowser ! LeanFMServer : MarkerboardPatchRejected
```

18.19 The grammar as an executable canvas

Future interface. The complete grammar is one navigable canvas rather than one permanently huge drawing. A task or production node may be closed to its typed boundary or opened to show its interaction messages, residual states, parameters, constraints, paired measurements and nested productions. Opening and closing is presentation state only. It never changes the grammar or conceals a failing obligation.

Each editable parameter has one typed definition and explicit dependency edges. Changing a queue capacity, actor population, timeout, probability or other parameter would mark every transitive dependent stale and regenerate the affected productions, reducers, proofs, codecs, charts and views, showing that cascade before the revision is committed. None of this is implemented yet.

A production known to cross HTTP may attach its transport renderer at that production. The binding names the method, path template, request and response protobuf types, status alternatives, content type, byte encoder and byte decoder. Generated fixtures emit the exact request and response bytes and read them back. Conformance requires decoded emitted values to equal their originals and their message atoms to be accepted by the selected residual grammar. Thus the high-level canvas can be progressively opened all the way down to bytes without confusing transport with the message value.

18.20 What is checked, and what remains

Lean elaboration checks the actors, message schemas, task states, transitions, grammars, constraints, desired-output provenance, and implementation mappings. The chapter generator rejects an accepted HTTP request with no named response mapping and rejects incomplete running-route records. The ordinary repository checks also validate the protobuf source against the typed requirements.

This does not yet prove that every accepted route has a handler: the running table makes that gap explicit. Nor does the current server yet append every browser exchange to the specified binary event journal. Completing those handlers and the schema-specific codec remains implementation work. Until then, the accepted grammar is the protocol contract, the running table is the honest conformance statement, and neither may be substituted for the other.

Chapter 19

The LeanFM Project Bootstrap Contract

This chapter defines the minimum machine-readable context that makes a LeanFM project self-hosting. It belongs in the machine reference because an agent must apply it before it can turn the rest of the book into checked artifacts. The bootstrap defines the language and its tools; it must not contain a hidden copy of the product requirements or implementation.

The intended handoff is deliberately simple: send another person the URL of this PDF, ask Claude, Gemini, Codex, or another capable construction agent to read it, and ask that agent to make the software. New ChatUI projects are provisioned from built-in book and specification templates; they do not depend on an example project directory. The live language-reference tools and read-only MCP surface supply the generic authoring types and commands named below. An agent must not copy the existing LeanFM application source or prior ChatUI specifications as an answer key.

19.1 Required project inputs

A copyable project root contains the following stable inputs:

`book/main.pdf`
: Central product-design argument and maintained system-prompt context.

`book/main-dark.pdf`
: Accessible rendering of the same source.

`LeanFM/`
: Versioned generic authoring types and deterministic validators. Product actors, messages, tasks and routes do not belong in this directory.

Bootstrap programs
: Whatever prints the language references and interrogation checklist, validates the conventional specification, and derives review views from its typed value. In this repository these are the `make` targets and the LeanFM server's routes and MCP tools listed below; a portable project may ship a `bootstrap/` directory instead.

`lean-toolchain` **and** `lakefile.toml`
: The exact compiler selection and module roots needed to run the bootstrap locally.

`project.json`
: Project identity, stable artifact paths, and the classification of design versus language inputs.

All project-owned state and decisions belong below the exclusive root: inputs, conversations, artifact revisions, source, tests, generated views, traces, reports, logs, and project-specific caches. Choose the root before session work; temporary files and atomic promotion stay inside it. Reject configured outputs and symlinks that escape. Export declared artifacts explicitly; import by copying into the root or recording an immutable, content-addressed dependency. Shared tools and caches may live outside only without project-owned state. A global index must be reconstructible from manifests, and deleting one root must remove all its state without an external search.

The project root is also the complete input to a fresh agent conversation. The agent discovers the stable paths from the manifest and repository instructions; the stakeholder need not paste a transcript or restate a bootstrap procedure. If the root has no durable problem statement, LeanFM creates the minimal editable book shell and begins with the visible question “What problem are you trying to solve?” The answer starts requirements discovery. Named ambiguity questions, book revisions and validation evidence then accumulate beneath that same root.

19.2 Conventional generated interface

The requirements candidate lives at `spec/Requirements.lean` (this repository keeps its own in `LeanFM/LLMGenerated/`), imports `LeanFM.Artifacts`, and defines:

```
namespace LeanFMProject.Requirements
def spec : LeanFM.RequirementSpec := ...
def all : List LeanFM.GeneratedRequirement := [.requirement spec]
end LeanFMProject.Requirements
```

The byte vocabulary lives beside it at `spec/Requirements.proto`. Once the requirements check, the separate implementation mapping lives at `spec/Implementation.lean`. Capitalization is significant because these are Lean module paths, not informal filenames.

19.3 Required commands and order

The project supplies deterministic equivalents of these commands. In this repository the references and checklist come from the running server (`/tools/leanfm-language/requirements-reference`, `/tools/requirements/interrogation`, or the MCP resources of the same name), and `make check` validates and renders:

```
make reference       # print the complete requirements-language reference
make interrogation   # print questions that expose missing observations
make validate        # elaborate and run LeanFM plus protobuf validation
make render          # derive interaction and residual-state review views
make check           # validate, then render
```

An agent runs `reference` and `interrogation` before authoring. It must not infer constructors from prose or invent a smaller Lean lookalike. The validator consumes the exported `RequirementSpec`; successful Lean elaboration by itself is not LeanFM validation. The renderer also consumes that same value. A separately written diagram is not evidence that it matches the requirements.

19.4 Three layers, reviewed in git

Work proceeds in three layers: observable requirements, then their realization mapping, then source. Each layer is checked before the next is built on it, and application source follows the security review in Chapter 14. Review and acceptance are ordinary git commits: the stakeholder reads the diff and commits it. LeanFM records no approvals and never makes an edit, a source write or a launch wait for one; its project status shows only whether each layer passes its canonical check and whether it is committed.

`Requirements.lean` defines the contract. Its multi-actor wire grammar is the conformance oracle: complete decoded traffic is legal or has a counterexample; an incomplete capture cannot establish conformance. `Requirements.proto` fixes byte values without choosing transport. `Implementation.lean` records choices needed to realize accepted behavior—for example queues, transports, journals or browser mechanisms—each mapped to its requirement. Answers to implementation questions belong there when they leave observable behavior unchanged. An answer changing messages, fields, ordering or another observable must instead revise Requirements. Remove implementation answers made unnecessary by later requirements changes rather than preserving them as requirements.

The bootstrap therefore solves a circularity. The book explains what is being built; the generic distribution makes the LeanFM language executable; committed project artifacts preserve the decisions. None is a substitute for the other two.

19.5 Construction is not a runtime dependency

The construction agent is not part of the delivered server. After construction, the user starts LeanFM, visits its root URL in an ordinary browser, creates or logs into an account, supplies a model-provider key, and conducts the requirements argument through LeanFM's own ChatGPT-like interface. The server talks to the selected model provider through its documented message protocol; it does not require the construction agent, its conversation, or local tooling.

Consequently, a reconstruction is not successful merely because it can be driven from a coding-agent terminal. It must build a standalone executable, serve the complete browser workflow, start without a platform API key, isolate concurrent users and their credentials, and pass its conformance tests without Codex being present.

19.6 Build, runtime, and publication dependency inventory

The repository does not treat one developer's workstation as an implicit build input. The machine-readable `toolDependencyInventory` in `LeanFM/ChatUI/Requirements.lean` is normative; this section is its human review view. "Required" means that the default check, server, book, or stakeholder-media path invokes the dependency. "Optional" means the named workflow is outside the default product build. JavaScript versions are fixed by `ui/package-lock.json`, Python ranges by `scripts/chatui_requirements.txt`, and Lean and Go by `lean-toolchain` and the top-level `Makefile`.

Lean and Lake — required.

Lean 4.35.0 from `lean-toolchain` compiles the DSL, validators, renderers, tests, and server.

Lake resolves modules and drives native linking.

C toolchain and libc headers — required.
A compiler and linker supported by Lean 4.35 link the native executables emitted by Lean.

GNU Make, Bash, and coreutils — required.
The workflows use `timeout`, `cmp`, `cp`, `tar`, `gzip`, and `sha256sum` for bounded commands and verified tool downloads.

Node and npm — required.
Node 22.12 or newer and `npm ci` type-check and bundle the React UI. The lockfile fixes React, ReactDOM, TypeScript, Vite, KaTeX, and their transitive packages.

Mermaid renderer — required for task diagrams.
The existing Mermaid 11 renderer is served locally. `assets/vendor/mermaid.json` records its resolved version, source, MIT license and SHA-256 digest, so diagrams and screenshot scenarios do not require a CDN connection.

Python packages — required.
Python 3.12 or newer with Cryptography 41–50, Pillow 10–12, and Protobuf 5–6 protects retained keys, performs protobuf round trips and reductions, and generates animated GIFs. CairoSVG 2.7–2.x and defusedxml 0.7–0.x normalize static SVG attachments without external resources; Pillow supplies bounded raster decoding.

Protocol Buffers compiler — required.
A `protoc` supporting proto3 and Python output generates the temporary byte-round-trip binding. Target-language generators are declared separately when needed.

Graphviz — required.
`dot` with PNG output rasterizes generated DOT diagrams during `make diagrams`.

TeX Live and pdfLaTeX — required.
Install the base, recommended, extra, pictures, and recommended-font collections. The source uses `lmodern`, TeX Gyre Heros, geometry, microtype, xcolor, graphicx, tagpdf, AMS math, mathtools, booktabs, longtable, enumitem, listings, natbib, imakeidx, TikZ, pgfplots, hyperref, and cleveref.

Go — required.
Go 1.23.12, whose archive hash is fixed by the Makefile, compiles and tests the Go example projects, such as `projects/PaxosKV`.

Bubblewrap — required on the Linux server.
`bwrap` with user, process, network, and mount namespaces isolates executable Lean checks, project commands, persistent applications, and project-local media generators. Ubuntu/Debian supplies it in the `bubblewrap` package. Run `make sandbox-check` before the full suite to verify that the host's namespace policy permits the actual worker. Missing or denied sandbox support fails explicitly; it never authorizes execution outside the sandbox.

FFmpeg — required for shareable video.
FFmpeg with `libx264` H.264 and AAC encoders converts GIF/WebM to validated MP4, adds fast-start metadata, and muxes optional narration.

curl, ripgrep, and iproute2 — required for development checks.
`curl`, `rg`, and `ss` exercise HTTP contracts, select validation evidence, and identify the local server.

Java and TLA+ tools — optional.
A Java runtime and the checksummed `tla2tools.jar` 1.7.4 run the separate `tla-check` example.

Manim and Piper — optional.
A project-local video Python environment and selected voice model render explanatory videos and synthetic narration.

Target compilers and SDKs — conditional.
The accepted `ImplementationSpec` inventories every compiler, code generator, SDK, runtime, native library, and version bound before source generation.

An agent can reconcile this declaration with the current host and report each item as satisfied, missing, incompatible, or optional. Installing or upgrading host software remains an explicit bounded action; inspection alone never silently mutates the machine. DuckDB is not currently invoked by a checked LeanFM target and is therefore not a build dependency. If a DuckDB-backed reducer is accepted, its CLI or library, binding, storage format, and version contract must first enter this inventory and the implementation mapping. The same rule prevents a locally installed compiler or media program from silently becoming part of the product contract.

19.7 Local and server-hosted references

The local commands are authoritative when no server is running. A running LeanFM server additionally exposes the requirements and implementation language references, the interrogation checklist, and read-only MCP context without requiring a user login. These public endpoints contain language documentation, not project or account data. Projects are shared; credentials stay private to their account, and model calls require that account's session.

Selected Books and Further Reading

These annotations identify the role each work plays in this book; they are not reviews of the books.

Richards and Hamburger, *Logic and Language Models for Computer Science*.
A source of the author's background in formal languages and logic, not the originators of those methods [21].

Gensler, *Introduction to Logic*.
Background for symbolization, validity, proof, quantification, and modal reasoning [8].

Ryan et al., *The Modelling and Analysis of Security Protocols*.
The CSP treatment of protocol analysis behind the approach of the protocol exercises, whose examples are the author's own [22].

Honda; Honda, Vasconcelos, and Kubo; Honda, Yoshida, and Carbone.
The origin of binary and then multiparty session types, the backbone of the method [10–12].

Yoshida, Hu, Neykova, and Ng, *The Scribble Protocol Language*.
The source of the `from`/`to`/`choice` surface style used for global types in this book (LeanFM drops Scribble's `at`: the role that decides is the one that starts every branch) [25].

Deniélou and Yoshida; Brand and Zafiropulo.
The link between session types and communicating finite-state machines, and why unbounded queues make verification undecidable, which motivates LeanFM's declared capacities [4, 6].

Barwell, Scalas, Yoshida, and Zhou.
One line of work that adds failure to session types, here crash-stop roles; LeanFM instead declares failure per channel [2].

Kröger and Merz, *Temporal Logic and State Systems*.
The author's principal source for temporal-logic background [13].

Lamport, *Specifying Systems*.
A reference point for the book's comparison with TLA+ state and action specifications [15].

Gunther, *Guerrilla Capacity Planning*.
The cited background for the Universal Scalability Law and capacity planning [9].

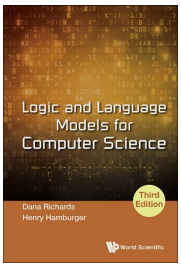
Menascé, Almeida, and Dowdy, *Performance by Design*.
Background for the queueing and capacity-planning discussion [18].

Mancosu, Galvan, and Zach, *An Introduction to Proof Theory*.
Further reading on natural deduction, sequent calculus, normalization, and cut elimination [17].

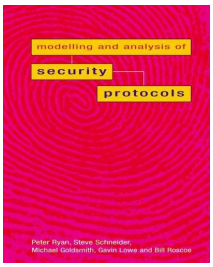
Pavlovic, *Programs as Diagrams*.
Related further reading; the current book does not rely on a specific claim from it [20].

Book-cover snapshots

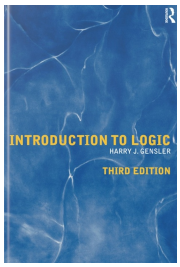
These cover snapshots identify the referenced editions.



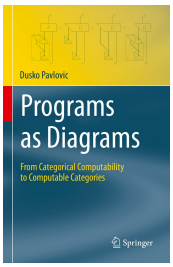
Logic and Language Models for Computer Science
Dana Richards and Henry Hamburger [21]



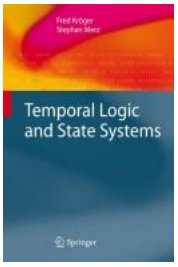
The Modelling and Analysis of Security Protocols
Peter Ryan, Steve Schneider, Michael Goldsmith, Gavin Lowe, and Bill Roscoe [22]



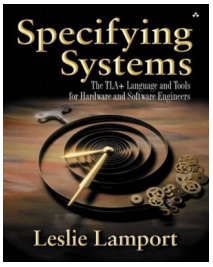
Introduction to Logic
Harry J. Gensler [8]



Programs as Diagrams
Dusko Pavlovic [20]



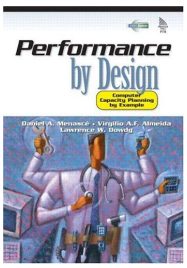
Temporal Logic and State Systems
Fred Kröger and Stephan Merz [13]



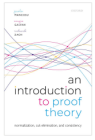
Specifying Systems
Leslie Lamport [15]



Guerrilla Capacity Planning
Neil J. Gunther [9]



Performance by Design
Daniel A. Menascé, Virgilio A. F. Almeida, and Lawrence W. Dowdy [18]



An Introduction to Proof Theory
Paolo Mancosu, Sergio Galvan, and Richard Zach [17]

Cover-image sources, accessed September 25 and 30, 2026: World Scientific edition via Kobo for Richards and Hamburger; Open Library for Ryan et al.; Apple Books for Gensler; and Kobo's Springer edition for Pavlovic. The Kröger and Merz cover snapshot is from Open Library. The Lamport and Menascé–Almeida–Dowdy snapshots are from Open Library; the Gunther snapshot is from Springer Nature; and the Mancosu–Galvan–Zach snapshot is from Google Books. Cover artwork remains the property of its respective publisher.

Bibliography

[1] FizzBee: Performance modeling. `https://fizzbee.io/tutorials/performance-modeling`, 2026. Probabilities and counters are supplied in a separate YAML file passed to the performance checker.

[2] Adam D. Barwell, Alceste Scalas, Nobuko Yoshida, and Fangyi Zhou. Generalised multiparty session types with crash-stop failures. In *33rd International Conference on Concurrency Theory (CONCUR 2022)*, volume 243 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 35:1–35:25, 2022. doi: 10.4230/LIPIcs.CONCUR.2022.35.

[3] Robert Braden. Requirements for internet hosts – application and support. RFC 1123, 1989. Section 4.2.3.1 describes and fixes the Sorcerer's Apprentice Syndrome.

[4] Daniel Brand and Pitro Zafiropulo. On communicating finite-state machines. *Journal of the ACM*, 30(2):323–342, 1983. doi: 10.1145/322374.322380.

[5] Edmund M. Clarke, E. Allen Emerson, and A. Prasad Sistla. Automatic verification of finite-state concurrent systems using temporal logic specifications. *ACM Transactions on Programming Languages and Systems*, 8(2):244–263, 1986. doi: 10.1145/5397.5399.

[6] Pierre-Malo Deniélou and Nobuko Yoshida. Multiparty session types meet communicating automata. In *Programming Languages and Systems (ESOP 2012)*, volume 7211 of *Lecture Notes in Computer Science*, pages 194–213. Springer, 2012. doi: 10.1007/978-3-642-28869-2_10.

[7] Robert Fielding and LeanFM contributors. LeanFM: Grammar-based formal methods in Lean. GitHub repository, 2026. URL `https://github.com/rfielding/leanFM`. Accessed 2026-09-21.

[8] Harry J. Gensler. *Introduction to Logic*. Routledge, 3 edition, 2017. ISBN 978-1-138-91058-4. doi: 10.4324/9781315693361.

[9] Neil J. Gunther. *Guerrilla Capacity Planning: A Tactical Approach to Planning for Highly Scalable Applications and Services*. Springer, Berlin, Heidelberg, 2007. ISBN 978-3-540-31010-5. doi: 10.1007/978-3-540-31010-5.

[10] Kohei Honda. Types for dyadic interaction. In *CONCUR '93: 4th International Conference on Concurrency Theory*, volume 715 of *Lecture Notes in Computer Science*, pages 509–523. Springer, 1993. doi: 10.1007/3-540-57208-2_35.

[11] Kohei Honda, Vasco T. Vasconcelos, and Makoto Kubo. Language primitives and type discipline for structured communication-based programming. In *Programming Languages and Systems*

(ESOP 1998), volume 1381 of *Lecture Notes in Computer Science*, pages 122–138. Springer, 1998. doi: 10.1007/BFb0053567.

[12] Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 2008)*, pages 273–284. ACM, 2008. doi: 10.1145/1328438.1328472.

[13] Fred Kröger and Stephan Merz. *Temporal Logic and State Systems*. Texts in Theoretical Computer Science. An EATCS Series. Springer, Berlin, Heidelberg, 2008. ISBN 978-3-540-68635-4. doi: 10.1007/978-3-540-68635-4.

[14] Marta Kwiatkowska, Gethin Norman, and David Parker. PRISM 4.0: Verification of probabilistic real-time systems. In *Computer Aided Verification (CAV 2011)*, volume 6806 of *Lecture Notes in Computer Science*, pages 585–591. Springer, 2011. doi: 10.1007/978-3-642-22110-1_47.

[15] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. ISBN 978-0-321-14306-8.

[16] LeanFM. Kerberos: Time, knowledge, and obligation. YouTube video, 2026. Illustration of the proposed multimodal semantics, not an implemented general checker. `https://www.youtube.com/watch?v=f2efwxq8BaY`.

[17] Paolo Mancosu, Sergio Galvan, and Richard Zach. *An Introduction to Proof Theory: Normalization, Cut-Elimination, and Consistency Proofs*. Oxford University Press, Oxford, 2021. ISBN 978-0-19-289593-6. doi: 10.1093/oso/9780192895936.001.0001.

[18] Daniel A. Menascé, Virgilio A. F. Almeida, and Lawrence W. Dowdy. *Performance by Design: Computer Capacity Planning by Example*. Prentice Hall PTR, Upper Saddle River, NJ, 2004. ISBN 978-0-13-090673-1.

[19] Denis Merigoux, Nicolas Chataing, and Jonathan Protzenko. Catala: A programming language for the law. *Proceedings of the ACM on Programming Languages*, 5(ICFP), 2021. doi: 10.1145/3473582.

[20] Dusko Pavlovic. *Programs as Diagrams: From Categorical Computability to Computable Categories*. Springer Nature Switzerland, Cham, Switzerland, 2023. ISBN 978-3-031-34826-6. doi: 10.1007/978-3-031-34827-3.

[21] Dana Richards and Henry Hamburger. *Logic and Language Models for Computer Science*. World Scientific, 3 edition, 2017. ISBN 978-981-322-920-4. doi: 10.1142/10507.

[22] Peter Y. A. Ryan, Steve A. Schneider, Michael H. Goldsmith, Gavin Lowe, and A. W. Roscoe. *The Modelling and Analysis of Security Protocols: The CSP Approach*. Addison-Wesley, Harlow, England, 2001. ISBN 978-0-201-67471-2.

[23] Karen R. Sollins. The TFTP protocol (revision 2). RFC 783, 1981.

[24] Karen R. Sollins. The TFTP protocol (revision 2). RFC 1350, 1992.

[25] Nobuko Yoshida, Raymond Hu, Rumyana Neykova, and Nicholas Ng. The Scribble protocol language. In *Trustworthy Global Computing (TGC 2013)*, volume 8358 of *Lecture Notes in Computer Science*, pages 22–41. Springer, 2014. doi: 10.1007/978-3-319-05119-2_3.

Index

agreement, 151
alphabet
  vague, 48
API boundary, 5

bytes, 147

channel declaration, 5
channel kind, 40
chart specification, 25
choice
  directed, 36
Codex, 65
communicating finite-state machines, 5
composition
  requirement intersection, 62
CSP, 146
CTL, 114
cut, 12

deadline, 154
device pairing, 153

event, 19

failure
  masked, 7
  surfaced, 7
fairness, 46
formal methods, 2

generated examples, 129
global type, 3, 34
grammar
  context-free, 34
  multiparty, 34

hiding, 10

HTTP
  call discipline, 12
  protobuf adapter, 163

interaction diagram
  collapsed, 10
interface, 7
  hierarchy, 7
internal choice, 13
intruder model, 151
island, 7

knowledge
  derived from events, 111

Lean, 138
local type, 3, 39
Lowe's attack, 155
LTL, 114

modal logic, 114
modality, 154
Model Context Protocol, 65
  logic consultation, 65
multiparty session type, 47

naming
  implicit, 7
Needham–Schroeder, 155

OAuth, 99
observable event, 48
on-call handoff, 148

parameter flow, 39
past-time logic, 122
physics of a channel, 5, 40, 44
probability, 49
probability overlay, 129, 154

measured, 29
profit
derived from events, 29
project
directory boundary, 81
projection, 3, 39, 146
protocol
LeanFM web server, 163
protocol role, 146
protocol syntax, 34

queue capacity, 5, 45

realizability, 13
requirements
durable, 63

scenario, 48
sealed-bid auction, 149
secrecy, 151
session identifier, 41
session type, 2
multiparty, 2
run of, 19
sharding, 6
Sorcerer's Apprentice, 44

temporal logic, 114
TFTP, 44
time, 24
topology, 7
trace, 19

UDP, 45

vibecoding, 63

waste
derived from events, 29

XY plot, 29